Munich Quantum Toolkit and Optimizations of Quantum Programs

Table of Contents

Compiler Optimization for Quantum Computing Using Reinforcement Learning **2**

Assertion-Based Optimization of Quantum Programs **8**

Differentiable quantum computational chemistry with PennyLane **26**

The Munich Quantum Toolkit Flyer **44**

Quantum compiling by deep reinforcement learning **46**

MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing **53**

MQT Core: The Backbone of the Munich Quantum Toolkit (MQT) **75**

Quantum Circuit Optimization with AlphaTensor **77**

Structure optimization for parameterized quantum circuits **116**

END **128**

Compiler Optimization for Quantum Computing Using Reinforcement Learning

Nils Quetschlich* Lukas Burgholzer† Robert Wille*‡
*Chair for Design Automation, Technical University of Munich, Germany
†Institute for Integrated Circuits, Johannes Kepler University Linz, Austria
‡Software Competence Center Hagenberg GmbH (SCCH), Austria
nils.quetschlich@tum.de lukas.burgholzer@jku.at robert.wille@tum.de
https://www.cda.cit.tum.de/research/quantum

***Abstract*—Any quantum computing application, once encoded as a quantum circuit, must be compiled before being executable on a quantum computer. Similar to classical compilation, quantum compilation is a sequential process with many compilation steps and numerous possible optimization passes. Despite the similarities, the development of compilers for quantum computing is still in its infancy—lacking mutual consolidation on the best sequence of passes, compatibility, adaptability, and flexibility. In this work, we take advantage of decades of classical compiler optimization and propose a *reinforcement learning* framework for developing optimized quantum circuit compilation flows. Through distinct constraints and a unifying interface, the framework supports the combination of techniques from different compilers and optimization tools in a single compilation flow. Experimental evaluations show that the proposed framework—set up with a selection of compilation passes from IBM's Qiskit and Quantinuum's TKET—significantly outperforms both individual compilers in 73% of cases regarding the expected fidelity. The framework is available on GitHub (https://github.com/cda-tum/MQTPredictor) as part of the Munich Quantum Toolkit (MQT).**

I. Introduction

In classical computing, each *Central Processing Unit* (CPU) comes with its own *Instruction Set Architecture* (ISA)—describing the native operations the CPU can process. High-level programming languages such as *C++*, *Rust*, or *Go* are used to develop programs in a platform-agnostic way. These high-level descriptions are then *compiled* to a specific device's ISA by a *compiler*. Prominent examples are *LLVM* [1] or *GCC* for C++, *rustc* for Rust, and *gc* or *gccgo* for Go. In each of these compilers, a plethora of passes for analyzing and transforming the source code is available. The "quality" of the code generated by compilers heavily depends on the selection of the individual passes applied during the compilation process as well as their execution order. Furthermore, one particular combination is highly unlikely to be the most effective sequence for every program on a particular machine. Even nowadays, after decades of classical compiler research, determining good sequences of compilation passes turns out to be a challenging task. In the literature, this problem is commonly referred to as *Phase Ordering*.

Research on the phase ordering problem already started roughly four decades ago [2]–[4] and is still very actively pursued. In the recent years, *autotuning* [5] (trying out different compilation parameters and evaluating the results with respect to a particular metric) and machine learning [6] have shown promising results. A particularly promising branch of research is the application of *Reinforcement Learning* (RL, [7], [8]) for this purpose. In [9], the authors show that their RL-based approach to phase ordering outperforms LLVM's optimization scheme. Reinforcement learning has also successfully been applied for inlining optimization [10] and vectorization [11].

The compilation of quantum circuits/programs follows a very similar scheme as the compilation of classical programs. All major quantum SDKs (such as IBM's Qiskit [12], Quantinuum's TKET [13], AWS Braket [14], or Microsoft's Azure Quantum [15]) implement some sort of compilation flow for making circuits executable on particular devices (see, e.g., [16]). However, while research on classical compilers has been conducted for decades, compilers for quantum computing are still in early development. This manifests in several ways:

1) There is no mutual consolidation on what the best methods and combinations thereof are.
2) It is not straightforward to mix and match different options from various frameworks—creating something similar to a vendor lock-in.
3) Major SDKs can be quite slow in adopting the newest research findings—effectively restraining users from taking advantage of state-of-the-art methods.
4) Users can hardly customize the optimization goal or the target metric for the compilation process. Instead, they have to rely on predefined combinations of compilation passes provided by the SDKs—typically in the form of various optimization levels.

As a consequence, we are quickly heading towards the same problems as in classical compiler development and, despite the similarities, techniques from the classical realm cannot be straightforwardly applied in the quantum realm due to the inherent differences of both computational paradigms.

In this work, we aim to mitigate the drawbacks discussed above. Instead of trying to reinvent the wheel, the solution proposed in this work builds upon the decades of research on compiler optimization in the classical realm. To this end, we adapt an established classical technique to the quantum domain—modeling the compilation flow as a *Markov Decision Process* (MDP, [17]) and applying *Reinforcement Learning* to learn the best order in which the different compilation passes and optimizations can be applied.

The resulting framework (which is available on GitHub (https://github.com/cda-tum/MQTPredictor) as part of the Munich Quantum Toolkit (MQT)) is platform agnostic and easily extendable with further compilation methods from different SDKs and software tools. This is accomplished by defining distinct constraints that can be efficiently checked for each state in the process and a unifying interface for each compilation step. In contrast to slowly-adapting compilers in major SDKs, this allows to develop optimized compilation flows that can keep up with the fast-paced development and research in this domain.

Experimental evaluations demonstrate the feasibility of the proposed approach. Compared to the compilation flows offered by IBM's Qiskit and Quantinuum's TKET, the proposed framework—set up with a selection of compilation passes from both tools—is shown to outperform both individual compilers with respect to expected fidelity, critical depth, and their combination in 73%, 84%, 75% of cases.

The rest of this paper is structured as follows. Section II briefly reviews quantum circuit compilation. Then, Section III describes how compiler optimization for quantum circuits can be modeled as an MDP and how it can be tackled with reinforcement learning. Based on that, Section IV provides details on the implementation of the proposed framework and evaluates its performance. Finally, Section V concludes the paper.

II. Background: Quantum Circuit Compilation

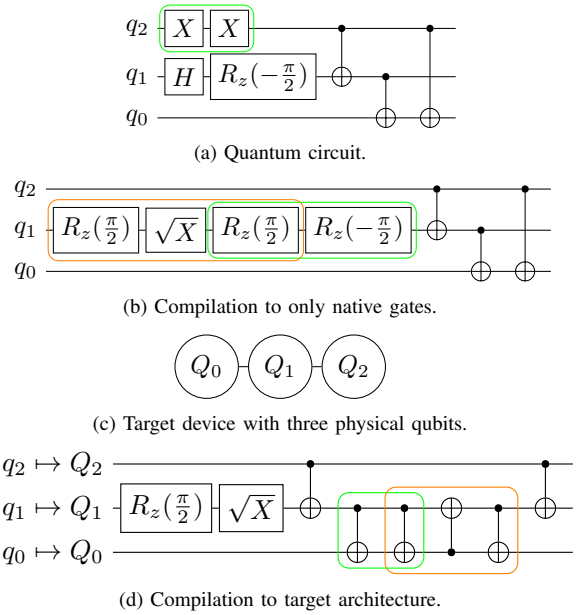
Quantum programs are commonly described in the form of *quantum circuits*, i.e., sequences of quantum operations (also called quantum gates) acting on the qubits of a quantum system.

Example 1. *A simple quantum circuit operating on three qubits (q_0, q_1, q_2) is shown in Fig. 1a. It contains four different kinds of gates: Two NOT gates (denoted as* X*), one Hadamard gate (denoted as* H*), one Z-rotation gate (denoted as R_z), and three two-qubit controlled-NOT or CNOT gates (with their control and target qubits denoted as $\bullet$ and $\oplus$, respectively).*

Executing such programs on an actual *Quantum Processing Unit* (QPU) requires *compilation* conducted by *compilers*. They usually first employ device-independent optimizations to reduce the complexity of the original circuit description. Examples of such optimizations include gate-cancellation, gate-commutation, or high-level synthesis routines, e.g., proposed in [18]–[22].

Example 2. *Consider again the circuit shown in Fig. 1a. Employing device-independent optimizations would, e.g., lead to the cancellation of the consecutive X gates on q_2 (highlighted by the green box) since $X \cdot X = I$.*

After these initial optimizations, QPU-specific circuits are generated. To this end, each quantum processor offers a particular set of *native gates*, i.e., operations that can be directly executed on the device. Consequently, all non-native gates need to be *synthesized* using these native gates for the circuit to be executable. This synthesis step creates opportunities for further optimizations.



(a) Quantum circuit.

(b) Compilation to only native gates.

(c) Target device with three physical qubits.

(d) Compilation to target architecture.

Fig. 1. Compilation of a quantum circuit to a targeted device.

Example 3. *Recent devices offered by IBM provide a native gate-set consisting of R_z, $\sqrt{X}$,* X*, and* CNOT *gates. To execute the circuit from Fig. 1a on such a device, e.g., the Hadamard gate needs to be expressed in terms of these gates. A suitable decomposition into two $R_z(\frac{\pi}{2})$ and a $\sqrt{X}$ gate is shown in Fig. 1b (highlighted by the orange box). Incidentally, one of the $R_z(\frac{\pi}{2})$ gates can, then, be cancelled with the existing $R_z(-\frac{\pi}{2})$ gate (highlighted by the green box).*

Some QPUs (e.g., those based on superconducting qubits or neutral atoms) only feature a limited connectivity between their qubits, so that multi-qubit gates can only be applied to qubits that are connected on the device. In these cases, a *mapping* step is required. Mapping is frequently split into *layouting*, i.e., assigning the circuit's logical qubits to the device's physical qubits, and *routing*, i.e., ensuring the connectivity constraints are satisfied by dynamically changing the assignment of logical to physical qubits. Again, optimizations may be applied after mapping the circuit to reduce the induced overhead.

Example 4. *Assume the circuit from Fig. 1b shall be executed on an IBM device with three physical qubits (Q_0, Q_1, Q_2) arranged in a line (as depicted in Fig. 1c), i.e., only interactions between Q_0 and Q_1 as well as Q_1 and Q_2 are possible. Then, no layout exists that makes this circuit executable right away since the circuit contains interactions between all pairs of qubits. Thus, a SWAP gate (eventually realized as a sequence of three CNOT gates highlighted by the orange box) needs to be inserted during the routing step to make the circuit executable (as shown in Fig. 1d). In this case, some of the induced overhead can be reduced by cancelling two CNOT gates (highlighted by the green box).*

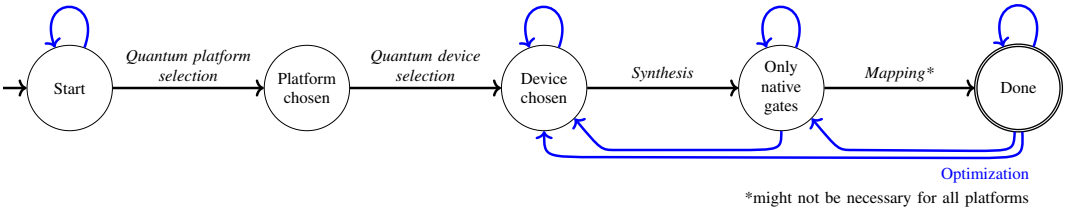


Fig. 2. Quantum compilation flow modeled as an MDP.

With the number of different synthesis, mapping, and optimization techniques for quantum circuits rapidly increasing, the landscape of compiler passes is becoming more and more complex—leading to several obstacles as discussed in Section I. In this work, we propose a framework that aims to overcome these obstacles.

III. Proposed Framework

Fortunately, the underlying problem is not new per-se, since classical compilers have seen a similar trend in the past and lots of research has been conducted to find efficient solutions there. In the following, we describe how a classically-inspired reinforcement learning technique can be adapted to quantum circuit compilation. To this end, we first describe how to programmatically model quantum circuit compilation in order to, then, apply reinforcement learning to try to learn the best sequence of compilation passes depending on characteristics of the initial circuit—resulting in an *optimized compiler* for quantum circuits.

A. Modeling as a Markov Decision Process

The quantum circuit compilation flow reviewed above can be seen as a sequential procedure that takes an initial (high-level) quantum circuit and, step by step, applies certain actions/transformations to eventually arrive at a circuit representation that can be executed on a target device. Such a procedure can be formally modeled as a *Markov Decision Process* (MDP, [17]). To this end, it is split into states and corresponding actions that can be performed in each of the states. The model proposed in this work is illustrated in Fig. 2—with states being denoted as circles and actions as arrows. In the following, the process with its respective actions is described one by one.

Initially, the process starts in the "Start" state and stores the original quantum circuit to be compiled. At this point, there are two possible types of actions. On the one hand, device-independent *optimizations* can be applied to the circuit (indicated by the blue arrow). As reviewed in Section II, this includes techniques such as single-qubit gate fusion, gate cancellation and commutation, as well as high-level synthesis methods, e.g., proposed in [18]–[22]. After such an optimization, the process remains in the same state as before, but now stores a new optimized quantum circuit. On the other hand, the process can proceed to a different state by *Quantum platform selection* actions, e.g., choosing *IBM*, *Rigetti*, or *IonQ*—effectively fixing the targeted native gate-set. In the classical domain, this roughly corresponds to deciding, e.g., between *x86* and *ARM* architectures.

Once a platform has been chosen (i.e., the process is in the "Platform chosen" state), only a single type of action can be applied. Namely, the *Quantum device selection* actions offered by the selected platform. This step fixes the number of qubits and their topology, i.e., which qubits can interact with each other. Depending on the choice of the platform, a varying range of devices with different characteristics is available. At the time of writing, IBM for example offers 23 different quantum computers that each vary in their qubit count, their quantum volume, and their *Circuit Layer Operations Per Second* (CLOPS). In the classical domain, this roughly corresponds to targeting a specific CPU model family, such as *Intel Alder Lake* or *AMD Zen 4*.

After a particular device has been selected (i.e., the process is in the "Device chosen" state), the device-dependent compilation begins. The goal of the remaining steps is to produce a circuit that can be executed on the chosen device. To this end, the circuit needs to satisfy two conditions:

1) it must only use gates native on the selected platform, and
2) it must conform to the topology of the selected device.

While the first condition can be ensured by using *Synthesis* methods, e.g., as proposed in [23]–[26], the second condition is realized by corresponding *Mapping* methods, e.g., as proposed in [27]–[29]. Both of these tasks are highly non-trivial and there is a vast variety of methods available in major quantum SDKs as well as individual research tools. At the time of writing, IBM's Qiskit alone offers four different layout and five routing algorithms—leading to 20 different mapping schemes. Once both conditions are satisfied, the process is in the "Done" state, i.e., the circuit is executable. Optimizations may be applied in any state during the device-dependent compilation. Depending on whether the resulting circuit satisfies none, the native-gates, or both constraints, the process respectively continues in the "Device chosen", "Only native gates", or "Done" state (again, indicated by the blue arrows).

Overall, the formulation described above leads to a modular framework with two characteristic traits:

1) Each state has *distinct constraints* that can be efficiently checked, e.g., whether the circuit only contains certain native gates or only acts on qubits connected on the device. Thus, it is straight-forward to determine the state after a certain action has been applied.
2) All actions have a *unified interface*, i.e., all of them use a quantum circuit as the main representation for their input and output. Moreover, the same format is used independent of the SDK or tool the actions originate from.

As a consequence, it is possible to mix and match compiler passes from different SDKs and tools and integrate them into a single *optimized* compiler. Such a framework also allows to mitigate vendor lock-ins and can help to keep up with the rapid pace of the field by allowing one to quickly integrate new methods into the overall compilation flow.

B. Compiler Optimization Using Reinforcement Learning

After a framework as described above is set up, it can be used as the basis for learning the best sequence of actions (i.e., compilation passes) depending on characteristics of the initial circuit. Motivated by its success in classical compiler optimization [9]–[11], we propose to apply reinforcement learning for this task. RL-methods aim to learn an *action policy* that describes which action to apply based on certain *observations* on the current state such that some cumulative *reward function* is maximized. In contrast to supervised machine learning methods, no training data must be provided. Instead, only an environment describing the underlying MDP (i.e., the set of states and corresponding actions), the features used as observations, and a corresponding reward function needs to be provided. Based on the rewards for certain actions in certain states, the action policy improves through the training process and learns how to maximize the expected cumulative reward.

In the proposed framework, observations are realized by extracting certain features from the quantum circuits in each state. In addition, a sparse reward function is used that reflects whether a final state has been reached and, then, quantifies the "quality" of the resulting circuit, e.g., in terms of the resulting gate count, circuit depth, or expected fidelity. This flexibility in the target metric allows one to design specialized compilers for dedicated use cases. After successful training, the learned action policy can be used as an *optimized compiler* for any quantum circuit.

C. Related Work

Machine learning has been successfully applied to quantum circuit design/compilation in the past. In [30], the authors propose a machine learning based layout and routing scheme and claim to be on-par with state-of-the-art methods. Another possible application is given in [31], where the graph structure of quantum circuits is used to predict their expected fidelities for certain devices considering their respective noise characteristics. An idea related to the one proposed in this work, which follows a more coarse-grained approach, has been demonstrated in [32]. There, a supervised machine learning model is used to predict good combinations of devices and existing compilation flows. In [33], RL has been applied to learn a strategy for applying individual gate transformation rules to optimize quantum circuits.

While these works either optimize one specific type of action or predict compilation options on a high abstraction level within one quantum SDK, the framework proposed in this work results in fine-grained steps that allow one to stitch together methods from multiple SDKs with a customizable optimization objective into a fully-fledged compilation flow.

IV. Feasibility Study

In the following, the feasibility of the proposed framework is demonstrated. To this end, we first describe the particular instantiation that has been developed as part of this work. Then, the performance of the resulting optimized compiler is evaluated and compared against the state-of-the-art compilers available in IBM's Qiskit [12] and Quantinuum's TKET [13]. All developments and results are made publicly available on GitHub (https://github.com/cda-tum/MQTPredictor) as part of the Munich Quantum Toolkit (MQT).

A. Instantiation

The basis of the framework is formed by the open-source *OpenAI Gym* [34] library, which is used to set up an environment for the MDP described in Section III (as illustrated in Fig. 2). In this regard, the main endeavor is the selection and integration of all compilation passes that should be considered as actions in the learning process. As modeled in Fig. 2, we distinguish five kinds of actions: Quantum platform selection, Quantum device selection, Synthesis, Mapping, and Optmization. Due to the unified action interface of the proposed framework described earlier, methods from more than one SDK or toolkit can be incorporated. For each kind of action, suitable representatives from IBM's Qiskit (version 0.39.2) and/or Quantinuum's TKET (version 1.8.1) have been chosen—leading to the following list of available actions.

- *Platforms* and corresponding *Devices*:
  - IBM (superconducting): *ibmq_montreal* (27 qubits) and *ibmq_washington* (127 qubits)
  - Rigetti (superconducting): *Aspen-M-2* (80 qubits)
  - IonQ (trapped-ions): *Harmony* (11 qubits)
  - OQC (superconducting): *Lucy* (8 qubits)
- *Synthesis*: Qiskit's *BasisTranslator*
- *Mapping*: Any combination of the following methods
  - Layout:
    - Qiskit's *TrivialLayout*
    - Qiskit's *DenseLayout*
    - Qiskit's *SabreLayout*
  - Routing:
    - Qiskit's *BasicSwap*
    - Qiskit's *StochasticSwap*
    - Qiskit's *SabreSwap*
    - TKET's *RoutingPass*
- *Optimization*:
  - Qiskit's *Optimize1qGatesDecomposition*
  - Qiskit's *CXCancellation*
  - Qiskit's *CommutativeCancellation*
  - Qiskit's *CommutativeInverseCancellation*
  - Qiskit's *RemoveDiagonalGatesBeforeMeasure*
  - Qiskit's *InverseCancellation*
  - Qiskit's *OptimizeCliffords*
  - Qiskit's *Collect2qBlocks* + *ConsolidateBlocks*
  - TKET's *PeepholeOptimise2Q*
  - TKET's *CliffordSimp*
  - TKET's *FullPeepholeOptimise*
  - TKET's *RemoveRedundancies*

Inspired by the feature selection and demonstrated relevance in [32], the observations used to guide the reinforcement learning agent are based on seven features—namely the *number of qubits*, the *depth* of the circuit, and the five composite features of *program communication*, *critical-depth*, *entanglement-ratio*, *parallelism*, and *liveness* originally proposed in [35]. For the learning itself, three different reward functions have been implemented:

1) *Expected fidelity*: provides an estimate on the reliability of the circuit based on characteristics of the chosen device. A fidelity of 1 indicates an error-free result, while a fidelity of 0 indicates that the result is completely wrong.
2) *Critical Depth*: describes how many two-qubit gates in a quantum circuit lie along the critical path and contribute to the overall circuit depth. A critical depth close to 1 indicates a highly sequential quantum circuit. Since, a lower value is desirable, the corresponding reward function is defined as $1 -$ critical depth.
3) *Combination*: adds up both mentioned reward functions (divided by two to result in a value between 0 and 1).

Finally, the *Proximal Policy Optimization (PPO)* algorithm [36] provided by *Stable-Baselines3* [37] is used for the reinforcement learning.

B. Results

To evaluate the performance of the framework instantiation described above, three models—one for each reward function—were trained on a selection of 200 circuits ranging from 2 to 20 qubits taken from the *target-independent* level of the MQT Bench library [38]. In total, $100,000$ time steps were executed during the training for each reward function with runtimes of the order of hours. Then, the trained models were evaluated on the same circuits and the resulting quality in terms of the respective reward function was measured. To put this performance into perspective, all circuits are additionally compiled to the *ibmq_washington* device using Qiskit and TKET with the highest optimization level ($O3$ for Qiskit and $O2$ for TKET).

The results are summarized in Fig. 3. To this end, Fig. 3a, Fig. 3b, and Fig. 3c, respectively show the distribution of *absolute* reward differences of the proposed approach compared to Qiskit and TKET with respect to the three quality metrics. An x-value larger than zero indicates a positive improvement, e.g., a value of 0.2 for the fidelity means that the proposed approach managed to boost the fidelity by 20%. In an ideal scenario, all results would accumulate on the right-side of the zero-line.

The results demonstrate that, in the vast majority of cases, this is the case, i.e., the proposed method produces circuits of equal or better quality. In particular, the proposed method outperforms the Qiskit/TKET compiler in 73%/80%, 84%/86%, and 75%/78.5% of cases regarding expected fidelity, critical depth, and their combination.

The advantage of the proposed method becomes even clearer when looking at the average reward difference per benchmark algorithm (subsuming several instances with varying amounts of qubits) as shown in Fig. 3d, Fig. 3e, and Fig. 3f respectively. There, y-values higher than zero indicate a net gain in quality. Again, in the vast majority of cases, a substantial improvement can be observed. Compared to Qiskit/TKET, an average *absolute* increase in fidelity, critical depth, and their combination of 4.9%/10.7%, 22.6%/22.8%, and 5.5%/8.5% is achieved.

TABLE I
COMPARISON OF REINFORCEMENT LEARNING MODELS.

Model trained for...	Average result for... Fidelity	Critical depth	Combination
Fidelity	**0.48**	0.27	0.37
Critical depth	0.18	**0.47**	0.33
Combination	0.45	0.33	**0.39**

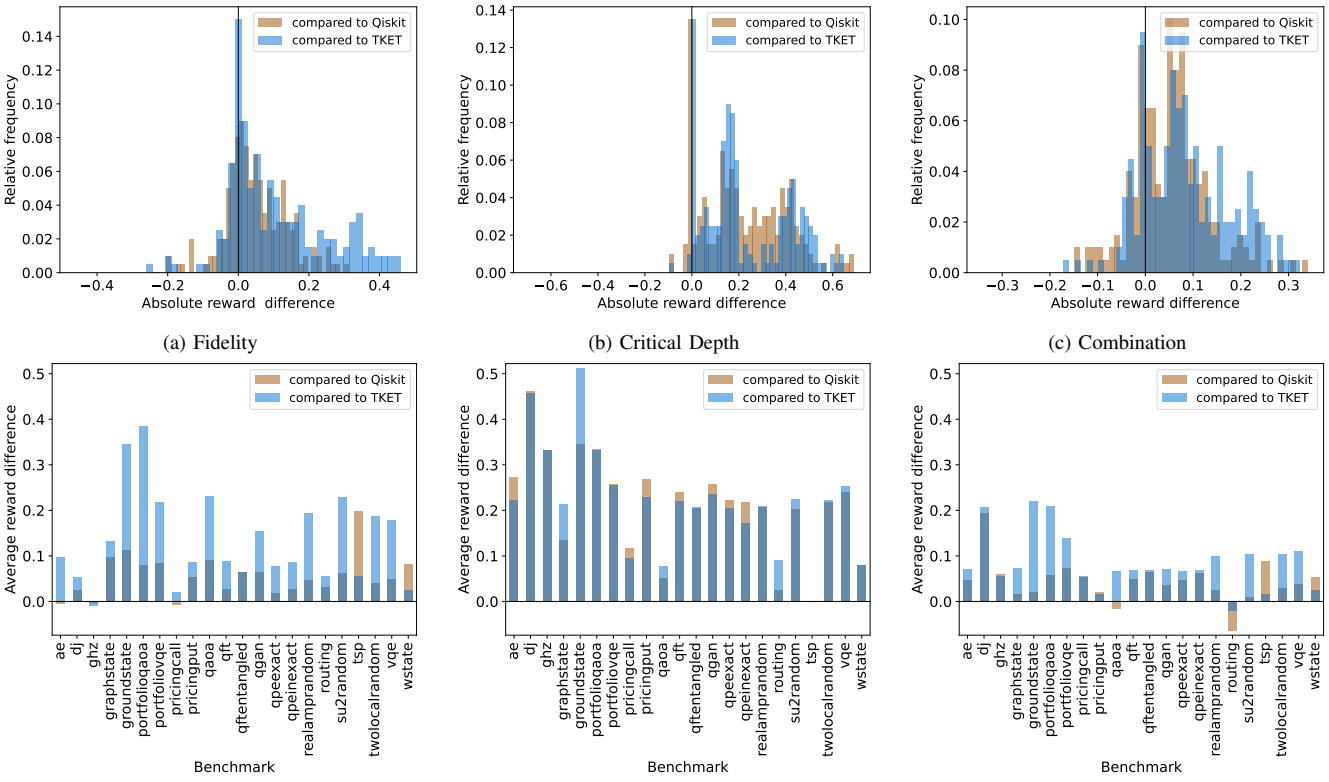
To further underline the proposed framework's ability to adapt to different target metrics, we additionally calculated the average reward for each metric by all three trained models—not only the one trained for the respective metric. The resulting numbers are denoted in Table I. They confirm that the model trained for a particular purpose indeed results in the compiled circuits with the highest rewards for all three cases.

V. CONCLUSIONS

Quantum circuit compilation flows are becoming more and more complex. This leads to a similar situation observed in classical compilation where there are so many options and it is unclear which options to choose and in which order to apply them. In this work, we proposed a framework that builds on the decades of research on classical compiler optimization to overcome these obstacles. To this end, the quantum circuit compilation flow was modeled as a Markov Decision Process with distinct constraints for each state and a unified interface for all actions. Based on that, reinforcement learning was applied to derive a complete compilation flow out of actions from multiple SDKs and toolkits. Experimental evaluations on the developed framework and three different optimization objectives demonstrate the advantage of the proposed method over Qiskit's and TKET's most optimized compilation sequences and the general feasibility of automatically creating compilations flows targeting a customizable objective. In 73%, 84%, 75% of the cases, the trained models outperformed Qiskit/TKET with an average *absolute* improvement of 4.9%/10.7%, 22.6%/22.8%, and 5.5%/8.5% regarding expected fidelity, critical depth, and their combination respectively. These results clearly demonstrate that adapting techniques from the classical realm can, indeed, yield substantial improvements in the quantum realm. By making the developed framework publicly available as open-source, we hope to lay the foundation for further exploration of this important area of research.

ACKNOWLEDGMENTS

This work received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (grant agreement No. 101001318), was part of the Munich Quantum Valley, which is supported by the Bavarian state government with funds from the Hightech Agenda Bayern Plus, and has been supported by the BMWK on the basis of a decision by the German Bundestag through project QuaST, as well as by the BMK, BMDW, and the State of Upper Austria in the frame of the COMET program (managed by the FFG).



(a) Fidelity (b) Critical Depth (c) Combination

(d) Fidelity (e) Critical Depth (f) Combination

Fig. 3. Experimental evaluation.

REFERENCES

[1] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization*, 2004.

[2] S. R. Vegdahl, "Phase coupling and constant generation in an optimizing microcode compiler," in *Workshop on Microprogramming*, 1982.

[3] D. B. Loveman and R. A. Faneuf, "Program optimization - theory and practice," *SIGPLAN Not.*, 1975.

[4] D. L. Whitfield and M. L. Soffa, "An approach for exploring code improving transformations," *ACM Trans. Program. Lang. Syst.*, 1997.

[5] A. H. Ashouri *et al.*, "A survey on compiler autotuning using machine learning," *ACM Comput. Surv.*, 2018.

[6] H. Leather and C. Cummins, "Machine learning in compilers: Past, present and future," in *Forum on Specification and Design Languages*, 2020.

[7] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of Artificial Intelligence Research*, 1996.

[8] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[9] Q. Huang *et al.*, *Autophase: Juggling hls phase orderings in random forests with deep reinforcement learning*, 2020. arXiv: 2003.00671.

[10] M. Trofin *et al.*, *MIgo: A machine learning guided compiler optimizations framework*, 2021. arXiv: 2101.04808.

[11] A. Haj-Ali *et al.*, *Neurovectorizer: End-to-end vectorization with deep reinforcement learning*, 2019. arXiv: 1909.13639.

[12] Qiskit contributors, *Qiskit: An open-source framework for quantum computing*, 2023.

[13] S. Sivirajah *et al.*, "TKET: A Retargetable Compiler for NISQ devices," *Quantum Science and Technology*, 2020.

[14] *Amazon Braket Python SDK*, Amazon Web Services, 2022. [Online]. Available: https://github.com/aws/amazon-braket-sdk-python.

[15] *Azure Quantum documentation*, Microsoft, 2022. [Online]. Available: https://learn.microsoft.com/en-us/azure/quantum/.

[16] R. Wille, R. Van Meter, and Y. Naveh, "IBM's Qiskit tool chain: Working with and developing for real quantum computers," in *Design, Automation and Test in Europe*, 2019.

[17] R. Bellman, "A markovian decision process," *Journal of Mathematics and Mechanics*, 1957.

[18] W. Hattori and S. Yamashita, "Quantum circuit optimization by changing the gate order for 2D nearest neighbor architectures," in *Int'l Conf. of Reversible Computation*, 2018.

[19] P. Niemann, R. Wille, and R. Drechsler, "Advanced exact synthesis of clifford+t circuits," *Quantum Information Processing*, 2020.

[20] G. Vidal and C. M. Dawson, "Universal quantum circuit for two-qubit transformations with three controlled-NOT gates," *Physical Review A*, 2004.

[21] T. Itoko *et al.*, "Quantum circuit compilers using gate commutation rules," in *Asia and South Pacific Design Automation Conf.*, 2019.

[22] D. Maslov, G. Dueck, D. Miller, and C. Negrevergne, "Quantum circuit simplification and level compaction," *IEEE Trans. on CAD of Integrated Circuits and Systems*, 2008.

[23] B. Giles and P. Selinger, "Exact synthesis of multiqubit Clifford+T circuits," *Physical Review A*, 2013.

[24] M. Amy, D. Maslov, M. Mosca, and M. Roetteler, "A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits," *IEEE Trans. on CAD of Integrated Circuits and Systems*, 2013.

[25] D. M. Miller, R. Wille, and Z. Sasanian, "Elementary quantum gate realizations for multiple-control Toffoli gates," in *Int'l Symp. on Multi-Valued Logic*, 2011.

[26] S. Schneider, L. Burgholzer, and R. Wille, "A SAT encoding for optimal Clifford circuit synthesis," in *Asia and South Pacific Design Automation Conf.*, 2023.

[27] M. Y. Siraichi, V. F. dos Santos, S. Collange, and F. M. Q. Pereira, "Qubit allocation," in *Int'l Symp. on Code Generation and Optimization*, 2018.

[28] A. Zulehner, A. Paler, and R. Wille, "An efficient methodology for mapping quantum circuits to the IBM QX architectures," *IEEE Trans. on CAD of Integrated Circuits and Systems*, 2019.

[29] A. Matsuo and S. Yamashita, "An efficient method for quantum circuit placement problem on a 2-D grid," in *Int'l Conf. of Reversible Computation*, 2019.

[30] A. Paler, L. M. Sasu, A.-C. Florea, and R. Andonie, "Machine learning optimization of quantum circuit layouts," *ACM Transactions on Quantum Computing*, 2022.

[31] H. Wang *et al.*, "Quest: Graph transformer for quantum circuit reliability estimation," 2022. arXiv: 2210.16724.

[32] N. Quetschlich, L. Burgholzer, and R. Wille, *Predicting good quantum circuit compilation options*, 2022. arXiv: 2210.08027.

[33] T. Fösel, M. Y. Niu, F. Marquardt, and L. Li, *Quantum circuit optimization with deep reinforcement learning*, 2021. arXiv: 2103.07585.

[34] G. Brockman *et al.*, *OpenAI Gym*, 2016. arXiv: 1606.01540.

[35] T. Tomesh *et al.*, *Supermarq: A scalable quantum benchmark suite*, 2022. arXiv: 2202.11045.

[36] J. Schulman *et al.*, *Proximal policy optimization algorithms*, 2017. arXiv: 11707.06347.

[37] A. Raffin *et al.*, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, 2021.

[38] N. Quetschlich, L. Burgholzer, and R. Wille, *MQT Bench: Benchmarking software and design automation tools for quantum computing*, MQT Bench is available at https://www.cda.cit.tum.de/mqtbench/, 2022. arXiv: 2204.13719.

Assertion-Based Optimization of Quantum Programs

THOMAS HÄNER*, ETH Zürich, Switzerland
TORSTEN HOEFLER, ETH Zürich, Switzerland
MATTHIAS TROYER, Microsoft, USA

Quantum computers promise to perform certain computations exponentially faster than any classical device. Precise control over their physical implementation and proper shielding from unwanted interactions with the environment become more difficult as the space/time volume of the computation grows. Code optimization is thus crucial in order to reduce resource requirements to the greatest extent possible. Besides manual optimization, previous work has adapted classical methods such as constant-folding and common subexpression elimination to the quantum domain. However, such classically-inspired methods fail to exploit certain optimization opportunities across subroutine boundaries, limiting the effectiveness of software reuse. To address this insufficiency, we introduce an optimization methodology which employs annotations that describe how subsystems are entangled in order to exploit these optimization opportunities. We formalize our approach, prove its correctness, and present benchmarks: Without any prior manual optimization, our methodology is able to reduce, e.g., the qubit requirements of a 64-bit floating-point subroutine by 34×.

Additional Key Words and Phrases: quantum computing, quantum circuit optimization

1 INTRODUCTION

Quantum computers promise to solve certain computational tasks exponentially faster than classical computers. As a result, significant resources are being spent in order to make quantum computing become reality. In anticipation of the first quantum computers, the most promising applications are being identified and manually optimized for specific problems of practical interest, resulting in great resource savings [Gidney and Ekerå 2019; Häner et al. 2017; Hastings et al. 2015; Kutin 2006; Reiher et al. 2017]. Such improvements are crucial, given the considerable overhead due to quantum error correction [Fowler et al. 2012] and the difficulty of engineering large-scale quantum computers.

To identify promising applications for quantum computing, it is necessary to develop a detailed understanding of all components of the quantum algorithm being studied. One possible approach is to implement the algorithm in a quantum programming language, as this also enables testing and debugging. To this end, a host of software packages, programming languages, methodologies, and compilers for quantum computing have been developed [Chong et al. 2017; Green et al. 2013; IBM 2018; JavadiAbhari et al. 2014; Paykin et al. 2017; Smith et al. 2016; Steiger et al. 2018; Svore et al. 2018]. In addition to providing the necessary layers of abstraction to facilitate software development, these packages include optimizing compilers that aim to reduce width and depth of the resulting quantum circuit, e.g., by merging operations at various layers of abstraction [Häner et al. 2018b]. Further optimization opportunities can be created by employing a set of commutation relations [Nam et al. 2018] to reorder operations. Moreover, several methods have been developed for exact circuit synthesis with certain optimality guarantees [Amy et al. 2013; Große et al. 2007, 2009; Kliuchnikov et al. 2013; Meuli et al. 2018]. While these methods alone are not suitable for optimization of large-scale quantum circuits, they can be combined with heuristics [Amy et al. 2014; Nam et al. 2018].

*Also with Microsoft Quantum, Zürich.

Authors' addresses: Thomas Häner, ETH Zürich, Switzerland; Torsten Hoefler, ETH Zürich, Switzerland; Matthias Troyer, Microsoft, USA.

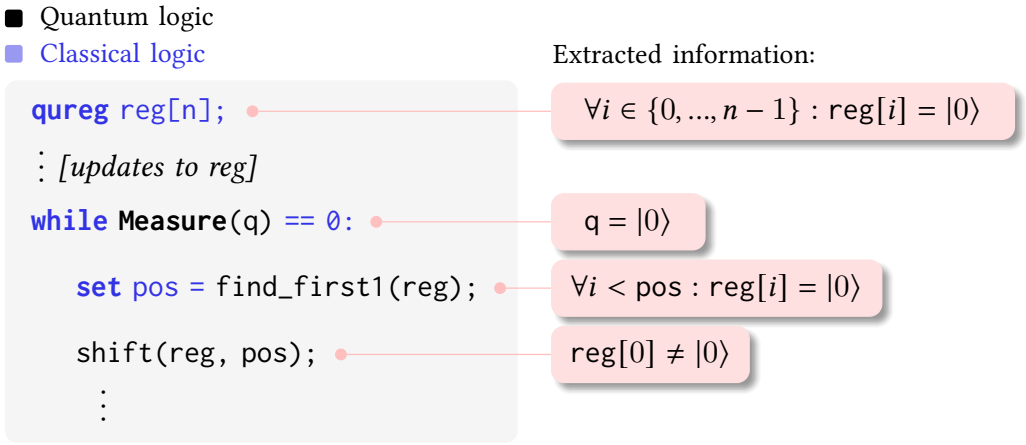


Fig. 1. Example depicting a quantum program and the corresponding information that our methodology infers from the postconditions of both classical and quantum subroutines. It uses this information to exploit optimization opportunities that arise due to subroutine composition. Here, the `shift` subroutine may be optimized, as discussed in Section 5.

Despite these efforts, most of the progress made in, e.g., quantum chemistry has been due to manual optimization [Hastings et al. 2015; Jones et al. 2012]. This suggests that the capabilities of optimizing compilers may still be significantly improved; especially at higher levels of abstraction where manual optimizations are carried out today. Such optimization opportunities usually arise when two or more existing subroutines are combined instead of directly implementing the desired functionality more efficiently. Exploiting these opportunities is only possible when optimizing across the boundaries of both quantum and classical subroutines. To enable such transformations, an optimizing compiler requires access to additional information such as the circumstances under which a given subroutine is invoked. While compilers may not be able to infer the semantics of a given program and its subroutines, such information may be extracted from assertions, and then used for program *optimization*. Specifically, we propose to use the pre- and postconditions of each subroutine in order to gather information about the state of the quantum computer before and after completion of the subroutine. Fig. 1 depicts a mixed quantum/classical code example and annotations of postconditions and derived facts for both classical and quantum subroutines. The gathered information may then be combined, e.g., with conditions specifying the circumstances under which a given subroutine acts trivially. If these conditions are met, our methodology removes such operations and is thus able to reduce both space and time requirements of a given quantum program.

Contributions. We develop a formalism that allows us to express (1) how subsystems (subsets of qubits) of the quantum computer are entangled and (2) under what circumstances the program may be optimized. Our methodology then uses this information for quantum program optimization. We prove its correctness and present a prototype implementation that successfully reduces the quantum resource requirements of common arithmetic subroutines by up to 410× (for a 64-bit floating-point subroutine). The chosen subroutines are frequently used in a wide range of quantum algorithms, including Shor's algorithm for factoring [Shor 1994], HHL for solving linear systems of equations [Harrow et al. 2009], algorithms for quantum chemistry [Babbush et al. 2016], and Grover's algorithm [Grover 1996] when used for optimization. Our proposed automatic program-level optimizations are thus beneficial for a wide range of quantum computing applications.

Related work. By carrying out optimizations across different subroutines of the quantum program, our methodology complements available tools for lower-level circuit optimization and synthesis [Amy et al. 2014, 2013; Nam et al. 2018]. Crucially, our methodology enables more effective use of abstractions when implementing libraries for quantum computing because it is able to remove the resulting overheads. Furthermore, previous work on high-level quantum program optimization [Häner et al. 2018b; Steiger et al. 2018], which adapted common subexpression elimination and constant-folding to the quantum domain, cannot handle the examples we present in Section 5. Moreover, the verification of quantum programs has been addressed through the introduction of a quantum Hoare logic [Ying 2012]. In contrast, our methodology uses the information that it gathers from pre- and postconditions of subroutines for optimization purposes.

2 PRELIMINARIES

In this section, we provide some background on quantum computing and quantum programs. For a more in-depth treatment of quantum computing, we refer to the textbook by Nielsen and Chuang [Nielsen and Chuang 2010].

2.1 Qubits and Gates

Whereas classical computers manipulate bits in order to solve a certain computational task, their quantum counterparts operate on so-called quantum bits, or *qubits*. A qubit is a two-level quantum system, i.e., a system which can be in two distinguishable states. An example would be the ground and first excited state of an ion, where we can denote the ground state by 0 and the first excited state as 1.

The principle of *quantum superposition* states that a single qubit can be in a complex superposition of its two levels. This means that there are two complex numbers associated with the quantum state of a qubit: the contribution from the 0-state and another one from the 1-state. Let us denote these two complex numbers by α_0 and α_1, respectively, where we require that $|\alpha_0|^2 + |\alpha_1|^2 = 1$. Given a qubit in a quantum state which is described by these two values, the probability of observing the qubit in state 0 or 1 is $|\alpha_0|^2$ or $|\alpha_1|^2$, respectively. Note that the normalization condition above ensures that the two probabilities sum up to 1.

If we add a single qubit to a system of $n-1$ qubits, the resulting system must be described in general using twice as many complex values due to *quantum entanglement*. Two subsystems being entangled means that one cannot write down the state of the entire system as a product state of the two subsystems. As a result, operations that act on one subsystem (such as measurement) may have nontrivial effects on the other. The state of an n-qubit quantum computer can be described using 2^n complex amplitudes that correspond to the contributions stemming from the all-zero state, the all-zero state but where the last bit is 1, up to the all-one state. We denote the corresponding amplitudes by $\alpha_{0\cdots00}, \alpha_{0\cdots01}, \ldots, \alpha_{1\cdots11}$ and, as a simplification, we interpret the indices as a number written in binary format, allowing to write $\alpha_0, \alpha_1, \ldots, \alpha_{2^n-1}$. When reading out, or *measuring*, all n qubits, the probability of observing the binary representation of i is given by $|\alpha_i|^2$. Once observed, the entire quantum system collapses onto the observed outcome, meaning that $\alpha_i = 1$ and $\alpha_j = 0$ for all $j \neq i$. In particular, repeated measurement results in the same answer.

When dealing with quantum systems, one typically employs the so-called *Dirac notation*, where state vectors correspond to so-called *kets* that are denoted by $|\cdot\rangle$, e.g., $|\psi\rangle$. Continuing the example from above, let $|\psi\rangle$ denote the quantum state of an n-qubit quantum computer. Using $|i\rangle$ with $i \in \{0, 1, \ldots, 2^n - 1\}$, we can write

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle \ ,$$

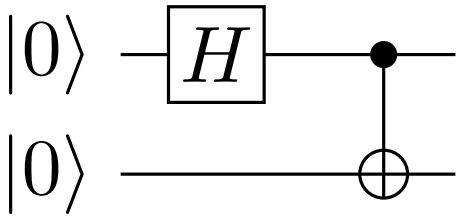


Fig. 2. Quantum circuit example: Each line represents a qubit and operations are drawn as boxes (e.g., the Hadamard operation H) or other symbols such as the controlled NOT or CNOT, which is depicted as $\oplus$ connected and attached to the filled circle on the control qubit. Time advances from left to right.

with $\alpha_i \in \mathbb{C}$ the contribution from the $|i\rangle$-state and $\sum_i |\alpha_i|^2 = 1$. As above, the binary representation of i lets us determine the value of each of the n qubits in the i-th basis state $|i\rangle$. We note that the set $\{|i\rangle, i \in \{0, ..., 2^n - 1\}\}$ is called the *computational basis* in the quantum computing literature.

Due to the relationship between amplitudes and probabilities, all operations on qubits, so-called *quantum gates*, must preserve inner-products. As a result, quantum gates must be *unitary*, which means that for a quantum gate U,

$$U^\dagger U = U U^\dagger = \mathbb{1} ,$$

where $U^\dagger$ denotes the Hermitian adjoint of U. Note that this also implies that all quantum gates must be reversible and, therefore, that only reversible operations can be implemented using quantum gates. Such unitary operations may also be applied *controlled* on another qubit, meaning that they are applied if the control qubit is 1. Formally, the controlled version of U is

$$U^c := |0\rangle\langle 0| \otimes \mathbb{1} + |1\rangle\langle 1| \otimes U ,$$

where $|c\rangle\langle c|$ is the projector onto the subspace in which the control qubit has the value $c \in \{0, 1\}$ and $\otimes$ denotes the tensor product. Since the control qubit may be in a superposition, the state after applying U^c is in a superposition of having and not having applied U.

2.2 Quantum Programs

A quantum program is a classical program that, in addition to classical instructions, executes so-called *quantum circuits* on a quantum co-processor. In a quantum circuit, each qubit is represented as a horizontal line and quantum gates are denoted by boxes or other symbols on these lines, with time moving from left to right. See Fig. 2 for an example. It consists of a Hadamard gate H and a controlled NOT or CNOT gate. The CNOT is drawn as a NOT gate (denoted by $\oplus$) that is connected to a filled circle on the control qubit.

Definition 2.1. Quantum instruction. Let $O\,|q_1, ..., q_k\rangle$ denote a *quantum instruction*. It consists of an operation O and a k-tuple of qubits $(q_1, ..., q_k)$, where the operation may be a quantum gate or a classical instruction (allocation, deallocation, measurement).

Every circuit consists of the following 4 steps:

(1) Allocate n qubits in state $|0\rangle^{\otimes n} := |0 \cdots 0\rangle$ (n zeros)
(2) Apply quantum gates to these qubits
(3) Measure some or all of the qubits
(4) Deallocate measured qubits

Upon completion, the quantum co-processor returns a set of classical bits, the so-called *measurement results*. Depending on these results, the classical processor may then provide further quantum

circuits to evaluate in order to solve the computational problem at hand. At the end of the entire quantum program, all qubits are deallocated again.

Since qubits are an extremely scarce resource, it is crucial to keep the number of allocated qubit minimal at any given point throughout the circuit and to deallocate all qubits which are no longer in use. Using the principle of deferred measurement [Nielsen and Chuang 2010], this means that as soon as the last operation on a given qubit has finished, the qubit can be measured and then freed for further use in the ongoing computation.

3 QUANTUM PROGRAM OPTIMIZATION USING ASSERTIONS

In this section, we introduce the basic idea of our assertion-based optimization methodology, followed by a proof of its correctness.

3.1 Entanglement Description Assertions for Quantum Program Optimization

Our goal is to use the knowledge of how subsystems are entangled for the purpose of quantum program optimization. To this end, we introduce the concept of *entanglement description assertions* that capture this information.

In particular, we introduce a formalism to describe the entanglement between qubits of the quantum computer throughout the execution of the quantum circuit. This entails statements that assert *entanglement descriptions* (to be defined next), that is, statements of the form

$$\text{“}\mathtt{q} == f(\mathtt{q},\ \mathtt{r})\text{”},\ \text{“}\mathtt{q} \geq f(\mathtt{q},\ \mathtt{r})\text{”},\ \text{etc.},$$

where q and r refer to quantum registers and f is a function of two registers returning one register of bits. Since q and r refer to quantum registers, they may be in superposition and entangled with other qubits in the system. The following definition assigns a precise meaning to these *entanglement description assertions* with respect to the state vector of the entire n-qubit quantum computer,

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i \, |i\rangle \ .$$

Definition 3.1. Entanglement description assertion. Let $\square_{\text{cmp}}$ denote a comparison operator, $f : \{0,1\}^k \times \{0,1\}^m \to \{0,1\}^k$ a function on $k+m$ bits returning k bits, and let q, r be quantum registers consisting of k and m qubits, respectively. The *entanglement description assertion* $A(q,r) =$ $\mathtt{q}\ \square_{\text{cmp}}\ f(\mathtt{q},\ \mathtt{r})$ on the n-qubit quantum state $|\psi\rangle$ asserts that

$$\forall i \in \{0,...,2^n-1\} : (\ |\alpha_i| > 0 \implies A\,(\mathscr{Q}(i), \mathscr{R}(i))\ ) \ ,$$

where the functions $\mathscr{Q} : \{0,1\}^n \to \{0,1\}^k$ and $\mathscr{R} : \{0,1\}^n \to \{0,1\}^m$ extract the bits corresponding to the quantum registers q and r, respectively, from the n-bit index i of the computational basis state $|i\rangle = |i_{n-1}, ..., i_0\rangle$.

With this definition in place, let us revisit the $|0\rangle$-control qubit example from the previous section and cast it as an *entanglement description assertion.*

Example 3.2. To express that a control qubit (denoted by $|c\rangle$) is in a definite state $|0\rangle$, let $f(\cdot,\cdot) = 0$ and $\square_{\text{cmp}}$ be the equals comparison operator in the above definition. Then $A(c,\cdot) = (\mathtt{c} == 0)$. For the corresponding state $|\psi\rangle$, this means that $\alpha_i = 0$ whenever i corresponds to a state where the control qubit is 1. As a result, the action of a controlled gate U^c on $|\psi\rangle$ is always trivial.

This shows that such assertions can be used to express knowledge about qubits that are in a definite state. This piece of information can, when combined with classical constant-folding, be used for optimization. However, in order to do so in a more general setting, the optimizer also needs

information which specifies the conditions for an operation to be trivial. We call this information *triviality conditions*.

Definition 3.3. Triviality condition. Let $A(q, r)$ be an entanglement description assertion on the quantum state $|\psi\rangle$. $A(q, r)$ is a *triviality condition* of a quantum operation U if

$$A(q, r) \implies U|\psi\rangle = |\psi\rangle \ ,$$

meaning that U acts as the identity if $A(q, r)$ is satisfied by $|\psi\rangle$.

Example 3.4. Continuing the $|0\rangle$-control qubit example, the triviality condition of the controlled unitary U^c would read $\{c == 0\}$ and, if this is satisfied as in the previous example, U^c may be removed from the circuit.

Therefore, using these two definitions, we can describe and carry out classical constant-folding. In order to see that this approach is strictly more powerful than classical constant-folding, consider the following example.

Example 3.5. Let $|\psi\rangle$ denote the quantum state of a two-qubit quantum computer. Initially, $|\psi\rangle = |00\rangle$ and our quantum program consists of two operations: 1) Prepare a Bell-pair and 2) swap the two qubits by applying a Swap gate. The Bell-pair preparation routine has $\{q_0 == 0, q_1 == 0\}$ as preconditions and ensures that $\{q_0 == q_1\}$ as a postcondition. In particular, given that the preconditions are satisfied, the Bell-pair preparation circuit in Fig. 2 transforms the state $|00\rangle$ to

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \ ,$$

The amplitudes of this quantum state are $\alpha_{00} = 1/\sqrt{2}$, $\alpha_{01} = \alpha_{10} = 0$, and $\alpha_{11} = 1/\sqrt{2}$. It is easy to check that the postcondition holds, i.e., that

$$\forall i \in \{0, 1, 2, 3\} : |\alpha_i| > 0 \implies \{i_0 == i_1\},$$

where i_0 and i_1 denote the 0th and 1st bit of i, respectively. Since swaps are trivial if $q_0 == q_1$, which is satisfied by the state above, the Swap gate can be removed from the circuit. We note that the same reasoning applies if the Hadamard gate in the Bell-pair preparation circuit is replaced by an arbitrary rotation gate. In this case, $\alpha_{01} = \alpha_{10} = 0$ still holds, and so the Swap gate may be removed.

Since the quantum state in the example above is in a superposition, it is clear that regular constant-folding cannot successfully perform this optimization. Using our entanglement description assertions, however, this becomes feasible. This shows that optimization using entanglement description assertions is strictly more powerful than classical constant-folding.

3.2 Correctness of our Methodology

We now prove that optimizations based on entanglement description assertions and triviality conditions leave the action of the overall quantum program invariant.

Theorem 3.6 (Correctness). *Let C be a quantum circuit that gets sent to a perfect*[1] *quantum device during execution of the quantum program $\mathcal{P}$. Applying assertion-based optimization to C will not change the output of $\mathcal{P}$.*

[1]Since circuit optimization may improve, e.g., success probability for a real device, we assume a hypothetical, perfect device.

Proof. Let $A(q, r)$ denote an entanglement description assertion on the state $|\psi\rangle$ of the quantum device at a given point during the execution of C and let U be the next subroutine to be executed. Moreover, let $A'(q, r)$ be a triviality condition of U such that

$$A(q, r) \implies A'(q, r). \tag{1}$$

Then, applying U is equivalent to

$$\begin{aligned} |\psi\rangle \mapsto U|\psi\rangle &= \sum_{i=0}^{2^n-1} \alpha_i U|i\rangle \\ &= \sum_{i:A(\mathscr{Q}(i),\mathscr{R}(i))} \alpha_i U|i\rangle + \sum_{i:\neg A(\mathscr{Q}(i),\mathscr{R}(i))} \alpha_i U|i\rangle \end{aligned}$$

We know that $\neg A(\mathscr{Q}(i), \mathscr{R}(i)) \implies \alpha_i = 0$, since $|\psi\rangle$ satisfies $A(q, r)$. Thus,

$$U|\psi\rangle = \sum_{i:A(\mathscr{Q}(i),\mathscr{R}(i))} \alpha_i U|i\rangle .$$

Now, for all i in this superposition, $A'(\mathscr{Q}(i), \mathscr{R}(i))$ holds due to (1) and U acts as the identity, i.e.,

$$\forall i : A(\mathscr{Q}(i), \mathscr{R}(i)) \implies U|i\rangle = |i\rangle ,$$

which lets us conclude that

$$U|\psi\rangle = \sum_{i:A(\mathscr{Q}(i),\mathscr{R}(i))} \alpha_i |i\rangle = |\psi\rangle .$$

Removing U from C does thus not affect $|\psi\rangle$ and, in particular, the final outcome of running $\mathcal{P}$ will not change by performing this optimization. Furthermore, our methodology will not modify the circuit if (1) does not hold. □

4 FORMALIZATION AND GENERALIZATION

In this section, we formalize the pre- and postconditions that are necessary to handle all examples in this paper, including the practical examples that will be introduced in the next section. Furthermore, we introduce a generalization of our methodology that is strictly more powerful.

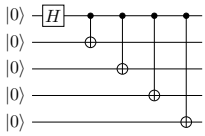
4.1 Formalization of our Basic Methodology

In order to formalize the basics of our methodology, we first define the pre- and postconditions of the quantum subroutines that are required for our examples in Table 1, where $X(q)$ denotes application of a Pauli-X [Barenco et al. 1995] gate to qubit q.

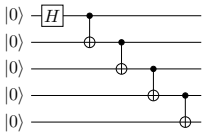
Operation	Preconditions	Postconditions
q=Alloc(n)	$\{q = \emptyset; n \in \mathbb{N}\}$	$\{q = \|0\rangle^{\otimes n}\}$
Dealloc(q)	$\{q = \|0\rangle^{\otimes n}\}$	$\{q = \emptyset\}$
Swap(q_i,q_j)	$\{q_i = A, q_j = B\}$	$\{q_i = B, q_j = A\}$
X(q)	$\{q = A, A \in \{0, 1\}\}$	$\{q = A \oplus 1\}$

Table 1. Pre- and postconditions of the quantum subroutines that are required to optimize our examples.

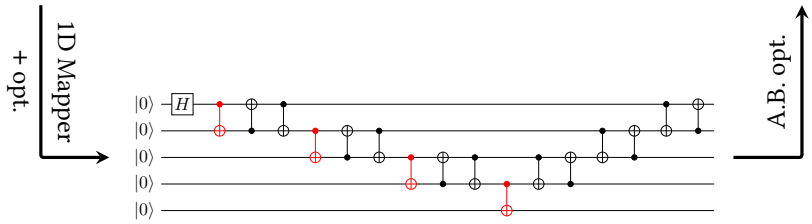
From the pre- and postconditions of the Swap operation, it is also apparent that a Swap is trivial if $q_i == q_j$; a fact that we already used in Example 3.5.



(a) Original circuit for entangling all qubits



(c) Circuit for LNN after optimization.



(b) Circuit for LNN before A.B. opt.

Fig. 3. Optimizing a chain of CNOTs for a linear nearest-neighbor (LNN) architecture such as the 9-qubit chip by Google [Kelly et al. 2015] by employing the pre- and postconditions of CNOT gates. The benefit of our assertion-based optimization (A.B. opt.) can be seen clearly when comparing the circuits in **(b)** and **(c)**: No extra CNOTs due to Swaps [Kutin et al. 2007] are necessary in **(c)**, resulting in much lower gate count and circuit depth.

In addition to the pre- and postconditions above, we require a formal description of the control modifier, which turns a given quantum subroutine U into its controlled version U^c, where c refers to the control qubit. The postcondition corresponding to

$$control(U)(c, q)$$

is $\{q = U^c |\psi\rangle\}$, where U^c is U on the subspace where $c = |1\rangle$ and $U^c = \mathbb{1}$ on the subspace where $c = |0\rangle$, and $|\psi\rangle$ denotes the state of the q-register before applying $control(U)$.

The pre- and postconditions of higher-level subroutines may be defined in a similar fashion. However, when combined, the pre-/postconditions above are sufficient for our examples. E.g., combining the control modifier with the NOT or Pauli X gate allows us to optimize the Bell-pair example where, after an initial Hadamard gate H [Barenco et al. 1995] on $|00\rangle$, the controlled NOT gate was applied as follows

$$H_1 |0\rangle |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) |0\rangle \overset{CNOT}{\mapsto} \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) .$$

Since the controlled NOT gate flips the qubit in $|0\rangle$ if the control qubit is one, we immediately get the postconditions for the two qubits q_0 and q_1

$$\{q_1 = X^{q_0} |0\rangle\} \implies \{q_1 == q_0\} ,$$

by combining the pre- and postconditions of the control modifier and the Pauli X gate. Together with the triviality condition of the Swap gate acting on two qubits q_i and q_j,

$$\{q_i == q_j\} ,$$

we can again remove the Swap gate from the circuit of the Bell-pair example. Similarly, the pre- and postconditions for CNOT can be used to identify the optimization opportunity in the following example.

Example 4.1. In addition to circuit optimizations at the logical level, entanglement description assertions and triviality conditions can be used to optimize the circuit for a specific target architecture. Consider the compilation steps outlined in Fig. 3. After mapping the circuit in **(a)** to a linear nearest-neighbor connectivity with additional optimizations to cancel intermediate partial Swap chains results in the circuit **(b)**. As before, we can employ our assertion-based optimizer to remove trivial CNOT gates using the fact that after each red CNOT gate acting on q_i and q_{i+1}, it holds that $q_i == q_{i+1}$. The optimized circuit is shown in Fig. 3**(c)**.

4.2 Generalized Optimization Methodology

By generalizing the basic methodology above, we can greatly increase its optimization capabilities. So far, our optimizer considers single gates at any given point together with all available postconditions of previously executed subroutines. For each such gate, it then determines whether it can be removed from the circuit without altering its output. The generalized strategy considers multiple gates and checks whether the supplied postconditions allow to deduce that the combined action of these gates is trivial, in which case all of these gates can be removed from the circuit. In order to properly introduce our generalized methodology, we first require a few definitions.

Definition 4.2. Set of control qubits. For an instruction $U\,|q_1, ..., q_k\rangle$ acting with a (unitary) gate U on k qubits, a set of qubits $\mathcal{S} \subset \{q_1, ..., q_k\}$ is called a *set of control qubits* if there exists a sequence of Swap gates $s_1, ..., s_t$ acting on pairs from $\{q_1, ..., q_k\}$ and a unitary U' such that with S denoting the unitary which performs $s_1, ..., s_t$, the following three statements hold.

(1) $SUS^\dagger = (\mathbb{1} - |1\cdots1\rangle\langle1\cdots1|) \otimes \mathbb{1} + |1\cdots1\rangle\langle1\cdots1| \otimes U'$
(2) the sequence of Swaps $(s_1, ..., s_t)$ permutes $(q_1, ..., q_k)$ such that the first $|\mathcal{S}|$ qubits of the resulting tuple are in $\mathcal{S}$
(3) $\mathcal{S}$ is the largest such set.

For instructions featuring a non-unitary operation (measurement, allocation, deallocation), the set of control qubits is empty.

We note that there may be multiple distinct sets of control qubits for a given instruction (as in the following example). For instructions where multiple choices exist, we choose a set of control qubits once and keep it invariant throughout the optimization process.

Example 4.3. As an example of an instruction where multiple choices exist for the set of control qubits, consider the Z^c operation applied to $|q_1 q_0\rangle$, where Z acts with a (-1)–phase on $|1\rangle$ and leaves $|0\rangle$ invariant. It is easy to check that

$$\begin{aligned} Z^c &= |0\rangle\langle0| \otimes \mathbb{1} + |1\rangle\langle1| \otimes Z \\ &= \mathbb{1} \otimes |0\rangle\langle0| + Z \otimes |1\rangle\langle1| \ , \end{aligned}$$

since for Z^c to be nontrivial, both qubits need to be in $|1\rangle$. Either qubit can thus be chosen to be the control qubit and, thus, the set of control qubits is not unique.

Definition 4.4. Target qubit. A qubit q in an instruction $U\,|..., q, ...\rangle$ is called a *target qubit* if it is not in the set of control qubits of the instruction $U\,|..., q, ...\rangle$.

Definition 4.5. Target-successive instructions. Two instructions I_1, I_2 with identical target qubits are called *target-successive* if no other instructions are scheduled to be executed between I_1 and I_2 that involve the target qubits in a way that does not commute with neither I_1 nor I_2.

Our generalized methodology considers $M \geq 1$ target-successive instructions at once, where all M instructions have the same t target qubits and arbitrary controls. Ignoring the control qubits, let

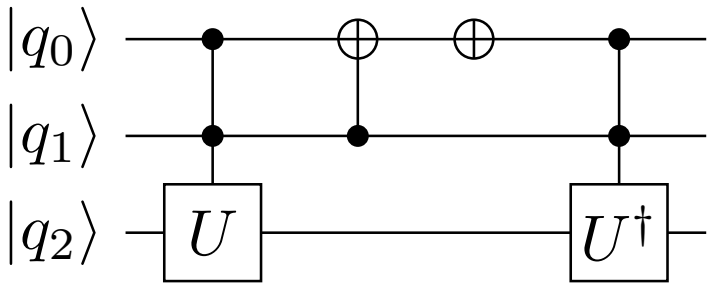


Fig. 4. Simple example of our multi-gate optimization methodology: The first gate is applied if and only if the last gate is applied (irrespective of the input state $|q_2, q_1, q_0\rangle$). Since the U gate is the inverse of $U^\dagger$, we can cancel the two doubly-controlled gates.

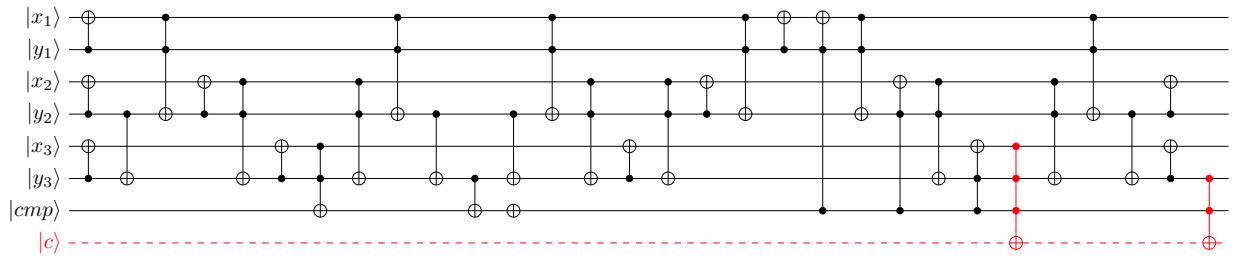


Fig. 5. Three-qubit example of a modular adder subroutine which performs the modular reduction. It consists of a comparison, the result of which is stored in the qubit $|cmp\rangle$, and a conditional subtraction. In this setting, our generalized methodology is able to deduce that the two red multi-controlled NOT gates can be canceled, allowing to completely remove the carry qubit $|c\rangle$. In a modular multiplier, n qubits would be saved since qubit reuse is not generally possible without uncomputation [Bennett 1973].

$U_1, ..., U_M$ denote the t-qubit gate matrices of these instructions. An optimization can be performed, for example, if

$$U_M \cdots U_1 = \mathbb{1}_{2^t \times 2^t}$$

and the postconditions on the control qubits are such that either all or none of the gates get executed. A simple example with $M = 2$ and $t = 1$ is depicted in Fig. 4, where the two doubly-controlled gates can be canceled using this reasoning.

We now give a practical example where our multi-gate optimization strategy performs better than the single-gate methodology discussed thus far.

Example 4.6. Consider a circuit that performs addition modulo a quantum number N (stored in another quantum register), i.e.,

$$|a\rangle\,|b\rangle\,|N\rangle \mapsto |(a+b) \bmod N\rangle\,|b\rangle\,|N\rangle \ .$$

A possible implementation is to first perform the regular addition, followed by a modular reduction if the result is greater than $|N\rangle$. Since we only subtract N if $(a + b) \geq N$, the result will always be non-negative and, as a consequence, the final carry qubit will always be zero and it can thus be removed from the subtraction circuit. When using the addition circuit by Takahashi et al. [Takahashi et al. 2010], the optimizer needs to remove the two red multi-controlled NOT gates in Fig. 5 which act on the carry qubit in order to exploit this optimization opportunity. Neither of these gates is trivial on its own, but in this setting, either both or none of the two gates are triggered. As a result, this optimization can only be performed using our generalized approach. The achieved reduction

in circuit width and depth can be found in Section 7, which discusses the results obtained using our implementation.

5 PRACTICAL EXAMPLES

Example 3.5 illustrates that optimization using entanglement description assertions is more powerful than classical constant-folding. Furthermore, Example 4.6 shows that our generalized methodology is strictly more powerful than regular assertion-based optimization. In this section, we discuss several practical examples that can be optimized using entanglement description assertions, but not using existing approaches for quantum circuit optimization. We mainly consider subroutines for quantum arithmetic, which is essential for most applications.

Perhaps surprisingly, most of the quantum gates required to run Shor's algorithm for factoring [Shor 1994] are due to the evaluation of modular exponentiation. In contrast to their classical counterparts, quantum computers must evaluate such classical functions on a superposition of inputs. Because the input is in a superposition, these functions cannot simply be evaluated on a classical computer. This would require reading out the state of the system, which would collapse the superposition and, thus, destroy any quantum speedup. Rather, these functions have to be implemented in terms of quantum gates in order to run them directly on the quantum computer. Further examples where the evaluation of such classical functions on a quantum computer is necessary are 1) the HHL algorithm for solving linear systems of equations, which requires computing the reciprocal [Harrow et al. 2009] and 2) certain algorithms for solving quantum chemistry problems: Babbush et al. [Babbush et al. 2016] have reduced the asymptotic runtime of a chemistry simulation algorithm by computing the entries of the Hamiltonian on-the-fly. This involves evaluating the Coulomb potential and various other mathematical functions which, e.g., describe the chosen orbitals.

In order to enable execution of such classical functions on a quantum computer, one may start by implementing subroutines for basic arithmetic such as addition and multiplication [Haener et al. 2018] in terms of quantum gates. These modules can then be combined to enable evaluating polynomials and further higher-level mathematical functions. We use the resulting subroutines as benchmarks and show how our methodology is able to reduce the quantum resource requirements.

5.1 Floating-Point Arithmetic Subroutine

As a first example in this section, we consider a subroutine that is omnipresent in floating-point arithmetic, namely that of *renormalization*. Renormalization is used during floating-point computations in order to bring intermediate results back into proper floating-point form. This can be achieved using two subroutines: The first subroutine determines the position p of the first nonzero bit of the mantissa. The second subroutine then shifts the mantissa to the left by the output of the first subroutine. A quantum circuit which determines the position of the first nonzero bit is shown in Fig. 6 and a circuit which shifts the mantissa $|x\rangle$ by $|p\rangle$ positions is depicted in Fig. 7. In order for the shift circuit to work properly for any input, it must allocate $2^{n_p} - 1$ extra work qubits in order to catch the overflow from the shifted $|x\rangle$, where n_p is the number of qubits in the position register $|p\rangle$. However, in the case where the input to the shift circuit gets initialized by the circuit which determines the position of the first one, such an overflow never occurs. As a result, the $2^{n_p} - 1$ work qubits can be eliminated from the combined circuit.

Identifying this optimization opportunity in the complete program is nontrivial and without some description of the action of gates or entire subroutines, such an optimization becomes completely infeasible for large circuits (as it would require simulation thereof for all inputs). We thus introduce a notion of how gates and subroutines interact by providing appropriate entanglement description assertions.

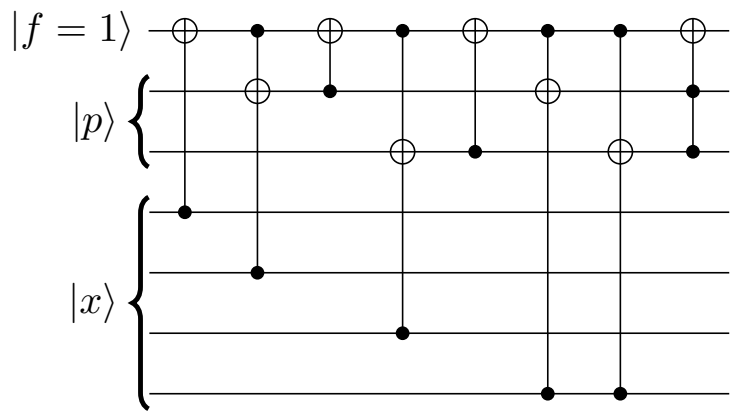


Fig. 6. Example of a circuit which finds the first nonzero bit of $|x\rangle$ and stores its position in $|p\rangle$ where $|x\rangle$ is a 4-qubit register and the position register $|p\rangle$ consists of two qubits [Haener et al. 2018]. The flag qubit $|f\rangle$ is one as long as the first one has not been found.

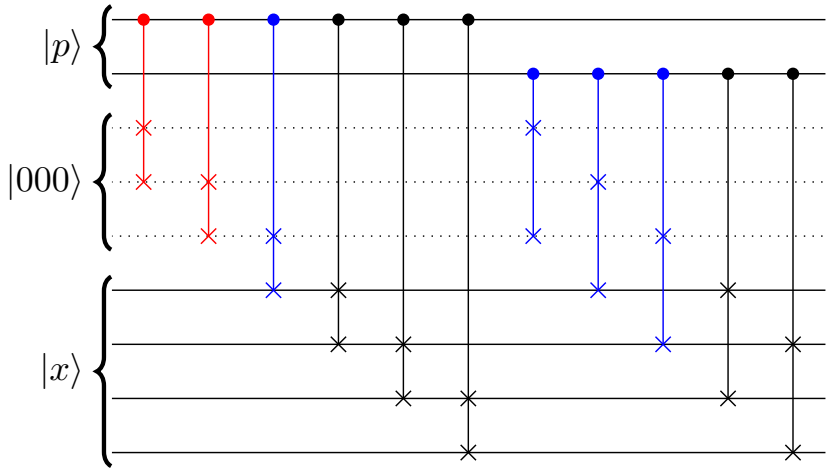


Fig. 7. Optimization of the shift circuit that can be performed if $|p\rangle$ contains the position of the first nonzero bit. Red Fredkin gates can be removed via regular constant-folding. Blue Fredkin gates can be removed using the postconditions of the subroutine depicted in Fig. 6 . As a result, all $2^{n_p} - 1$ work qubits can be eliminated (dotted lines).

For this concrete example, consider the postcondition of the subroutine which determines the position `pos` of the first nonzero bit of $|x\rangle$. It asserts that the first `pos` qubits of `x` are zero, i.e.,

$$\forall i \in 0..\texttt{pos-1} : \texttt{x[i]} == \texttt{0}\,,$$

where `pos` and `x` are entangled quantum variables. We can express this equivalently as an entanglement description assertion with

$$A_{FO}(x, p) = (x < 2^{n-p})\,,$$

where x is interpreted as an integer with x_0 as the most-significant bit (MSB) and p corresponds to the position register `pos` from above with p_0 being the least-significant bit (LSB). Using this postcondition, we now optimize the circuit in Fig. 7, which achieves the desired shift. Clearly, the red controlled Swap gates – also known as Fredkin gates – can be removed since they act on newly allocated qubits which are zero (the postcondition of qubit-allocation says that $q = |0\rangle^{\otimes n}$). The

left-most blue Fredkin gate is a Swap gate controlled on the 0-th bit of $|p\rangle$ and thus acts trivially if $p_0 = 0$. Furthermore, the Swap itself is trivial if $x_0 = 0$ because all ancilla qubits are still in $|0\rangle$. Combining these two triviality conditions of the controlled Swap gate with the postcondition above yields that the blue Fredkin gate may act nontrivially only if

$$(p > 0) \wedge (x < 2^{n-p}) \wedge (x_0 \neq 0) \,,$$

where x_0 denotes the MSB of the n-qubit register x. Clearly, these conditions cannot hold simultaneously and, as a result, the first blue Fredkin gate in Fig. 7 can be removed. Combining the postconditions of the Fredkin gates with $A_{FO}(x, p)$ yields a new assertion with

$$\begin{aligned} A_{new}(x, p) &= (2^{-p_0} x < 2^{n-p}) \\ &= (x < 2^{n-p+p_0}) \,, \end{aligned}$$

because if the first bit of the position register p_0 is one, we have just shifted all of x by one position. Since we successfully removed the first blue Fredkin gate, we can employ regular constant-folding to cancel the second blue Fredkin gate as well (all ancilla qubits are still in $|0\rangle$). For the final two blue Fredkin gates, note that they act nontrivially only if

$$(p_1 \neq 0) \wedge ((x_0 \neq 0) \vee (x_1 \neq 0)) \,.$$

From which we can use $p_1 \neq 0$ and combine it with the updated postcondition with a case-distinction on p_0: If p_0 is zero, then $p \geq 2$ and if p_0 is one, we have that $p \geq 3$ and that there is a shift of +1 in the exponent of the updated postcondition. Thus, in both cases,

$$x < 2^{n-2} \,,$$

and hence, the two most-significant bits x_0, x_1 of x must be zero. The action of the remaining two blue Fredkin gates is therefore always trivial and they can also be removed from the circuit. Finally, since none of the allocated overflow qubits will be used anymore throughout the computation (as their content is always trivial in this application), they will eventually get deallocated without any operations having acted on them. It is then a simple local optimization to cancel allocations with subsequent deallocations, allowing to reduce the width of the resulting circuit by $2^{n_p} - 1$ qubits, as desired.

5.2 Fixed-Point Arithmetic Subroutine

Similar optimization opportunities arise when using a fixed-point representation. As an example, consider the evaluation of a function using a range reduction, e.g., evaluating the function $f(x) = \sqrt{x}$ for $x \in [0, 2)$.

One approach to evaluate $f(x)$ is to approximate the function on the interval $[1, 2)$ by a polynomial. We can then perform range reduction for every input x to $y := x \cdot 2^k \in [1, 2)$, for $k \in \mathbb{N}$, evaluate the polynomial for y and then use that

$$f(x) = \sqrt{x \cdot 2^k} 2^{-k/2} \,,$$

where both multiplications by powers of two can be implemented using shifts and an additional multiplication by $\sqrt{2}$ for the $k/2$ exponent with an odd k. The function $f(x)$ can thus be evaluated on a quantum computer as follows:

1. Determine the position of the first non-zero qubit of x (starting from the most-significant bit)
2. Shift all bits of the fixed-point number such that the position is aligned with the binary point (and the number is now in the interval $[1, 2)$)
3. Evaluate the polynomial approximating $f(x)$ on $[1, 2)$ and store the result in a new quantum register

4. On the result register, undo half of the shift and multiply by $\sqrt{2}$ for odd shifts

We know that x was shifted by k positions in step 2. Therefore, the last k qubits of x are zero (where k is a quantum integer). These qubits can be used as work qubits when undoing half of the shift on the result register and our assertion-based optimizer can thus save an entire quantum fixed-point register.

More specifically, after having shifted the qubits of x toward the MSB, we have the same entanglement description assertion as in the previous example, $A_{FO}(x, p)$, between the x register and the position register. Now, the library implementation of the shift circuit should be able to use these "free" qubits of x as scratch space when shifting the output of the polynomial evaluation subroutine. This can be achieved through weakening of the preconditions on the work qubits of the shift circuit that catch potential overflow: Instead of requiring $2^{n_p} - 1$ qubits in $|0\rangle$, it is sufficient that the first k qubits be zero, where k is a quantum integer denoting the distance of the shift. While this precondition can always be satisfied by allocating $2^{n_p} - 1$ work qubits in $|0\rangle$, stating the weakened version allows the compiler to perform this optimization.

5.3 Integer Addition and Dirty Qubits

Recently, it was shown that the cost of an integer addition subroutine can be halved using the fact that the target qubit after an uncompute Toffoli gate is back in $|0\rangle$ [Gidney 2018]. Using the pre- and postconditions of allocation and deallocation, our optimization methodology can identify and exploit this optimization opportunity.

A similar optimization can be applied for subroutines that employ so-called *dirty qubits* [Barenco et al. 1995; Häner et al. 2017]. While such implementations use fewer (clean) work qubits, they typically cause an increase in circuit depth. Thus, when compiling the program for a specific architecture, the compiler should use as many clean qubits as possible in order to reduce this negative effect on the runtime. Because our assertion-based optimizer is able to identify when a dirty qubit gets mapped to a qubit that is actually clean, it can then optimize the resulting circuit.

For n-ary controlled NOT operations [Barenco et al. 1995], this translates to canceling Toffolis that are guaranteed not to be applied because one of the control qubits is in $|0\rangle$ (since it is a clean qubit). For an addition-by-constant circuit that uses dirty qubits, this translates to removing both the controlled inversions and a controlled incrementer [Häner et al. 2017, Fig. 5]. These automatic conversions from dirty to clean qubits allow savings of approximately $2\times$ in the number of gates and, crucially, allow for more modularity when implementing libraries for quantum computing.

As a technical detail, note that gates can be removed both after allocation and before deallocation: By definition, a dirty qubit must be returned to its original state before deallocation and so the clean qubit will have been brought back to $|0\rangle$.

6 IMPLEMENTATION USING PROJECTQ AND Z3

In this section, we discuss our implementation of our optimization methodology. We implement our methodology using the ProjectQ software framework for quantum computing [Steiger et al. 2018]. ProjectQ features an extensible compiler framework, allowing to easily integrate custom compiler passes such as our optimization methodology.

For each quantum operation for which we would like to add nontrivial optimization capabilities using our approach, we add the corresponding post- and triviality conditions. Additionally, preconditions may be supplied which would allow to test the program for correctness. To add support for our generalized methodology, we only require the triviality condition of the control modifier, in addition to information which lets us determine whether a sequence of operations $U_1, ..., U_M$ acts as the identity. The latter is already available in ProjectQ.

We extend the definitions of several quantum gates in ProjectQ with the corresponding entanglement descriptions (both post- and triviality conditions). Specifically, we add member functions that use functionality from the Z3 Theorem Prover package [de Moura and Bjørner 2008] to express these conditions. Our custom compiler pass uses these member functions in combination with the Z3 solver in order to check whether certain operations are guaranteed to be trivial, in which case they can be removed.

While we do not elaborate on the details of the ProjectQ compilation framework, we point out that optimization and compilation is carried out during circuit generation time. As a result, all parameters of the circuit are already known. In particular, the lengths of all quantum registers are known since all classical inputs to the quantum program have been supplied. The circuit can thus be optimized specifically to the problem instance in question. Furthermore, this enables more powerful optimizations when employing our methodology because we do not require parametric proofs. It is of course theoretically possible to prove such statements by induction, however, there is only limited support in automatic theorem provers such as Z3 [de Moura and Bjørner 2008] due to the difficulty of, e.g., constructing appropriate induction rules [Bundy 1999]. Since all classical parameters have a definite value upon circuit generation, we can unroll quantified statements and thereby generate claims that are easier to prove.

As an example, we show how the definition of the ProjectQ Swap operation was altered in order to enable our optimization engine to carry out the optimizations discussed so far. The definition of `SwapGate` was extended by merely the following two member functions:

```
class SwapGate(SelfInverseGate):
    [...]
    def trivial_if(self, x1, x2):
        return (x1 == x2)

    def postconditions(self, x1, x2, y1, y2):
        return And(x1 == y2, x2 == y1)
```

Clearly, these are very minor modifications that provide exactly the information required: Postconditions and triviality conditions of the Swap gate. The `trivial_if` member function of every gate is invoked by the optimizer with one symbolic boolean variable for each target qubit of the gate (two in this case). The returned expression is negated and then added to the solver together with the expression `ctrls_one = And(v[cqb`$_1$`], v[cqb`$_2$`], ...)`, which is true if and only if all variables `v[cqb`$_i$`]` corresponding to control qubits `cqb`$_i$ that are true / equal to one:

```
solver.push()
solver.add(And(ctrls_one, Not(cmd.gate.trivial_if(*target_vars))))
if solver.check() == unsat:
  ... # skip current operation
solver.pop()
```

where `target_vars` are the Z3 variables corresponding to the target qubits of the current gate before it is executed. If the solver finds a solution that satisfies all previous conditions and the negated conditions of `trivial_if`, the gate cannot be removed since it may have a nontrivial effect on the state of the quantum computer $|\psi\rangle$ at that point. If there is no such solution, on the other hand, this means that the gate is trivial and it can thus be removed from the circuit. After this triviality check, the conditions of the Z3 solver are updated according to the postconditions of the operation which hold irrespective of whether the gate was removed: For each target qubit, a new boolean Z3 variable is created and the `postconditions` member function of the gate relates

the old variables (before applying the gate) to the new ones. In particular, operations are handled by adding two Z3 `Implies(...)` statements:

(1) The control qubit(s) being all ones implies that the new target variables are now related to the old ones via the `postconditions` function, i.e.,

```
Implies(ctrls_one, cmd.gate.postconditions(*(target_vars+
    new_target_vars)))
```

is added to the solver, where `new_target_vars` are the Z3 variables that correspond to the target qubits after applying the gate.

(2) The control qubit(s) not being all ones implies that the new target variables are equal to the old ones, i.e., for all i we add the expression

```
Implies(Not(ctrls_one), new_target_vars[i] == target_vars[i]))
```

to the solver.

If there are no control qubits, (1) and (2) are of the form

$$\{\texttt{true} \implies y = f(x)\} \text{ and } \{\texttt{false} \implies y = x\}\,,$$

respectively and, therefore, are equivalent to stating that $y = f(x)$ holds after the gate has been applied, where f is given by the `postconditions` member function. As a technical detail, note that the ProjectQ Swap gate derives from `SelfInverseGate`, stating that the Swap operation is its own inverse. This information is useful for our generalized optimization approach, which is employed whenever the circuit buffer size of the optimizer exceeds a user-defined threshold. When this happens, the stored circuit is traversed in order to identify target-successive operations which may be removed from the circuit. For each such sequence of gates, the Z3 solver is used to determine whether there is an assignment to the control qubits that agrees with all previous postconditions and that causes $0 < m < M$ operations to be executed. If there is no such assignment, either all or none of these M operations are executed, meaning that they always act trivially. As a result, the entire sequence of gates can be removed from the circuit.

7 RESULTS

In this section, we report the results that were obtained using our prototype implementation of the proposed optimization methodology. We demonstrate the performance of our optimizer using three different quantum subroutines. The first subroutine performs floating-point mantissa renormalization, see Figs. 6 and 7, the second entangles a linear chain of qubits, see Fig. 3, and the third performs modular reduction, see Fig. 5, which is a subroutine that is used in constructing a modular adder.

For all subroutines, we compare two ProjectQ compiler setups—one which features a local optimizer capable of merging/canceling subsequent operations that act on the same qubits, and a second configuration which additionally contains our assertion-based optimizer. We choose $\{\text{CNOT, X, H, S, } T, T^\dagger\}$ as the target gate set for both configurations. In order to compare these different configurations, we use circuit width and depth as benchmark numbers. The circuit width corresponds to the maximal number of alive qubits at any point throughout the execution of the circuit. The *circuit depth* is equal to the delay of the circuit assuming that all quantum gates in the target gate set take unit time.

The comparisons can be found in Table 2 and Fig. 8 for the floating-point renormalization circuit and the entangling circuit, respectively. Both cases clearly demonstrate the benefits of our assertion-based optimizer, which is able to reduce the circuit area (width $\times$ depth) by a factor of up to $410\times$ and $5\times$ for the first and second circuit, respectively.

n	width	depth	optimized width	optimized depth	area reduction
8	15	235	7	107	4.7×
16	47	922	15	388	7.4×
32	287	5538	31	1080	47.5×
64	2111	38903	62	3229	410.2×

Table 2. Optimizer comparison for the floating-point renormalization circuit for 8-, 16-, 32-, and 64-qubit floating-point, where $n_p = 3, 5, 8, 11$, respectively, and the mantissa contains $n - n_p - 1$ qubits. Our entanglement description based optimizer achieves a reduction in circuit area (width×depth) of up to 410×.

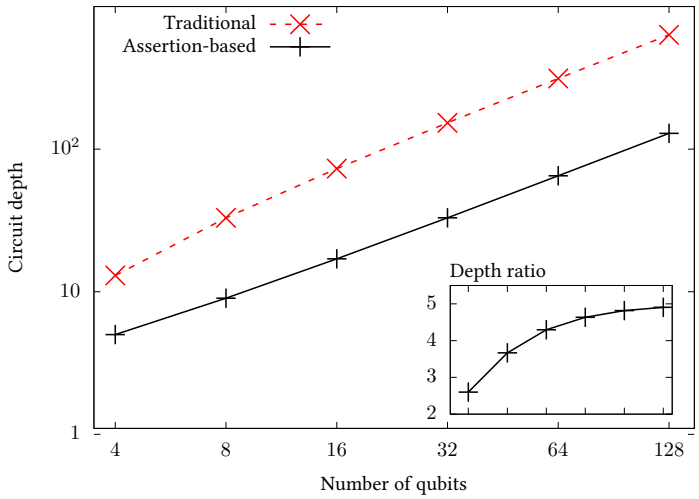


Fig. 8. Optimizer comparison for the entangling circuit on n qubits depicted in Fig. 3. The assertion-based optimizer achieves a 5× improvement in circuit depth for large n.

For the floating-point renormalization circuit, our optimizer is able to eliminate $2^{n_p} - 1$ ancilla qubits in addition to several Fredkin gates. Due to the elimination of Fredkin gates, the depth is also significantly reduced. We note that these savings are obtained without any prior manual optimizations such as limiting the maximal shift to the number of bits in the mantissa, as this describes a realistic scenario for software reuse.

For the entangling circuit, all CNOT gates resulting from swap operations can be removed when using the assertion-based optimization strategy (see Example 3.5 for more details). Therefore, the circuit depth would grow by $4(n-2)$ gates for $n \geq 2$ when turning off assertion-based optimization (see Fig. 3). The ratio between the resulting circuit depths for $n \geq 2$ is thus

$$\frac{4(n-2)+n}{n} = \frac{5n-8}{n} \overset{n\to\infty}{\to} 5\,,$$

which agrees with the experimental results in Fig. 8 and constitutes an up to 5× improvement over state-of-the-art optimizers.

The modular reduction circuit, which is a subroutine for modular addition, is optimized by identifying a pattern similar (but more complex) to the one shown in Fig. 4. In this case, the target qubit is the carry qubit of the controlled subtraction and upon removing the two multi-controlled

NOT operations, no operations on the carry qubit remain. As a result, our methodology removes this qubit from the circuit. Furthermore, our methodology achieves a slight depth reduction due to the removal of the two generalized Toffoli gates. In Shor's algorithm for factoring an n-bit number, n calls to such a modular reduction are required. The total savings would thus be equal to n qubits, where $n \sim 2000$ for practical applications.

8 SUMMARY AND FUTURE WORK

We have presented an optimization methodology that extends the scope of automatic circuit optimizations. In particular, our methodology carries out high-level optimizations across subroutine boundaries that are typically performed by humans, enabling more efficient code reuse. This is achieved by taking into account pre-, post-, and triviality conditions of all subroutines that get invoked by the quantum program that is being optimized.

Our generalized methodology currently performs optimizations if the overall action of a sequence of gates is trivial. Future work could address more general cases where, e.g., control qubits are in a state that only triggers subsets of these gates that, when combined, correspond to trivial operations. Additionally, symbolic computation on entanglement description assertions may be incorporated. This would allow to optimize iterative procedures such as the Newton-Raphson method which can be used to evaluate high-level arithmetic functions on a quantum computer [Cao et al. 2013]: For many such functions, the initial guesses can be chosen to be very simple (e.g., integer powers of two). The first iteration of a Newton-Raphson method may then be applied symbolically to the output of the initial guess routine. Such optimizations have been shown to yield significant resource savings when performed manually [Häner et al. 2018a]. Automating such procedures would thus result in the same benefits without the need for labor-intensive manual code optimization. Moreover, the focus of the present work lies on optimizing permutation-type subroutines. Future work could extend this to target phase-oracles by supporting pre- and postconditions in multiple bases.

ACKNOWLEDGMENTS

We thank the anonymous referees for their valuable comments and suggestions. This work was supported by Microsoft and by the Swiss National Science Foundation through the National Competence Center for Research NCCR QSIT.

REFERENCES

M. Amy, D. Maslov, and M. Mosca. 2014. Polynomial-Time T-Depth Optimization of Clifford+T Circuits Via Matroid Partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33, 10 (Oct 2014), 1476–1489. https://doi.org/10.1109/TCAD.2014.2341953

Matthew Amy, Dmitri Maslov, Michele Mosca, and Martin Roetteler. 2013. A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 32, 6 (2013), 818–830. https://doi.org/10.1109/TCAD.2013.2244643

Ryan Babbush, Dominic W Berry, Ian D Kivlichan, Annie Y Wei, Peter J Love, and Alán Aspuru-Guzik. 2016. Exponentially more precise quantum simulation of fermions in second quantization. *New Journal of Physics* 18, 3 (2016), 033032. https://doi.org/10.1088/1367-2630/18/3/033032

Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. 1995. Elementary gates for quantum computation. 52 (03 1995).

CH Bennett. 1973. Logical reversibility of computation. *Maxwell's Demon. Entropy, Information, Computing* (1973), 197–204.

Alan Bundy. 1999. *The automation of proof by mathematical induction.* Technical Report.

Yudong Cao, Anargyros Papageorgiou, Iasonas Petras, Joseph Traub, and Sabre Kais. 2013. Quantum algorithm and circuit design solving the Poisson equation. *New Journal of Physics* 15, 1 (2013), 013021. http://stacks.iop.org/1367-2630/15/i=1/a=013021

Frederic T Chong, Diana Franklin, and Margaret Martonosi. 2017. Programming languages and compiler design for realistic quantum hardware. *Nature* 549, 7671 (2017), 180. https://doi.org/10.1038/nature23459

Differentiable quantum computational chemistry with PennyLane

Juan Miguel Arrazola,[1] Soran Jahangiri,[1] Alain Delgado,[1] Jack Ceroni,[1] Josh Izaac,[1] Antal Száva,[1] Utkarsh Azad,[1] Robert A. Lang,[2,3] Zeyue Niu,[1] Olivia Di Matteo,[1] Romain Moyard,[1] Jay Soni,[1] Maria Schuld,[1] Rodrigo A. Vargas-Hernández,[2,4] Teresa Tamayo-Mendoza,[2,5] Cedric Yen-Yu Lin,[3] Alán Aspuru-Guzik,[2,4,6,7] and Nathan Killoran[1]

[1] *Xanadu, 777 Bay Street, Toronto, Canada*

[2] *Chemical Physics Theory Group, Department of Chemistry, University of Toronto, Canada*

[3] *AWS Quantum Technologies, Seattle, Washington 98170, USA*

[4] *Vector Institute for Artificial Intelligence, Toronto, Canada*

[5] *Department of Chemistry and Chemical Biology, Harvard University*

[6] *Department of Computer Science, University of Toronto, Canada*

[7] *Canadian Institute for Advanced Research (CIFAR) Lebovic Fellow, Toronto, Canada*

This work describes the theoretical foundation for all quantum chemistry functionality in PennyLane, a quantum computing software library specializing in quantum differentiable programming. We provide an overview of fundamental concepts in quantum chemistry, including the basic principles of the Hartree-Fock method. A flagship feature in PennyLane is the differentiable Hartree-Fock solver, allowing users to compute exact gradients of molecular Hamiltonians with respect to nuclear coordinates and basis set parameters. PennyLane provides specialized operations for quantum chemistry, including excitation gates as Givens rotations and templates for quantum chemistry circuits. Moreover, built-in simulators exploit sparse matrix techniques for representing molecular Hamiltonians that lead to fast simulation for quantum chemistry applications. In combination with PennyLane's existing methods for constructing, optimizing, and executing circuits, these methods allow users to implement a wide range of quantum algorithms for quantum chemistry. We discuss how PennyLane can be used to implement variational algorithms for calculating ground-state energies, excited-state energies, and energy derivatives, all of which can be differentiated with respect to both circuit and Hamiltonian parameters. We provide an example workflow describing how to jointly optimize circuit parameters, nuclear coordinates, and basis set parameters for quantum chemistry algorithms. We discuss a functionality for reducing the number of qubits by using symmetries and explain how PennyLane can be used to estimate quantum resources needed to implement several quantum algorithms. By combining insights from quantum computing, computational chemistry, and machine learning, PennyLane is the first library for differentiable quantum computational chemistry.

Introduction

Differentiable programming, which is central to machine learning and artificial intelligence, has benefited both from new hardware architectures such as graphical and tensor processing units, and from specialized software libraries for automatic differentiation [1–4]. An analogous trend is occurring for quantum computing. Quantum devices across different platforms are now accessible through the cloud, and considerable efforts are ongoing to scale up the number of components and to improve their performance [5–13]. Simultaneously, quantum software platforms continue their development with the goal of providing functionality across the entire spectrum of quantum computing [14–22]. This includes libraries designed for quantum differentiable programming, with native support for computing gradients of hybrid quantum-classical models [18, 19].

The motivation for building quantum software originates from the underlying interest in quantum computing due to its importance to fundamental science and to its potential to outperform existing approaches, notably for problems related to the simulation of quantum systems [23, 24]. Quantum chemistry is recognized as one of the key potential application areas of quantum computing because simulating properties of molecules and materials plays a central role in many industrial processes [25–29]. This has led to considerable interest in developing new and improved quantum algorithms for quantum chemistry, ranging from proof-of-principle methods that can be implemented in simulators and small-scale quantum computers [30–34], to advanced techniques designed to run on large-scale fault-tolerant machines [35–40]. Researchers and enthusiasts working in this field have created a demand for quantum software tools designed for quantum chemistry applications, leading to the appearance of software libraries aimed specifically at fulfilling their needs [41–43].

This work describes quantum chemistry functionality available in PennyLane, a quantum software library for quantum differentiable programming (`www.pennylane.ai`) [18]. By combining methodologies from quantum computing, computational chemistry, and machine learning, PennyLane is the first library built specifically for differentiable quantum computational chemistry. PennyLane users can natively compute gradients of all relevant methods, allowing for a flexible and general approach to the optimization of hybrid quantum-classical algorithms for quantum chemistry. This document is a summary of the theoretical foundation of the software, serving as a technical complement to the online documentation (`pennylane.readthedocs.io`). It is intended as a living manuscript to be periodically updated to reflect major new features from all contributing developers.

A distinguishing feature of PennyLane's quantum chemistry capabilities is a differentiable implementation of the Hartree-Fock method, as pioneered in Ref. [44] and also implemented in Ref. [45]. PennyLane's differentiable solver enables users to compute exact gradients of atomic and molecular orbitals, Hartree-Fock energies, and molecular Hamiltonians, which can then be employed in quantum algorithms. PennyLane provides custom gates and operations for quantum chemistry, including excitation gates as Givens rotations, templates for chemically-inspired circuits, built-in functionality for computing observables, and sparse matrix techniques for representing molecular Hamiltonians. Together with PennyLane's existing methods for designing, optimizing, and executing quantum circuits across simulators and hardware devices, these methods allow users to implement a wide range of quantum algorithms for quantum chemistry, which can be optimized with respect to both circuit and Hamiltonian parameters. PennyLane also includes a collection of tutorials aimed specifically at quantum chemistry, suitable for both beginners and experts.

The rest of this manuscript is organized as follows. The first section covers basic principles of quantum chemistry such as the electronic structure problem and methods based on linear combinations of atomic orbitals. We then focus on the differentiable implementation of the Hartree-Fock method, describing fundamental principles of automatic differentiation and explaining how they can be used to construct differentiable implementations of electronic integrals, Fock matrices, and solvers for self-consistent field equations. We also explain how to use these calculations to construct qubit representations of molecular Hamiltonians, which are also fully differentiable.

We continue by providing an overview of quantum algorithms for quantum chemistry. We outline the core idea of using quantum computers to build many-body wavefunctions of molecules and compute expectation values of molecular Hamiltonians. We summarize the main methods for achieving this with a focus on variational approaches. The rest of the section describes how to design quantum circuits, compute expectation values, calculate ground and excited-state energies, and obtain exact energy derivatives, which can be used to perform geometry optimization and to compute vibrational normal modes and frequencies of molecules. We describe how to use PennyLane to implement fully differentiable workflows for quantum chemistry capable of simultaneously optimizing gate parameters, nuclear coordinates, and basis set parameters. We also describe how symmetries present in a Hamiltonian can be used to taper off qubits and perform more efficient

simulations. Finally, we demonstrate how to estimate quantum resources, such as the number of qubits, gates, and shots, for the variational quantum eigensolver (VQE) and quantum phase estimation (QPE) algorithms. Examples of PennyLane code are used throughout the manuscript as illustrations of the user interface.

Quantum chemistry

This section reviews basic principles of quantum chemistry that are central to the entire manuscript. This includes a discussion of central concepts such as fermionic ladder operators, the occupation-number representation, molecular Hamiltonians, molecular orbitals, and electron integrals.

The electronic structure problem

The underlying laws of quantum mechanics that govern the properties of molecules and materials have been understood for decades, yet the challenge remains to perform efficient and accurate calculations of these properties. A first step in taming the computational challenges of quantum chemistry is to decouple the electronic and nuclear degrees of freedom. This is known as the Born-Oppenheimer approximation, which is supported by the fact that electrons are largely responsible for the chemical properties of molecules, and nuclei are orders-of-magnitude heavier than electrons. The result is the electronic structure problem: solving the time-independent Schrödinger equation

$$H\left|\Psi\right\rangle = E\left|\Psi\right\rangle, \tag{1}$$

for a collection of interacting electrons in the presence of an electrostatic field generated by the nuclei, which are treated as stationary point particles fixed to their positions. Here H is the electronic Hamiltonian depending parametrically on the positions of the nuclei and $|\Psi\rangle$ is the quantum state of the electronic degrees of freedom. Knowledge of the eigenvalues E_i and eigenstates $|E_i\rangle$ of the Hamiltonian is enough to compute important properties of molecules. Typically it is sufficient to consider only the ground state and the first few excited states.

We focus on the second-quantization approach where the antisymmetry of fermionic systems is captured by properties of operators, and states are represented in terms of occupation numbers. More concretely, we consider a set of functions $\{\phi_p(\mathbf{x})\}$, where $\mathbf{x} = (\mathbf{r}, \sigma)$ includes spatial ($\mathbf{r}$) and spin (σ) degrees of freedom. These functions are referred to as spin-orbitals. We introduce the notation

$$\left|n_1, n_2, \ldots, n_M\right\rangle, \tag{2}$$

to denote an electronic state in the space of M spin-orbitals, where $n_p = 1$ if ϕ_p is occupied by an electron and $n_p = 0$ otherwise. Within the restrictions of the chosen set of spin-orbitals, any possible electronic state can be expressed as a superposition of states of this form.

To represent general operators, it is useful to define the fermionic ladder operators $a_p, a_p^\dagger$. They satisfy the anticommutation relations

$$\{a_p, a_q^\dagger\} = \delta_{pq}, \tag{3}$$

$$\{a_p, a_q\} = \{a_p^\dagger, a_q^\dagger\} = 0. \tag{4}$$

Their action on states in the occupation number representation is given by

$$a_p\left|n_1, \ldots, n_M\right\rangle = \delta_{1,n_p}(-1)^{N_p}\left|n_1, \ldots, n_p \oplus 1, \ldots, n_M\right\rangle, \tag{5}$$

$$a_p^\dagger\left|n_1, \ldots, n_M\right\rangle = \delta_{0,n_p}(-1)^{N_p}\left|n_1, \ldots, n_p \oplus 1, \ldots, n_M\right\rangle, \tag{6}$$

where $N_p = \sum_{i=1}^{p} n_i$ and $\oplus$ denotes addition modulo 2. The phase $(-1)^{N_p}$ applied to the state encodes the exchange symmetry of fermions, meaning that the states themselves do not need to be explicitly antisymmetrized — an important property that both classical and quantum algorithms benefit from.

In the regime where relativistic effects are not important and we can decouple electronic and nuclear degrees of freedom,

a molecular Hamiltonian can be represented in terms of these operators as

$$H = \sum_{pq} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s. \tag{7}$$

The coefficients h_{pq} and h_{pqrs} are known respectively as the one- and two-electron integrals. They will be described in more detail in future sections.

Linear combination of atomic orbitals

For very simple systems like the hydrogen atom, it is possible to derive analytical solutions of the Schrödinger equation. For example, the ground-state wavefunction of the electron in the hydrogen atom is

$$\Psi(\mathbf{r}) = \frac{1}{\sqrt{\pi}} e^{-r}, \tag{8}$$

where $\mathbf{r} = (x, y, z)$ is the electron coordinate vector, $r = \sqrt{x^2 + y^2 + z^2}$, and we use atomic units that set the Bohr radius equal to one. Analytical solutions for more complicated systems quickly become intractable, requiring the use of numerical methods. Any function can be represented in terms of a complete basis, for example the set of all plane waves of the form $e^{-i\mathbf{k}\cdot\mathbf{r}}$. Since exact representations require an infinitely large basis set, it is customary to define a restricted basis set that can be used as an approximation to the continuum limit. The challenge is to find basis sets containing few elements that can still provide accurate representations.

The most common approach in quantum chemistry is to introduce localized basis sets that are appropriate for describing functions centered at the nuclei of each atom. These are used to represent localized functions known as atomic orbitals. By taking linear combinations of atomic orbitals, it is possible to represent molecular orbitals that describe how electrons are distributed around the entire molecule. This strategy is known as the linear combination of atomic orbitals (LCAO) [46].

Atomic orbitals $\chi(\mathbf{r})$ are expanded in terms of primitive functions $\psi(\mathbf{r})$ as

$$\chi(\mathbf{r}) = \mathcal{N} \sum_{i=1}^{n} a_i \psi_i(\mathbf{r}, \alpha_i), \tag{9}$$

where the a_i are coefficients, α_i denote possible internal parameters of the primitive functions, and $\mathcal{N}$ is a normalization constant. Here we are implicitly assuming that these functions are centered at the origin, but their centers can be translated to the nuclear coordinates $\mathbf{R}$ by setting $\mathbf{r} \to \mathbf{r} - \mathbf{R}$. This choice is a key feature of the LCAO approach: electron wave functions have cusps centered at the nuclei where the Coulomb potential peaks, so it is useful to employ nuclei-centered orbitals that can replicate this behaviour. Molecular orbitals are denoted by $\phi(\mathbf{r})$ and can be expressed as

$$\phi(\mathbf{r}) = \sum_i C_i \chi_i(\mathbf{r}), \tag{10}$$

where the C_i are real coefficients and the index i runs over all atomic orbitals. Molecular orbitals can thus be viewed as a double expansion: they are a linear combination of atomic orbitals, which are themselves linear combinations of primitive basis set functions. In the restricted case, particles of different spin occupy the same spatial orbitals, allowing us to drop the spin label and consider both types of spin-orbitals on the same footing.

Experts have worked for decades to develop a vast array of increasingly sophisticated basis sets for chemistry applications [47, 48]. The coefficients a_i, which have been optimized to describe the electronic structure of isolated atoms, are typically used regardless of the molecule containing the atoms. The parameters that are individually optimized for each molecule are the coefficients C_i. Large basis sets are needed for highly-accurate simulations, which leads to increased computational costs. This establishes a fundamental trade-off between accuracy and efficiency which is particularly important for quantum algorithms, where the number of qubits typically increases with the number of elements in the basis set.

The one- and two-electron integrals h_{pq}, h_{pqrs} introduced in Eq. (7) can be interpreted as matrix elements of the Hamil-

tonian in the basis of molecular orbitals. Their full expressions are

$$h_{pq} = \int d\mathbf{x}\, \phi_p^*(\mathbf{x}) \left(-\frac{\nabla^2}{2} - \sum_{i=1}^{N} \frac{Z_i}{|\mathbf{r} - \mathbf{R}_i|} \right) \phi_q(\mathbf{x}), \tag{11}$$

$$h_{pqrs} = \int d\mathbf{x}_1 d\mathbf{x}_2 \, \frac{\phi_p^*(\mathbf{x}_1)\phi_q^*(\mathbf{x}_2)\phi_r(\mathbf{x}_2)\phi_s(\mathbf{x}_1)}{|\mathbf{r}_1 - \mathbf{r}_2|}, \tag{12}$$

where Z_i is the atomic number of the i-th nucleus and the sum is taken over all atoms in the molecule. The one-electron integral contains contributions from the electronic kinetic energy and the electron-nuclear attraction, while the two-electron integral represents energy contributions from the Coulomb repulsion between pairs of electrons.

Molecular Hamiltonians are defined in terms of the integrals in Eqs. (11) and (12), which depend explicitly on the choice of molecular spin-orbitals. In the following section, we discuss the Hartree-Fock method, which can be employed to obtain optimized spin-orbitals. In PennyLane, the Hartree-Fock method is implemented in a fully differentiable manner, allowing users to compute gradients of all relevant quantities with respect to basis set parameters and nuclear coordinates. This is achieved using the techniques of automatic differentiation, which we describe briefly in comparison to other strategies such as symbolic and numerical differentiation.

Differentiable Hartree-Fock

The ability to compute derivatives of functions is a central task in science and mathematics. Symbolic differentiation obtains derivatives of an input function by direct mathematical manipulation, for example using standard strategies of differential calculus. These can be performed by hand or with the help of computer algebra software. The resulting expressions are exact, but the symbolic approach is of limited scope, particularly since many functions are not known in explicit analytical form. Symbolic methods also suffer from the expression swell problem where careless usage can lead to exponentially large symbolic expressions. Numerical differentiation is a versatile but unstable method, often relying on finite differences to calculate approximate derivatives. This is problematic especially for large computations consisting of many differentiable parameters [49].

Automatic differentiation is a computational strategy where a function implemented using computer code is differentiated by expressing it in terms of elementary operations for which derivatives are known [49, 50]. The gradient of the target function is then obtained by applying the chain rule through the entire code. In principle, automatic differentiation can be used to calculate derivatives of a function using resources comparable to those required to evaluate the function itself. This strategy has gained significant attention in machine learning, and consequently several software packages for automatic differentiation have been developed [1–4].

In this section, we describe the Hartree-Fock method for quantum chemistry and outline its implementation in PennyLane using principles of automatic differentiation. Building upon work pioneered in Ref. [44], PennyLane provides built-in methods for constructing atomic and molecular orbitals, building Fock matrices, and solving the self-consistent Hartree-Fock equations to obtain optimized orbitals, which can be used to construct molecular Hamiltonians. PennyLane allows users to natively compute derivatives of all these objects with respect to the underlying parameters. The implementation makes use of differentiable transforms, allowing the entire workflow to be differentiable while maintaining a simple and flexible user interface. All physical quantities in this context are defined in atomic units.

The main goal of the Hartree-Fock method is to obtain molecular spin-orbitals that minimize the energy of a state where electrons are treated as independent particles occupying the lowest-energy orbitals. Here we provide an overview of the main steps of this process and their PennyLane implementation. An in-depth pedagogical review can be found in Ref. [46]. We focus on the restricted Hartree-Fock method, where each spatial orbital is occupied by two electrons with opposite spin.

Basis sets, orbitals, and molecules

The design and choice of basis sets is largely guided by their usefulness in computational methods. Part of the challenge in optimizing molecular orbitals, building Hamiltonians, and performing electronic structure calculations is to compute integrals over primitive basis functions. Most modern methods in quantum chemistry employ Gaussian functions as the

primitive basis because they are simple to evaluate analytically and numerically [51]. These are functions of the form

$$\psi(\mathbf{r}, \alpha) = x^l y^m z^n e^{-\alpha r^2}, \tag{13}$$

where for simplicity we assume are centered at the origin. Gaussian functions $e^{-\alpha r^2}$ are smoother at the origin and decay more quickly than Slater-type functions $e^{-\alpha r}$, as in Eq. (8). Nonetheless, linear combinations of Gaussians can be used to approximate Slater-type orbitals. This is the strategy of the STO-nG class (Slater-type orbitals with n primitive Gaussians), of which the simplest example that is commonly used is the STO-3G basis set. This consists of atomic orbitals of the form

$$\chi(\mathbf{r}) = \sum_{i=1}^{3} a_i \psi(\mathbf{r}, \alpha_i), \tag{14}$$

where the contraction coefficients a_i and the exponents α_i are typically predefined for each atom, but in principle can be optimized for specific molecules. PennyLane provides native support for the STO-3G, 6-31G, 6-311G and cc-pVDZ basis sets in the differentiable solver, while also allowing users to import other basis sets. Moreover, PennyLane provides native support for differentiating atomic orbitals with respect to contraction coefficients, exponents, and nuclear coordinates.

The `Molecule` class can be used to encapsulate all relevant information about a system. By specifying a list of atomic symbols and nuclear coordinates, we can instantiate a molecule object containing as attributes the charge of the molecule, number of electrons, nuclear charges, number of orbitals, Gaussian exponents α, Gaussian centers, contraction coefficients a, and angular momentum values (l, m, n) of the basis functions, which can also be optionally passed by the user. This enables sophisticated functions that depend on all these parameters to simply take a molecule object as an argument.

```
import pennylane as qml
from pennylane import numpy as np

symbols = ['H', 'H']
geometry = np.array([[0.0, 0.0, -0.6614], [0.0, 0.0, 0.6614]])
mol = qml.qchem.Molecule(symbols, geometry)
```

Codeblock 1: *Constructing a molecule object for the hydrogen molecule.*

Matrices and integrals

To group all coefficients in a single object, we can define the coefficient matrix $\mathbf{C}$ with entries $C_{\mu i}$ such that the i-th molecular orbital can be expressed as

$$\phi_i(\mathbf{r}) = \sum_{\mu} C_{\mu i} \chi_\mu(\mathbf{r}), \tag{15}$$

where Greek letters are used to denote sums over basis functions. We also define the matrix $\mathbf{P}$ with entries

$$P_{\mu\nu} = \sum_{i} C_{\mu i} C_{\nu i}, \tag{16}$$

which leads to a concise expression for the Hartree-Fock energy

$$E = 2 \sum_{\mu\nu} P_{\mu\nu} H_{\mu\nu} + \sum_{\mu\nu} P_{\mu\nu} \left(2J_{\mu\nu} - K_{\mu\nu}\right). \tag{17}$$

The matrices appearing in this expansion are the core Hamiltonian $\mathbf{H}$ with entries

$$H_{\mu\nu} = \int dr\, \chi_\mu^*(\mathbf{r}) \left(-\frac{1}{2}\nabla^2 - \sum_{i=1}^{N} \frac{Z_i}{|\mathbf{r} - \mathbf{R_i}|} \right) \chi_\nu(\mathbf{r}), \tag{18}$$

the Coulomb matrix $\mathbf{J}$ with entries

$$J_{\mu\nu} = \sum_{\eta\gamma} P_{\eta\gamma} (\mu\nu|\eta\gamma), \tag{19}$$

and the exchange matrix $\mathbf{K}$ with entries

$$K_{\mu\nu} = \frac{1}{2}\sum_{\eta\gamma} P_{\mu\nu}(\mu\eta|\nu\gamma), \tag{20}$$

where the electron repulsion integral is defined as

$$(\mu\nu|\eta\gamma) = \int d\mathbf{r}_1\, d\mathbf{r}_2\, \frac{\chi_\mu^*(\mathbf{r}_1)\chi_\nu^*(\mathbf{r}_2)\chi_\eta(\mathbf{r}_1)\chi_\gamma(\mathbf{r}_2)}{|\mathbf{r}_1 - \mathbf{r}_2|}. \tag{21}$$

The core Hamiltonian does not depend on the coefficients $C_{\mu i}$, but the total energy depends on them through the matrix $\mathbf{P}$.

The goal of the Hartree-Fock method is to find the coefficients $C_{\mu i}$ that minimize the total energy function. While this problem could be tackled through different strategies, a closed-form solution is known that can be used to directly obtain the optimized molecular orbitals [51]. Central to this approach is the Fock matrix $\mathbf{F}$, whose entries are defined as

$$F_{\mu\nu} := \frac{\partial E}{\partial P_{\mu\nu}} = H_{\mu\nu} + 2J_{\mu\nu} - K_{\mu\nu}. \tag{22}$$

PennyLane provides functionality to compute core Hamiltonians, Coulomb matrices, exchange matrices, repulsion integrals, Fock matrices, and total energy, all of which are differentiable with respect to nuclear coordinates, contraction coefficients, and Gaussian exponents. By expanding the atomic orbitals in terms of the primitive Gaussian functions, the core Hamiltonian integrals of Eq. (18) and the electron repulsion integrals of Eq. (21) can be expressed as linear combinations of Gaussian integrals, for which there exist closed-form analytical formulas [52]. This enables an implementation that is directly compatible with automatic differentiation techniques. PennyLane currently builds on Autograd [1] as a lightweight approach to computing derivatives.

Self-consistent equations

The problem of finding the optimal coefficients $C_{\mu i}$ can be reduced to solving the Roothaan-Hall equation [46, 53, 54]

$$\mathbf{FC} = \mathbf{SCE}, \tag{23}$$

where $\mathbf{F}$ and $\mathbf{C}$ are respectively the Fock and coefficient matrix, $\mathbf{E}$ is a diagonal matrix of eigenvalues, and $\mathbf{S}$ is the overlap matrix with entries

$$S_{\mu\nu} = \int d\mathbf{r}\, \chi_\mu(\mathbf{r})^* \chi_\nu(\mathbf{r}). \tag{24}$$

Equation (23) can be further simplified by transforming the Fock and coefficient matrices. Diagonalizing the overlap matrix as

$$\mathbf{S} = \mathbf{V}\mathbf{D}\mathbf{V}^T, \tag{25}$$

where $\mathbf{D}$ is a diagonal matrix, we define $\mathbf{X} = \mathbf{V}\mathbf{D}^{-1/2}\mathbf{V}^T$, $\mathbf{C} = \mathbf{X}\tilde{\mathbf{C}}$, and $\tilde{\mathbf{F}} = \mathbf{X}^T\mathbf{F}\mathbf{X}$. The equation can then be cast in its standard form

$$\tilde{\mathbf{F}}\tilde{\mathbf{C}} = \tilde{\mathbf{C}}\mathbf{E}. \tag{26}$$

This is an eigenvalue problem that can be solved using standard techniques in linear algebra, which are compatible with automatic differentiation.

The Fock matrix depends implicitly on the coefficient matrix. By setting an initial value for $\tilde{\mathbf{C}}_0$, computing the resulting matrix $\tilde{\mathbf{F}}_0$, and solving Eq. (26), it is possible to end with a solution $\tilde{\mathbf{C}}_1$ that is inconsistent with the original choice of $\tilde{\mathbf{C}}_0$. Instead, we seek a self-consistent solution by iteratively repeating the process until $\mathbf{C}^k = \mathbf{C}^{k+1}$. Once a self-consistent solution is found, the resulting coefficients can be used to construct optimized molecular orbitals, compute the Hartree-Fock energy, and build molecular Hamiltonians. Different initialization strategies are possible, but a simple and effective approach is to set the first coefficient matrix equal to zero, meaning the only contribution to the energy arises from the core Hamiltonian. The example code below shows how to calculate the Hartree-Fock energy and its gradient with respect

to nuclear coordinates for the simple case of the hydrogen molecule, which we focus on for subsequent code examples:

```
import pennylane as qml

energy = qml.qchem.hf_energy(mol)(geometry)
g = qml.grad(qml.qchem.hf_energy(mol))(geometry)
```

Codeblock 2: *Calculating the Hartree-Fock energy and its derivative with respect to nuclear coordinates. This uses the same quantities defined in Codeblock 1. The nuclear coordinates that define the molecular geometry are differentiable by default.*

This simple user interface for constructing molecule objects makes use of default values for the basis set, charge, multiplicity, and active space of the molecule. Advanced users can also specify custom values for all these quantities.

Qubit Hamiltonians

The starting point of many quantum algorithms for quantum chemistry is the Hamiltonian describing the system of interest. The molecule itself is described by the set of its constituent atoms and their coordinates in three-dimensional space. Within a given basis set, this information is sufficient to build the Fock matrix and implement the Hartree-Fock method. The resulting optimized orbitals are then used to compute the one- and two-electron integrals that specify the molecular Hamiltonian in terms of fermionic ladder operators.

To execute a quantum algorithm, it is necessary to express the Hamiltonian as a qubit operator while ensuring that it retains all the required fermionic exchange symmetries. This in turn requires a description of fermionic states in terms of qubit states. These transformations are performed using fermionic-to-qubit mappings [29]. The most widely used is the Jordan-Wigner transformation [55]. Here, a state $|n_1, \dots, n_M\rangle$ in the occupation number representation on M spin-orbitals can be directly mapped to a state $|n_1\rangle|n_2\rangle \cdots |n_M\rangle$ on M qubits, where the state of the i-th qubit is $|1\rangle$ if the corresponding spin-orbital is occupied, and $|0\rangle$ otherwise. Fermionic ladder operators are transformed as

$$a_p = \frac{1}{2} Z_0 \otimes \cdots Z_{p-1} \otimes (X_p + iY_p), \tag{27}$$

$$a_p^\dagger = \frac{1}{2} Z_0 \otimes \cdots Z_{p-1} \otimes (X_p - iY_p), \tag{28}$$

where X, Y, Z are Pauli matrices. This results in a Hamiltonian of the form

$$H = \sum_j h_j P_j, \tag{29}$$

where P_j is a tensor product of the Pauli matrices I, X, Y, Z. In PennyLane, a qubit Hamiltonian can be easily obtained with the `molecular_hamiltonian()` function:

```
H = qml.qchem.molecular_hamiltonian(symbols, geometry)[0]
```

Codeblock 3: *Constructing the Hamiltonian for the hydrogen molecule, using the same definitions as in Codeblock 1.*

As an alternative to the differentiable Hartree-Fock solver, PennyLane can also interface with the external packages PySCF [56] or Psi4 [57] to calculate the one- and two-electron integrals and use functionality in OpenFermion [41] to construct the qubit Hamiltonian. Users can also construct OpenFermion Hamiltonians themselves and map them to PennyLane Hamiltonians.

The `molecular_hamiltonian()` function can take differentiable parameters as input to construct the Hamiltonian. Under the hood, this function uses the differentiable solver to compute the optimized Hartree-Fock orbitals, calculate the one- and two-electron integrals, and perform the Jordan-Wigner fermionic-to-qubit mapping to construct a PennyLane Hamiltonian.

Differentiable qubit Hamiltonians can be constructed by defining the atomic symbols and nuclear coordinates for a given molecule and specifying the molecular parameters that will be differentiated. Currently, the exponents, contraction coefficients, and centers of the Gaussian basis functions can be specified to be differentiable by setting `requires_grad=True`

when their initial values are defined. The following example code shows how to build a differentiable qubit Hamiltonian when the atomic coordinates and the basis set parameters are all differentiable:

```
from pennylane import numpy as np

symbols = ['H', 'H']
geometry = np.array([[0.0, 0.0, -0.6614], [0.0, 0.0, 0.6614]], requires_grad=True)

# basis set exponents
alpha = np.array([[3.4, 0.6, 0.2], [3.4, 0.6, 0.2]], requires_grad = True)

# basis set contraction coefficients
coeff = np.array([[0.2, 0.5, 0.4], [0.2, 0.5, 0.4]], requires_grad = True)

mol = qml.qchem.Molecule(symbols, geometry, alpha=alpha, coeff=coeff)

args = [geometry, alpha, coeff]

H = qml.qchem.molecular_hamiltonian(symbols, geometry, alpha=alpha, coeff=coeff,
    args=args)[0]
```

Codeblock 4: *Constructing the differentiable Hamiltonian for the hydrogen molecule.*

PennyLane also provides functionality to construct sparse matrix representations of molecular Hamiltonians. In simulators, this is used by PennyLane to perform fast computations of expectation values. PennyLane also allows users to directly diagonalize the target Hamiltonian from its sparse-matrix representation, which can be used to benchmark the performance of quantum algorithms without the need to perform additional quantum chemistry calculations, such as full-configuration interaction. Besides molecular Hamiltonians, PennyLane also provides support for constructing other observables, which currently require interfacing with external packages. These include total particle number, total spin, spin projection, dipole moment, and user-specified observables.

Quantum circuits

Quantum algorithms for quantum chemistry construct states of many-body fermionic systems and extract relevant information from them. The most popular methods to achieve this are quantum phase estimation and variational algorithms. In quantum phase estimation [58], the eigenvalues of a Hamiltonian H are encoded in the eigenvalues of a unitary U, for example through the time-evolution operator $U = e^{-iHt}$. The algorithm samples from the spectrum of U by performing repeated calls to an oracle implementing U controlled on the state of auxiliary qubits. The auxiliary qubits are then rotated using an inverse quantum Fourier transform and measured to reveal a sampled eigenvalue. If the input state has a sufficiently large overlap with a target eigenstate of H, quantum phase estimation can probabilistically prepare the target eigenstate and compute its corresponding eigenvalue.

This is a remarkable capability that underlies the long-term potential of quantum computing for quantum chemistry. However, quantum phase estimation requires a large number of qubits and long-depth circuits that make it challenging to implement in both hardware and simulators, even for small systems. Still, PennyLane provides a general template for quantum phase estimation that can be leveraged by advanced users to study prototype applications to quantum chemistry.

In variational algorithms, the goal is to design quantum circuits that can be optimized to prepare fermionic states of interest. This allows for shorter-depth circuits without the need for auxiliary qubits, making them more appropriate for simulation and implementation in noisy hardware. However, variational algorithms lack the theoretical guarantees of quantum phase estimation and struggle to compute expectation values efficiently when implemented in hardware [59]. Optimization can often be carried out using gradient-based methods, meaning variational algorithms are amenable to the methodologies of quantum differentiable programming that are PennyLane's core strength. Below we discuss the various strategies that can be used in PennyLane to create, evaluate, and optimize variational quantum circuits for quantum chemistry.

Givens rotations

Under the Jordan-Wigner transformation, a qubit is associated with each spin-orbital and its states are used to represent occupied $|1\rangle$ or unoccupied $|0\rangle$ spin-orbitals. When designing quantum circuits for quantum chemistry, it is useful to employ only particle-number conserving gates guaranteeing that the trial space is within the correct particle number subspace. The simplest non-trivial particle-conserving gate is a two-qubit transformation that couples the states $|01\rangle, |10\rangle$ as

$$U|01\rangle = a|01\rangle + b|10\rangle, \tag{30}$$
$$U|10\rangle = c|10\rangle + d|01\rangle, \tag{31}$$

while acting as the identity on $|00\rangle$ and $|11\rangle$. Here $|a|^2 + |c|^2 = |b|^2 + |d|^2 = 1$ and $ab^* + cd^* = 0$ to ensure unitarity. This is an example of a Givens rotation: a two-dimensional rotation in the subspace of a larger Hilbert space. Restricting to the case of real parameters, this gate can be written as

$$G(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta/2) & -\sin(\theta/2) & 0 \\ 0 & \sin(\theta/2) & \cos(\theta/2) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \tag{32}$$

where we use the ordering $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ of computational basis states. We call this a single-excitation gate because the states $|10\rangle, |01\rangle$ differ by the excitation of a single particle. Similarly, we can consider the four-qubit gate that couples the states $|1100\rangle$ and $|0011\rangle$ as

$$G^{(2)}(\theta)|0011\rangle = \cos(\theta/2)|0011\rangle + \sin(\theta/2)|1100\rangle, \tag{33}$$
$$G^{(2)}(\theta)|1100\rangle = \cos(\theta/2)|1100\rangle - \sin(\theta/2)|0011\rangle. \tag{34}$$

This is an example of a double-excitation gate. Single-excitation and double-excitation gates are implemented natively in PennyLane, together with decompositions into elementary gate sets and analytical gradient rules [60, 61].

It was proven in Ref. [62] that controlled single-excitation gates are universal for particle-conserving unitaries, establishing the role of Givens rotations as the ideal building blocks of quantum circuits for quantum chemistry. PennyLane places an emphasis on single and double-excitation gates in the construction of variational circuits, as shown in the example code below. Using the `qnode` decorator, quantum circuits can be connected to a specific device performing its execution, whether hardware or simulators. In this case the circuit is executed by the built-in `default.qubit` simulator, which returns the expectation value of the Hamiltonian:

```
import pennylane as qml

dev = qml.device("default.qubit", wires=4)

@qml.qnode(dev)
def circuit(params):
    qml.BasisState(np.array([1, 1, 0, 0]), wires=range(4))
    qml.DoubleExcitation(params[0], wires=[0, 1, 2, 3])
    qml.SingleExcitation(params[1], wires=[0, 2])
    qml.SingleExcitation(params[2], wires=[1, 3])
    return qml.expval(H)
```

Codeblock 5: *An example circuit built using excitation gates based on Givens rotations. The circuit is initialized in the Hartree-Fock state for a system with two electrons in two spin-orbitals. It then applies a double-excitation gate and two single-excitation gates that are chosen to preserve spin.*

Templates

A common approach in the design of quantum circuits for quantum chemistry is to propose a fixed circuit architecture that can be used for a variety of molecules. Also known as ansätze, these circuits aim to provide a general strategy for creating states of interest in quantum chemistry. Circuits composed of multiple gates can be pre-coded in PennyLane as templates, which can then be called in the same way as individual operations. This allows for a simple interface to build circuits that implement various ansätze for quantum chemistry.

PennyLane currently provides templates for the unitary coupled-cluster singles and doubles (UCCSD) [63, 64], particle-conserving gate fabrics [65], k-UpCCGSD [66], and the AllSinglesDoubles [62] ansatz consisting of all spin-conserving single and double-excitation gates, which is shown in the example code below:

```
import pennylane as qml
from pennylane import numpy as np

singles = [[0, 2], [1, 3]]
doubles = [[0, 1, 2, 3]]
hf = np.array([1, 1, 0, 0])

dev = qml.device("default.qubit", wires=4)

@qml.qnode(dev)
def circuit(params, wires):
    qml.AllSinglesDoubles(params, wires, hf, singles, doubles)
    return qml.expval(H)
```

Codeblock 6: *Using a template to implement a circuit consisting of all spin-conserving single- and double-excitation gates acting on the Hartree-Fock state defined on a system with two electrons and four spin-orbitals, as in the hydrogen molecule in a minimal basis set.*

Adaptive circuits

A fixed circuit ansatz may work well in many cases but it will not be optimized for the specific problem at hand. This motivates strategies that select gates using information from the system being studied, which can lead to more efficient circuits. The most common approach for adaptively creating circuits is to select gates that have a large gradient with respect to the relevant cost function. By design, PennyLane allows the computation of gradients with respect to all parametrized gates in a quantum circuit.

The strategy of the ADAPT-VQE algorithm [32, 67] is to define a pool of gates, apply each to an initial state (typically the Hartree-Fock state), and select the gate with the largest gradient. This gate is then optimized and the resulting gate parameter fixed, which defines a new initial state. The procedure is repeated until a convergence criterion is reached. The `AdaptiveOptimizer()` functionality in PennyLane can be used to implement algorithms such as ADAPT-VQE:

```
import pennylane as qml
from pennylane import numpy as np

@qml.qnode(qml.device("default.qubit", wires=4))
def circuit():
    qml.BasisState(np.array([1, 1, 0, 0]), wires=range(4))
    return qml.expval(H)

operator_pool = [qml.DoubleExcitation(0.0, wires=[0, 1, 2, 3]),
                 qml.SingleExcitation(0.0, wires=[0, 2]),
                 qml.SingleExcitation(0.0, wires=[1, 3])]

opt = qml.AdaptiveOptimizer()
for i in range(len(operator_pool)):
    circuit, energy, gradient = opt.step_and_cost(circuit, operator_pool)
```

Codeblock 7: *Building a circuit adaptively by selecting and adding gates from a user-defined collection of operators. The initial state is defined on a system with two electrons and four spin-orbitals, as in the hydrogen molecule in a minimal basis set.*

Ground and excited-state energies

Once a quantum circuit has been defined, the goal of variational algorithms is to optimize the circuit with respect to an appropriate cost function. As discussed before, it is often sufficient to compute ground and excited-state energies of molecular Hamiltonians. This section discusses algorithms for performing these calculations and their PennyLane implementation.

Ground states

The variational quantum eigensolver (VQE) is an algorithm that computes approximate ground-state energies by optimizing the parameters of a quantum circuit with respect to the expectation value of a molecular Hamiltonian. More precisely, using $\theta = (\theta_1, \ldots, \theta_n)$ to denote the parameters of a variational circuit, $|\psi(\theta)\rangle$ to denote the state prepared by the circuit, and H for the molecular Hamiltonian, the goal of VQE is to optimize the quantum circuit in order to minimize the cost function

$$C(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle . \tag{35}$$

This is a broad approach that encompasses a variety of methods for circuit design and parameter optimization. As discussed above, PennyLane provides functionality to build molecular Hamiltonians and design quantum circuits using individual gates, templates, or adaptive methods. To perform the circuit optimization, users have access to a wide array of gradient-based optimizers that leverage PennyLane's built-in ability to compute gradients of quantum circuits. This includes quantum-aware optimizers such as the quantum natural gradient [68], Rotosolve [61, 69, 70], and Rotoselect [70].

To compute expectation values of Hamiltonians using simulators, PennyLane converts the Hamiltonian to a sparse matrix, then uses the vector representation of the state to compute the expectation value $\langle H \rangle = \langle \psi | H | \psi \rangle$ directly using matrix-vector multiplication. It is also possible to directly pass a sparse matrix representation of the Hamiltonian to PennyLane's `expval()` function. Gradients are obtained using parameter-shift rules [71], backpropagation, or the adjoint method.

To calculate expectation values in hardware, it is necessary to compute the expectation of each term in the Hamiltonian's expansion as a linear combination of Pauli words

$$\langle H \rangle = \sum_j h_j \langle P_j \rangle . \tag{36}$$

The expectation values $\langle P_j \rangle$ can be calculated by performing only additional single-qubit rotations because the Pauli words P_j are tensor products of local qubit operators. One major challenge is the large number of terms in this expansion, which increases rapidly for larger molecules. This leads to the requirement for a total number of samples that scales dramatically with system size; a major obstacle for scaling these algorithms. To partially alleviate this problem, it is possible to group Pauli words into sets of mutually-commuting operators whose expectation values can then be calculated from the same measurement statistics. This functionality is provided in PennyLane's `grouping` module, and can be activated by passing the keyword argument `optimize=True` when computing the expectation value.

The code below shows a full VQE workflow in PennyLane for computing the ground state of the hydrogen molecule in the default minimal basis set.

```
import pennylane as qml
from pennylane import numpy as np

symbols = ['H', 'H']
geometry = np.array([[0.0, 0.0, -0.6614], [0.0, 0.0, 0.6614]], requires_grad=False)
mol = qml.qchem.Molecule(symbols, geometry)
H = qml.qchem.molecular_hamiltonian(symbols, geometry)[0]

@qml.qnode(qml.device("default.qubit", wires=4))
def circuit(param):
    qml.BasisState(np.array([1, 1, 0, 0]), wires=range(4))
    qml.DoubleExcitation(param, wires=[0, 1, 2, 3])
    return qml.expval(H)

opt = qml.GradientDescentOptimizer(stepsize=0.4)
theta = np.array(0.0, requires_grad=True)

for n in range(20):
    theta, energy = opt.step_and_cost(circuit, theta)
```

Codeblock 8: *VQE workflow to compute the ground-state energy of the hydrogen molecule.*

By performing ground-state calculations for Hamiltonians $H(\mathbf{R})$ defined on different nuclear coordinates $\mathbf{R}$, it is possible to compute ground-state potential energy surfaces of molecules, which are defined by the energy function

$$E(\mathbf{R}) = \min_{|\psi\rangle} \langle\psi| H(\mathbf{R}) |\psi\rangle = \langle\psi_0(\mathbf{R})| H(\mathbf{R}) |\psi_0(\mathbf{R})\rangle , \tag{37}$$

where $|\psi_0(\mathbf{R})\rangle$ is the ground state, which depends on the nuclear coordinates $\mathbf{R}$. The minima of a potential energy surface correspond to equilibrium geometries, while local maxima correspond to transition states. Computing the energies at these configurations allows us to calculate activation energies and chemical reaction rates.

Excited states

The simplest method to extend ground-state algorithms to excited states is to leverage the fact that eigenstates are mutually orthogonal [72, 73]. Excited-state energies can be computed by adding penalty terms to the cost function that enforce orthogonality with lower-lying eigenstates. For example, to optimize a circuit that prepares the first excited state we employ the cost function

$$C^{(1)}(\theta) = \langle\psi(\theta)| H^{(1)} |\psi(\theta)\rangle , \tag{38}$$

$$H^{(1)} = H + \beta |\psi_0\rangle \langle\psi_0| , \tag{39}$$

where $\beta > 0$ is an adjustable penalty parameter. The ground state $|\psi_0\rangle$ is also an eigenstate of $H^{(1)}$ with eigenvalue $E_0 + \beta$, where E_0 is the ground-state energy of H. Setting $\beta > E_1 - E_0$ ensures that the lowest-energy eigenstate of $H^{(1)}$ is actually the first excited state of H. This procedure can be iterated to find the k-th excited state by minimizing the cost function

$$C^{(k)}(\theta) = \langle\psi(\theta)| H^{(k)} |\psi(\theta)\rangle , \tag{40}$$

$$H^{(k)} = H + \sum_{i=0}^{k-1} \beta_i |\psi_i\rangle \langle\psi_i| . \tag{41}$$

PennyLane users can perform excited-state calculations using the same functionalities as for ground-state computations by adjusting the cost function accordingly. It is also possible to employ the built-in spin and dipole moment observables to add penalties that select excited states corresponding to desired transitions.

These extended cost functions can be readily evaluated using simulators, but require additional resources in hardware. Penalty terms of the form $|\langle\psi(\theta)|\psi_i\rangle|^2$ can be computed using a SWAP test, which requires roughly doubling the number of qubits for the same circuit depth. An alternative is to apply a unitary $V^\dagger$ at the end of the circuit, where $V|0\rangle = |\psi_i\rangle$. The desired overlap can be computed as $|\langle\psi(\theta)|\psi_i\rangle|^2 = |\langle 0| U(\theta) V^\dagger |0\rangle|^2$, which is the probability of measuring the all-zero state in the adjusted circuit and $U(\theta)$ is a unitary preparing the trial ground state. This maintains the same number of qubits while roughly doubling the circuit depth.

Energy derivatives

Ground and excited-state energies provide valuable information about the properties of molecules, but they are not exhaustive in revealing all relevant information. Further insights can be obtained by studying gradients of the total energy. This provides the ability to calculate forces on nuclei, perform molecular geometry optimization, and compute vibrational normal modes and frequencies in the harmonic approximation. This section describes the theory of energy derivatives in quantum chemistry and explains how to implement related algorithms in PennyLane.

Nuclear forces and geometry optimization

The force experienced by the nuclei of each atom in a molecule is given by the gradient of the total energy with respect to the nuclear coordinates

$$F(\mathbf{R}) = -\nabla_{\mathbf{R}} E(\mathbf{R}), \tag{42}$$

where $E(\mathbf{R})$ is defined as in Eq. (37). From the Hellmann-Feynman theorem, the derivative of the ground-state energy with respect to the coordinates of the i-th atom can be written as

$$\frac{\partial E(\mathbf{R})}{\partial R_i} = \left\langle \psi_0(\mathbf{R}) \middle| \frac{\partial H(\mathbf{R})}{\partial R_i} \middle| \psi_0(\mathbf{R}) \right\rangle. \tag{43}$$

This is an analytical expression that can be calculated from a circuit that has been optimized to prepare $|\psi_0(\mathbf{R})\rangle$. Crucially, it requires a method to compute the Hamiltonian derivatives $\partial H(\mathbf{R})/\partial R_i$. One of the key advantages of PennyLane's differentiable Hartree-Fock solver is that energy can be calculated directly and exactly using methods of automatic differentiation, as illustrated in the example code below:

```python
import pennylane as qml
from pennylane import numpy as np

symbols = ['H', 'H']
geometry = np.array([[0.0, 0.0, -0.6614], [0.0, 0.0, 0.6614]], requires_grad=True)
mol = qml.qchem.Molecule(symbols, geometry)
args = [geometry]

dev = qml.device("default.qubit", wires=4)

def energy(mol):
    @qml.qnode(dev)
    def circuit(*args):
        qml.BasisState(np.array([1, 1, 0, 0]), wires=range(4))
        # applies gate with optimal parameter
        qml.DoubleExcitation(0.209733, wires=[0, 1, 2, 3])
        H = qml.qchem.molecular_hamiltonian(symbols, geometry, args=args)[0]
        return qml.expval(H)
    return circuit

forces = -qml.grad(energy(mol))(*args)
```

Codeblock 9: *The nuclear forces can be computed exactly using the differentiable Hartree-Fock solver to compute gradients of an optimized circuit that calculates the ground-state energy.*

More broadly, as described in Ref. [74], Hamiltonian derivatives permit the study of cost functions that depend both on circuit parameters θ and Hamiltonian parameters such as the nuclear coordinates $\mathbf{R}$:

$$C(\theta, \mathbf{R}) = \langle \psi(\theta) | H(\mathbf{R}) | \psi(\theta) \rangle . \tag{44}$$

PennyLane users can compute gradients of this cost function with respect to both set of parameters, allowing for a joint optimization of the cost function. The results are optimal circuit parameters θ^* that prepare an approximate ground state and optimal nuclear coordinates $\mathbf{R}^*$ that describe the equilibrium geometry of the molecule.

Hessians and vibrational modes

Expressions for higher-order energy derivatives can also be obtained [33, 75], but in this case there are contributions arising from derivatives of the ground state $|\psi_0(\mathbf{R})\rangle$, which themselves depend on derivatives of the optimal parameters $\theta^*(\mathbf{R})$. The second-order derivatives that define the Hessian of the energy can be calculated as [33]:

$$\frac{\partial^2 E(\mathbf{R})}{\partial R_i \partial R_j} = \sum_a \left[\frac{\partial \theta_a^*(\mathbf{R})}{\partial R_i} \frac{\partial}{\partial \theta_a} \frac{\partial E(\mathbf{R})}{\partial R_j} \right] + \left\langle \psi(\theta^*(\mathbf{R})) \middle| \frac{\partial^2 H(\mathbf{R})}{\partial R_i \partial R_j} \middle| \psi(\theta^*(\mathbf{R})) \right\rangle.$$

To evaluate this expression, it is necessary to compute the expectation value of the Hessian of the Hamiltonian $\partial^2 H(\mathbf{R})/\partial R_i \partial R_j$, which again can be performed exactly using PennyLane's differentiable Hartree-Fock solver. The need for exact calculations is even more important in this case since approximate techniques such as finite-difference are notoriously problematic for Hessians.

To compute $\frac{\partial}{\partial\theta_a}\frac{\partial E(\mathbf{R})}{\partial R_j}$, it suffices to take the circuit that has been optimized to prepare the ground-state energy and compute its gradient with respect to the expectation vale of $\partial H(\mathbf{R})/\partial R_j$. This can be performed natively with PennyLane by computing gradients of a circuit that has been optimized to calculate the ground-state energy. The remaining term $\partial\theta_a^*(R)/\partial R_i$ can be obtained by solving the response equations [33]:

$$\sum_b \frac{\partial}{\partial\theta_a}\frac{\partial E(\mathbf{R})}{\partial\theta_b}\frac{\partial\theta_b^*(\mathbf{R})}{\partial R_i} = -\frac{\partial}{\partial\theta_a}\frac{\partial E(\mathbf{R})}{\partial R_j}. \tag{45}$$

We previously discussed how $\frac{\partial}{\partial\theta_a}\frac{\partial E(\mathbf{R})}{\partial R_j}$ can be computed. The term $\frac{\partial}{\partial\theta_a}\frac{\partial E(\mathbf{R})}{\partial\theta_b}$ is the second-order derivative of the circuit with respect to the expectation value of the Hamiltonian, evaluated at the optimal circuit parameters. It can also be performed natively with PennyLane by expressing the Hessian as the Jacobian of the gradient. Therefore all terms in the equation can be calculated except for $\partial\theta_b^*(R)/\partial R_i$. This can be obtained by constructing the response equations of Eq. (46) and solving the resulting system of linear equations.

Computing energy Hessians provides information about the vibrational structure of molecules, which can be used as inputs to algorithms for calculating properties such as vibronic spectra and vibrational dynamics [76–79]. In the harmonic approximation, the potential experienced by the nuclei is approximated by a quadratic function that leads to harmonic motion. The eigenvectors of the energy Hessian correspond to the normal modes of vibration in the harmonic approximation, and the eigenvalues are the vibrational normal mode frequencies.

Fully-differentiable workflows

This section combines concepts described throughout the manuscript to illustrate how PennyLane can be used to create differentiable quantum chemistry workflows for electronic structure calculations capable of simultaneously optimizing circuit parameters, nuclear coordinates, and basis set parameters. The example code below illustrates this for the H_3^+ cation.

```
import pennylane as qml
from pennylane import numpy as np

symbols = ["H", "H", "H"]

# non-equilibrium geometry
geometry = np.array([[0.0, 0.0, 0.0], [1.0, 1.0, 0.0],
                     [2.0, 0.0, 0.0]], requires_grad=True)

# basis set exponents
alpha = np.array([[3.4253, 0.6239, 0.1689], [3.4253, 0.6239, 0.1689],
                  [3.4253, 0.6239, 0.1689]], requires_grad=True)

# basis set contraction coefficients
coeff = np.array([[0.1543, 0.5353, 0.4446], [0.1543, 0.5353, 0.4446],
                  [0.1543, 0.5353, 0.4446]], requires_grad=True)

params = [np.array([0.0], requires_grad=True)] * 2

dev = qml.device("default.qubit", wires=6)

def energy(mol):
    @qml.qnode(dev)
    def circuit(*args):
        qml.BasisState(np.array([1, 1, 0, 0, 0, 0]), wires=range(6))
        qml.DoubleExcitation(*args[0][0], wires=[0, 1, 2, 3])
        qml.DoubleExcitation(*args[0][1], wires=[0, 1, 4, 5])
        H = qml.qchem.molecular_hamiltonian(mol.symbols, mol.coordinates,
            charge = 1, alpha=mol.alpha, coeff=mol.coeff, args=args[1:])[0]
        return qml.expval(H)
    return circuit
```

```

mol = qml.qchem.Molecule(symbols, geometry, alpha=alpha, coeff=coeff)
args = [params, geometry, alpha, coeff]

# gradient for circuit parameters
gradient_params = qml.grad(energy(mol), argnum = 0)(*args)

# gradient for nuclear coordinates
forces = -qml.grad(energy(mol), argnum = 1)(*args)

# gradient for basis set exponents
gradient_alpha = qml.grad(energy(mol), argnum = 2)(*args)

# gradient for basis set contraction coefficients
gradient_coeff = qml.grad(energy(mol), argnum = 3)(*args)
```

Codeblock 10: *Workflow for simultaneously optimizing circuit parameters, nuclear coordinates, and basis set parameters for the H_3^+ cation.*

Qubit tapering

The number of qubits required to simulate a molecular Hamiltonian can be reduced in the presence of symmetries [80, 81]. PennyLane provides functions for tapering off qubits from Hamiltonians containing multiple $\mathbb{Z}_2$ symmetries [80]. The qubit tapering method requires finding a unitary operator that transforms the molecular Hamiltonian to a new Hamiltonian that has the same eigenvalues. The new Hamiltonian always acts trivially, e.g., with an identity or a Pauli-X operator, on a set of qubits $q(j)$. This guarantees that each term of the transformed Hamiltonian commutes with each of the Pauli-X operators applied to the same set of qubits. These commuting terms have the same eigenvectors, which allows removing the qubits from the Hamiltonian terms and replacing them with eigenvalues of the Pauli-X operators ± 1. The unitary operator U is constructed as a Clifford operator given by [80]

$$U = \Pi_j \left[\frac{1}{\sqrt{2}} \left(X^{q(j)} + \tau_j \right) \right], \tag{46}$$

where τ denotes the generators of the symmetry group of the Hamiltonian, and the Pauli-X operators act on those qubits $q(j)$ that will be ultimately tapered off from the Hamiltonian. The following code shows how to obtain τ and X for a given Hamiltonian in PennyLane and use them to taper off qubits. The same symmetries can be used to taper the initial state and the excitation gates to perform a full VQE simulation with a smaller number of qubits.

```
import pennylane as qml
from pennylane import numpy as np

symbols = ["He", "H"]
geometry = np.array([[0.0, 0.0, -0.9], [0.0, 0.0, 0.9]])

charge, n_electrons = 1, 2

H, qubits = qml.qchem.molecular_hamiltonian(symbols, geometry, charge=charge)

generators = qml.symmetry_generators(H)

paulixops = qml.paulix_ops(generators, qubits)

paulix_sector = qml.qchem.optimal_sector(H, generators, n_electrons)

H_tapered = qml.taper(H, generators, paulixops, paulix_sector)
```

Codeblock 11: *Tapering off qubits in the presence of multiple $\mathbb{Z}_2$ symmetries.*

The original Hamiltonian in this code example requires four qubits, two of which can be tapered off using the symmetries.

Resource estimation

Quantum algorithms such as quantum phase estimation (QPE) and the variational quantum eigensolver (VQE) can be applied to problems that are intractable for conventional computers. These problems require quantum computers capable of implementing large-scale versions of these algorithms, which are not yet available. This makes it difficult to properly explore the accuracy and efficiency of quantum algorithms for problem sizes where the actual advantage of the algorithms can potentially occur. However, it is still possible to estimate the resources required to implement these algorithms for a given problem.

PennyLane has a functionality for estimating quantum resources required to simulate a molecular system. For example, PennyLane can be used to estimate the total number of shots required to estimate the expectation value of an observable [82]. This is relevant for the VQE algorithm, where expectation values are needed to compute gradients and also the final output of the circuit. The following code shows how to estimate the number of shots needed to compute the expectation value of the water Hamiltonian:

```
import pennylane as qml
from pennylane import numpy as np

symbols = ['O', 'H', 'H']

geometry = np.array([[0.0,  0.0,  0.3],
                     [0.0,  1.5, -1.0],
                     [0.0, -1.5, -1.0]], requires_grad=False)

H = qml.qchem.molecular_hamiltonian(symbols, geometry)[0]

shots = qml.resource.estimate_shots(H.coeffs)
```

Codeblock 12: *Estimating the number of shots needed to compute the ground state energy with VQE.*

Resource estimation in PennyLane can also be used to compute the total number of non-Clifford gates and logical qubits required to run quantum phase estimation algorithms for quantum chemistry. The following example shows how to perform such estimation for the water molecule using a double low-rank factorization algorithm [83, 84] for the QPE simulation:

```
import pennylane as qml
from pennylane import numpy as np

symbols = ['O', 'H', 'H']

geometry = np.array([[0.0,  0.0,  0.3],
                     [0.0,  1.5, -1.0],
                     [0.0, -1.5, -1.0]], requires_grad=False)

mol = qml.qchem.Molecule(symbols, geometry, basis_name='6-31g')

core, one, two = qml.qchem.electron_integrals(mol)()

algo = qml.resource.DoubleFactorization(one, two)

gates, qubits = algo.gates, algo.qubits
```

Codeblock 13: *Estimating the number of non-Clifford gates and logical qubits with the double low-rank factorization algorithm.*

The total number of non-Clifford gates and logical qubits can also be estimated for first-quantized Hamiltonians of periodic materials in a plane wave basis [40]. The cost of these algorithm depends only on the number of plane waves, the volume of the unit cell, and the number of electrons in the material. The following code uses dilithium iron silicate Li_2FeSiO_4 as an example, whose unit cell consists of 156 electrons and has a volume of $1,145a_0^3$, where a_0 is the Bohr radius [25]:

```python
import pennylane as qml

planewaves = 100000
electrons = 156
volume = 1145

algo = qml.resource.FirstQuantization(planewaves, electrons, volume)

gates, qubits = algo.gates, algo.qubits
```

Codeblock 14: *Estimating the number of non-Clifford gates and logical qubits for a periodic material.*

Conclusion

Quantum chemistry is arguably the leading application of quantum computing: it combines (i) a mathematical foundation underpinning the long-term potential for quantum advantage, and (ii) an area of significant industrial interest. Scientists working at the interface between quantum computing and quantum chemistry face the challenge of gaining expertise in these two fields, both of which have a considerable barrier of entry. Software plays a central role in lowering these obstacles and boosting scientific progress. This is one of the main goals of PennyLane: to create a tool that can enhance progress in quantum computing. With this technical manuscript, we aim to provide an additional resource for users that benefit from a deeper dive into the scientific concepts that underly PennyLane's quantum chemistry functionality, enabling them to better employ the software and advance research combining differentiable programming, quantum computing, and quantum chemistry.

The functionality described in this manuscript reflects PennyLane's capabilities at the time of writing. Our goal is to continue working on further development to expand its scope and enhance its performance. As an open-source software library, PennyLane welcomes contributions from users across the world. This effort will constitute incorporation of state-of-the-art techniques in the scientific literature as well as innovations from the team of PennyLane developers. Quantum computational chemistry is a relatively young field and there still remains much to be discovered.

[1] Dougal Maclaurin, David Duvenaud, and Ryan P Adams, "Autograd: Effortless gradients in numpy," in *ICML 2015 AutoML workshop*, Vol. 238 (2015) p. 5.

[2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, *et al.*, "Tensorflow: A system for large-scale machine learning," in *12th USENIX symposium on operating systems design and implementation (OSDI 16)* (2016) pp. 265–283.

[3] James Bradbury, Roy Frostig, Peter Hawkins, Matthew J. Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang, "JAX: composable transformations of Python+NumPy programs," (2018).

[4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," Adv. Neural Inf. Process. Syst. **32**, 8026–8037 (2019).

[5] Matthew Reagor, Christopher B Osborn, Nikolas Tezak, Alexa Staley, Guenevere Prawiroatmodjo, Michael Scheer, Nasser Alidoust, Eyob A Sete, Nicolas Didier, Marcus P da Silva, *et al.*, "Demonstration of universal parametric entangling gates on a multi-qubit lattice," Sci. Adv. **4**, eaao3603 (2018).

[6] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando GSL Brandao, David A Buell, *et al.*, "Quantum supremacy using a programmable superconducting processor," Nature **574**, 505–510 (2019).

[7] K Wright, KM Beck, Sea Debnath, JM Amini, Y Nam, N Grzesiak, J-S Chen, NC Pisenti, M Chmielewski, C Collins, *et al.*, "Benchmarking an 11-qubit quantum computer," Nat. Commun. **10**, 5464 (2019).

[8] JM Arrazola, V Bergholm, K Brádler, TR Bromley, MJ Collins, I Dhand, A Fumagalli, T Gerrits, A Goussev, LG Helt, *et al.*, "Quantum circuits with many photons on a programmable nanophotonic chip," Nature **591**, 54–60 (2021).

[9] Petar Jurcevic, Ali Javadi-Abhari, Lev S Bishop, Isaac Lauer, Daniela F Bogorin, Markus Brink, Lauren Capelluto, Oktay Günlük, Toshinari Itoko, Naoki Kanazawa, *et al.*, "Demonstration of quantum volume 64 on a superconducting quantum computing system," Quantum Sci. Technol. **6**, 025020 (2021).

[10] Sepehr Ebadi, Tout T Wang, Harry Levine, Alexander Keesling, Giulia Semeghini, Ahmed Omran, Dolev Bluvstein, Rhine Samajdar, Hannes Pichler, Wen Wei Ho, *et al.*, "Quantum phases of matter on a 256-atom programmable quantum simulator," Nature **595**, 227–232 (2021).

Chair for Design Automation
TUM School of Computation, Information and Technology
Technical University of Munich

The Munich Quantum Toolkit (MQT)

Design Automation Tools and Software for Quantum Computing

Robert Wille and Team
Contact: robert.wille@tum.de
https://www.cda.cit.tum.de/research/quantum/mqt

Abstract

Quantum computers are becoming a reality. But designing applications for these devices requires automated, efficient, and user-friendly software tools that cater to the needs of end-users, engineers, and physicists at every level of the design flow. The Munich Quantum Toolkit (MQT) is a collection of design automation tools and software for quantum computing developed at the Chair for Design Automation at the Technical University of Munich. This flyer provides an overview of the provided solutions. For each step in the design flow, numbered nodes indicate the correspondingly available software repositories (summarized on the back of this flyer). All software is available as open-source.

Data Structures / Core Methods 11 12 13 14 15 16

In order to tackle the complexity of important design tasks, the MQT utilizes efficient data structures (e.g., for the representation and manipulation of quantum states and operations) as well as dedicated core methods (e.g., allowing to realize optimal methods) including:

- Decision Diagrams
- Tensor Networks
- ZX-Calculus
- SAT/SMT Solvers
- Machine Learning
- Heuristics

For performance reasons, all tools are implemented in C++ with convenient Python bindings and compatibility to other tools.

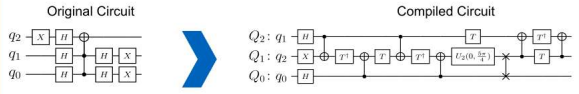
MUNICH QUANTUM TOOLKIT

Application 1 2

- Workflow from classical problem to quantum solution
- Automated problem encoding, execution, and decoding

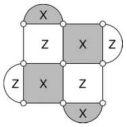
Compilation 4 5 6 7 8

- Determining good compilation options
- Reversible circuit/quantum oracle synthesis
- Technology-specific mapping
  - Quantum circuit mapping/SWAP gate insertion
  - Shuttling for Trapped Ions
- Multi-level (Qudit) Compilation



Error Correction 10

- Decoding algorithms
- Automated code construction and numerical simulations



All tools are available as open-source implementations on GitHub.

Application

Simulation

Compilation

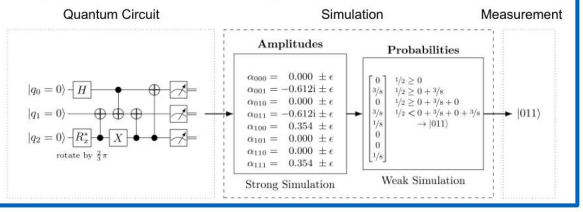
Verification

Error Correction

Hardware

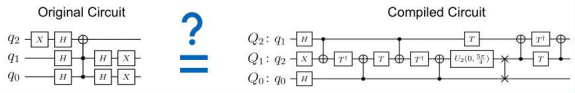
Simulation 3

- Simulation of gate-based quantum circuits based on decision diagrams
- Includes sampling, noise-aware simulation, Hybrid Schrödinger Feynman approaches, approximation strategies, etc.



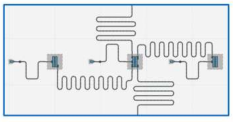
Verification 9

- Equivalence checking of quantum circuits
- Verification of compilation results



Hardware

- Application-specific physical design for superconducting platform



Chair for Design Automation
TUM School of Computation, Information and Technology
Technical University of Munich

MUNICH QUANTUM TOOLKIT

1 MQT ProblemSolver — Application

A Tool for Solving Problems Using Quantum Computing

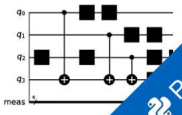
github.com/cda-tum/mqtproblemsolver

PyPI

2 MQT Bench — Application

A Quantum Circuit Benchmark Suite

www.cda.cit.tum.de/mqtbench
github.com/cda-tum/mqtbench

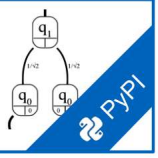


PyPI

3 MQT DDSIM — Simulation

A Tool for Classical Quantum Circuit Simulation based on Decision Diagrams

github.com/cda-tum/ddsim



PyPI

4 MQT Predictor — Compilation

A Tool for Determining Good Quantum Circuit Compilation Options

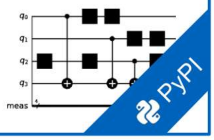
github.com/cda-tum/mqtpredictor

PyPI

5 MQT SyReC — Compilation

A Tool for the Synthesis of Reversible Circuits/Quantum Computing Oracles

github.com/cda-tum/syrec



PyPI

6 MQT QMAP — Compilation

A Tool for Quantum Circuit Mapping

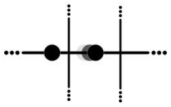
github.com/cda-tum/qmap

PyPI

7 MQT IonShuttler — Compilation

A Tool for Generating Shuttling Schedules for QCCD Architectures

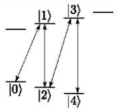
github.com/cda-tum/ion-shuttler



8 MQT Qudits — Compilation

A Tool for Compiling High-Dimensional Quantum Systems

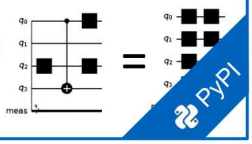
github.com/cda-tum/qudit-compilation
github.com/cda-tum/qudit-entanglement-compilation



9 MQT QCEC — Verification

A Tool for Quantum Circuit Equivalence Checking

github.com/cda-tum/qcec

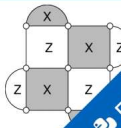


PyPI

10 MQT QECC — QECC

A Tool for Quantum Error Correcting Codes

github.com/cda-tum/qecc

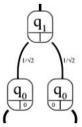


PyPI

11 MQT DDPackage — Data Structures

A Decision Diagram Package for Quantum Computing

github.com/cda-tum/dd_package



12 MQT DDVis — Data Structures

A Web-Application visualizing Decision Diagrams for Quantum Computing

www.cda.cit.tum.de/app/ddvis
github.com/cda-tum/ddvis

13 MQT QFR — Data Structures

An Intermediate Representation for Quantum Circuits and Computations

github.com/cda-tum/qfr

14 MQT Logic Blocks — Data Structures

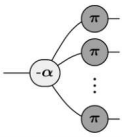
An Interface Library for SAT/SMT Abstractions

github.com/cda-tum/logicblocks

15 MQT ZX — Data Structures

A ZX-Calculus Package

github.com/cda-tum/zx



16 MQT QuSAT — Core Methods

A Tool for Encoding Quantum Computing using Satisfiability Testing (SAT) Techniques

github.com/cda-tum/qusat

$$F \wedge (x_1 \wedge \neg x_2)$$
$$F \wedge (x_3 \wedge x_$$

PyPI

Technische Universität München

European Research Council
Established by the European Commission

Munich Quantum Valley

communications physics

ARTICLE

https://doi.org/10.1038/s42005-021-00684-3 OPEN

Quantum compiling by deep reinforcement learning

Lorenzo Moro[1,2], Matteo G. A. Paris[3], Marcello Restelli[1] & Enrico Prati[2]✉

The general problem of quantum compiling is to approximate any unitary transformation that describes the quantum computation as a sequence of elements selected from a finite base of universal quantum gates. The Solovay-Kitaev theorem guarantees the existence of such an approximating sequence. Though, the solutions to the quantum compiling problem suffer from a tradeoff between the length of the sequences, the precompilation time, and the execution time. Traditional approaches are time-consuming, unsuitable to be employed during computation. Here, we propose a deep reinforcement learning method as an alternative strategy, which requires a single precompilation procedure to learn a general strategy to approximate single-qubit unitaries. We show that this approach reduces the overall execution time, improving the tradeoff between the length of the sequence and execution time, potentially allowing real-time operations.

[1] Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milano, Italy. [2] Istituto di Fotonica e Nanotecnologie, Consiglio Nazionale delle Ricerche, Milano, Italy. [3] Quantum Technology Lab, Dipartimento di Fisica Aldo Pontremoli, Università degli Studi di Milano, Milano, Italy. ✉email: enrico.prati@cnr.it

Quantum computation takes place at its lowest level by means of physical operations described by unitary matrices acting on the state of qubits. However, gate-model quantum computers may in practice provide just a limited set of transformations according to the constraints in their architecture[1–4]. Therefore, the computation is achieved as circuits of quantum gates, which are ordered sequences of unitary operators, acting on a few qubits at once[5]. Although the Solovay Kitaev theorem[6] ensures that any computations can be approximated, within an arbitrary tolerance, as a circuit based on a finite set of operators, there is no optimal strategy to establish how to compute such a sequence. The problem is known as quantum compiling and the algorithms to compute suitable approximating circuits as quantum compilers.

Every quantum compiler has its own trade-off between the length of the sequences, which should be as short as possible, the precompilation time, i.e., the time taken by the algorithm to be ready for use, and finally the execution time, i.e, the time the algorithm takes to return the sequence[7].

Previous works[7–10], mostly based on the Solovay–Kiteav theorem, addressed the problem by providing algorithms that return the approximating sequence with lengths and execution times that scale polylogarithmic as $\mathcal{O}(\log^{c}(1/\delta))$, where δ is the accuracy and c is a constant between 3 and 4. For instance, the Dawson–Nielsen (DNSK) formulation[11] provides sequences of length $\mathcal{O}(\log^{3.97}(1/\delta))$ in a time of $\mathcal{O}(\log^{2.71}(1/\delta))$. Additional performance gains can be achieved by selecting unique sets of quantum gates[7], reaching lengths that scale as $\mathcal{O}(\log^{\log(3)/\log(2)}(1/\delta))$ at the cost of increasing the precompilation time. Hybrid approaches involving a planning algorithm[12], in some cases boosted by deep neural networks[13], could achieve better performance. However, the planning algorithm raises the execution time, which could scale suboptimally for high accuracy. Despite the strategy considered, no algorithm can return the sequence using less than $\mathcal{O}(\log(1/\delta))$ gates, as shown in[9] by a geometrical proof. While existing quantum compilers are characterized by high execution and precompilation times[7,11], which make them impractical to compute during online operations, deep learning suggests an alternative approach.

Deep reinforcement learning is a subset of machine learning that exploits deep neural networks to learn optimal policies in order to achieve specific goals in decision-making problems[14–16]. Such techniques can be effective in high-dimensional control tasks and to address problems where limited or no prior knowledge of the configuration space of the system is available.

The fundamental assumptions and concepts in the reinforcement learning theory are built upon the idea of continuous interactions between a decision-maker called agent and a controlled system named environment, typically defined in the form of a Markov decision process[17] (MDP). According to a policy function that fully determines its behavior, the former interacts with the latter at discrete time steps, performing an action based on an observation related to the current state of the environment. Therefore, the environment evolves changing its state and returning a reward signal that can be interpreted as a measure of the adequateness of the action the agent has performed. The only purpose of the agent is to learn a policy to maximize the reward over time. The learning procedure can be a highly time-consuming task, but it has to be performed once. Then, it is possible to exploit the policy encoded in the deep neural network, with low computational resources in minimal time.

Recently, deep learning has been successfully applied to physics[18–21], where unprecedented advancements have been achieved by combining reinforcement learning[22] with deep neural networks into deep reinforcement learning (DRL). DRL, thanks to its ability to identify strategies for achieving a goal in complex configuration spaces without prior knowledge of the system[23–28], has recently been proposed for the control of quantum systems[15,18,29–33]. In this context, some of us previously applied deep reinforcement learning to control and initialize qubits by continuous pulse sequences[34,35] for coherent transport by adiabatic passage (CTAP)[36] and by digital pulse sequences for stimulated Raman passage (STIRAP)[37,38], respectively. Furthermore, it has proven effective as a control framework for optimizing the speed and fidelity of quantum computation[39] and in control of quantum gates[40].

In this work, we propose an approach to quantum compiling, exploiting deep reinforcement learning to approximate single-qubit unitary operators as circuits made by an arbitrary initial set of elementary quantum gates. As examples, we show how to steer quantum compiling for small rotations of $\pi/128$ around the three-axis of the Bloch sphere and for the Harrow–Recht–Chuang efficiently universal gates (HRC)[9], by employing two alternative DRL algorithms, depending on the nature of the base. After training, agents can generate single-qubit logic circuits within a tolerance of 0.99 average-gate fidelity (AGF). The adopted strategy for training the agent consists of generating a uniform distribution of single-qubit unitary matrices, where to sample the training targets. The agents are not told how to approximate such targets, but instead, they are asked to establish a suitable policy to complete the task. The agents' final performance is then measured using a validation set of unitary operators not previously seen by the agent.

To summarize, the DRL agents learn a policy to approximate single-qubit unitary transformation at the cost of a precompilation procedure, which is done only once. The method is effective for both sets of small-angle rotations and sparse sets of unitary operators. Average gate fidelity achieves $\varepsilon = 0.9999$ in the best cases for small rotations, for which the execution time empirically scales as $\mathcal{O}(\log^{1.25}(1/(1-\varepsilon)))$. Although the method does not guarantee finding the solution, it has the advantage of operating independently from the specific hardware and that its speed would enable real-time computing.

Results and discussion

Deep reinforcement learning as quantum compiler. The quantum compilation is a fundamental problem in the quantum computation theory, consisting of approximating any unitary transformation as a finite sequence of unitary operators A_j is chosen from a universal set of gates $\mathcal{B}$.

In this work, we ask the agent to approximate any single-qubit unitary matrix $\mathcal{U}$, within a fixed tolerance ε. Therefore, the goal of the agent is to find a unitary matrix $U_n = \prod_{j=1}^{n} A_j$, resulting from the composition of the elements in the sequence, that is sufficiently close to $\mathcal{U}$. Although the DRL framework allows exploiting any distance between matrices to evaluate the accuracy of the solutions, the average gate fidelity is wildly used for the purpose, mainly due to the modest computational demands needed to compute it. Alternative choices are possible, such as the diamond norm[41,42].

In the framework of quantum compiling, the environment consists of a quantum circuit that starts as the identity at the beginning of each episode. It is built incrementally at each time step by the agent, choosing a gate from $\mathcal{B}$ according to the policy π encoded in the deep neural network, as shown in Fig. 1. Therefore, the available actions that the agent can perform correspond to the gates in the base $\mathcal{B}$.

The observation used as input at time step n corresponds to the vector of the real and imaginary parts of the elements of the matrix O_n, where $\mathcal{U} = U_n \cdot O_n$. Such representation encodes all the information needed by the agent to build a suitable approximating sequence of gates, i.e., the current composition

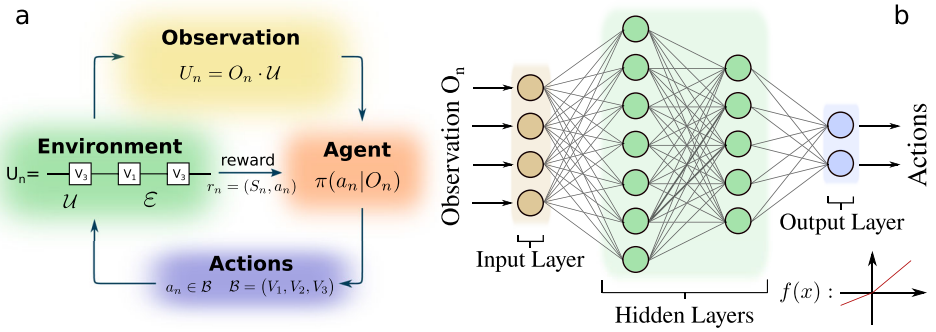


Fig. 1 The deep reinforcement learning (DRL) architecture. a The DRL environment can be described as a quantum circuit modeled by the approximating sequence U_n, the fixed tolerance ε, and the unitary target to approximate $\mathcal{U}$, that generally changes at each episode. At each time step n, the agent receives the current observation O_n and based on that information, it chooses from the base $\mathcal{B}$ the next gate an to apply on the quantum circuit. Therefore, the environment returns the real-valued reward r_n to the agent, which is a function of the state S_n and the action a_n. **b** The policy π of the agent is encoded in a deep neural network (DNN). The policy of the agent is encoded in a deep neural network. At each time-step, the DNN receives as input a vector made by the real and imaginary parts of the observation O_n. Such information is processed by the hidden layers and returned through the output layer. The neurons in the output layer are associated with the action the agent will perform in the next time step. In the bottom-right corner is reported an example of the nonlinear activation function, i.e., the rectified linear unit function RELU.

of gates and the unitary target to approximate. No information on the tolerance is given to the agent since it is fixed and thus it can be learned indirectly during the training.

Designing a suitable reward function is challenging, potentially leading to unexpected or unwanted behavior if not defined accurately. Therefore, two reward functions have been designed, depending on the different characteristics of the gate base considered, which can be identified as quasi-continuous-like sets of small rotations and discrete sets. The former are inspired by gates available on superconductive and trapped ions architecture[1,2,4], where the latter is the standard set of logic gates, typically used to write quantum algorithms, e.g., the Clifford+T library[43,44]. Both reward functions are negative at each time step, so that the agent will prefer shorter episodes.

In this work, we exploit Deep Q-Learning (DQL)[45] and Proximal Policy Optimization (PPO)[46] algorithms to train the agents, depending on the reward function. Such algorithms differ in many aspects, as described in Supplementary Note 1. The former is mandatory for the case of sparse reward, since such reward requires off-policy methods to be exploited, while the latter has been chosen for its robustness and tunability. More details on the rewards are given in Supplementary Note 2.

Training neural networks for approximating a single-qubit gate. To demonstrate the exploitation of DRL as a quantum compiler, we first considered the problem of decomposing a single-qubit gate $\mathcal{U}$, into a circuit of unitary transformations that can be implemented directly on quantum hardware. The base of gates corresponds to six small rotations of $\pi/128$ around the three-axis of the Bloch sphere, i.e., $\mathcal{B} = (R_{\hat{x}}(\pm\frac{\pi}{128}), R_{\hat{y}}(\pm\frac{\pi}{128}), R_{\hat{z}}(\pm\frac{\pi}{128}))$.

It is essential to choose the tolerance ε and the fixed target accurately to appreciate the learning procedure. The former should be small enough and the latter sufficiently far from the identity not to be solved by chance. However, if the target is too difficult to approximate, the agent will fail and no learning occurs. To be sure that at least one solution does exist, we build $\mathcal{U}$ as a composition of 87 elements selected from $\mathcal{B}$. The resulting unitary target is

$$\mathcal{U} = \begin{pmatrix} 0.76749896 - 0.43959894\mathrm{i} & -0.09607122 + 0.45658344\mathrm{i} \\ 0.09607122 + 0.45658344\mathrm{i} & 0.76749896 + 0.43959894\mathrm{i} \end{pmatrix}. \quad (1)$$

Table 1 List of the hyperparameters and their values used in the fixed-target problem. We used the scaled exponential linear units (SELU) as nonlinear activation functions in the hidden layers of the neural network.

Area	Hyperparameter	Value
Neural network	# hidden layers	128, 128
	activations	SELU, SELU, linear
	initializers	lecun, lecun, glorot
Training	optimizer	Adam
	learning rate	0.0005
	batch size	10^3
	training frequency	every one episode
Algorithm	epsilon decay	0.99976
	memory size	10^4 experiences
	max length episode	130

We tested a nonlearning agent that acts randomly to ensure not to deal with a trivial task, setting the tolerance at 0.99 average gate fidelity and limiting the maximum length of the episode at 130. No solution was found after 10^4 episodes. Then, the problem was addressed by exploiting a DQL agent, using the same thresholds for the tolerance and the length of the episode. We exploited the dense reward function

$$r(S_n, a_n) = \begin{cases} (L - n) + 1 & \text{if } d(U_n, \mathcal{U}) < \varepsilon \\ -d(U_n, \mathcal{U})/L & \text{otherwise} \end{cases} \quad (2)$$

where L is the maximum length of the episode, a_n, S_n, and $d(U_n, \mathcal{U})$ are the action performed, the state of the environment, and the distance between the target and the approximating sequence at time n, respectively. Such reward performs adequately if small rotations are used as base only.

Table 1 reports some additional information about the network architecture and the hyperparameter set, while Fig. 2 shows the performance and the solutions found by the agent during the training time. The agent learns how to approximate the target after about 10^4 episodes, while improving the solution over time. At the end of the learning, the agent discovered an approximating circuit made by 76 gates only, within the target tolerance.

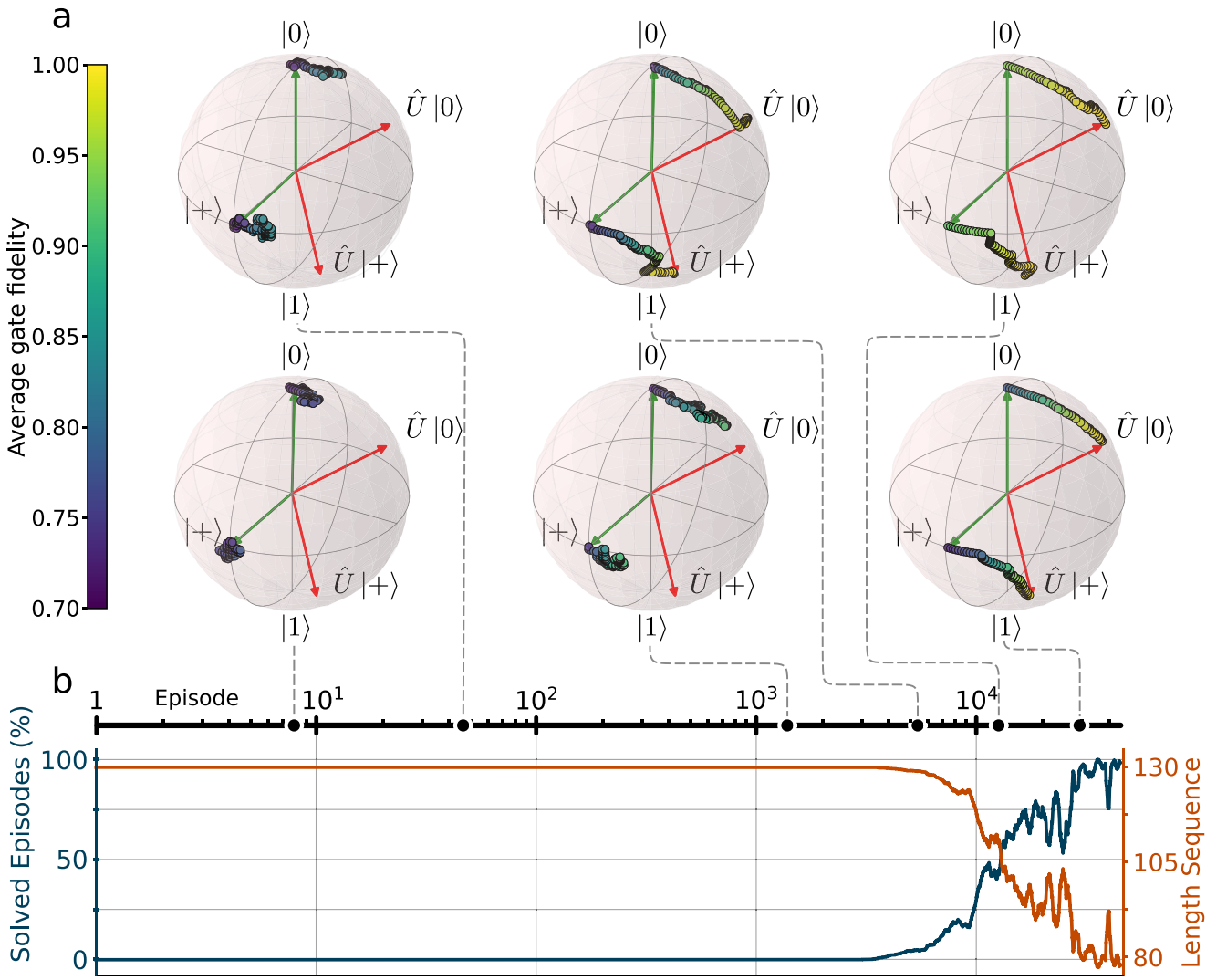


Fig. 2 The deep reinforcement learning agent learns how to approximate a single-qubit gate. a Best sequences of gates discovered by the agent during the training at different epochs. The dashed lines connecting the Bloch spheres to the Episode axis indicate the episode at which the sequences were found for the first time. Each approximating sequence is represented by two trajectories of states (colored points) on the Bloch sphere. They are obtained by applying the unitary transformations associated with the circuit at the time step n on two representative states, namely $|0\rangle$ and $|+\rangle$ respectively. The agent is asked to transform the starting state (green arrows) in the corresponding ending state (red arrows), i.e., $|0\rangle$ to $\mathcal{U}|0\rangle$ and $|+\rangle$ to $\mathcal{U}|+\rangle$ respectively, where $\mathcal{U}$ corresponds to the unitary target. **b** Performance of the agent during training. The plot represents the percentage of episodes for which the agent was able to find a solution (blue line) and the average number of the sequence of gates (orange line). The agent learns how to approximate the target after about 104 episodes and then improves the solution over time.

Quantum compiling by rotation operators. The DRL approach can be generalized to the quantum compilation of a larger class of unitary transformations. Instead of limiting to approximating one matrix only, we aim at exploiting the knowledge of a trained agent to approximate any single-qubit unitary transformation, without requiring additional training. Therefore, we used as training targets Haar unitary matrices, since they form an unbiased and a general data set which is ideal to train neural networks, as described in the "Methods" section. If additional information on the type and distribution of targets is known, it is possible to choose a different set of gates for training, potentially increasing the performance of the agent as described in Supplementary Note 3.1.

Such task is tougher to solve compared with the fixed-target problem. Therefore, we exploited the Proximal Policy Optimization algorithm (PPO)[46] being more robust and easy to tune than DQL. We fixed the tolerance ε at 0.99 AGF and limited the maximum length of the approximating circuits (time step per episode) at 300 gates, as reported in Table 2. Figure 3b shows the performance of the agent during the training time (blue lines). The agent starts to approximate unitaries after 10^5 episodes, but it requires much more time to achieve satisfactory performance.

We tested the performance of the agents at the end of the learning, using a validation set of 10^6 Haar unitary targets. The agent is able to approximate more than 96% of the targets within the tolerance requested. Complete results are reported in Table 3.

Quantum compiling by the HRC efficiently universal base of gates. In order to fully exploit the power of DRL, we now turn to the problem of compiling single-qubit unitary matrices using a base of discrete gates. The agent can perform the set of HRC efficiently universal base of gates[9]:

$$V_1 = \frac{1}{\sqrt{5}}\begin{pmatrix} 1 & 2i \\ 2i & 1 \end{pmatrix} V_2 = \frac{1}{\sqrt{5}}\begin{pmatrix} 1 & 2 \\ -2 & 1 \end{pmatrix} V_3 = \frac{1}{\sqrt{5}}\begin{pmatrix} 1+2i & 0 \\ 0 & 1-2i \end{pmatrix} \quad (3)$$

Such unitary matrices implement quantum transformations that are very different from the ones performed by small rotations. The agent has to learn how to navigate in the high-dimensional space of unitary matrices, exploiting counterintuitive movements that could lead it close to the target at the last time step of the episode only. Therefore, the dense reward function (2) is no longer useful to guide the agent toward the targets. We exploited a "sparse" reward (binary reward):

$$r(S_n, a_n) = \begin{cases} 0 & \text{if } d(U_n, \mathcal{U}) < \varepsilon \\ -1/L & \text{otherwise.} \end{cases} \quad (4)$$

Such function lowers the reward of the agent equally for every action it takes, bringing no information to the agent on how to find the solution. Therefore, it requires advanced generalization techniques to be effective, such as Hindsight Experience Replay (HER)[47]. Since HER requires an off-policy reinforcement learning algorithm, we chose the DQL agent to address the problem using the hyperparameters reported in Table 4.

We fixed the tolerance ε at 0.99 AGF and limited the maximum length of the approximating circuits at 130 gates. Figure 3 shows the performance of the agent during the training time (orange lines) and the length distribution of the solved sequences obtained using a validation set of 10^6 Haar random unitaries. Although the agent receives a noninformative reward signal at each time-step, it surprisingly succeeds to solve roughly more than 95% of the targets, using on average less than 36 gates as reported in Table 3. It is worth noting that the agent can build significantly shorter circuits, compared with the case of rotation matrices. It is not unexpected, since the HRC base allows to explore the space of unitary matrices quite efficiently.

Table 2 List of the hyperparameters and their values used in the problem of quantum compiling by rotations operators. The proximal policy optimization agent (PPO) exploits scaled exponential linear units (SELU) as nonlinear activation functions in the hidden layers of the neural network.

Area	Hyperparameter	Value
Neural network	# hidden layers	128, 128
	activations	SELU, SELU
Training	learning rate	0.0001
	batch size	128
	# agents	40
Algorithm	max length episode	300

Performances of the DRL quantum compiler. Our results show that deep reinforcement learning-based quantum compilers can approximate single-qubit gates by a set of quantum gates without prior knowledge. We now turn to the evaluation of the performances demonstrated by our method.

We point out that our method differs from existing quantum compilers for its flexibility since it can be applied to any basis. Indeed, Y–Z–Y gate decomposition can only manage a basis consisting of y and z rotations, while KAK decomposition[48] is limited to two-qubits and CNOT and y and z rotations. Machine-learning methods based on A*[49] algorithms could suffer from high execution time that could scale suboptimally for high accuracy[12,13]. Instead of designing a tailored quantum compiling algorithm, we exploited a DRL agent to learn a general strategy to approximate single-qubit unitary matrices and store it within an artificial neural network.

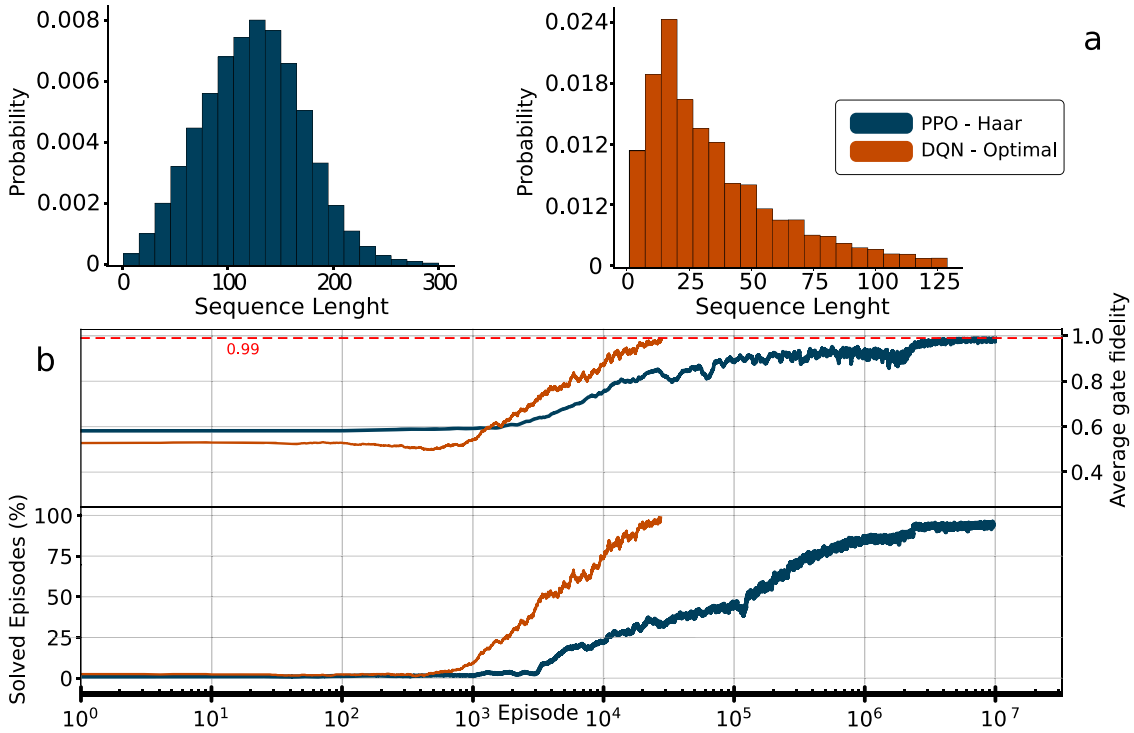


Fig. 3 Deep reinforcement learning agents learn how to approximate single-qubit unitaries using different base of gates. A proximal policy-optimization agent (PPO) (blue color) and a deep Q-learning hindsight-experience replay agent DQL+HER (orange color) were trained to approximate single-qubit unitaries using two different bases of gates, i.e., six small rotations of $\pi/128$ around the three-axis of the Bloch sphere and the Harrow–Recht–Chuang efficient base of gates (HRC), respectively. The tolerance was fixed to 0.99 average gate fidelity. **a** The length distributions of the gates sequences discovered by the agents at the end of the learning. The HRC base generates shorter circuits as expected. **b** Performance of the agent during training on the tasks.

Table 3 Performance of the proximal policy optimization (PPO) and deep Q-learning (DQL) agents in approximating Haar unitary matrices. The performances are measured after the training procedures over a validation set of 10^6 targets. We exploit the Harrow-Recht-Chuang efficient base of gates (HRC) and the rotations gates. The 95th and 95th columns refer to the 95th and 95th percentile of the distribution of the length of the solved sequences.

Base	Solved (%)	Mean length	95th percentile	99th percentile
HRC	95.0	35	94	120
Rotations	96.4	124	204	245

Table 4 List of the hyperparameters and their values used in the Harrow-Recht-Chuang efficient base of gates (HRC) problem. The deep Q-learning agent employs scaled exponential linear units (SELU) as nonlinear activation functions in the hidden layers of the neural network.

Area	Hyperparameter	Value
Neural network	# hidden layers	128, 128
	activations	SELU, SELU, linear
	initializers	lecun, lecun, glorot
Training	optimizer	Adam
	learning rate	0.0001
	batch size	200
	training frequency	every one episode
Algorithm	epsilon decay	0.99931
	memory size	$5 \cdot 10^5$ experiences
	max length episode	130

One of the critical questions to consider when measuring the performance of a quantum compiler is how a classical computer can efficiently return the sequence of gates. Inefficient strategies[50] can neutralize any quantum advantage over classical counterparts if the execution time and the length of the sequence scale suboptimally with the accuracy.

Our DRL quantum compiler can produce sequences that length scales as $\mathcal{O}(\log^{1.25}(1/\delta))$, as demonstrated by empirically measuring the performance on a specific task and shown in Fig. 4. Although such an approach has no guarantee to return a suitable solution, DRL quantum compilers can return solutions after a precompilation procedure to be performed once, improving the execution time and potentially enabling online quantum compilation. In fact, by writing the policy into a deep neural network, the execution time depends on the complexity of the network and the episode length only. Therefore, it scales proportionally to the sequence length, i.e., as $\mathcal{O}(\log^{1.25}(1/\delta))$. At the end of the training procedure, the agent returns the whole approximating sequence in a fraction of a second ($5.4 \cdot 10^{-4}s$ per time step) on a single CPU core. The speed-up gain could be further enhanced by reducing the size of the neural networks and being easily parallelizable by exploiting specialized hardware to run them, such as GPUs or Tensor Processing Unit[51].

As examples, we trained and employed two deep reinforcement learning quantum compilers to build quantum circuits within a final tolerance of 0.99 AGF, using two different sets of quantum logic gates. We accounted for the diverse characteristics of the bases designing a dense and a sparse reward function. We addressed Haar distributed unitary matrices as targets to be as general as possible, but if additional information on the targets is available, it is possible to achieve higher tolerance without fine-tuning neural network architectures or the RL hyperparameters, as shown in Supplementary Note 3.1.

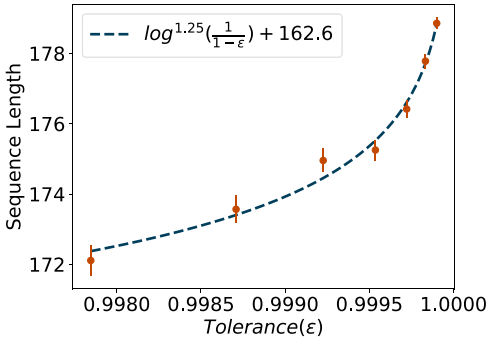


Fig. 4 Relation between sequence length and tolerance. Each data point is obtained by averaging the length of the approximating sequence of gates found by a trained agent using a validation set of 10^7 unitary targets. The error bars report the standard deviation. The agent was trained to achieve a final tolerance of 0.9999 average gate fidelity (AGF). The targets are built as compositions of small rotations around the three axes of the Bloch sphere, as described in Supplementary Note 3.1. The data are fitted by a polylogarithmic function (dashed blue line) with $R^2 = 0.986$ and $RMSE = 0.26$.

Our method could be employed in larger qubit spaces, as shown by an early prototype in Supplementary Note 3.2. The DRL compiler can approximate two-qubit logic gates with consistent performance compared with the one-qubit gates of 0.99 AGF. Supplementary Note 4 reports examples of single and two-qubit circuits discovered by the DRL quantum compilers.

As a concluding remark, we observe that this approach can be specialized, taking into account any hardware constraints that limit operations, integrating them directly into the environments. These tests, as well as the extension of this approach to n-qubits, will be the objective of future work.

Methods

Generation of Haar random unitary matrices. The strategy used to generate the training data set should be chosen opportunely, depending on the particular set of gates of interest, since deep neural networks are very susceptible both to the range and the distribution of the inputs. Therefore, Haar random unitaries have been used as training targets. Pictorially, picking a Haar unitary matrix from the space of unitaries can be thought as choosing a random number from a uniform distribution[52]. More precisely, the probability of selecting a particular unitary matrix from some region in the space of all unitary matrices is directly proportional to the volume of the region itself. Such matrices form an unbiased data set that is ideal to train neural networks.

Learning by HER. Many RL problems may be efficiently addressed employing sparse rewards only, since engineering efficient and well-shaped reward functions can be extremely challenging. Such rewards are binary, i.e., the agent receives a constant signal until it achieves the goal. However, if the agent gets the same reward almost every time, it cannot learn any relationships of cause and effect that its actions have on the environment. Therefore, it might take an extremely long time to learn something, if anything at all.

HER is a technique introduced by OPENAI that allows to mitigate the sparse-reward problem[47]. The basic idea of HER is to exploit the ability that humans have to learn from failure. Specifically, even if the agent always failed to solve the task, it can reach different objectives. Exploiting this information, it is possible to train the agent to reach different targets. Although the agent receives a reward signal to achieve a distinct goal from the original one, this procedure, if iterated, can help the agent to learn how to generalize the policy to reach the primary task we want to solve.

The implementation of HER in the Q-learning algorithm is straightforward. After an entire episode is completed, the experiences associated with that episode are modified selecting a new goal. Then, the q-function is updated as usual. There are several strategies to choose the goals[47]. We designed a strategy to select the new goals, consisting in randomly selecting k-percent of the states that come from the same episode.

Average gate fidelity. The average fidelity $\bar{F}(\mathcal{U}, U)$ between two gates $\mathcal{U}$ and U is defined by

$$\bar{F}(\mathcal{U}, U) = \int \langle\psi|\mathcal{U}^\dagger U|\psi\rangle\langle\psi|U^\dagger\mathcal{U}|\psi\rangle d\psi, \qquad \int d\psi = 1 \tag{5}$$

where the integral is over all the state spaces using a Haar measure.

Neural network architectures. The architecture of the deep neural network directly affects the performance of the DRL agent. However, choosing the optimal architecture is a trial-and-error task and can be an exceptionally time-consuming procedure, since it depends on the specific problem the agent is addressing. Therefore, in this work, we did not focus on the optimization, but on finding the smallest neural network architecture that can lead to satisfactory performance. We started with one hidden layer only and a few neurons, gradually increasing the depth and the width of the network. We found that a relatively small architecture made by two hidden layers of 128 neurons is sufficient to achieve the tasks.

Software and hardware. All the code in this work was developed using Python language. The Stable Baseline[53] library has been employed for the implementation of PPO agent only. Most of the simulation has been run by using GNU parallel[54] on an Intel Xeon W-2195 and a Nvidia GV100.

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Code availability

The code and the algorithm used in this study are available from the corresponding author upon reasonable request.

Received: 22 February 2021; Accepted: 20 July 2021;
Published online: 06 August 2021

References

1. Linke, N. M. et al. Experimental comparison of two quantum computing architectures. *Proc. Natl Acad. Sci. USA* **114**, 3305–3310 (2017).
2. Maslov, D. Basic circuit compilation techniques for an ion-trap quantum machine. *New J. Phys.* **19**, 023035 (2017).
3. Leibfried, D., Knill, E., Ospelkaus, C. & Wineland, D. J. Transport quantum logic gates for trapped ions. *Phys. Rev. A* **76**, 032324 (2007).
4. Debnath, S. et al. Demonstration of a small programmable quantum computer with atomic qubits. *Nature* **536**, 63 (2016).
5. Maronese, M. & Prati, E. A continuous rosenblatt quantum perceptron. *Int. J. Quantum Inf.* https://doi.org/10.1142/S0219749921400025 (2021).
6. Kitaev, A. Y. Quantum computations: algorithms and error correction. *Russian Math. Surv.* **52**, 1191–1249 (1997).
7. Zhiyenbayev, Y., Akulin, V. & Mandilara, A. Quantum compiling with diffusive sets of gates. *Phys. Rev. A* **98**, 012325 (2018).
8. Barenco, A. et al. Elementary gates for quantum computation. *Phys. Rev. A* **52**, 3457 (1995).
9. Harrow, A. W., Recht, B. & Chuang, I. L. Efficient discrete approximations of quantum gates. *J. Math. Phys.* **43**, 4445–4451 (2002).
10. Kitaev, A. Y., Shen, A., Vyalyi, M. N. & Vyalyi, M. N. *Classical and quantum computation*. 47 (American Mathematical Soc., 2002).
11. Dawson, C. M. & Nielsen, M. A. The solovay-kitaev algorithm. *Quantum Info. Comput.* **6**, 81–95 (2006).
12. Davis, M. G. et al. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 223–234 (IEEE, 2020).
13. Zhang, Y.-H., Zheng, P.-L., Zhang, Y. & Deng, D.-L. Topological quantum compiling with reinforcement learning. *Phys. Rev. Lett.* **125**, 170501 (2020).
14. Tognetti, S., Savaresi, S. M., Spelta, C. & Restelli, M. In *2009 IEEE Control Applications,(CCA) & Intelligent Control (ISIC)*, 582–587 (IEEE, 2009).
15. Niu, M. Y., Boixo, S., Smelyanskiy, V. N. & Neven, H. Universal quantum control through deep reinforcement learning. *npj Quantum Inf.* **5**, 1–8 (2019).
16. Castelletti, A., Pianosi, F. & Restelli, M. A multiobjective reinforcement learning approach to water resources systems operation: Pareto frontier approximation in a single run. *Water Resour. Res.* **49**, 3476–3486 (2013).
17. Sutton, R. S., Barto, A. G. et al. *Introduction to reinforcement learning*, vol. 135 (MIT press Cambridge, 1998).
18. Fösel, T., Tighineanu, P., Weiss, T. & Marquardt, F. Reinforcement learning with neural networks for quantum feedback. *Phys. Rev. X* **8**, 031084 (2018).
19. Dunjko, V. & Briegel, H. J. Machine learning & artificial intelligence in the quantum domain: a review of recent progress. *Rep. Prog. Phys.* **81**, 074001 (2018).
20. Sarma, S., Deng, D.-L. & Duan, L.-M. Machine learning meets quantum physics. *Phys. Today* **72**, 48–54 (2019).
21. Carleo, G. et al. Machine learning and the physical sciences. *Rev. Mod. Phys.* **91**, 045002 (2019).
22. Sutton, R. S., Barto, A. G. et al. *Reinforcement learning: An introduction* (MIT press, 1998).
23. Mnih, V. et al. Human-level control through deep reinforcement learning. *Nature* **518**, 529 (2015).
24. Melnikov, A. A. et al. Active learning machine learns to create new quantum experiments. *Proc. Natl Acad. Sci.* **115**, 1221–1226 (2018).
25. Nautrup, H. P., Delfosse, N., Dunjko, V., Briegel, H. J. & Friis, N. Optimizing quantum error correction codes withreinforcement learning. *Quantum* **3**, 215 (2019).
26. Sweke, R., Kesselring, M. S., van Nieuwenburg, E. P. & Eisert, J. Reinforcement learning decoders for fault-tolerantquantum computation. *Mach. Learn. Sci. Technol.* **2**, 025005 (2020).
27. Reddy, G., Celani, A., Sejnowski, T. J. & Vergassola, M. Learning to soar in turbulent environments. *Proc. Natl Acad. Sci. USA* **113**, E4877–E4884 (2016).
28. Colabrese, S., Gustavsson, K., Celani, A. & Biferale, L. Flow navigation by smart microswimmers via reinforcement learning. *Phys. Rev. Lett.* **118**, 158004 (2017).
29. August, M. & Hernández-Lobato, J. M. Taking gradients through experiments: Lstms and memory proximal policy optimization for black-box quantum control. In *International Conference on High Performance Computing*, 591–613 (Springer, 2018).
30. Niu, M. Y., Boixo, S., Smelyanskiy, V. N. & Neven, H. Universal quantum control through deep reinforcement learning. *npj Quantum Inf.*, **5**, 33 (2019).
31. Albarrán-Arriagada, F., Retamal, J. C., Solano, E. & Lamata, L. Measurement-based adaptation protocol with quantum reinforcement learning. *Phys. Rev. A* **98**, 042315 (2018).
32. Andreasson, P., Johansson, J., Liljestrand, S. & Granath, M. Quantum error correction for the toric code using deep reinforcement learning. *Quantum* **3**, 183 (2019).
33. Prati, E. Quantum neuromorphic hardware for quantum artificial intelligence. *J. Phys. Conf. Ser*, **880**, 012018 (2017).
34. Porotti, R., Tamascelli, D., Restelli, M. & Prati, E. Coherent transport of quantum states by deep reinforcement learning. *Commun. Phys.* **2**, 61 (2019).
35. Porotti, R., Tamascelli, D., Restelli, M. & Prati, E. Reinforcement learning based control of coherent transport by adiabatic passage of spin qubits. *J. Phys. Conf. Ser.* **1275**, 012019 (2019).
36. Ferraro, E., De Michielis, M., Fanciulli, M. & Prati, E. Coherent tunneling by adiabatic passage of an exchange-only spin qubit in a double quantum dot chain. *Phys. Rev. B* **91**, 075435 (2015).
37. Paparelle, I., Moro, L. & Prati, E. Digitally stimulated Raman passage by deep reinforcement learning. *Phys. Lett. A* 384, 126266 (2020).
38. Moro, L., Paparelle, I. & Prati, E. Using deep learning for digitally controlled STIRAP. *Int. J. Quantum Inf.* https://doi.org/10.1142/S0219749921410021 (2021).
39. Niu, M. Y., Boixo, S., Smelyanskiy, V. N. & Neven, H. Universal quantum control through deep reinforcement learning. *npj Quantum Inf.* **5**, 1–8 (2019).
40. An, Z. & Zhou, D. Deep reinforcement learning for quantum gate control. *EPL* **126**, 60002 (2019).
41. Aharonov, D., Kitaev, A. & Nisan, N. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, STOC '98, 20–30 (Association for Computing Machinery, 1998).
42. Watrous, J. Semidefinite programs for completely bounded norms. *Theory Comput.* **5**, 217–238 (2009).
43. Nielsen, M. & Chuang, I. *Quantum Computation and Quantum Information*. Cambridge Series on Information and the Natural Sciences (Cambridge University Press, 2002). https://books.google.it/books?id=xnI9PgAACAAJ.
44. Tolar, J. In *Journal of Physics: Conference Series*, vol. 1071, 012022 (IOP Publishing, 2018).
45. Watkins, C. J. & Dayan, P. Q-learning. *Mach. Learn.* **8**, 279–292 (1992).
46. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. Proximal policy optimization algorithms. CoRRabs/1707.06347 (2017).
47. Andrychowicz, M. et al. In *Advances in Neural Information Processing Systems*, 5048–5058 (NIPS, 2017).
48. Vatan, F. & Williams, C. Optimal quantum circuits for general two-qubit gates. *Phys. Rev. A* **69**, 032315 (2004).
49. Hart, P. E., Nilsson, N. J. & Raphael, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Syst. Sci. Cybern.* **4**, 100–107 (1968).
50. Lloyd, S. Almost any quantum logic gate is universal. *Phys. Rev. Lett.* **75**, 346 (1995).

MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing

NILS QUETSCHLICH, Technical University of Munich, Germany
LUKAS BURGHOLZER, Technical University of Munich, Germany
ROBERT WILLE, Technical University of Munich, Germany and Software Competence Center Hagenberg GmbH, Austria

Fueled by recent accomplishments in quantum computing hardware and software, an increasing number of problems from various application domains are being explored as potential use cases for this new technology. Similarly to classical computing, realizing an application on a particular quantum device requires the corresponding (quantum) circuit to be *compiled* so that it can be executed on the device. With a steadily growing number of available devices—each with their own advantages and disadvantages—and a wide variety of different compilation tools, the number of choices to consider when trying to realize an application is quickly exploding. Due to missing tool support and automation, especially end-users who are not quantum computing experts are easily left unsupported and overwhelmed.

In this work, we propose a methodology that allows one to *automatically select* a suitable quantum device for a particular application *and* provides an *optimized compiler* for the selected device. The resulting framework—called the *MQT Predictor*—not only supports end-users in navigating the vast landscape of choices, it also allows *mixing and matching* compiler passes from various tools to create optimized compilers that transcend the individual tools. Evaluations of an exemplary framework instantiation based on more than 500 quantum circuits and seven devices have shown that—compared to both Qiskit's and TKET's most optimized compilation flows for all devices—the MQT Predictor produces circuits within the top-3 out of 14 baselines in more than 98% of cases while frequently outperforming any tested combination by up to 53% when optimizing for *expected fidelity*. Additionally, the framework is trained and evaluated for *critical depth* as another *figure of merit* to showcase its flexibility and generalizability—producing circuits within the top-3 in 89% of cases while frequently outperforming any tested combination by up to 400%. MQT Predictor is part of the *Munich Quantum Toolkit* (MQT) and publicly available as open-source on GitHub (https://github.com/cda-tum/mqt-predictor) and as an easy-to-use *Python* package (https://pypi.org/p/mqt.predictor).

1 INTRODUCTION

Quantum computing is an emerging computational technology and has seen considerable accomplishments in both hardware and software—especially in recent years. New quantum devices with an increasing number of qubits and higher fidelity rates are emerging almost on a daily basis. A similar trend can be observed for the development of quantum *Software Development Kits* (SDKs) such as, e.g., IBM's Qiskit [1], Quantinuum's TKET [2], Google's Cirq [3], Xanadu's PennyLane [4], or Rigetti's Forest [5]. This sparks interest not only in academia but also in industry—with promising applications of quantum computing emerging in various domains, such as finance [6], optimization [7], machine learning [8], or chemistry [9]. However, realizing such quantum computing-based solutions remains a challenge that is not straight-forward to solve and, to date, requires several manual steps [10].

First, a suitable quantum device must be *selected* for the execution of the developed quantum algorithm. This alone is non-trivial since new quantum devices based on various underlying technologies emerge almost daily—each with their own advantages and disadvantages. There are hardly any practical guidelines on which device to choose based on the targeted application. As such, the best guess in many cases today is to simply try out many (if not all) possible devices and, afterwards, choose the best results—certainly a time- and resource-consuming endeavor that is not sustainable for the future.

Once a target device has been selected, the quantum circuit, which is typically designed in a device-agnostic fashion that does not account for any hardware limitations (such as a limited gate-set or limited connectivity), must be *compiled* accordingly so that it actually becomes executable on that device. This is similar to classical computing, where high-level, platform-agnostic code (e.g., written in C++) is being *compiled* to low-level, platform-specific machine code before being executed on a particular classical computer. Compilation itself is a sequential process consisting of a sequence of *compilation passes* that, step-by-step, transform the original quantum circuit so that it eventually conforms to the limitations imposed by the target device. Since many of the underlying problems in compilation are computationally hard (e.g., satisfying the connectivity limitations of a certain device with the least amount of overhead being NP-complete [11]), there is an ever-growing variety of compilation passes available across several quantum SDKs and software tools—again, each with their own advantages and disadvantages. As a result of the sheer number of options, choosing the best sequence of compilation passes for a given application is nearly impossible. Consequently, most quantum SDKs (such as Qiskit and TKET) provide easy-to-use high-level function calls that encapsulate "their" sequence of compilation passes into a single compilation flow. While this allows to conveniently compile circuits, it has several drawbacks. For one, it creates a kind of *vendor lock* that limits the available compilation passes to those available in the SDK offering the compilation flow. Furthermore, the respective compilation flows are designed to be broadly applicable and, hence, are neither device-specific nor circuit-specific. On top of that, no means are provided to optimize for a specific *figure of merit*—only the degree of desired optimization can be specified without any explicit optimization criterion.

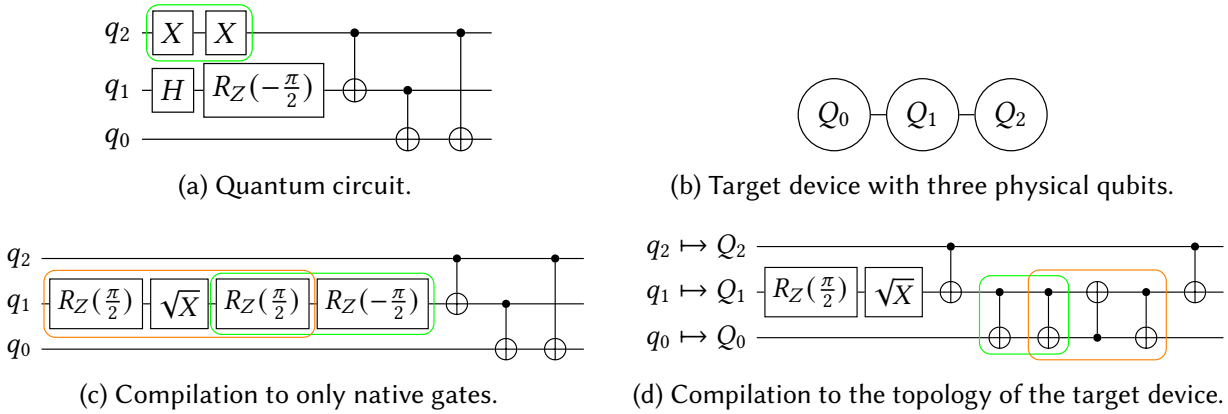
Overall, this leaves anyone trying to realize a quantum computing application with more options than they could ever feasibly explore. What makes things worse is that the choice of the quantum device and the subsequent compilation flow heavily affect the quality of the resulting circuit and its execution. Choosing the right combination can often make the difference between a useful result and pure noise. This is especially unfortunate for end-users who are not experts in quantum computing and just want to use the technology to solve their application domain problem.

In this work[1], a new approach to *automate* the quantum device selection and according compilation is proposed to free end-users from this task and to prevent a scenario where quantum computers can only be utilized by quantum computing experts—hindering the overall adoption of that technology. To this end, we tackle this problem from two angles:

(1) We propose a *prediction* method (based on *Supervised Machine Learning*) that, without performing any compilation, automatically predicts the most suitable device for a given application. This completely eliminates the manual and laborious task of determining a suitable target device and guides end-users through the vast landscape of choices without the need for quantum computing expertise.
(2) We propose a method (based on *Reinforcement Learning*) that produces device-specific quantum circuit compilers by combining compilation passes from various compiler tools and learning optimized sequences of those passes with respect to a customizable *figure of merit*. This *mix-and-match* of compiler passes from various tools allows one to eliminate vendor locks and to create optimized compilers that transcend the individual tools.

Combining both of these methods into a holistic framework—the *MQT Predictor*—allows one to *automatically select* a suitable quantum device for a particular application while also providing an *optimized compiler* for the selected device and a customizable figure of merit.

[1]Preliminary versions of this work have been published in [12], [13].



(a) Quantum circuit.

(b) Target device with three physical qubits.

(c) Compilation to only native gates.

(d) Compilation to the topology of the target device.

Fig. 1. Compilation of a quantum circuit to a targeted device.

To show its benefits, an instantiation of the proposed approach is implemented based on more than 500 quantum circuits ranging from 2 to 90 qubits and considering seven different quantum devices based on two qubit technologies. As a baseline to compare against, the most optimized compilation flows available in Qiskit and TKET have been used—leading to a total of $7 \times 2 = 14$ possible combinations of device and compiler. Compared to these reference results, the MQT Predictor produces circuits with an expected fidelity that is *on par with the best* possible result that could be achieved by trying out all combinations of devices and compilers. Specifically, the *automatic device selection* and *optimized circuit compilation* produced circuits within the top-3 in more than 98% of cases while frequently outperforming any tested combination by up to 53%. Additionally, MQT Predictor is trained and evaluated for *critical depth* as another figure of merit to showcase its flexibility and generalizability. The respective results show that the proposed method also achieves a similar performance—producing circuits within the top-3 in 89% of cases while frequently outperforming any tested combination by up to 400%. The corresponding framework (which is part of the *Munich Quantum Toolkit* (MQT) [14]) including the pre-trained models and classifiers is publicly available as open-source on GitHub (https://github.com/cda-tum/mqt-predictor) and as an easy-to-use *Python* package (https://pypi.org/p/mqt.predictor).

The rest of this work is structured as follows: In Section 2, quantum circuit compilation flows and related work are reviewed. Based on that, Section 3 introduces the proposed approach and the underlying methodologies with details given in Section 4 for the compilation using reinforcement learning and Section 5 for the device selection using supervised machine learning. The resulting combined approach is then described in Section 6, evaluated in Section 7, and discussed in Section 8. Section 9 concludes this work.

2 BACKGROUND AND MOTIVATION

To keep this work self-contained, this section gives a brief introduction to quantum circuit compilation, discusses its challenges, and reviews the related work.

2.1 Quantum Circuit Compilation

Solving any problem using quantum computing requires encoding the problem as a quantum algorithm. These algorithms are typically described as sequences of quantum operations (often referred to as *quantum gates*) that form a *quantum circuit*. Usually, one distinguishes between *single-qubit* gates affecting only one qubit and *multi-qubit* gates affecting more than one qubit at the same time.

Example 1. *Fig. 1a shows a small exemplary quantum circuit with three qubits (q_0, q_1, q_2) and four different kinds of quantum gates acting on them. There are two single-qubit gates on q_2 (two* NOT *gates denoted as* X*) and two single-qubit gates on q_1 (a Hadamard gate and a Z-rotation gate denoted as* H *and R_Z, respectively). Additionally, there are multi-qubit gates between all possible qubit pairs, namely* controlled-NOT *(CNOT) gates denoted as $\bullet$ and $\oplus$ for their control, respectively, target qubit.*

The actual compilation can only be started as soon as a suitable quantum device is *selected* and, by that, its native gate-set and connectivity constraints are known. The device selection itself is already challenging due to several factors:

- Various underlying quantum technologies are currently pursued in parallel with no clear winner yet and all of them have their own advantages and disadvantages. *Ion trap*-based devices, e.g., feature an all-to-all connectivity but provide rather slow gate execution times while, e.g., *superconducting*-based devices provide fast gate execution times but usually come with a restricted connectivity.
- Even devices based on the same technology significantly vary in their hardware characteristics, such as qubit count, their respective error rates, coherence times, qubit connectivity, and gate execution times.
- On top of all that, the domain of quantum computing is fast-paced and constantly changing—quickly and frequently redefining the state of the art.

Example 2. *Fig. 1b shows the topology of a three-qubit quantum device. This topology indicates that two-qubit gates may only be applied to (Q_0, Q_1) or (Q_1, Q_2), but not to (Q_0, Q_2).*

As introduced in Section 1, executing such quantum circuits on an actual quantum device requires compilation, which is usually conducted by *compilers* that are part of quantum SDKs as easy-to-use high-level function calls. However, compilation actually is a sequential process that can be roughly divided into three kinds of *compilation passes*: *Synthesis* passes compile a given quantum circuit to the native gate-set of a selected quantum device, *mapping* passes enforce its connectivity constraints, and *optimization* passes exploit various possibilities to reduce the circuit's complexity. Quantum SDKs usually offer a wide range of device-independent optimization passes with the goal of creating a more efficient quantum circuit which still realizes the same functionality by, e.g., gate-cancellation, gate-commutation, or high-level synthesis routines.

Example 3. *Since $X \cdot X = I$, the circuit shown in Fig. 1a can be simplified by canceling the two subsequent* X *gates (denoted by the green box).*

Usually, those device-independent optimization passes are followed by the *device-dependent* compilation passes. Every quantum device can only directly execute a subset of all possible quantum gates—its so-called native gates. These native gates usually comprise a handful of single-qubit gates and one multi-qubit gate to become a *universal* gate-set. Therefore, all non-native gates must be *synthesized* into a sequence of native gates of that respective quantum device—a procedure that is NP-complete to solve in an optimal fashion [15]. Consequently, this alters the quantum circuit—leading to further room for optimization.

Example 4. *Assume that the targeted device from Example 2 has a native gate-set consisting of R_Z, $\sqrt{X}$,* X*, and* CNOT *gates and that the circuit from Fig. 1a shall be executed on it. Only the Hadamard gate on qubit q_1 is not a native gate of the device and, hence, must be synthesized into a sequence of such gates. The orange box in Fig. 1c indicates one such decomposition into two $R_Z(\frac{\pi}{2})$ and a $\sqrt{X}$ gate. Again, this circuit alteration leads to further optimization potential since the two subsequent R_Z gates in the green box, again, cancel each other out.*

In addition to the limited gate-set, many quantum devices (e.g., based on such as superconducting qubits) only have a *limited connectivity* between their underlying qubits—requiring that all multi-qubit gates must be placed on qubits which are actually connected on the respective device. To adapt a quantum circuit with gates operating between arbitrary qubits to such a limited connectivity, a *mapping* pass is necessary. This pass is often split into *layouting* and *routing*. The former assigns each (logical) qubit of the circuit to a (physical) qubit on the device and the latter ensures that all connectivity constraints are satisfied by changing the qubit assignment throughout the circuit, e.g., using *SWAP* gates—a procedure that is, again, NP-complete to solve in an optimal fashion [11]. Then, again, optimization passes can be applied to reduce the complexity of the resulting quantum circuit.

Example 5. *Consider again the circuit shown in Fig. 1c which shall be executed on the three-qubit quantum device shown in Fig. 1b. Since the quantum circuit contains multi-qubit gates between all qubit pairs, there is no trivial layout that satisfies the connectivity constraints throughout the entire circuit. Therefore, the circuit must be mapped by introducing a* SWAP *gate as denoted in the orange box in Fig. 1d (again, compiled into its native gate-set sequence of three* CNOT *gates). Although the resulting quantum circuit is now fully executable, a successive optimization pass can, again, eliminate two subsequent* CNOT *gates.*

Although the compilation process is rather straight-forward conceptually—consisting of *synthesis*, *mapping*, and *optimization* passes—it comes with practical challenges. For each step, a variety of different techniques and approaches have been proposed, such as [15]–[20] for synthesis, [21]–[29] for mapping, and [30]–[33] for optimization. Therefore, end-users are easily overwhelmed with selecting the *best* combination of provided SDKs and respective compilation passes—not even mentioning the effort of becoming acquainted with multiple (often poorly documented) SDKs and, respectively, needed conversions between them. Every SDK usually implements a subset of these compilation passes with varying degrees of effectiveness and efficiency. Those passes are provided as pre-configured, fixed sequences of compilation passes (for various levels of desired optimization)—independent of the given quantum circuit.

2.2 Related Work

A handful of techniques have already been explored to support the quantum device selection and compilation. In [34]–[37], approaches that compile a given circuit for *all* devices in a brute-force fashion and compare their resulting quality (also referred to as *autotuning*) are proposed. A similar approach to select the best sequence of compilation passes by evaluating all possible combinations has been proposed in [38]. Furthermore, in [39], a methodology has been proposed to determine whether compiled circuits can be reliably executed on certain devices to give some guidance. The same authors have further explored how *Multi-Criteria Decision Analysis* methods can be used to select the most suitable compiled circuit from a given set of all compiled circuits [40] and how machine learning can be used to predict the goodness of a device and compiler combination [41].

Supervised Machine Learning (ML) techniques have also been applied to quantum compilation, e.g., in [42], an ML-based layout and routing scheme is proposed, while in [43], an ML model is trained to predict the optimization potential of a given quantum circuit to decide whether to apply (potentially costly) optimization passes or not. Furthermore, *Reinforcement Learning* (RL) has been explored in this context, e.g., in [44] where a strategy for applying individual gate transformation rules to optimize quantum circuits is learned or in [45] where an RL model is trained to learn how to estimate the circuit fidelity for specific devices. Moreover, environments to train RL models for various compilation tasks are proposed in [46]—similar to the environment proposed in [13].

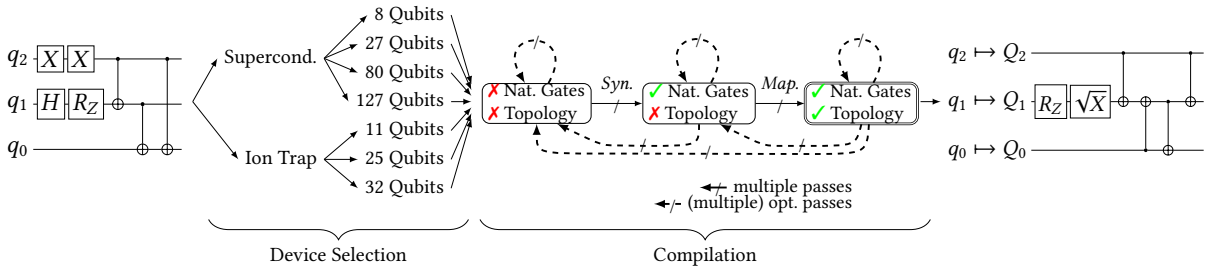


Fig. 2. The device selection and compilation process.

In classical compilation, machine learning has been explored for a much longer time which led to sophisticated compiler optimization techniques with overviews given in, e.g., [47], [48] and established these methods as state-of-the-art techniques in this domain. Additionally, even a combination of these techniques has been explored in [49] by using a machine learning model to determine promising areas of optimization space which then can be further investigated using autotuning. On top of that, reinforcement learning approaches targeting the LLVM [50] compiler emerged especially in recent years with encouraging results [51]–[53].

So far, many of the approaches toward both quantum device selection and compilation [34]–[40] follow a naive brute-force approach that, obviously, is not a long-term solution due to the steadily increasing number of quantum devices and compilers. Furthermore, existing ML-based tools to support end-users [41]–[45] do not consider the entire compilation or do not provide a broad coverage in terms of supported devices, range of qubits, or considered algorithms, and frequently lack the necessary means to scale to significantly more devices and compilers. On top of that, most approaches focus on using current compilers *as-is* and hardly make use of the sophisticated ML-based approaches for classical compiler optimization—leading to lots of untapped potential.

3 GENERAL IDEA

In this work, we propose a new approach to *automate* the device selection and compilation process that learns from previously conducted compilations and *predicts* the most suitable target device for a given quantum circuit *as well as* provides an *optimized compiler* for the selected device. To this end, this section motivates the proposed approach and describes its general ideas, the chosen methodology, and its utilization.

3.1 Proposed Approach

In an ideal scenario, a quantum circuit would be compiled for all possible sequences of compilations on all possible devices to determine the best compilation flow. However, since compilation comes with considerable complexity both in time and resources, this is practically infeasible.

Example 6. *Assume that we want to find the best compiled version of the quantum circuit shown in Fig. 1a given access to seven different devices (based on two qubit technologies) and various compiler passes provided by multiple SDKs as illustrated in Fig. 2. Manually determining the best possible combination of options is not only challenging, but close to impossible due to the number of possible combinations—which, in theory, is infinitely large due to backward loops in the compilation process that are introduced by optimization passes. End-users could just choose* any *(random) path from the left-hand side of Fig. 2 to the right-hand side. However, this is highly unlikely to yield a sufficiently good result (not to dare, the best possible result).*

In the proposed approach, we first consider finding the best sequence of *compilation* passes for a particular device as an isolated problem that is independent of the device selection task itself. To this end, compilation is modeled as a sequential (*Markov Decision)* process of synthesis, mapping, and optimization passes for a given native gate-set and qubit connectivity. Using *Reinforcement Learning* (RL), respective models can be trained to *learn* the sequences that lead to the most promising compilation result for each supported device according to some *figure of merit* such as expected fidelity or critical depth. These models can then be used as optimized black-box compilers for the respective devices.

Based on these optimized compilers, the task of choosing the most promising device is tackled. To this end, *Supervised Machine Learning* (ML) is applied by training an agent that *learns* the best suited device for a given circuit assuming that it will be compiled using the trained RL models. Thus, a holistic approach is created that takes a to-be-compiled quantum circuit as input and predicts which quantum device to use, compiles it accordingly using the trained RL model, and returns the compiled circuit with detailed compilation information. All of this is conducted in a fully automated fashion which is especially helpful for end-users who are not experts in quantum computing. Additionally, compilation passes from various sources can be integrated—effectively eliminating vendor locks by mixing and matching compiler passes from various SDKs.

To learn how to *best* compile quantum circuits, it is necessary to give the notion of "best" a meaning by defining a *figure of merit* to optimize for. As this is an essential part of the proposed approach, more details on this way of customizing the compilation are provided in the following.

3.2 Figures of Merit

In general, the figure of merit can be completely customizable and consider *anything* from (compiled) circuit characteristics such as gate counts to *soft* factors—applicable to the device selection—such as actual execution costs or the queue length for a specific device vendor. Nevertheless, a comprehensive figure of merit should consider at least the characteristics of the compiled quantum circuit and the characteristics of the selected quantum device. Therefore, the respective hardware information needs to be collected, such as, e.g., gate/readout fidelities, gate execution times, or decoherence times. While for some devices, this information may be publicly available, for other devices, it may be estimated from comparable devices, previous records, or insider knowledge.

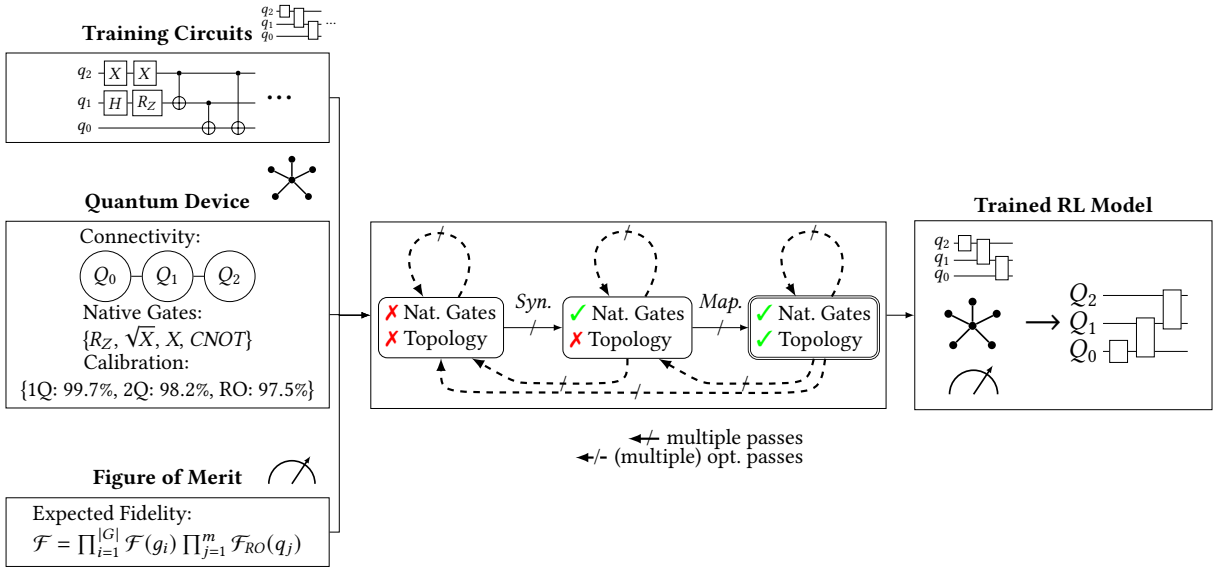
Example 7. *Consider the figure of merit that takes into account the following two aspects:*

(1) *If the selected device is not large enough to execute a given quantum circuit with respective to its number of qubits, the worst-possible score is assigned.*
(2) *If the device is large enough, the evaluation score is calculated using the formula:*

$$\mathcal{F} = \prod_{i=1}^{|G|} \mathcal{F}(g_i) \prod_{j=1}^{m} \mathcal{F}_{RO}(q_j)$$

with $\mathcal{F}(g_i)$ being the expected execution fidelity of gate g_i on its corresponding qubit(s), $\mathcal{F}_{RO}(q_j)$ being the expected execution fidelity of a measurement operation q_j on its corresponding qubit and $|G|$ respectively m being the number of gates and measurements in the compiled circuit.

This figure of merit determines an estimate of the probability that a quantum circuit will return the expected result, the so-called expected fidelity*, which ranges between* 0.0 *and* 1.0 *with higher values being better.*



(a) Input. (b) MDP for quantum compilation. (c) Output.

Fig. 3. Training process of an RL model for quantum compilation.

However, the expected fidelity introduced above is just one exemplary figure of merit. A potential alternative could be the *critical depth* (taken from [54])—a measure to describe the percentage of multi-qubit gates on the longest path through a compiled quantum circuit (determining the depth). A respective value close to 1 would indicate a very sequential circuit while a value of 0 would indicate a highly parallel one. On top of that, even combinations with customizable weighting are possible, such as 25% of the expected fidelity and 75% critical depth. Furthermore, different figures of merit could be used for the device selection and the respective compilation itself. While for the former, soft factors such as actual execution costs or queue lengths might be sensible to consider as well, the latter should only focus on technical factors that describe the quality of the compilation itself.

In the following, detailed descriptions of both the RL-based approach to the compilation and the ML-based approach to the device selection are given in Section 4 and Section 5, respectively. Afterwards, Section 6 describes the combination of both techniques into a holistic framework.

4 COMPILATION USING REINFORCEMENT LEARNING

Compilation, fortunately, is not new per-se, since classical compilers have seen a similar trend of increasing complexity and variety in the past. Instead of reinventing the wheel, we propose to make use of the decades of research on classical compiler optimization and model quantum circuit compilation in a similar fashion. Based on that, classical reinforcement learning is used to learn optimized sequences of compilation passes for a given figure of merit. The resulting model can then be used as a compiler.

4.1 Quantum Circuit Compilation Modeled as a Markov Decision Process

Quantum circuit compilation as described in Section 2.1 can be interpreted as a *Markov Decision Process* (MDP, [55]) that transforms a high-level circuit into an executable circuit by applying synthesis, mapping, and optimization passes. For that, the compilation process is broken down into *states* and corresponding *actions* that can be executed in those states to advance the process. Each

state captures which of the requirements for a circuit to be executable are satisfied—specifically, whether the circuit only contains native gates and whether it matches the topology of the device. This results in three states[2]:

(1) The circuit contains non-native gates and does not respect the device topology,
(2) The circuit only contains native gates but does not respect the device topology, and
(3) The circuit only contains native gates and respects the device topology, i.e., it is executable.

The resulting MDP is depicted in Fig. 3b—with states being denoted as rounded rectangles and actions as arrows. In the following, the individual states and actions will be described in more detail alongside an example.

Given a high-level quantum circuit and a target device (determining the native gate-set and qubit topology), compilation typically begins in the state where the circuit contains non-native gates and does not respect the device topology. In this state, two kinds of actions are possible: Either a *synthesis action* that transforms the circuit so that it only contains gates native to the target device, or an *optimization action* that applies any kind of optimization to the circuit.

Example 8. *To make this modeling more tangible, consider again the compilation flow shown in Fig. 1 with the initial circuit depicted in Fig. 1a. Since it comprises a non-native gate, the compilation starts from the left-most state in Fig. 3b. First, an optimization pass is applied that cancels the two redundant* X *gates. Since the circuit still contains non-native gates, this does not change the overall state—as indicated by the dashed self-loop above the state. Then, a synthesis pass is applied, which yields the circuit depicted in Fig. 1c and advances the state of the MDP to the next state.*

Whenever the circuit only contains native gates but does not yet respect the device topology, three different actions are possible: First, the circuit could be mapped using any *mapping action*—making the circuit executable and, thus, advancing the MDP to the final state. Additionally, the circuit may be optimized again. Depending on whether the optimization pass preserves the native gates, the state after the action either stays the same (*gate-set-preserving optimization action*) or goes back to the previous, non-native gates state (*general optimization action*).

Example 9. *Consider again the synthesized quantum circuit from Fig. 1c. As before in Example 8, a similar optimization is conducted to cancel the two rotation gates. Since the resulting circuit still only consists of native gates, this optimization does not change the state of the MDP—again represented by a dashed self-loop in Fig. 3b. Subsequently, a mapping action is applied that leads to the quantum circuit depicted in Fig. 1d and advances the state of the MDP to the final state.*

In the final state, both the device's native gate-set and qubit connectivity constraints are fulfilled and any circuit in this state is already executable. However, additional *optimization* actions can be applied to further reduce complexity. Again, those actions might violate any of the two constraints and the circuit might end up in a state where it is not executable any more.

Example 10. *The quantum circuit resulting from Example 9 is already executable. However, another optimization action is applied to the circuit depicted in Fig. 1d to cancel two further gates and, by that, reducing the complexity. Since this does not violate the device's connectivity, the circuit remains executable and the MDP state does not change—again represented by a dashed self-loop in Fig. 3b.*

[2]We purposely omit the state where the circuit respects the topology but contains non-native gates as there are hardly any compilation passes available that take advantage of this specific combination of conditions.

This MDP formulation enables a modular framework based on two properties:

(1) Each of the constraints that characterize an MDP state is *efficiently computable* (via a single traversal of the circuit).
(2) All actions have a *unified interface* based on a common intermediate representation (IR) for their input and output—independent of the origin of a certain action.

The former ensures that, at any stage during the compilation, state transitions within the MDP can be computed efficiently. The latter ensures interoperability between different SDKs and makes it possible to *mix and match* compilation passes provided by multiple SDKs. Consequently, they can be combined and integrated in a single compilation framework—mitigating vendor lock-ins, allowing one to quickly adapt and integrate further compilation passes, and helping to keep up with the fast-paced development in quantum computing software.

4.2 Learning Optimal Compilation Sequences Using Reinforcement Learning

In classical compilation, different problems have been tackled very successfully using *Reinforcement Learning* (RL) [51]–[53]. In general, these techniques aim to learn an *action policy* that determines which actions should be applied based on *observations* of the current state with the goal of maximizing a cumulative *reward function*. In contrast to providing labeled training data (as in *Supervised Machine Learning*, ML), a *training environment* representing the underlying MDP including its states and actions must be defined.

To avoid reinventing the wheel, we also propose using such an RL approach for quantum circuit compilation based on the MDP shown in Fig. 3b. To this end, three things must be provided as input (which are depicted in Fig. 3a):

(1) *Training Circuits*: Used in the training process to learn an action policy.
(2) *Quantum Device*: Defines the native gate-set, the qubit connectivity, and certain characteristics (such as gate fidelities) for the compilation process.
(3) *Figure of Merit*: Defines the evaluation score to guide the learning process.

An *RL agent* can then be employed to learn a corresponding action policy that maximizes the expected cumulative reward. For the training itself, a *sparse* reward function is used, i.e., as long as the circuit is not executable, the reward is always zero, and whenever the circuit becomes executable, the chosen figure of merit (e.g., expected fidelity as described in Section 3.2) is used to assign a score to the compiled circuit.

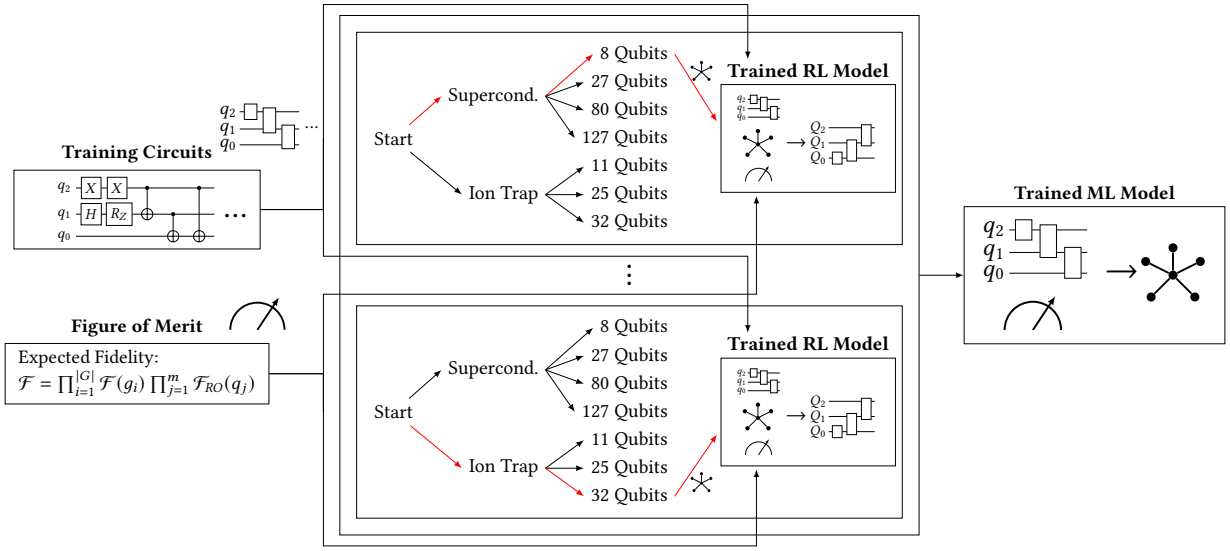
The resulting trained RL model then acts as *compiler* itself that can compile any given quantum circuit for the target device it was trained for—as illustrated in Fig. 3c. This creates a compiler that is fine-tuned for a specific device, optimizes for a customizable figure of merit, and factors in the input circuit when determining which compilation passes to apply—three major advances over current quantum circuit compilers.

5 DEVICE SELECTION USING SUPERVISED MACHINE LEARNING

As discussed in Section 2.1, selecting the best quantum device for a particular application is itself a challenging task. This section describes how that task can be interpreted as a classification problem and how supervised machine learning can predict the most promising qubit technology and device for a given quantum circuit—therefore, automatically make this decision for an end-user.

5.1 Quantum Device Selection Modeled as a Classification Task

A naive approach to selecting the best quantum device for a given quantum circuit would be to first compile it for all devices using a specific compiler, e.g., the trained RL models from Section 4. Afterwards, the resulting compiled circuits must be evaluated according to some *figure of merit*



(a) Input. (b) Classification task for the quantum device selection. (c) Output.

Fig. 4. Training process of an ML model for the quantum device selection.

to identify the most promising device. However, doing this for each and every to-be-compiled quantum circuit is practically infeasible since compilation is a time-consuming task and the number of available devices is steadily growing.

Example 11. *There are various underlying technologies how to realize quantum computers such as, e.g., superconducting- and ion trap-based qubits. Even within a technology, respective devices may significantly vary in their characteristics such as the connectivity scheme, native gate-set, and fidelity rates. Assuming that an end-user has access to seven devices based on two technologies, the task of determining the best device for a given circuit and figure of merit already becomes challenging since seven different compilations would have to be conducted—as insinuated in Fig. 4b.*

Therefore, we propose an approach to *learn* from previously conducted compilations and interpret the selection of the most promising device for a circuit and figure of merit as a statistical *classification* task—a perfect fit for supervised machine learning. By that, end-users are freed from answering the following questions themselves:

- Which *qubit technology* is best suited for the application at hand?
- Which particular *device*, e.g., from IBM or Rigetti, fits the quantum algorithm best?

5.2 Predicting a Quantum Device Using Supervised Machine Learning

Supervised Machine Learning (ML) has proven a promising methodology in classical compilation (as described in [47], [48]) but also in various other domains, such as, e.g., medicine, cyber security, and predictive analytics [56]. A crucial part to utilize this technology is gathering suitable training data which are representative for the whole problem domain space to facilitate generalizability—in this case, representative training circuits covering a broad range of applications. Subsequently, each training circuit must be compiled for *all* possible devices—using the trained RL models that act as compilers—to be able to identify the most promising according to the chosen figure of merit as shown in Fig. 4b. That device then acts as the *classification label* and, together with the training circuit itself, constitutes one *training sample*. To generate all the *labeled training data* necessary

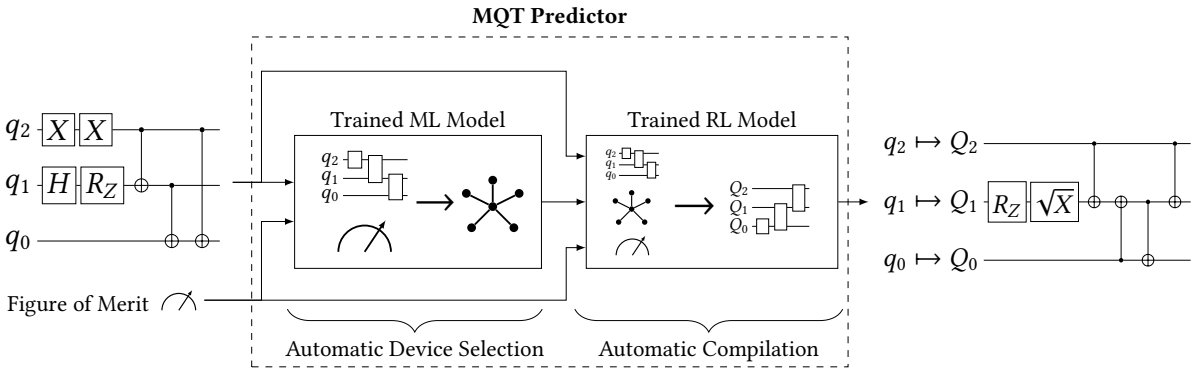


Fig. 5. Automatic compilation flow selection and execution using MQT Predictor.

to train respective ML models, this process is repeated for each training circuit and requires two things that must be provided (which are depicted in Fig. 4a):

(1) *Training Circuits*: Similar to the RL models, used here to generate labeled training data[3].
(2) *Figure of Merit*: Determines the most promising device based on all conducted compilations.

Each labeled circuit must then be transformed into a *feature vector* to be suitable for training a classifier. There are many degrees of freedom when choosing the respective feature and it is very much an active field of research to determine features that best characterize a given quantum circuit. In practice, these features should at least include the number of qubits and some statistics about the interactions of gates in the circuit, such as depth, parallelism, or interaction frequency.

Subsequently, the actual ML model can be trained. This methodology with the respectively trained ML model (as illustrated in Fig. 4c) then acts as a tool to automatically predict the most promising quantum device for a given circuit and figure of merit. As a result, the user gets a recommendation that is circuit-specific, tuned for a specific figure of merit, and requires no compilation at all—again, three major advances over the established workflow.

6 PROPOSED APPROACH: MQT PREDICTOR

Together with the trained RL models from Section 4, the trained ML model automatically solves the device selection and compilation process—forming a holistic framework. In the following, this framework, called the *MQT Predictor*, and its implementation are described in more detail.

6.1 Approach

The proposed approach is shown in Fig. 5. It takes a quantum circuit and a customizable figure of merit as input. Then, the framework itself predicts which quantum device is the most promising using the trained ML model—in *realtime* without conducting any compilation at all. The outcome of this is the predicted device. Based on that, the respectively trained RL model compiles the given circuit for the predicted device optimizing for the chosen figure of merit. Finally, it returns the compiled quantum circuit. By that, the MQT Predictor completely frees end-users from the daunting tasks of selecting a proper quantum device and finding a good compiler and can be used with just a single *Python* function call—providing the same ease of use as the state-of-the-art compilers such as Qiskit [1]. On top of that, it combines the advantages of both individual techniques into a single powerful tool.

Example 12. *Consider again the original scenario shown in Fig. 2, where an end-user wants to execute the circuit shown again on the left-hand side of Fig. 5. Instead of manually finding the*

[3]To reduce the effort needed for regenerating labeled training data after changes, all compiled circuits including their evaluation score are persistently stored in a database.

most promising device and compiler passes, the circuit can be simply fed into the MQT Predictor framework together with the expected fidelity (as defined in Section 3.2) as its figure of merit. Then, the MQT Predictor predicts a respective device and compiles the circuit accordingly using the trained RL model—returning the compiled circuit shown on the right-hand side of Fig. 5.

The following sections describe the particular instantiation of the framework that has been implemented as part of this work.

6.2 Implementation of Automatic Compilation Using RL

The training environment described in Section 4 and shown in Fig. 3b is built on top of the open-source library *OpenAI Gym* [57]. For that, all compilation passes from different sources that should be considered in learning promising compilation sequences must be implemented through a *unified interface*.

To this end, *Qiskit's QuantumCircuit* is used as the *intermediate representation*. Therefore, a compilation pass can be incorporated if the following requirement is fulfilled: Either it must provide an interface for both the input and the compiled output circuit that supports *Qiskit's QuantumCircuit* or a parser that supports *OpenQASM* (version 2 or 3)—which, then, can be used to create and serialize a *Qiskit's QuantumCircuit* again.

As modeled in Fig. 3, three different types of actions are considered: synthesis, mapping, and optimization passes. Using the described unified interface for both input and output of a compilation action, compilation passes from various quantum SDKs can easily be incorporated. To demonstrate this, representative compilation passes from IBM's Qiskit (version 0.43.3) and Quantinuum's TKET (version 1.17.1) have been integrated—leading to the following list of available actions.

- *Synthesis*: Qiskit's *BasisTranslator*
- *Mapping*: Qiskit's *Sabre* mapping or any combination of the following methods
  - Layout:
    - Qiskit's *TrivialLayout*
    - Qiskit's *DenseLayout*
    - Qiskit's *SabreLayout*
  - Routing:
    - Qiskit's *BasicSwap*
    - Qiskit's *StochasticSwap*
    - Qiskit's *SabreSwap*
    - TKET's *RoutingPass*
- *Optimization*:
  - Qiskit's *Optimize1qGatesDecomposition*
  - Qiskit's *CXCancellation*
  - Qiskit's *CommutativeCancellation*
  - Qiskit's *CommutativeInverseCancellation*
  - Qiskit's *RemoveDiagonalGatesBeforeMeasure*
  - Qiskit's *InverseCancellation*
  - Qiskit's *OptimizeCliffords*
  - Qiskit's *Collect2qBlocks* + *ConsolidateBlocks*
  - Qiskit's *O3 Fixpoint Optimization Routine*
  - TKET's *PeepholeOptimise2Q*
  - TKET's *CliffordSimp*
  - TKET's *FullPeepholeOptimise*
  - TKET's *RemoveRedundancies*

For the training process, more than 500 training circuits from *MQT Bench* [58] on the target-independent level have been used ranging from 2 to 30 qubits. Lastly, a maskable version of the *Proximal Policy Optimization (PPO)* algorithm [59] provided by *Stable-Baselines3* [60] is used for the reinforcement learning training process itself and also takes care of potential infinite sequences due to the backward loops within the MDP.

6.3 Implementation of Automatic Device Selection Using ML

For the quantum device selection, the ML model described in Section 5 and shown in Fig. 4b is trained using scikit-learn [61]. In this regard, seven different underlying algorithms are implemented and evaluated in [12]—with grid-searched and 5-fold cross-validated parameter values—to determine the most promising one:

- *Random Forest* [62]
- *Gradient Boosting* [63]
- *Decision Tree* [64]
- *Nearest Neighbor* [65]
- *Multilayer Perceptron* [66]
- *Support Vector Machine* [67]
- *Naive Bayes* [68]

In [69], an overview of today's use of those techniques is given. All seven machine learning classifiers have been experimentally evaluated. Since the *Random Forest* classifier has given the best results, it is used in the following as the ML algorithm.

As indicated in Fig. 2, seven devices from two qubit technologies are considered as representatives:

- Superconducting: four devices with 8, 27, 80, and 127 qubits
- Ion Trap: three devices with 11, 25, and 32 qubits

To generate respective training data, more than 500 training circuits from *MQT Bench* [58] on the target-independent level are used (ranging from 2 to 90 qubits) with a 70/30 train-test-split. The compilations to generate the labeled training data are conducted based on the trained RL models acting as respective compilers.

6.4 Quantum Circuit Representation

Both the RL and the ML training process require quantum circuits to be represented as so-called *feature vectors*–a vectorized representation based on integer and floating point values that makes them suitable for training a respective model. Similar to the figure of merit described in Section 3.2, these features can be arbitrarily complex. In this instantiation, various characteristics are used to describe a quantum circuit for both models: the *number of qubits*, the *depth* of the circuit, and the five composite features of *program communication*, *critical-depth*, *entanglement-ratio*, *parallelism*, and *liveness* originally proposed in [54]:

- *Program Communication*: Metric to measure the average degree of interaction for all qubits. A value of 1 indicates that each qubit interacts at least once with all other qubits.
- *Critical Depth*: Metric to measure how many of all multi-qubit gates are on the longest path (defining the depth of a quantum circuit). A value of 1 indicates that all multi-qubit gates are on the longest path.
- *Entanglement Ratio*: Metric to measure how many gates in a quantum circuit are multi-qubit gates. A value of 1 indicates that the quantum circuit consists of only multi-qubit gates.
- *Parallelism*: Metric to measure how much parallelization within the circuit is possible due to simultaneous gate execution. A value of 1 describes large parallelization.

- *Liveness*: Metric to measure how often the qubits are idling and waiting for their next gate execution. A value of 1 describes a circuit in which there is a gate execution on each qubit at each time step.

Furthermore, for the ML model, the number of gates for each gate type according to the *OpenQASM 2.0* specification [70] are used as features. For the RL model, this was omitted to improve the learning efficiency, since the gate count features have been shown to be less representative than the composite ones [12].

7 EVALUATION

Based on the instantiation described above, this section demonstrates the feasibility of the MQT Predictor framework by means of numerical evaluation. All used pre-trained models and classifiers are publicly available as open-source on GitHub (https://github.com/cda-tum/mqt-predictor) and as an easy-to-use *Python* package (https://pypi.org/p/mqt.predictor).

7.1 Setup

To assess the quality of the proposed framework, it is evaluated on more than 150 benchmarks from 2 to 90 qubits taken from *MQT Bench* [58] using the target-independent level circuits and the MQT Predictor instantiation described in Section 6. For each of those evaluation circuits—which have not been part of the (ML) training circuits—the tool predicts the most promising target device and its optimized compiler to compile the given circuit to the selected device. To assess quality, two figures of merit are used:

(1) The expected fidelity as introduced in Section 3.2 and
(2) The critical depth describing the ratio of multi-qubit gates on the longest path of the compiled quantum circuit[4].

Note that the used figures of merit are examples and the MQT Predictor framework allows end-user to provide *any* metric to optimize for. In this case, the focus rather lies on technical factors to determine both the best device and compilation sequence while it might be sensible to also factor in soft factors such as the execution costs or the queue length for the device selection. Since the focus of this work is to provide a framework that optimizes for *any* given figure of merit, the following evaluations focus on how well the framework optimizes for these exemplary figures of merit and not on how suited these figures of merit are to assess the compilation task itself.

To generate baseline values, all evaluation circuits are compiled for all seven currently supported devices using both Qiskit's and TKET's most optimized (pre-configured and fixed) compilation settings *O3* respectively *O2*—leading to 14 different compiled circuits as baselines.

7.2 Results

Fig. 6 shows the results obtained by using the proposed framework to optimize the expected fidelity. Each tick on the x-axis denotes one evaluation circuit that is assessed using the 14 baseline compilation flows (7 devices × 2 compilers) and the MQT Predictor. Benchmarks leading to only zero values have been excluded from all evaluations. Since (maximally) 15 values per tick would be hard to read, the graph is reduced to only show the

- MQT Predictor evaluation score in blue,
- best evaluation score in green,
- median evaluation score in yellow, and
- worst evaluation score in red.

[4]Those ratios are actually subtracted from 1 such that higher values indicate a more parallel quantum circuit that is assumed to be better.

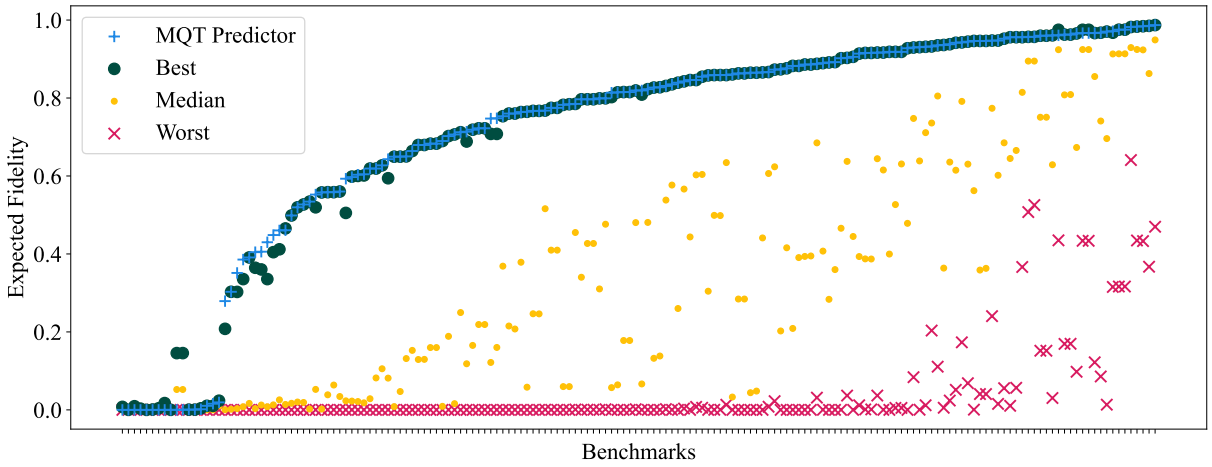


Fig. 6. Evaluation of the expected fidelity. All benchmarks are sorted by their MQT Predictor score for better readability.

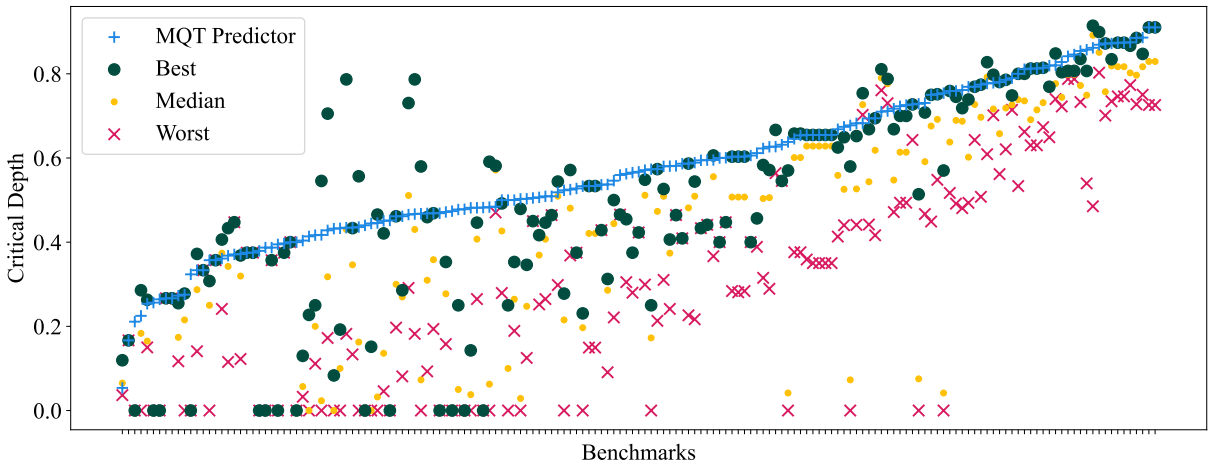


Fig. 7. Evaluation of the critical depth. All benchmarks are sorted by their MQT Predictor score for better readability.

In this setup, the proposed method is capable of producing compiled quantum circuits that are within the top-3 of all 14 baselines in over 98% of cases while frequently outperforming all baselines by up to 53%. Therefore, especially end-users who are not experts in quantum computing can use it to reliably and automatically compile quantum circuits whose quality otherwise can only be achieved by many manual experiments—often infeasible due to the necessary time effort and expert knowledge. These results also clearly highlight that choosing the right device and compiler can make the difference between a successful execution and obtaining completely random results.

To demonstrate the ability to optimize for customizable figures of merit, we additionally set up the framework to optimize for critical depth and show the respective results in Fig. 7. Again, benchmarks only leading to zero values have been excluded from the evaluations. Since it is not possible to define a custom figure of merit/optimization criterion for both Qiskit and TKET, the same compiled baseline evaluation circuits are used. The respective graph has been reduced in a similar fashion as described above. For 89% of cases, the compiled circuits produced by MQT Predictor are within the top-3 of the 14 baselines while frequently outperforming all baselines by up to 400%.

Table 1. Comparison of the influence of the figure of merit.

	Average result for...	
Model trained for...	Exp. Fidelity	Critical depth
Exp. Fidelity	**0.67**	0.42
Critical depth	0.12	**0.55**

These results describe the performance from the end-users' perspective, who use the MQT Predictor framework to automatically select the most promising device and compile accordingly. To provide more insight into the underlying training data, the ideal device distribution for the evaluated benchmarks is explored in Appendix A.1. Additionally, the impact of both the device selection as well as the compilation has been examined in an isolated fashion in Appendix A.2 and Appendix A.3, respectively.

To further investigate the frameworks ability to optimize for customizable figures of merit, the average evaluation scores for both figures of merit are calculated for both respectively trained frameworks. The resulting numbers are denoted in Table 1 and confirm that the framework trained for a particular figure of merit indeed results in the compiled circuits with the highest evaluation scores for both cases.

7.3 Generalizability

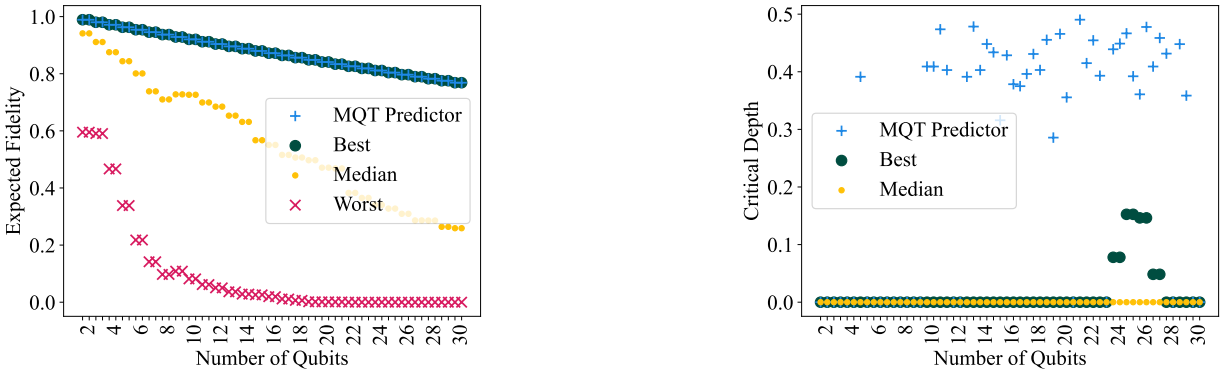
The performance of machine learning models, in general, is highly dependent on the quality of the training data. Therefore, training data should be as heterogeneous as possible to cover a wide variety of circuits with different characteristics to achieve a high *generalizability*. For that purpose, a multitude of different algorithms are considered in the training data for both the ML and RL models described above—ranging from quantum circuit building blocks such as, e.g., the *quantum fourier transform* and *amplitude estimation* towards rather complex quantum circuits such as, e.g., ground state estimation—all taken from *MQT Bench* [58].

To estimate how well the trained MQT Predictor works for not only the quantum circuits that are part of the training data but also for other algorithms that have not been considered during training, a further evaluation has been conducted. To evaluate generalizability, quantum circuits for the *Greenberger–Horne–Zeilinger* (GHZ) state, which have not been considered during training, are evaluated in a similar fashion as before in Section 7.2. The resulting scores for expected fidelity and critical depth are denoted in Fig. 8a and Fig. 8b, respectively.

Taking into account both figures of merit, a performance similar to the results described in Section 7.2 is achieved. Considering the expected fidelity, MQT Predictor leads to the same performance as the best baseline for all cases. In case of the critical depth as a figure of merit, MQT Predictor even outperforms all baselines in most cases. It should be noted that the best baseline results for critical depth frequently are 0.0 (since *all* two-qubit gates are on the longest path), while the MQT Predictor achieves significant improvements in many of these cases. Although this is only a small evaluation of the generalizability of the proposed methodology, it indicates that it also works well in untrained scenarios.

8 DISCUSSION

Classical machine learning has shown tremendous capabilities to solve problems from various domains such as classical compilation and beyond. However, its potential success inherently depends on gathering a sufficiently large amount of high-quality training data and, by that, it is not per-se well suited for changes, which often require to re-train the underlying machine learning models. This section specifically gives guidance how the MQT Predictor deals with such changes in various ways.



(a) Expected fidelity for GHZ circuits.

(b) Critical depth for GHZ circuits.

Fig. 8. Generalizability evaluation for GHZ circuits which were not part of the training processes.

8.1 Changes in Hardware Information used in Figure of Merit

Quantum computers are frequently calibrated to maximize the actual gate fidelities, e.g., used in the figure of merit introduced in Section 3.2. Thus, these calibration updates affect the fidelity data used throughout both the RL and ML training as part of the cost function. Consequently, the models need to be re-trained. Unfortunately, it is not feasible to always generate all training data from scratch whenever such a calibration is conducted since it may occur on a daily (or even hourly) basis, while the training of the proposed framework instantiation took approximately 24 hours on a consumer MacBook Pro. As with any AI solution currently, a natural way to speed up the training time would be to utilize supercomputers, e.g., using *High Performance Computing* clusters. Future work may focus on machine learning techniques to adjust existing models to new circumstances that do not involve re-training from scratch.

8.2 Additional Compilation Passes and Figures of Merit

The quantum software ecosystem is moving fast and quantum SDKs frequently release new versions of their toolchains. In addition, many new research methods for quantum circuit compilation are proposed regularly. In order to keep up with this pace, the MQT Predictor has been designed with a unified and modular interface that allows one to easily add and/or update compilation passes. The good news is that the degree of change—whether just another optimization pass has been added or whether all of it changed—does not affect the overall effort. The bad news is that any such change still requires re-training of all RL and ML models. Also here, future work may focus on finding ways to re-train models without starting from scratch.

A similar effort is necessary when changing or adapting the figure of merit. Since both the RL and the ML models optimize for it during training, again, the whole MQT Predictor framework must be re-trained and the persistently stored data must be refreshed.

8.3 Changes in Supported Quantum Devices

New quantum devices become available on a regular basis. Fortunately, adding a new device to the proposed framework is rather easy. It is only necessary to train the respective RL model and to determine the respective evaluation score based on the chosen figure of merit for all training circuits. Afterwards, the ML model can be re-trained in minutes. This is enabled by persistently storing the compiled circuits and evaluation scores of previously conducted compilations as part of the training data generation process. On the contrary, no additional effort is needed to exclude devices—either in case of a temporary downtime due to, e.g., calibration, or a permanent deprecation. These devices can just be masked out without any further effort necessary.

9 CONCLUSION

Using quantum computing as a technology to *just* solve a problem from any kind of application domain is challenging, since a suitable quantum device must be selected to solve a problem encoded as a quantum circuit and the circuit must be compiled accordingly. Deciding which is the best device and how to best compile for a certain application—according to a *figure of merit*—currently requires expert knowledge in quantum computing. So far, especially end-users from application domains are often left unsupported and overwhelmed due to missing automation and tool support. This situation is even worsened, since the result quality heavily fluctuates with the particular choice of options and often makes the difference between a useful result and a useless one due to too much noise.

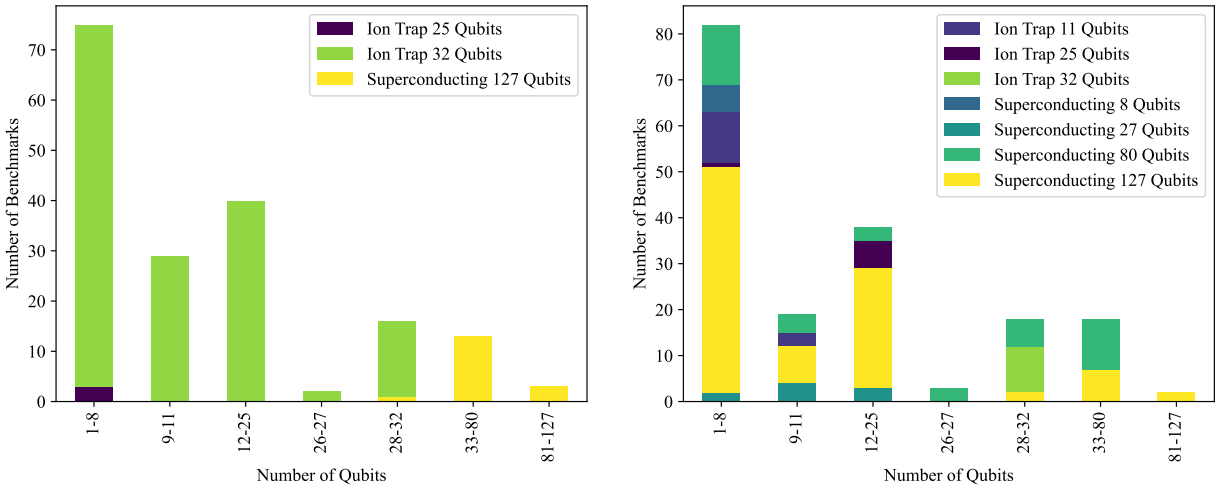
In this work, we proposed the MQT Predictor framework to automate the selection of suitable target devices and the subsequent compilation. By that, end-users are freed from this task with the goal of preventing a scenario where quantum computers can only be utilized by quantum computing experts—hindering the overall adoption of the technology. The proposed methodology allows learning on the basis of previously conducted compilations of quantum circuits for various devices. For that, the problem is tackled from two angles: Using *Reinforcement Learning*, optimal compilation pass sequences are learned for all available devices—mixing and matching compilation passes from various sources and quantum SDKs. Furthermore, using *Supervised Machine Learning*, a prediction of a quantum device is made for a given quantum circuit according to customizable figures of merit such as, e.g., the *expected fidelity* of the to-be-compiled quantum circuit.

The proposed framework has been implemented using more than 500 quantum circuits from 2 to 90 qubits for seven devices from two qubit technologies and has shown to yield compiled circuits that are within the top-3 in more than 98% of cases while frequently outperforming any tested combination by up to 53% when considering the expected fidelity as the figure of merit. Furthermore, trained and evaluated for *critical depth* as another figure of merit, the performance was within the top-3 in 89% of cases while frequently outperforming any tested combination by up to 400%. The corresponding framework (which is part of the *Munich Quantum Toolkit* (MQT)) including its pre-trained models and classifiers is publicly available on GitHub (https://github.com/cda-tum/mqt-predictor) and as an easy-to-use *Python* package (https://pypi.org/p/mqt.predictor).

These results clearly demonstrate that adapting technologies from the classical realm and applying them to quantum computing can significantly foster the progress of this new and promising technology. Especially since quantum computers are mostly utilized by experts at the moment due to the challenges within the current workflow for realizing quantum application on real machines. With this work, we hope to take a step forward in simplifying the utilization of quantum computers for end-users from application domains and the development of optimized compilers within the community.

ACKNOWLEDGMENTS

This work received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (grant agreement No. 101001318), was part of the Munich Quantum Valley, which is supported by the Bavarian state government with funds from the Hightech Agenda Bayern Plus, and has been supported by the BMWK on the basis of a decision by the German Bundestag through project QuaST, as well as by the BMK, BMDW, the State of Upper Austria in the frame of the COMET program, and the QuantumReady project within Quantum Austria (managed by the FFG).



(a) Expected fidelity.

(b) Critical depth.

Fig. 9. Device distributions.

A APPENDICES

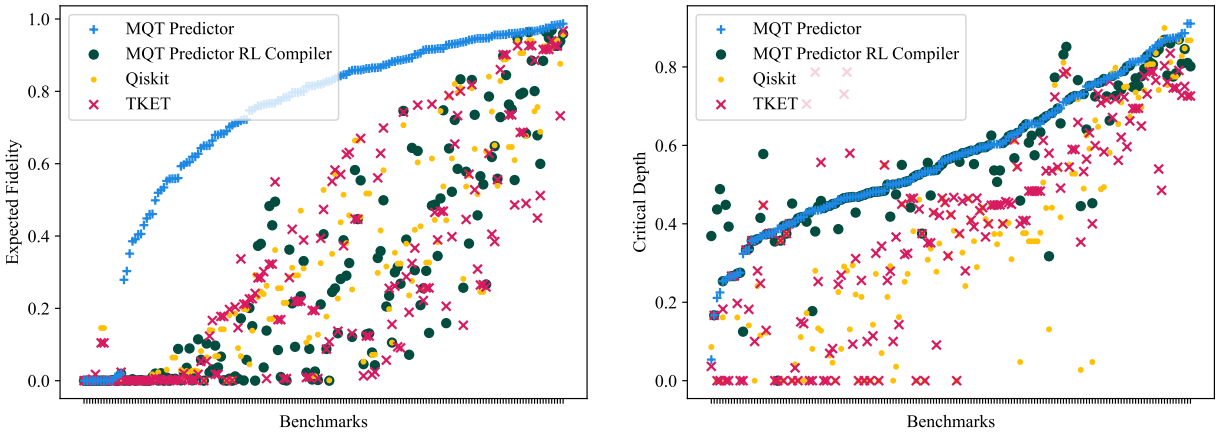
A.1 Device Distribution

To better understand the problem of selecting the best device, the underlying ground truth data used to train the MQT Predictor is further investigated. To this end, the frequency with which each device is selected as the best performing one is investigated. For that, each benchmark is compiled for all supported devices using the respectively trained RL compilers and the one leading to the best result is determined. The results are visualized in Fig. 9. Furthermore, the benchmarks are grouped by their number of qubits such that all benchmarks within one group can be executed on the same number of devices—leading to seven groups since currently seven devices are supported with a decreasing number of available devices for an increasing number of qubits.

When using the expected fidelity as the figure of merit, the results are rather distinct as visualized in Fig. 9a. For most benchmarks, the ion trap device with 32 qubits results in the best performance. Therefore, the rule of thumb known by quantum computing experts—taking an ion trap device when possible—has been confirmed. However, when all ion trap devices' capacities are exceeded, the largest superconducting device is best even when a smaller device would have been available as well. This showcases that another quantum expert rule of thumb—taking always the smallest but fitting device—cannot be confirmed by our experimental data.

When using the critical depth as the figure of merit, the device distribution is significantly more diverse as visualized in Fig. 9b[5]. Here, all seven devices are chosen at least sometimes and there is no apparent correlation between the best performing device and the benchmark groups since most devices result in the best performance for more than just one group. Consequently, no clear guidance can be derived from that—underlining the importance of software tools to guide this decision automatically.

[5]The benchmarks used for the evaluation differ between both figures of merit and, therefore, the number of benchmarks in each group differs as well.



(a) Expected fidelity.

(b) Critical depth.

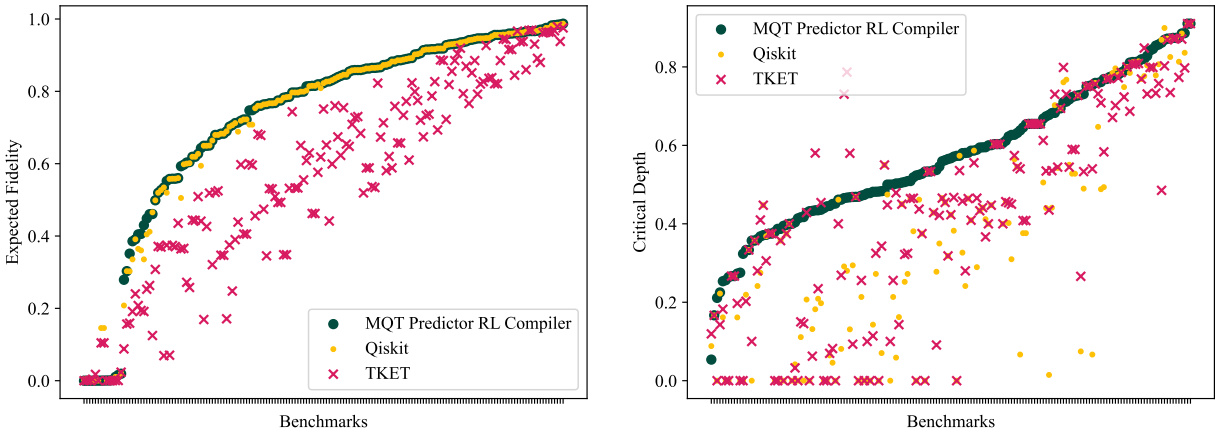
Fig. 10. Isolated evaluation of the device selection using ML.

A.2 Detailed Evaluation: Device Selection Using ML

The MQT Predictor framework automatically selects the most promising device and compiles accordingly. To assess the effect each of the two tasks has, they are evaluated in an isolated fashion—starting with the device selection and its respective results shown in Fig. 10. To this end, the largest device (superconducting device with 127 qubits) is used as the default device to fit all benchmarks. Then, the benchmarks are compiled for it using Qiskit, TKET, and the MQT Predictor RL compiler and, similarly to Fig. 6 and Fig. 7, the benchmarks are sorted by their MQT Predictor score for better readability. The resulting performances are then compared against the full MQT Predictor approach including the device selection.

For the expected fidelity, the results are visualized in Fig. 10a—showing a distinct gap between the results of the full MQT Predictor approach including the device selection compared to all three baselines using the default device. This highlights the importance of selecting the most suitable device, since that can make the difference between a successful execution and obtaining completely random results. Furthermore, it is interesting to see that there is no clear winner among Qiskit, TKET, and the MQT Predictor RL compiler.

In the case of critical depth, the situation is different as visualized in Fig. 10b since the full MQT Predictor approach less frequently leads to the best result. This suggests that the default device is a good choice for critical depth and the influence between it and the best one is smaller than it is the case for the expected fidelity. Also, this matches with Fig. 9b since often the largest device is the most suitable in the ground truth data. Furthermore, the MQT Predictor RL compiler approach outperforms both Qiskit and TKET in most cases—showcasing, that both are not optimizing for critical depth but optimize for a different implicit underlying criterion closer to the expected fidelity.



(a) Expected fidelity.

(b) Critical depth.

Fig. 11. Isolated evaluation of the compilation using RL.

A.3 Detailed Evaluation: Compilation Using RL

To assess the compilation quality of the RL part of the MQT Predictor framework, it is evaluated in a similar and isolated fashion as the device selection described in Appendix A.2. To this end, the device is set to be the one predicted for each benchmark and the effect of the different compilers is evaluated in Fig. 11. For that, the same compilers as before are used: Qiskit, TKET, and the MQT Predictor RL compiler—while, in this case, the latter corresponds to the full MQT Predictor approach because the default device is the predicted one. Therefore, this evaluation setup results in plots that show a subset of the data points plotted in Fig. 6 respectively Fig. 7 in which the compilers are evaluated for all seven devices while here only the predicted one is used. The benchmarks are sorted by their MQT Predictor RL compiler scores for better readability (which, in this case, corresponds to the scores of the full MQT Predictor approach).

For the expected fidelity visualized in Fig. 11a, the MQT Predictor RL compiler outperforms TKET and Qiskit, although there is only a small performance gap to the latter since Qiskit performs better than TKET for the predicted device. Compared to Fig. 10a where the default device was the largest superconducting one, this was not the case. Therefore, apparently the compilation quality significantly depends on the device—either by Qiskit being better for the predicted one (which is often an ion trap one), TKET being worse, or both. Nevertheless, the plot shows the (small) improvement of the MQT Predictor compilation—potentially growing when including more compilation passes from further providers.

In the case of the critical depth visualized in Fig. 11b, the performance differences between Qiskit and TKET are not as distinct as before. Furthermore, the performance difference of the MQT Predictor RL compiler compared to both of them is even larger—highlighting the impact it has.

The Journal of Open Source Software

MQT Core: The Backbone of the Munich Quantum Toolkit (MQT)

Lukas Burgholzer [1,2¶], **Yannick Stade** [1], **Tom Peham** [1], **and Robert Wille** [1,2,3]

1 Chair for Design Automation, Technical University of Munich, Germany **2** Munich Quantum Software Company GmbH, Garching near Munich, Germany **3** Software Competence Center Hagenberg GmbH, Hagenberg, Austria ¶ Corresponding author

DOI: 10.21105/joss.07478

Software

- Review
- Repository
- Archive

Editor: Daniel S. Katz

Reviewers:

- @1ucian0
- @edyounis
- @josh146

Submitted: 08 November 2024
Published: 07 April 2025

Summary

MQT Core is an open-source C++ and Python library for quantum computing that forms the backbone of the quantum software tools developed as part of the *Munich Quantum Toolkit (MQT, (Wille et al., 2024))* by the Chair for Design Automation at the Technical University of Munich as well as the Munich Quantum Software Company (MQSC). To this end, it consists of multiple components that are used throughout the MQT, including a fully fledged intermediate representation (IR) for quantum computations, a state-of-the-art decision diagram (DD) package for quantum computing, and a state-of-the-art ZX-diagram package for working with the ZX-calculus. Pre-built binaries are available via PyPI for all major operating systems and all modern Python versions. MQT Core is fully compatible with IBM's Qiskit 1.0 and above (Javadi-Abhari et al., 2024), as well as the OpenQASM format (Cross et al., 2022), enabling seamless integration with the broader quantum computing community.

Statement of Need

Quantum computing is rapidly transitioning from theoretical research to practice, with potential applications in fields such as finance, chemistry, machine learning, optimization, cryptography, and unstructured search. However, the development of scalable quantum applications requires automated, efficient, and accessible software tools that cater to the diverse needs of end users, engineers, and physicists across the entire quantum software stack.

The Munich Quantum Toolkit (MQT, (Wille et al., 2024)) addresses this need by leveraging decades of design automation expertise from the classical computing domain. Developed by the Chair for Design Automation at the Technical University of Munich, the MQT provides a comprehensive suite of tools designed to support various design tasks in quantum computing. These tasks include high-level application development, classical simulation, compilation, verification of quantum circuits, quantum error correction, and physical design.

MQT Core offers a flexible intermediate representation for quantum computations that forms the basis for working with quantum circuits throughout the MQT. The library provides interfaces to IBM's Qiskit (Javadi-Abhari et al., 2024) and the OpenQASM format (Cross et al., 2022) to make the developed tools accessible to the broader quantum computing community. Furthermore, MQT Core integrates state-of-the-art data structures for quantum computing, such as decision diagrams (Wille et al., 2023) and the ZX-calculus (Duncan et al., 2020; van de Wetering, 2020), that power the MQT's software packages for classical quantum circuit simulation (MQT DDSIM), compilation (MQT QMAP), verification (MQT QCEC), and more. As such, MQT Core has enabled more than 30 research papers over its first five

years of development (Burgholzer et al., 2020; Burgholzer, Bauer, et al., 2021; Burgholzer, Raymond, et al., 2021; Burgholzer, Kueng, et al., 2021; Burgholzer, Ploier, et al., 2022a, 2022b; Burgholzer, Schneider, et al., 2022; Burgholzer & Wille, 2020a, 2020b, 2021; Grurl, Pichler, et al., 2023; Grurl et al., 2020, 2021; Grurl, Fuß, et al., 2023; Hillmich et al., 2021, 2020, 2022; Hillmich, Markov, et al., 2020; Hillmich, Kueng, et al., 2020; Peham et al., 2022b, 2022a, 2023b, 2023a; Peham, Brandl, et al., 2023; Sander et al., 2023; Schmid, Locher, et al., 2024; Schmid, Park, et al., 2024; Wille et al., 2020, 2021, 2022, 2023; Wille & Burgholzer, 2022).

To ensure performance, MQT Core is primarily implemented in C++. Since the quantum computing community predominantly uses Python, MQT Core provides Python bindings that allow seamless integration with existing Python-based quantum computing tools. In addition, pre-built Python wheels are available for all major platforms and Python versions, making it easy to install and use MQT Core in various environments without the need for manual compilation.

Related Work

MQT Core builds on a rich history of research in quantum computing, design automation, and data structures. The design of its IR is heavily inspired by IBM's Qiskit (Javadi-Abhari et al., 2024), with which it has stayed compatible since `qiskit-terra` version 0.16.1 (released at the end of 2020). MQT Core remains one of the few libraries providing drop-in replacements for large parts of Qiskit's core data structures in C++. Alternative IRs, that come as part of larger quantum computing frameworks, include Quantinuum's C++-based t|ket⟩ (Sivarajah et al., 2021), LBNL's Python-based `bqskit` (Younis et al., 2021), Xanadu's MLIR-based `catalyst` (Ittah et al., 2024), and NVIDIA's MLIR-based `CUDA-Q` (*CUDA-q*, 2024).

The origin of the decision diagram package in MQT Core dates back to the seminal work on Quantum Multiple-Valued Decision Diagrams (QMDDs) (Niemann et al., 2016). It provides a state-of-the-art implementation of QMDDs that natively integrates with the MQT Core IR. Alternative types of quantum decision diagrams and related software packages include TDD's (Hong et al., 2022), Bit-Slicing Decision Diagrams (Tsai et al., 2021), and LIMDDs (Vinkhuijzen et al., 2023). In comparison to MQT Core, most of these libraries have remained academic prototypes and have not seen widespread adoption in the quantum computing community.

The ZX-diagram package in MQT Core is inspired by the `PyZX` library (Kissinger & Wetering, 2020) and the t|ket⟩ (Sivarajah et al., 2021) compiler. It provides an efficient C++ implementation of core ZX-calculus concepts and is tightly integrated with the MQT Core IR. Compared to other implementations, the ZX package in MQT Core is fine-tuned for verification use cases and provides dedicated support for handling qubit permutations as well as numerical inaccuracies that arise in practice.

Acknowledgements

We would like to thank all past and present contributors to the MQT Core project, including (in alphabetical order): Hartwig Bauer, Martin Fink, Ioan-Albert Florea, Elias Foramitti, Rebecca Ghidini, Thomas Grurl, Stefan Hillmich, Fabian Hingerl, Daniel Lummerstorfer, Joachim Marin, Katrin Muck, Christoph Pichler, Tobias Prienberger, Parham Rahimi, Janez Rotman, Damian Rovara, Roope Salmi, Aaron Sander, Ludwig Schmid, Sarah Schneider, Tianyi Wang, Theresa Wasserer, and Alwin Zulehner. Their contributions have been instrumental in making MQT Core a robust and feature-rich library.

The Munich Quantum Toolkit has been supported by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (grant agreement

Quantum Circuit Optimization with AlphaTensor

Francisco J. R. Ruiz*,[1] Tuomas Laakkonen*,[2] Johannes Bausch[1]
Matej Balog[1] Mohammadamin Barekatain[1] Francisco J. H. Heras[1]
Alexander Novikov[1] Nathan Fitzpatrick[3] Bernardino Romera-Paredes[1]
John van de Wetering[4] Alhussein Fawzi[1] Konstantinos Meichanetzidis[2]
Pushmeet Kohli[1]

[1] Google DeepMind, 6-8 Handyside Street, London N1C 4UZ, UK
[2] Quantinuum, 17 Beaumont Street, Oxford OX1 2NA, UK
[3] Quantinuum, Terrington House, 13–15 Hills Road, Cambridge CB2 1NL, UK
[4] Informatics Institute, University of Amsterdam, 1098 XH Amsterdam, NL

Abstract

A key challenge in realizing fault-tolerant quantum computers is circuit optimization. Focusing on the most expensive gates in fault-tolerant quantum computation (namely, the T gates), we address the problem of T-count optimization, i.e., minimizing the number of T gates that are needed to implement a given circuit. To achieve this, we develop AlphaTensor-Quantum, a method based on deep reinforcement learning that exploits the relationship between optimizing T-count and tensor decomposition. Unlike existing methods for T-count optimization, AlphaTensor-Quantum can incorporate domain-specific knowledge about quantum computation and leverage *gadgets*, which significantly reduces the T-count of the optimized circuits. AlphaTensor-Quantum outperforms the existing methods for T-count optimization on a set of arithmetic benchmarks (even when compared without making use of gadgets). Remarkably, it discovers an efficient algorithm akin to Karatsuba's method for multiplication in finite fields. AlphaTensor-Quantum also finds the best human-designed solutions for relevant arithmetic computations used in Shor's algorithm and for quantum chemistry simulation, thus demonstrating it can save hundreds of hours of research by optimizing relevant quantum circuits in a fully automated way.

1 Introduction

Quantum computation presents a fundamentally new approach to solving computational problems. Since its inception [1, 2], many potential applications in various fields have been proposed, including cryptography [3], drug discovery [4], and materials science and high energy physics [5]. Yet, fault-tolerant quantum computation introduce some expensive components that have a significant impact on the overall runtime and resource cost [6, 7]; thus it is important to minimize the use of these components in order to enable the execution of large computations that address these real-world problems.

*Equal contributors.

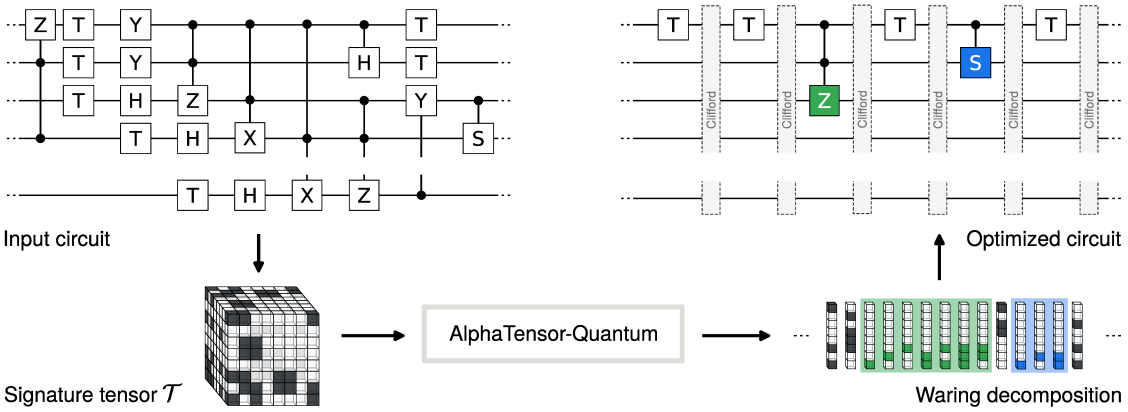


Figure 1: Pipeline of AlphaTensor-Quantum. We first extract the non-Clifford components of an input circuit and represent them as a symmetric *signature tensor* $\mathcal{T}$, a binary tensor depicted as a cube of solid and transparent blocks. We then use AlphaTensor-Quantum to find a low-rank Waring decomposition of that tensor, that is, a set of factors (displayed as columns of blocks) that can be mapped back into an optimized quantum circuit with reduced T-count. In general, there is a one-to-one correspondence between factors and T gates, but AlphaTensor-Quantum can also group factors into *gadgets* (highlighted in green and blue)—constructions that can be equivalently implemented with fewer T gates than the sum of the factors.

To be able to implement any quantum algorithm, universal quantum computers require a combination of two types of quantum gates: Clifford (e.g., Hadamard or CNOT) and non-Clifford gates (e.g., the T gate). Non-Clifford gates are required to achieve universality [8]; indeed, a quantum algorithm implemented using Clifford gates alone can be simulated on a classical computer efficiently, i.e., in polynomial time (Gottesman–Knill theorem [9]). Unfortunately, non-Clifford gates are the expensive components, as they are notoriously harder to implement than Clifford gates. This is because fault-tolerant quantum computation requires error correction schemes [10, 11], whose implementation requires distilling magic states [6], a process with high spacetime cost (qubit-seconds) [12]. For example, the spacetime cost of the T gate is about two orders of magnitude larger than the cost of a CNOT operation between two adjacent qubits [8]. Consequently, the cost of a quantum algorithm in the fault-tolerant era is arguably dominated by the cost of implementing the non-Clifford gates.

In this paper, we focus on the problem of T-count optimization, i.e., minimizing the number of T gates of quantum algorithms. T-count optimization is an NP-hard problem [13] that has been addressed in the literature using different approaches [14–23]. Like most of these works, and for the reasons laid out before, we consider the T-count as the *sole* metric of complexity of a quantum circuit.

With that aim, we first transform the problem into an instance of tensor decomposition, exploiting the relationship between T-count optimization and finding the symmetric tensor rank [16, 18]. We develop *AlphaTensor-Quantum*, an extension of AlphaTensor [24], a method that finds low-rank tensor decompositions using deep reinforcement learning (RL). AlphaTensor-Quantum tackles the problem from the tensor decomposition point of view and is able to find efficient algorithm implementations with low T-count. The approach is illustrated in Figure 1.

AlphaTensor-Quantum addresses three main challenges that go beyond the capabilities of AlphaTensor [24] when applied on this problem. First, we need to optimize the *symmetric* tensor rank, as opposed to the standard notion of tensor rank. Second, the approach must extend to *large tensor sizes*, since the size of the tensor corresponds directly to the number of qubits in the circuit to be optimized. Third, AlphaTensor cannot leverage domain knowledge that falls outside of the tensor decomposition framework. AlphaTensor-Quantum addresses these three challenges. First, it modifies the RL environment and actions to provide symmetric decompositions (called *Waring decompositions*) of the tensor; this has the beneficial side effect of reducing the action search space. Second, it scales up to larger tensors due to its innovative neural network architecture featuring *symmetrization layers*. Third, it incorporates domain knowledge by leveraging so-called *gadgets* (constructions that can save T gates by using auxiliary ancilla qubits); this is achieved through an efficient procedure embedded in the RL environment that exploits the Toffoli gadget [25] and the controlled S (CS) gadget [26].

We show that AlphaTensor-Quantum is a powerful method for finding efficient quantum circuits. On a benchmark of arithmetic primitives, AlphaTensor-Quantum outperforms all existing methods for T-count optimization, especially when allowed to leverage domain knowledge. For the relevant operation of multiplication in finite fields, which has applications in cryptography [27], AlphaTensor-Quantum finds an efficient quantum algorithm with the same complexity as the *classical* Karatsuba's method [28]. As quantum computations are reversible by nature, naive translations of classical algorithms commonly introduce overhead [29, 30]. AlphaTensor-Quantum thus finds the most efficient quantum algorithm for multiplication on finite fields reported to date. We also optimize quantum primitives for other relevant problems, ranging from arithmetic computations used, e.g., in Shor's algorithm [31], to Hamiltonian simulation in quantum chemistry, e.g., FeMoco simulation [32, 33]. Here, AlphaTensor-Quantum recovers the best known hand-designed solutions, demonstrating it can effectively optimize circuits of interest in a fully automated way. We envision this approach can significantly accelerate discoveries in quantum computation as it saves the numerous hours of research invested in the design of optimized circuits.

These results show that AlphaTensor-Quantum can effectively exploit the domain knowledge that is provided in the form of gadgets and state-of-the-art magic state factories [8], finding constructions that are optimal for the chosen metrics of interest. Because of its flexibility, AlphaTensor-Quantum can be readily extended in multiple ways, e.g., by considering different complexity metrics (other than the T-count), or by incorporating new domain knowledge. Thus, we expect that AlphaTensor-Quantum will become instrumental for automatic circuit optimization with new advancements in quantum computing.

2 AlphaTensor-Quantum for T-count Optimization

We describe AlphaTensor-Quantum, an RL agent to optimize the T-count of quantum circuits. It relies on the relation between T-count optimization and tensor decomposition (see Figure 1), which we briefly review next.

Quantum gates. Quantum computers work with *qubits*. The state of a qubit is representable as a L2-normalized vector in $\mathbb{C}^2$, typically denoted with the *bra-ket* notation as $|\psi\rangle$, with the basis vectors being $|0\rangle$ and $|1\rangle$. An N-qubit state is represented by a normalized vector in $\mathbb{C}^{2^N}$. Quantum circuits perform linear operations over a quantum state that preserve the normalization, thus they can be represented by *unitary matrices*. Quantum circuits are implemented by *quantum gates*.

Quantum operator	#Qubits	Gate symbol	Unitary matrix
Controlled NOT (CNOT)	2		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$
Controlled S (CS)	2		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & i \end{pmatrix}$
Controlled Z (CZ)	2		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$
Controlled controlled Z (CCZ)	3		$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \end{pmatrix}$
Toffoli (Controlled CNOT)	3		$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$

Table 1: Common controlled gates used in quantum computing, along with their symbols and unitary matrices.

Following convention, we focus on the Clifford+T gate set because it is the leading candidate for fault-tolerant implementation and it is approximately universal, meaning that any unitary matrix can be approximated arbitrarily well by a circuit consisting of these gates. In particular, we consider the following gate set: Hadamard (H) and controlled NOT (CNOT), which are Clifford gates, and T gates, given by the unitary matrices

$$T = \begin{pmatrix} 1 & 0 \\ 0 & \mathrm{e}^{i\pi/4} \end{pmatrix}, \qquad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad \text{and} \qquad CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

For convenience, we also define the following Clifford gates:

$$S = T^2, \qquad Z = S^2, \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \text{and} \qquad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}.$$

Each of these gates, except the CNOT, acts on a single qubit. The T, S, and Z gates perform phase operations, e.g., for a quantum state $|x\rangle$ with $x \in \{0, 1\}$, we have $T|x\rangle = \mathrm{e}^{i\frac{\pi}{4}x}|x\rangle$. The CNOT acts on two qubits: it flips the second qubit if the first qubit is in the $|1\rangle$ state, i.e., $CNOT|x, y\rangle = |x, x \oplus y\rangle$, where '$\oplus$' denotes the XOR operation. Controlled gates like the CNOT enable quantum entanglement; these gates are summarized in Table 1. For instance, the controlled CNOT, or Toffoli gate, acts on three qubits and applies the operation $Tof|x, y, z\rangle = |x, y, z \oplus (xy)\rangle$.

Quantum gates can be composed to implement arbitrary unitary operations: serial composition corresponds to matrix multiplication, and parallel composition corresponds to Kronecker product.

Signature tensor. Given a circuit composed of CNOT and T gates alone, we can encode the information about the non-Clifford components into a symmetric *signature tensor* $\mathcal{T} \in \{0,1\}^{N\times N\times N}$, with N being the number of qubits. Indeed, two CNOT + T circuits implement the same unitary matrix, up to Clifford gates, if and only if they have the same signature tensor [16, 18] (see Section 5.1).

To optimize the T-count of a circuit, we first find its representation in terms of the signature tensor $\mathcal{T}$. When the circuit has gates other than CNOT and T gates alone, to obtain the signature tensor we first split the circuit into two parts—one that consists only of Clifford gates, and a diagonal part that contains only CNOT and T gates. We do this using the techniques of Heyfron and Campbell [16] in combination with the algorithm by Vandaele et al. [34] (see Figure 3 for an example and Appendix C.1 for more details).

After that, we obtain $\mathcal{T}$ by first mapping each T gate to a vector $u^{(r)} \in \{0,1\}^N$, which is a straightforward step [35], and then constructing the tensor from the *Waring decomposition* (a symmetric form of the tensor rank decomposition) given by these vectors, i.e.,

$$\mathcal{T} = \sum_{r=1}^{R} u^{(r)} \otimes u^{(r)} \otimes u^{(r)},$$

where R is the number of T gates, '$\otimes$' denotes the outer product, and the additions are under modulo 2.

Crucially, any *alternative* Waring decomposition of $\mathcal{T}$ can *also* be mapped to a quantum circuit, as each vector $u^{(r)}$ corresponds one-to-one to a T gate. The resulting circuit is equivalent to the original one up to Clifford operations [18], albeit with a different number of T gates—according to the number of factors R in the decomposition. Hence, we recast the problem of T-count optimization as finding low-rank decompositions of $\mathcal{T}$.

TensorGame. AlphaTensor-Quantum is an RL-based tensor factorization approach that finds low-rank Waring decompositions of tensors. It builds upon AlphaTensor [24], which uses deep RL to find low-rank decompositions of 3-dimensional tensors. AlphaTensor-Quantum recasts the problem of tensor decomposition into a single-player game, called *TensorGame*, in which at each step s the player observes the *game state*, consisting of a tensor $\mathcal{T}^{(s)}$ and all the past played actions (initially, the tensor $\mathcal{T}^{(1)}$ is set to the signature tensor of interest $\mathcal{T}$, and there are no past actions). At step s, the player selects an *action* consisting of a vector (or factor) $u^{(s)} \in \{0,1\}^N$. Choosing an action affects the game state: the tensor $\mathcal{T}^{(s+1)} = \mathcal{T}^{(s)} - u^{(s)} \otimes u^{(s)} \otimes u^{(s)}$, and $u^{(s)}$ is included as the last played action. The goal of TensorGame is to reach the all-zero tensor in as few moves as possible. The game ends after S moves if that is the case and, by construction, the played actions form a rank-S Waring decomposition of the signature tensor, since $\mathcal{T} = \sum_{s=1}^{S} u^{(s)} \otimes u^{(s)} \otimes u^{(s)}$. (The game also ends after a user-specified maximum number of moves to avoid infinitely long games, in which case the game is declared "unsuccessful" as it did not find a decomposition of the tensor.) To encourage the RL agent to find low-rank decompositions or, equivalently, circuits with lower T-count, games are penalized based on the number of moves taken to reach the all-zero tensor (with an additional penalty for unsuccessful games): every played move incurs a reward of -1.

In AlphaTensor-Quantum, each action consists of a single factor $u^{(s)}$. This represents a significant advantage over standard AlphaTensor (where actions consist of factor triplets), because for a fixed tensor size N,

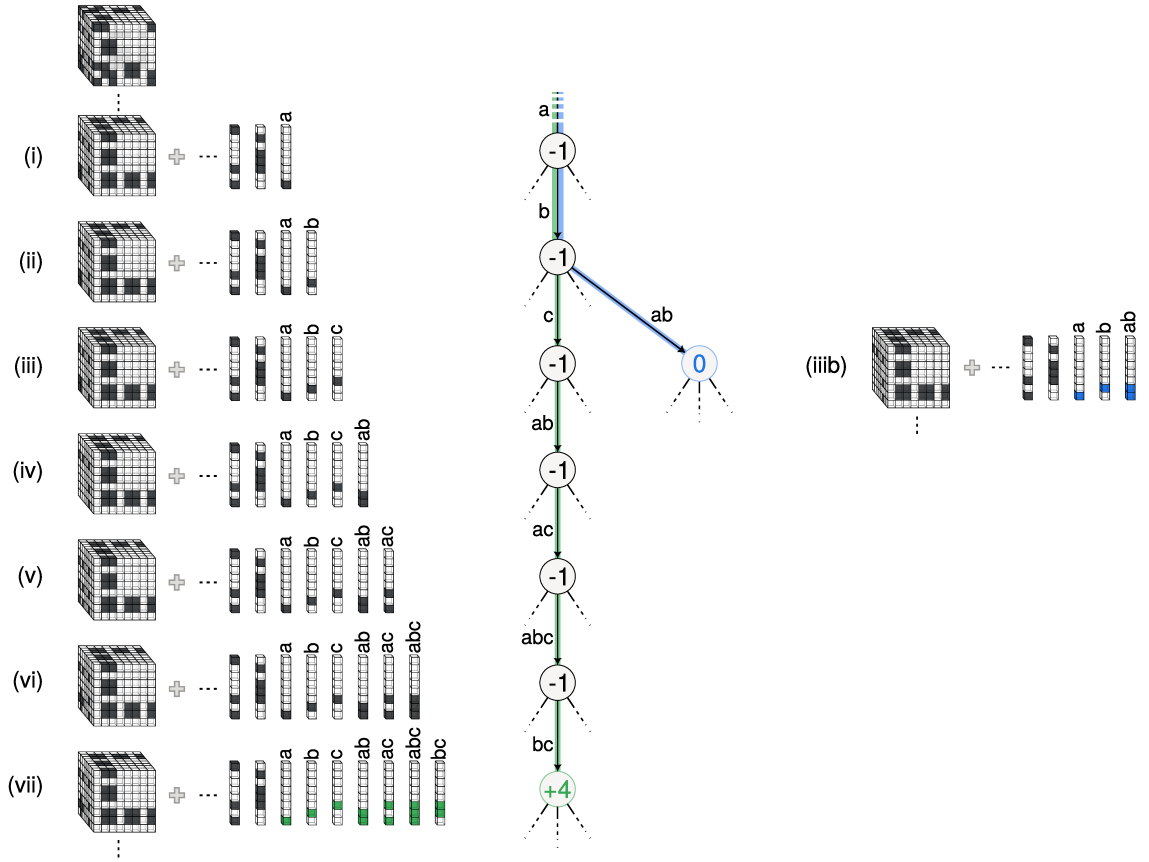


Figure 2: Illustration of gadgetization in TensorGame. Nodes in the tree correspond to states, and the number in each node is the immediate reward associated to the action leading to that node. In state (i), the last played action is labeled "a", and state (ii) is reached after playing another action (labeled "b"). From state (ii), playing action "c" leads to state (iii), incurring an additional -1 reward. If action "ab" is played instead from state (ii), the reward leading to state (iiib) is 0 because the move completes a CS gadget (blue path). Similarly, the sequence of moves in the green path completes a Toffoli gadget, and thus the immediate reward in state (vii) is $+4$ so that the last seven actions jointly receive a reward of -2 (see Section 5.2 for details on the gadgetization patterns). With its ability to plan, from state (i) the agent may decide to play actions in the green or blue paths and benefit from the adjusted rewards, or to play other actions that are not part of a gadget (black dashed paths). In this way, AlphaTensor-Quantum can automatically find trade-offs between playing actions that are part of a gadget and actions that are not.

the action space reduces from 2^{3N} to 2^N. This is one of the key properties that allows AlphaTensor-Quantum to scale up to tensors larger than the ones considered by Fawzi et al. [24]; we consider tensors of size up to $N = 72$ in Section 3.

Gadgets. In contrast to other state-of-the-art T-count optimization approaches, AlphaTensor-Quantum can leverage *gadgetization* techniques to find more efficient constructions. In this context, a gadget is an alternative implementation of a quantum gate that reduces the number of T gates at the cost of introducing additional (ancilla) qubits and operations such as Clifford gates and measurement-based corrections. (These extra qubits do not represent a significant limitation in practice, as they can be reused throughout the circuit.)

We consider two such gadgets. First, the Toffoli gate, a quantum gate that needs at least seven T gates [15] to implement without ancillae. The Toffoli gate admits a gadget to implement it using four T gates [25], or when considering a state-of-the-art magic state factory [8], a gadget to implement it at a cost equivalent to that of two T gates. Second, the CS gate, which can be built with a cost equivalent to that of two T gates due to the magic state factory, instead of the three T gates of its direct implementation [26, Appendix A.3].

Although the gadgetization techniques fall outside of the tensor decomposition framework, AlphaTensor-Quantum incorporates them into TensorGame as follows. Every time that an action is played, we check whether it completes a Toffoli when combined with the six previous actions, a CS gate when combined with the two previous actions, or none of them. (These checks are up to Clifford operations and they simply involve finding linear dependencies among the factors; see Section 5.2 and appendix C.3 for details.) If it completes a Toffoli, we assign a positive reward, so that the seven actions jointly incur a reward of -2 (corresponding to two T gates) as opposed to the reward of -7 corresponding to the un-gadgetized implementation. If it completes a CS, we adjust the reward so that the three actions jointly incur a reward of -2. See Figure 2 for an illustration of a TensorGame with gadgetization.

Other than the reward signal, AlphaTensor-Quantum has no further information about the gadgets. It learns to recognize and exploit those patterns of actions from the data, and it finds good trade-offs between playing actions that are part of a gadget and actions that are not.

3 Results

Section 3.1 demonstrates the effectiveness of AlphaTensor-Quantum on a benchmark of circuits widely used in the literature of T-count optimization, and Section 3.2 shows applications of AlphaTensor-Quantum in different domains.

3.1 Benchmark Circuits

We consider a benchmark set of quantum circuits originally proposed by Amy et al. [14] and widely used in the literature of T-count optimization [16–18, 23]. We take the circuits from Amy [36] (we also include GF(2^2)-mult and GF(2^3)-mult for completeness; see Appendix D.1) and apply two versions of AlphaTensor-Quantum: the full approach that includes gadgetization, and an alternative agent that does not consider any gadgets. We report the resulting T-count for each approach in Table 2.

Circuit compilation. We apply a standard compilation technique [16, 23] with an improvement [34] (see Appendix C.1 for details and Figure 3 for an example). To make the circuits amenable to the tensor decomposition approach, we replace the Hadamard gates by extra ancilla qubits and Clifford gates (we refer to this as *Hadamard gadgetization* [16] to distinguish it from the gadgetization described in Section 2). For circuits exceeding 60 qubits after this process, we split them into smaller sub-circuits of fewer than 60 qubits each, and apply AlphaTensor-Quantum on each sub-circuit independently. We split the circuits at the boundaries corresponding to Hadamard gates via a random-greedy algorithm designed to minimize the number of splits (see Appendix C.2).

Results. In Table 2, we compare the results of the non-gadgetized version of AlphaTensor-Quantum against the best T-count reported in the literature [14, 16–19, 22, 23, 37, 38], as the baselines do not consider any gadgets either. For the circuits that did not require splitting, AlphaTensor-Quantum matches or outperforms all the existing T-count optimization methods. For all small circuits (at most 20 T gates), we confirmed with the Z3 theorem prover [39] that AlphaTensor-Quantum obtains the best possible T-count achievable with a tensor decomposition approach. For the circuits that exceed 60 qubits, AlphaTensor-Quantum outperforms the baselines for Hamming_{15} (high) and Mod-Adder_{1024}, but not for the others, likely due to the lack of optimality of the splitting technique.

When considering gadgets, AlphaTensor-Quantum leverages the Toffoli and CS gates to *further drastically reduce* the T-count. When compared against the original construction of the circuits, consisting mostly of Toffoli gates, AlphaTensor-Quantum reduces the T-count in all except two cases (likely due to the splitting). When compared against the baselines, AlphaTensor-Quantum reduces the T-count for all circuits except Grover_5. Remarkably, for the GF(2^m)-mult circuits, AlphaTensor-Quantum finds constructions with significantly smaller T-count than the baselines or the original constructions; we discuss this further in Section 3.2.

Circuit	#Qubits		T-count without gadgets		T-count with gadgets	
	Original	Compiled	Baselines	AlphaTensor-Quantum	Original	AlphaTensor-Quantum
8-bit adder	24	⋆	**129** [16]	139	114 (57Tof)	**94** (33Tof + 28T)
Barenco Toff_3	5	8	13 [22, 23]	13*	8 (4Tof)	**4** (2Tof)
Barenco Toff_4	7	14	24 [16, 22]	**23**	16 (8Tof)	**8** (4Tof)
Barenco Toff_5	9	20	34 [16, 22]	**33**	24 (12Tof)	**12** (6Tof)
Barenco Toff_{10}	19	50	84 [16]	**83**	64 (32Tof)	**32** (16Tof)
CSLA-MUX_3	15	21	40 [22]	**39**	20 (10Tof)	**16** (8Tof)
CSUM-MUX_9	30	42	72 [16, 19]	**71**	56 (28Tof)	**28** (14Tof)
$\text{GF}(2^2)$-mult	6	6	17†	17*	8 (4Tof)	**6** (3Tof)
$\text{GF}(2^3)$-mult	9	9	31†	**29**	18 (9Tof)	**12** (6Tof)
$\text{GF}(2^4)$-mult	12	12	47 [22]	**39**	32 (16Tof)	**18** (9Tof)
$\text{GF}(2^5)$-mult	15	15	84 [22]	**59**	50 (25Tof)	**26** (13Tof)
$\text{GF}(2^6)$-mult	18	18	118 [22]	**77**	72 (36Tof)	**36** (18Tof)
$\text{GF}(2^7)$-mult	21	21	167 [23]	**104**	98 (49Tof)	**44** (22Tof)
$\text{GF}(2^8)$-mult	24	24	214 [19]	**123**	128 (64Tof)	**58** (29Tof)
$\text{GF}(2^9)$-mult	27	27	295 [16]	**161**	162 (81Tof)	**70** (35Tof)
$\text{GF}(2^{10})$-mult	30	30	350 [16]	**196**	200 (100Tof)	**92** (46Tof)
Grover_5	9	⋆	**44** [16]	152	96 (48Tof)	**66** (27Tof + 12T)
Hamming_{15} (high)	20	⋆	787† (1010 [16])	**773**	702 (351Tof)	**440** (173Tof + 2CS + 90T)
Hamming_{15} (low)	17	42	75 [16]	**73**	46 (23Tof)	**34** (17Tof)
Hamming_{15} (med)	17	⋆	156† (162 [16])	156	164 (82Tof)	**78** (35Tof + 8T)
HWB_6	7	27	51 [16]	51	30 (15Tof)	**20** (10Tof)
Mod-Adder_{1024}	28	⋆	798† (978 [16])	**762**	570 (285Tof)	**500** (141Tof + 15CS + 188T)
Mod-Mult_{55}	9	11	17 [16]	17*	14 (7Tof)	**6** (3Tof)
Mod-Red_{21}	11	28	55 [16, 22]	**51**	34 (17Tof)	**22** (11Tof)
Mod 5_4	5	5	7 [19, 22, 23]	7*	8 (4Tof)	**2** (1Tof)
QCLA-Adder_{10}	36	⋆	**116** [16]	135	**68** (34Tof)	94 (28Tof + 5CS + 28T)
QCLA-Com_7	24	42	59 [16]	59	58 (29Tof)	**24** (12Tof)
QCLA-Mod_7	26	⋆	**165** [16]	199	**118** (59Tof)	122 (43Tof + 36T)
QFT_4	5	43	55 [16]	**53**	59 (2Tof + 55T)	**44** (4Tof + 3CS + 30T)
RC-Adder_6	14	24	37 [16, 22]	37	22 (11Tof)	**12** (6Tof)
NC Toff_3	5	7	13 [16, 22, 23]	13*	6 (3Tof)	**4** (2Tof)
NC Toff_4	7	11	19 [16, 22, 23]	19*	10 (5Tof)	**6** (3Tof)
NC Toff_5	9	15	25 [16]	25	14 (7Tof)	**8** (4Tof)
NC Toff_{10}	19	35	55 [16]	55	34 (17Tof)	**18** (9Tof)
VBE-Adder_3	10	14	20 [16, 22, 23]	**19***	20 (10Tof)	**6** (3Tof)

Table 2: T-count achieved by different methods on a set of benchmark circuits. Even without gadgetization, AlphaTensor-Quantum matches or outperforms the considered baselines for all circuits that have not been split into sub-circuits (split circuits are marked with ⋆). When considering gadgets, AlphaTensor-Quantum further reduces the T-count drastically, and generally outperforms the original constructions with gadgets. The notation "aTof+bCS+cT" indicates a circuit with a Toffoli gates, b CS gates, and c T gates (when multiple solutions from AlphaTensor-Quantum achieve the same T-count but differ in the trade-off between gadgets and T gates, we report the result with the highest number of Toffoli gates). Baseline results marked with † were obtained with the methods in Appendix B and are better than the best published T-count (provided in parentheses where applicable). Results marked with * were proven to be optimal decompositions of the given tensor using the Z3 theorem prover.

3.2 Applications

We applied AlphaTensor-Quantum to circuits derived from several important use-cases for quantum computing, which fit into two broad categories: quantum implementations of classical reversible logic for arithmetic

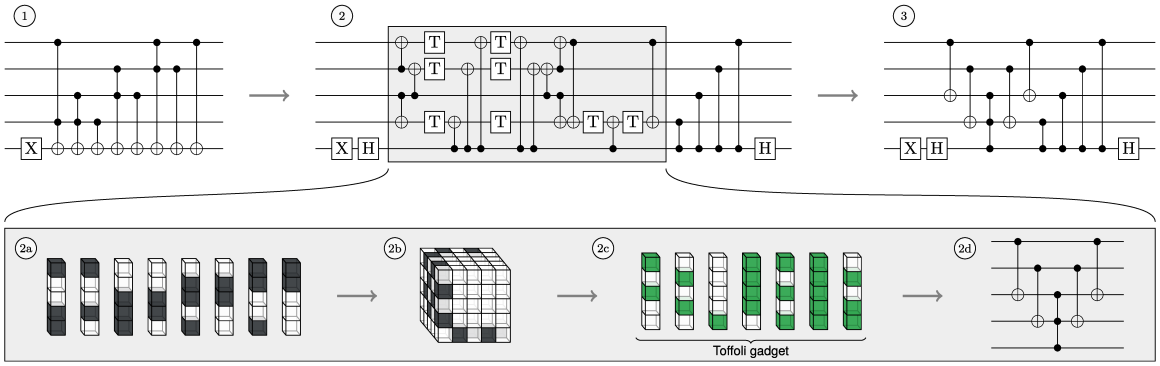
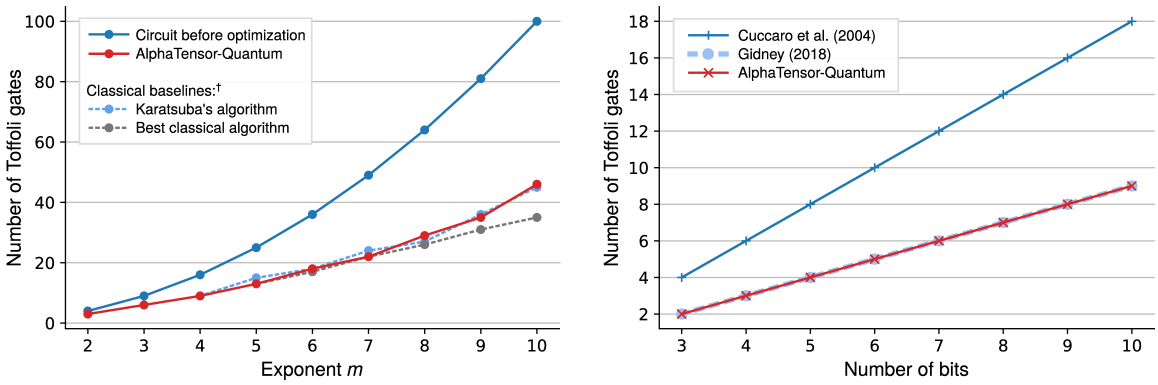


Figure 3: Results of AlphaTensor-Quantum on the Mod 5_4 circuit. **(1)** The original quantum circuit, which can be implemented with 4 Toffoli gates (with an equivalent cost of 8 T gates). **(2)** The compiled circuit, which has been split into Clifford gates and a subcircuit of just CNOT and T gates (in the highlighted box). **(2a)** The Waring decomposition corresponding to the highlighted CNOT+T circuit: each T gate corresponds precisely to one of the factors displayed as columns of blocks. **(2b)** The signature tensor formed from this decomposition. **(2c)** An alternative Waring decomposition of the tensor found by the RL agent. Again, each factor corresponds one-to-one to a T gate; however in this case the seven factors are grouped into a single Toffoli gadget. **(2d)** The quantum circuit obtained from this new decomposition. **(3)** The optimized circuit after having applied AlphaTensor-Quantum, which can be implemented using Clifford gates and a single Toffoli gate (i.e., its equivalent T-count is 2).

operations, and primitives important to quantum chemistry. We study the former because many quantum algorithms (for instance, Shor's algorithm and many variations of Grover's algorithm) require a quantum implementation of a classical oracle that performs some arithmetical or logical operation. We study the latter because quantum chemistry is a major application of quantum computing, and many of the state-of-the-art methods share similar circuit constructions.

Multiplication in finite fields. Multiplication in finite fields is a crucial operation for cryptography, since many elliptic-curve cryptography (ECC) implementations are built over such finite fields, and in fact these circuits are derived from a quantum algorithm that cracks ECC in polynomial time [27].

The results from Table 2 show that AlphaTensor-Quantum excels in optimizing the primitives for multiplication in finite fields of order 2^m, i.e., the circuits GF(2^m)-mult. In Figure 4a, we show that for $m = \{2, 3, 4, 5, 7\}$, AlphaTensor-Quantum actually achieves the same Toffoli gate count as the best known *upper bound* for the equivalent classical (non-reversible) circuits [40, 41]. For $m = \{2, 3, 4, 5\}$ this is also the best known *lower bound* [42], which suggests that the resulting quantum circuits are likely optimal. While the circuits for these targets were constructed using the naive $O(m^2)$ multiplication algorithm, the number of Toffoli gates of the optimized circuits follows $\sim m^{\log_2(3)}$ for the considered values of m. Thus, AlphaTensor-Quantum found a multiplication algorithm with the same complexity as Karatsuba's method [28], a *classical* algorithm for multiplication in finite fields. While it is conceivable to construct the resulting circuits by hand in principle, to the best of our knowledge no sub-quadratic constructions of quantum circuits for multiplication in finite fields have been discussed in the literature. AlphaTensor-Quantum finds them automatically, similar to Gidney's



(a) **(b)**

Figure 4: Number of Toffoli gates of the optimized circuits found by AlphaTensor-Quantum. **(a)** For multiplication in finite fields of order 2^m, AlphaTensor-Quantum finds efficient circuits that significantly outperform the original construction, which scales like $O(m^2)$. The number of Toffoli gates matches the best known lower bound of the *classical* circuits [42] for some values of m, and scales as $\sim m^{\log_2(3)}$, showing that AlphaTensor-Quantum found an algorithm with the same complexity as Karatsuba's method [28], a classical algorithm for multiplication on finite fields for which a quantum version has not been reported in the literature. (The baselines marked with † are classical circuits, and hence not directly comparable, as naive translations of classical to quantum circuits commonly introduce overheads. To compare, we assume the number of effective Toffoli gates is the number of 1-bit AND gates in the classical circuit.) **(b)** For binary addition, AlphaTensor-Quantum halves the cost of the circuits from Cuccaro et al. [43] matching the state-of-the-art circuits from Gidney [31]. Remarkably, it does so automatically without any prior knowledge of the measurement-based uncomputation technique, which was crucial to their results.

translation of Karatsuba's method for multiplication of integers [30], but in contrast to the latter without requiring additionally ancilla qubits.

Binary addition. We apply AlphaTensor-Quantum to two different quantum implementations of the binary addition operation, which is a relevant primitive in oracle construction because many other common operations are built from it (such as multiplication, comparators, and modular arithmetic). For example, controlled addition circuits are the dominant cost of Shor's algorithm. There are many types of quantum addition circuits (ripple-carry, carry-save, or block-carry-lookahead); we focus on ripple-carry addition circuits because they have fewer ancilla qubits generated by Hadamard gadgetization.

In particular, we optimize the circuits of Cuccaro et al. [43], which have a Toffoli count scaling as $2n + O(1)$ where n is the number of input bits, as well as the circuits of Gidney [31], with a Toffoli count of $n + O(1)$. As shown in Figure 4b, AlphaTensor-Quantum reduces the Toffoli count of Cuccaro et al.'s circuits, matching the Toffoli count of Gidney's construction of $n + O(1)$.

Prior to the publication of Gidney's results, Cuccaro et al.'s circuits attained the lowest Toffoli count of any addition circuit. To halve their cost, Gidney introduced a new approach called *measurement-based uncomputation*, which allows Toffoli gates that were performed on zero-initialized ancilla qubits to be uncomputed for free under certain conditions, by introducing measurements and classically-controlled Clifford

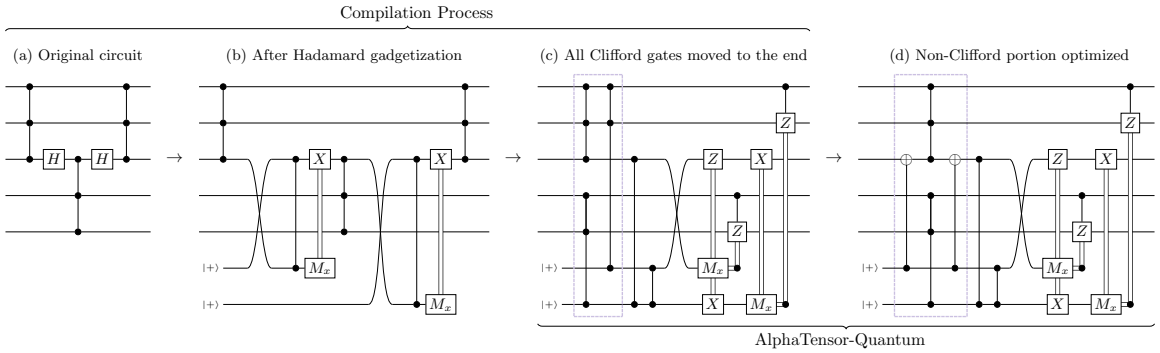


Figure 5: An example in which measurement-based uncomputation tricks can be recovered by AlphaTensor-Quantum. Here, a portion of the NC Toff_3 circuit is mapped to three CCZ gates, a pair of which share two inputs (in the highlighted box), by the Hadamard gadgetization process (detailed in Appendix C.1). AlphaTensor-Quantum optimizes this pair by expressing it as a single gadget, saving one Toffoli gate with respect to the original construction. This kind of optimization happens often because CCZ gates can usually be moved freely along the circuit, as the corresponding unitary matrix will be diagonal after compilation. This is reminiscent of the temporary logical-AND construction from Gidney [31], although some patterns with more than two CCZ gates can also be combined in this way.

gates. This non-trivial technique was invented more than a decade after the addition circuits of Cuccaro et al. [43], but is now widely used for optimizing fault-tolerant quantum circuits.

Importantly, for both circuit constructions, AlphaTensor-Quantum had no knowledge of measurement-based uncomputation, and all Toffoli gates were compiled naively using the standard unitary construction with seven T gates. Yet, AlphaTensor-Quantum automatically finds optimized circuits only accessible via measurement-based uncomputation. It does so with its ability to find Toffoli gadgets, by exploiting the fact that ancilla qubits from the Hadamard gadgetization allow the movement of Toffoli gates; see Figure 5 for an example.

Hamming weight and phase gradients. Applying small angle rotations to qubits is a common operation in quantum algorithms, e.g., in the quantum Fourier transform, data loading, and quantum chemistry. In fault-tolerant quantum circuits, any rotation can be implemented using the Solovay-Kitaev algorithm with a sequence of Hadamard and T gates. However, if the same small angle is to be applied repeatedly (such as in quantum chemistry), it is often more efficient to use a technique called Hamming weight phasing.

We use AlphaTensor-Quantum to optimize the Hamming weight phasing circuits from Gidney [31]. While the original circuits use measurement-based uncomputation, we instead compile a version where every Toffoli gate is constructed explicitly, as before. Again, AlphaTensor-Quantum halves the cost with respect to the naive input circuits, thus matching the complexity of the circuits with measurement-based uncomputation tricks.

Unary iteration. We also apply AlphaTensor-Quantum to a family of circuits called *unary iteration* [33]. These circuits allow selecting which Pauli operator (from a set of operators) to apply on the data qubits, based on the value of some control qubits. We apply this construction to the Pauli operators $P_1 = XI\cdots I$, $P_2 = IX\cdots I$, $\ldots$, $P_m = II\cdots X$ on $m = 8$, 16, and 32 qubits, therefore implementing the quantum equivalent of a classical n-to-2^n demultiplexer or decoder for $n = 3, 4, 5$ (see Figure 6). These circuits could be used to

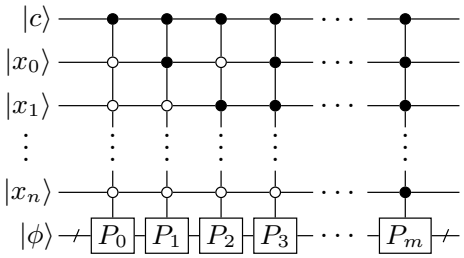


Figure 6: The action implemented by the unary iteration family of circuits. If the control qubit c is in the state $|1\rangle$, the circuit applies one of the Pauli operators from the set $\{P_1, \ldots, P_m\}$ to the multiqubit state $|\phi\rangle$, depending on the value of the qubits x_1 to x_n. If the control qubit c is in the state $|0\rangle$, no operation is applied to $|\phi\rangle$.

implement the same construction for any set of 8, 16, or 32 Pauli operators, without extra T-count, and because of their generality, they appear both in oracle construction (demultiplexers are commonly used in classical logic) and in quantum chemistry (as in the linear combination of unitaries method discussed further below).

The unary iteration circuits were implemented following Babbush et al. [33], except we compiled the Toffoli gates unitarily with seven T gates instead of using measurement-based uncomputation. The largest circuit ($n = 5$) has 72 qubits after compilation. As before, AlphaTensor-Quantum finds its own version of measurement-based uncomputation to match the best known T-count of these circuits.

We also apply a version of AlphaTensor-Quantum without gadgets. Remarkably, it finds optimized circuits that, for each value of n, have only three T gates more than the circuits from Babbush et al. [33] (where each Toffoli gate is implemented using four T gates), suggesting that the unary iteration circuits have an internal structure particularly amenable to this kind of optimization.

Quantum chemistry. Molecular simulation involves solving the Schrödinger equation, which requires a computational cost scaling exponentially with the number of particles in the system. This makes classical methods struggle to simulate strongly correlated molecules [44]. Quantum computers can theoretically handle these calculations more efficiently, with one compelling use case being the simulation of the FeMoco molecule, the active site in the nitrogenase enzyme responsible for nitrogen fixation. This use case has high potential industrial impact, but is challenging due to the highly complex electronic structure of FeMoco [45].

FeMoco simulation motivated Lee et al.'s [46] fault-tolerant implementation of a quantum walk-based algorithm. In this method, the Hamiltonian associated with the molecule is encoded into the quantum circuit with a technique known as *linear combination of unitaries* (LCU). This framework requires the implementation of two oracles called PREPARE and SELECT (each being an instance of the unary iteration operation discussed earlier). These are repeated many times, making up the dominant cost of the whole algorithm, and therefore optimizing these oracles can significantly impact the total resource requirements (see Appendix A for more details on the algorithm).

We run AlphaTensor-Quantum on portions of Lee et al.'s SELECT oracle construction (which is itself based on von Burg et al. [47]). Specifically, we set a range of parameters to keep the total qubit count below 60, and compile angle rotations of the state into the diagonal basis using the phase-gradient technique from Gidney [31]; see Figure 7 for details. As before, we remove all measurement-based uncomputation tricks and compile

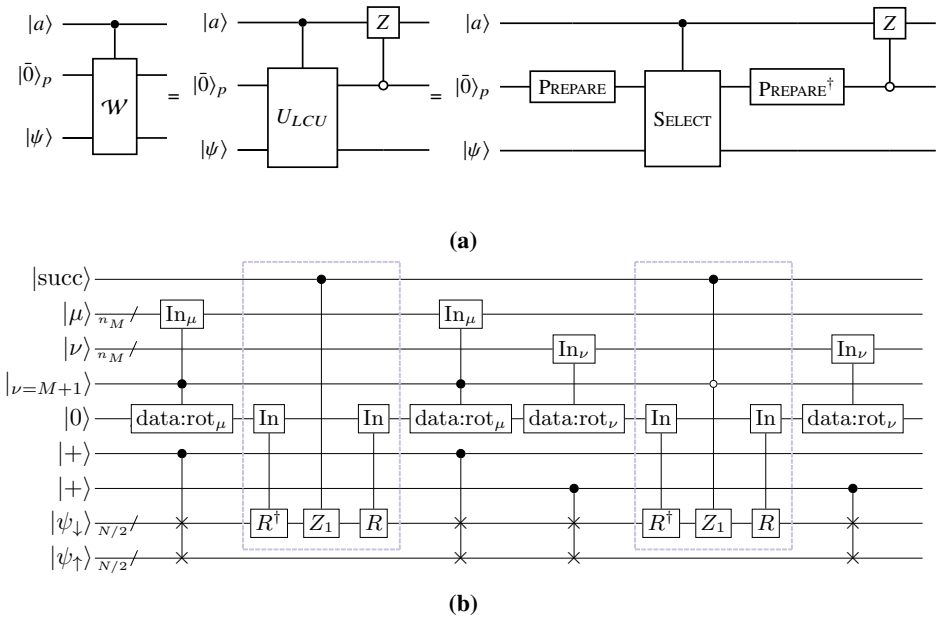


Figure 7: **(a)** The quantum walk operator $\mathcal{W}$ at the heart of state-of-the-art quantum chemistry algorithms [33, 46, 48] (see Figure 9) can be decomposed using the LCU technique into two oracles, PREPARE and SELECT, which encode the details of the molecule. This operator is repeated many times, so its cost dominates the overall algorithm. **(b)** Construction of the SELECT oracle. This figure is adapted from Lee et al. [46, Figure 5, pp. 18]. We compile and optimize the part in the dashed box, along with its constituent components, for a range of parameters (up to 10 bits of precision per rotation, and for systems of up to 10 spin-orbitals).

the Toffoli gates unitarily with seven T gates. We find that, for all parameter values, AlphaTensor-Quantum is able to match the T-count of Lee et al. [46], which relies heavily on measurement-based uncomputation.

Thus, AlphaTensor-Quantum automatically finds state-of-the-art constructions for multiple use cases. We expect that, as new improvements on quantum chemistry are made at a structural level, AlphaTensor-Quantum will be able to automatically find the best constructions (in terms of T-count), without additional human effort.

4 Discussion

We have demonstrated the effectiveness of AlphaTensor-Quantum at optimizing the T-count, a key metric in quantum computations, showing that it significantly outperforms existing approaches, being able to automatically discover constructions as efficient as the best human-designed ones across various applications. For those applications, the fact that very different techniques achieve the same T-count might hint to the optimality of the discovered constructions. In fact, although AlphaTensor-Quantum does not come with optimality guarantees, we proved that some decompositions in Section 3.1 are optimal in terms of the number of factors of the decomposition.

Compared to previous works that use RL to optimize quantum circuits [49–51] or to improve variational quantum algorithms [52–56], Compared to previous works that use RL to optimize quantum circuits [49–51],

AlphaTensor-Quantum is the the first one that scales to a large enough number of qubits to cover a wide range of applications (see Section 3.2), and the first one to consider fault-tolerant compilation. Compared to existing T-count optimization approaches, which are not RL-based, AlphaTensor-Quantum significantly outperforms them; however this comes at the cost of increased computational complexity (e.g., it took around 8 hours to optimize the addition circuits of Cuccaro et al. [43]) and loss of optimality due to extra compilation steps (e.g., if the circuits need to be split into parts). A possible future research avenue to accelerate the algorithm is to improve the neural network architecture; another avenue to scale up further is to handle Hadamard gadgetization differently, so that the number of qubits does not increase as much during the compilation process, or to apply RL to alternative circuit representations that can optimize Hadamard gates natively. However, we argue that its comparably increased computational complexity is outweighed by its results, especially considering the cost of manually designing optimized constructions. Moreover, it is a one-off cost: once AlphaTensor-Quantum has been applied, the optimized circuits can be reused.

Unlike existing approaches for T-count optimization, AlphaTensor-Quantum can leverage and exploit gadgets. These constructions are an active area of research [25, 26, 57, 58], and as fundamentally new gadgets are discovered, they can be readily incorporated into the RL environment, possibly allowing AlphaTensor-Quantum to discover even more efficient constructions for established quantum circuits. Similarly, due to its flexibility, AlphaTensor-Quantum can be extended to optimize metrics other than the T-count, such as T-depth [35] or weighted combinations of different types of gates [17], by changing the reward signal accordingly.

We envision AlphaTensor-Quantum will play a pivotal role as quantum chemistry and related fields advance, and new improvements at a structural level emerge. Similarly to how Gidney [31] optimized the addition circuits of Cuccaro et al. [43] over a decade later, AlphaTensor-Quantum finds and exploits non-trivial patterns, thus eliminating the need for manual construction design and saving research costs.

5 Methods

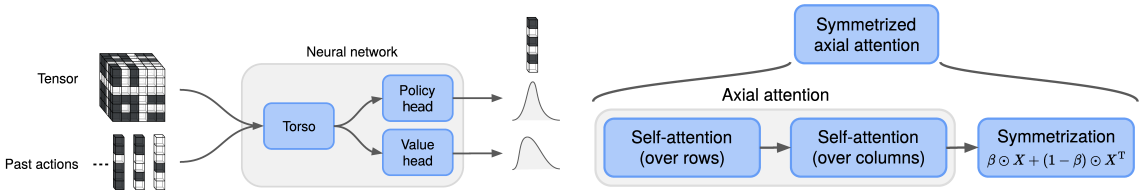
5.1 The Signature Tensor

The tensor representation of a circuit relies on the concept of *phase polynomials*. Consider a circuit with CNOT and T gates alone. Its corresponding unitary matrix U performs the operation $U|x_{1:N}\rangle = e^{i\phi(x_{1:N})}|Ax_{1:N}\rangle$, where $\phi(x_{1:N})$ is the phase polynomial and A is an invertible matrix that can be implemented with Clifford gates only. For example, applying a T gate to the second qubit after a CNOT gives $(I \otimes T)CNOT|x, y\rangle = e^{i\frac{\pi}{4}(x\oplus y)}|x, x \oplus y\rangle$, i.e., $\phi(x, y) = \frac{\pi}{4}(x \oplus y) = \frac{\pi}{4}(x + y - 2xy)$. After cancelling out the terms leading to multiples of 2π, the phase polynomial takes a multilinear form, i.e., $\phi(x_{1:N}) = \frac{\pi}{4}\left(\sum_i a_i x_i + 2\sum_{i<j} b_{ij}x_i x_j + 4\sum_{i<j<k} c_{ijk}x_i x_j x_k\right)$, where $a_i \in \{0, \ldots, 7\}$, $b_{ij} \in \{0, \ldots, 3\}$, and $c_{ijk} \in \{0, 1\}$. This maps directly to a circuit of T, CS and CCZ gates: each $a_i x_i$ term is implemented by a_i T gates acting on the i-th qubit, each $b_{ij}x_i x_j$ term corresponds to b_{ij} CS gates on qubits i and j, and each non-zero $c_{ijk}x_i x_j x_k$ term is a CCZ gate on qubits i, j, and k. Each of these terms corresponds to a non-Clifford operation only if the corresponding constant (a_i, b_{ij}, or c_{ijk}) is odd. Thus, two CNOT + T circuits implement the same unitary up to Clifford gates if and only if they have the same phase polynomial after taking the modulo 2 of their coefficients. This information about the polynomial can then be captured in a symmetric 3-dimensional binary *signature tensor* $\mathcal{T}$ of size $N \times N \times N$ [16, 18]. See Appendix A for more details.

5.2 System Description

AlphaTensor-Quantum builds on AlphaTensor, which is in turn based on AlphaZero [59]. It is implemented over a distributed computing architecture, composed of one *learner* and multiple *actors* that communicate asynchronously. The learner is responsible for hosting a neural network and updating its parameters, while the main role of the actors is to play games in order to generate data for the learner. When the learner updates the network parameters by gradient descent steps, it sends them to the actors. The actors employ Monte Carlo tree search (MCTS), using queries to the network to guide their policy, and play games whose actions tend to improve over the policy dictated by the neural network. Played games are sent to the learner and stored in a buffer, and they will serve as future data points to train the network parameters.

Neural network. The neural network at the core of AlphaTensor-Quantum takes as input the game state and outputs: (i) a policy, i.e., a probability distribution over the next action to play, and (ii) a probability distribution over the estimated return (i.e., the sum of rewards from the current state until the end of the game); see Figure 8a. The neural network has a key component, the *torso*, where a set of attention operations [60] are applied. In AlphaTensor-Quantum, the torso interleaves two types of layers: axial attention [61] and *symmetrization layers* (Figure 8b). Symmetrization layers are inspired by the empirical observation that, for a symmetric input $X \in \mathbb{R}^{N\times N}$ (with $X = X^\top$), preserving the symmetry of the output in-between axial attention layers significantly improves performance. Therefore, after each axial attention layer, we add a symmetrization operation, defined as $X \leftarrow \beta \odot X + (1 - \beta) \odot X^\top$, where '$\odot$' denotes element-wise product. For $\beta = \frac{1}{2}$, this makes the output X symmetric. In practice, we found that letting β be a learnable $N \times N$ matrix improves performance even further, even though X is no longer symmetric, due to the ability of the network to exchange information between rows and columns after each axial attention layer. (When the input is a tensor, $X \in \mathbb{R}^{N\times N\times C}$, we apply the symmetrization operations independently across the dimensions C, using



(a) **(b)**

Figure 8: **(a)** An overview of the neural network, where the input is the current state (tensor and past actions) and the outputs are distributions over the next action to play (given by the policy head) and over the estimated return (given by the value head). **(b)** A symmetrized axial attention layer, the building block of the neural network torso. Symmetrized axial attention incorporates a symmetrization operation after each axial attention layer to favour exchange of information between rows and columns.

the same matrix β.) We refer to this architecture as *symmetrized axial attention*. See Appendix C.4 for more details of the full neural network architecture.

Symmetrized axial attention performs better than the neural network of standard AlphaTensor (which requires attention operations across every pair of modes of the tensor) [24]. It is also one of the key ingredients that allow AlphaTensor-Quantum to scale up to larger tensors: for $N = 30$ and the same number of layers, symmetrized axial attention runs about 4x faster and reduces the memory consumption by a factor of 3x. (See also Appendix D.3 for further ablations.) We believe symmetrized axial attention may be useful for other applications beyond AlphaTensor-Quantum.

Gadgetization patterns. In TensorGame, every time an action is played, we check whether it completes a gadget to adjust the reward signal accordingly. If the action $u^{(s)}$ at step s does not complete a gadget, the immediate reward is simply -1. Otherwise, the reward is +4 if it completes a Toffoli, or 0 if it completes a CS gadget. These checks are done up to Clifford operations, and they involve finding certain linear dependencies between $u^{(s)}$ and the previous actions:

- If $s \geq 7$, we check whether the action $u^{(s)}$ completes a *Toffoli gadget*. Let $a \triangleq u^{(s-6)}$, $b \triangleq u^{(s-5)}$, and $c \triangleq u^{(s-4)}$. If the vectors a, b, and c are linearly independent and all the following relationships hold, then we declare that $u^{(s)}$ completes a Toffoli gadget:

$$u^{(s-3)} = a + b, \qquad u^{(s-2)} = a + c, \qquad u^{(s-1)} = a + b + c, \qquad \text{and} \qquad u^{(s)} = b + c.$$

- If $s \geq 3$, we check whether the action $u^{(s)}$ completes a *CS gadget*. Let $a \triangleq u^{(s-2)}$ and $b \triangleq u^{(s-1)}$. If the vectors a and b are linearly independent and $u^{(s)} = a + b$, then $u^{(s)}$ completes a CS gadget.

We refer to the relationships above as *gadgetization patterns* (see Appendix C.3 for a justification of these steps). Each factor (action) may belong to at most one gadget; thus, if the action at step s completes a gadget of any type, we only check for a new CS gadget from step $s + 3$ onward, and for a new Toffoli gadget from step $s + 7$ onward.

Synthetic demonstrations. Like its predecessor, AlphaTensor-Quantum uses a data set of randomly generated tensor/factorization pairs to train the neural network. Specifically, the network is trained on a mixture of a

supervised loss (which encourages it to imitate the synthetic demonstrations) and a standard RL loss (which encourages it to learn to decompose the target quantum signature tensor). For each experiment, we generate 5 million synthetic demonstrations. To generate each demonstration, we first randomly sample the number of factors R from a uniform distribution in $[1, 125]$ and then sample R binary factors $u^{(r)}$, such that each element has a probability of 0.75 of being set to 0.

After having generated the factors, we randomly overwrite some of them to incorporate the patterns of the Toffoli and CS gadgets, in order to help the network learn and identify those patterns. For each demonstration, we include at least one gadget with probability 0.9. The number of gadgets is uniformly sampled in $[1, 15]$ (the value is also truncated when R is not large enough), and for each gadget we sample the starting index r and the type of gadget (Toffoli with probability 0.6 and CS with probability 0.4), overwriting the necessary factors according to the gadget's pattern. We ensure gadgets do not overlap throughout the demonstration.

Change of basis. To increase the diversity of the played games, the actors apply a random change of basis to the target tensor $\mathcal{T}$ at the beginning of each game. From the quantum computation point of view, a change of basis can be implemented by a Clifford-only circuit, and therefore this procedure does not affect the resulting number of T gates.

A change of basis is represented by a matrix $M \in \{0, 1\}^{N \times N}$ such that $\det(M) = 1$ under modulo 2. We randomly generate 50 000 such basis changes and use them throughout the experiment, i.e., the actor first randomly picks a matrix M and then applies the following transformation to the target tensor $\mathcal{T}$:

$$\widetilde{\mathcal{T}}_{ijk} = \sum_{a=1}^{N} \sum_{b=1}^{N} \sum_{c=1}^{N} M_{ia} M_{jb} M_{kc} \mathcal{T}_{abc}.$$

The actor then plays the game in that basis (i.e., it aims at finding a decomposition of $\widetilde{\mathcal{T}}$). After the game has finished, we can map the played actions back to the standard basis by applying the inverse change of basis.

The diversity injected by the change of basis facilitates the agent's task, because it suffices to find a low-rank decomposition in any basis. Note that gadgets are not affected by the change of basis, since their patterns correspond to linear relationships that are preserved after linear transformations.

Data augmentation. For each game played by the actors, we obtain a new game by swapping two of the actions in it. Specifically, we swap the last action that is not part of a gadget with a randomly chosen action that is also not part of a gadget. Mathematically, swapping actions is a valid operation due to commutativity. From the quantum computation point of view, it is a valid operation because the considered unitary matrices are diagonal.

Besides the action permutation, we also permute the inputs at acting time every time we query the neural network. In particular, we apply a random permutation to the input tensor and past actions that represent the current state, and then invert this permutation at the output of the network's policy head. Similarly, at training time, we apply a random permutation to both the input and the policy targets, and train the neural network on this transformed version of the data. In practice, we sample 100 permutations at the beginning of an experiment, and use them thereafter.

Sample-based MCTS. In AlphaTensor-Quantum, the action space is huge, and therefore it is not possible to enumerate all the possible actions from a given state. Instead, AlphaTensor-Quantum relies on sample-based MCTS, similarly to sampled AlphaZero [62].

Before committing to an action, the agent uses a *search tree* to explore multiple actions to play. Each node in the tree represents a state, and each edge represents an action. For each state-action pair (s, a), we store a set of statistics: the visit count $N(s, a)$, the action value $Q(s, a)$, and the empirical policy probability $\widehat{\pi}(s, a)$. The search consists of multiple simulations; each simulation traverses the tree from the root state s_0 until a leaf state s_L is reached, by recursively selecting in each state s an action a that has high empirical policy probability and high value but has not been frequently explored, i.e., we choose a according to

$$\arg\max_a Q(s, a) + c(s)\widehat{\pi}(s, a)\frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)},$$

where $c(s)$ is an exploration factor. The MCTS simulation keeps choosing actions until a leaf state s_L is reached; when that happens, the network is queried on s_L — obtaining the policy probability $\pi(s_L, \cdot)$ and the return probability $z(s_L)$. We sample K actions a_k from the policy to initialize $\widehat{\pi}(s_L, a) = \frac{1}{K}\sum_k \delta_{a,a_k}$, and initialize $Q(s_L, a)$ from $z(s_L)$ using a risk-seeking value [24]. After the node s_L has been expanded, we update the visit counts and values on the simulated trajectory in a backward pass [62]. We simulate 800 trajectories using MCTS; after that, we use the visit counts of the actions at the root of the search tree to form a sample-based improved policy, following Fawzi et al. [24], and commit to an action sampled from that improved policy.

Training regime. We train AlphaTensor-Quantum on a TPU with 256 cores, with a total batch size of 2 048, training for 1 million iterations on average (the number of training iterations varies depending on the application, ranging between 80 000 and 3 million iterations). We use 3 600 actors on average per experiment, each running on a standalone TPU. Each experiment is ran on a set of target tensors, which we group by application (i.e., one experiment for addition, one for unary iteration, etc.); this allows the RL agent to recognize and exploit the decomposition patterns that might be present across multiple tensors within the same experiment. We find that the experiment variance is very low: a single experiment is enough for AlphaTensor-Quantum to find the best constructions for each application.

Other versions of AlphaTensor-Quantum. We found empirically that the GF(2^m)-mult circuits were particularly challenging to optimize, and they required additional training iterations. To reduce the size of the search space and accelerate the training procedure, we developed a variant of AlphaTensor-Quantum that significantly favours Toffoli gates. Specifically, every time that three consecutive and linearly independent actions (factors) are played, the environment automatically applies the remaining four actions that would complete the Toffoli gadget. We applied this version of AlphaTensor-Quantum to optimize these targets only.

5.3 Verification

In order to verify that the results of AlphaTensor-Quantum are correct, we used the `feynver` tool developed by Amy [36] after each step of the compilation process discussed in Appendix C.1. It uses the sum-over-paths formalism [63] to produce a proof of equality and can be applied to any circuit. However, it is a sound but not

complete method, so it may fail both to prove and to disprove equality for some pairs of circuits. Since the tool runs slowly for larger circuits, we ran it wherever practical (limiting each run to several hours). Each circuit was verified assuming that any intermediate measurements introduced by Hadamard gadetization always return zero, because our software pipeline does not generate the classically-controlled correction circuits (these can be easily constructed but are irrelevant to AlphaTensor-Quantum).

This allowed us to verify inputs up to the point where a tensor is obtained. In order to verify that the decompositions constructed by AlphaTensor-Quantum were correct, we computed the corresponding tensor and checked that it was equal to the original input tensor. While we did not use the optimized decompositions to produce circuits, they could be verified in the same way as discussed above.

Acknowledgements

The authors thank Steve Clark, Alexander L. Gaunt, Alexandre Krajenbrink, Luca Mondada, and Richie Yeung for their feedback on the paper; and Ryan Babbush, Sergio Boixo, Craig Gidney, Matt Harrigan, Tanuj Khattar, and Nicholas Rubin for insightful discussions and ideas.

References

[1] Richard P Feynman. Simulating physics with computers. *International journal of theoretical physics*, 21 (6/7):467–488, 1982.

[2] Yuri Manin. *Computable and noncomputable*. Sovetskoe Radio, 1980.

[3] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997. ISSN 1095-7111. doi: 10.1137/s0097539795293172.

[4] Nick S. Blunt, Joan Camps, Ophelia Crawford, Róbert Izsák, Sebastian Leontica, Arjun Mirani, Alexandra E. Moylett, Sam A. Scivier, Christoph Sünderhauf, Patrick Schopf, Jacob M. Taylor, and Nicole Holzmann. Perspective on the current state-of-the-art of quantum computing for drug discovery applications. *Journal of Chemical Theory and Computation*, 18(12):7001–7023, November 2022. ISSN 1549-9626. doi: 10.1021/acs.jctc.2c00574.

[5] Alexander M. Dalzell, Sam McArdle, Mario Berta, Przemyslaw Bienias, Chi-Fang Chen, András Gilyén, Connor T. Hann, Michael J. Kastoryano, Emil T. Khabiboulline, Aleksander Kubica, Grant Salton, Samson Wang, and Fernando G. S. L. Brandão. Quantum algorithms: A survey of applications and end-to-end complexities, 2023.

[6] Sergey Bravyi and Alexei Kitaev. Universal quantum computation with ideal Clifford gates and noisy ancillas. *Physical Review A*, 71:022316, 2005.

[7] Earl T. Campbell, Barbara M. Terhal, and Christophe Vuillot. Roads towards fault-tolerant universal quantum computation. *Nature*, 549(7671):172–179, September 2017. ISSN 1476-4687. doi: 10.1038/nature23460.

A Additional Background

A.1 Quantum Computing

Qubits. While a classical computer works on bits 0 and 1, a quantum computer works with qubits, which are superpositions of 0 and 1, representable by a normalized vector in the vector space $\mathbb{C}^2$. We write $|0\rangle, |1\rangle \in \mathbb{C}^2$ for the standard basis vectors. Generally, we adopt *bra-ket* notation, and write $|\psi\rangle = a|0\rangle + b|1\rangle$ (with $a, b \in \mathbb{C}$) for any normalized complex vector (i.e., with $|a|^2 + |b|^2 = 1$). When we have two qubits, its state space is given by tensor product of the spaces for a single qubit: $\mathbb{C}^2 \otimes \mathbb{C}^2 \cong \mathbb{C}^{2^2}$. Generally, the N-qubit state space consists of the normalized vectors in $\mathbb{C}^{2^N}$. A basis for this space is given by the states $|\vec{x}\rangle$ given by bit strings $\vec{x} \in \{0,1\}^N$.

Quantum gates. State transformations in quantum computing are linear operations that preserve the normalization, and hence map quantum states to quantum states. These operations are fully represented by unitary matrices. Besides the common examples from the main paper, such as the Hadamard gate or the X gate, we define the Z[α] phase gate as

$$Z[\alpha] \;=\; \begin{pmatrix} 1 & 0 \\ 0 & \mathrm{e}^{i\alpha} \end{pmatrix}.$$

We use the notation Z[α] to explicitly indicate that the gate is parameterized by $\alpha \in \mathbb{R}$, so that $Z[\alpha]|\psi\rangle = aZ[\alpha]|0\rangle + bZ[\alpha]|1\rangle = a|0\rangle + b\mathrm{e}^{i\alpha}|1\rangle$. In contrast, when we refer to the (unparameterized) Z gate, we assume the special case $\alpha = \pi$, i.e., $Z \equiv Z[\pi]$, such that $Z|\psi\rangle = a|0\rangle - b|1\rangle$. The T gate and the S gate are also phase gates, with $\alpha = \pi/4$ and $\alpha = \pi/2$, respectively.

Controlled gates. As explained in the main paper, the controlled gates are two-qubit gates that only perform a non-trivial action on the second qubit if the first qubit is in the $|1\rangle$ state. For instance, the CNOT gate performs a NOT gate on the second qubit if the first qubit is $|1\rangle$. We can write this succinctly as $CNOT|x, y\rangle = |x, x \oplus y\rangle$. Similarly, the CZ gate implements a Z phase gate on the second qubit if the first is in the $|1\rangle$ state, i.e., $CZ|x, y\rangle = (-1)^{xy}|x, y\rangle$, where here we write a juxtaposition of bits xy to denote their AND. More generally, the controlled $Z[\alpha]$ phase gate acts as $CZ[\alpha]|x, y\rangle = \mathrm{e}^{i\alpha xy}|x, y\rangle$. We also consider the controlled CNOT gate, also called Toffoli gate, whose action is $Tof|x, y, z\rangle = |x, y, (xy) \oplus z\rangle$. Similarly, the controlled CZ gate acts as $CCZ|x, y, z\rangle = (-1)^{xyz}|x, y, z\rangle$.

Universal gate sets. The Hadamard H, CNOT, and the Z phase gates $Z[\alpha]$ form a *universal* gate set, meaning that any unitary matrix, and hence any quantum computation, can be decomposed into these gates. For instance, we have $X = HZ[\pi]H$.

In fault-tolerant quantum computation we cannot use a continuous family of gates like $Z[\alpha]$, and instead work with a small discretized set of quantum gates. A particularly often used one is the T gate, $T \triangleq Z[\frac{\pi}{4}]$. The gate set consisting of H, CNOT and T is an *approximately universal* gate set, meaning that any unitary can be approximated arbitrarily well by a circuit consisting of these gates.

Clifford + T. While quantum computing is generally believed to be hard to classically simulate, there is a useful subset of quantum computations for which there is an efficient classical simulation algorithm. These are known as *Clifford* computations, and they are built out of Clifford gates. The Clifford gates are H, CNOT, and

S. For this reason, an often-used approximately universal gate set is {H, CNOT, S, T}, which is called the Clifford+T gate set. As the Clifford gates are efficiently simulable, the number of T gates is a measure of how hard it is to perform the computation classically.

More concretely, T gates are also the hardest gates to execute: in most fault-tolerant architectures for quantum computation, the Clifford gates are relatively easy to perform, while T gates cost orders-of-magnitude more resources. This is because Clifford gates can be executed "natively" in many error-correcting codes, while T gates are implemented by injecting a magic state that first has to be distilled to a desired precision.

T gate optimization. For this reason it is important to reduce the number of T gates required for a computation as much as possible. There are several ways to do this. For instance, since $S = T^2$, whenever two T gates appear in a row, they combine to form a Clifford gate. More generally, if we have T^k, then this costs just one T gate if k is odd, and is Clifford otherwise. This technique for combining T gates can be generalized to settings where they do not appear directly after one another on the same qubit —which is a restrictive consideration. The generalization can be used to combine T gates that are applied to the same *parity*.

Recall the actions of the Z[α] and CNOT gates. Considering $x, y \in \{0, 1\}$, we have $Z[\alpha]|x\rangle = \mathrm{e}^{i\alpha x}|x\rangle$ and $CNOT|x, y\rangle = |x, x \oplus y\rangle$. Thus, if we apply a $Z[\alpha]$ gate to the second qubit after a CNOT, it applies a phase to the parity $x \oplus y$, i.e., $(I \otimes Z[\alpha])CNOT|x, y\rangle = (I \otimes Z[\alpha])|x, x \oplus y\rangle = \mathrm{e}^{i\alpha(x\oplus y)}|x, x \oplus y\rangle$.

More generally, when we have a circuit consisting of just CNOT and Z[α] phase gates, the unitary matrix U that it implements acts as $U|\vec{x}\rangle = \mathrm{e}^{i\phi(\vec{x})}|A\vec{x}\rangle$, where $\phi : \{0, 1\}^n \to \mathbb{R}$ is a *phase polynomial*, and A is an invertible matrix that can be implemented with Clifford operations. Using this phase polynomial representation, phase gates that are implicitly acting on the same parity get combined. When we then resynthesize a quantum circuit from this representation, we hence possibly require fewer phase gates.

Multilinear phase polynomials. When we construct a phase polynomial from a CNOT + Z[α] circuit in this way, we get a phase polynomial consisting of XOR terms. This representation is not unique. As a simple example, we can see that $\mathrm{e}^{i\pi(x+y+x\oplus y)} = \mathrm{e}^{2\pi i} = 1$ for all $x, y \in \{0, 1\}$. If we were to directly synthesize the first expression, this would require three phase gates corresponding to the three terms (x, y, and $x \oplus y$), even though it implements an identity. We can rewrite the phase polynomial into a unique form by decomposing each of the XOR terms into a *multilinear* form (i.e., a polynomial where the degree of each variable in each term is either 0 or 1), e.g.,

$$x \oplus y = x + y - 2xy.$$

(Note that here '+' denotes regular addition in the real numbers.) We can generalize this expression to decompose an expression of N parities into a multilinear polynomial:

$$x_1 \oplus \cdots \oplus x_N \;=\; -\sum_{\vec{y}\neq\vec{0}} (-2)^{|\vec{y}|-1} x_1^{y_1} \cdots x_N^{y_N}. \tag{1}$$

Note that the weight of a degree-k term is 2^{k-1}. This means that if we had a phase polynomial ϕ where the smallest constant in its XOR form is $2\pi/2^k$, then all the terms in its multilinear form of degree greater than k will have a weight that is an integer multiple of 2π, and hence can be ignored. In the example above, when we write $\mathrm{e}^{i\pi(x+y+x\oplus y)}$ in multilinear form we get $\mathrm{e}^{i\pi(2x+2y-2xy)}$, so that it only contains expressions that are multiples of 2π.

In particular, if we have a circuit of CNOT and T gates only (i.e., $\alpha = \pi/4$), the multilinear phase polynomial involves terms of degree at most 3, after having removed all the terms leading to multiple of 2π. Hence, there is a one-to-one correspondence between N-qubit diagonal CNOT + T circuits and multilinear polynomials of the form

$$\phi(x_1, \ldots, x_N) \;=\; \frac{\pi}{4}\left(\sum_i a_i x_i + 2\sum_{i<j} b_{ij} x_i x_j + 4\sum_{i<j<k} c_{ijk} x_i x_j x_k\right), \tag{2}$$

where $a_i \in \{0, \ldots, 7\}$, $b_{ij} \in \{0, \ldots, 3\}$ and $c_{ijk} \in \{0, 1\}$.

We can now check whether two diagonal CNOT + T circuits implement the same unitary by converting them to the multilinear phase polynomial representation of Eq. 2. However, since we are considering Clifford operations to be free, we only need to check whether two CNOT + T circuits are equal up to Clifford operations.

Symmetric 3-tensors. The multilinear representation corresponds directly to a circuit of T, CS and CCZ gates: each $a_i x_i$ term is implemented by a_i T gates acting on the ith qubit; each $b_{ij} x_i x_j$ term corresponds to b_{ij} CS gates on qubits i and j; and each $c_{ijk} x_i x_j x_k$ term corresponds to one CCZ gate on qubits i, j and k if $c_{ijk} = 1$. Each of these terms only corresponds to a non-Clifford operation if the constant a_i, b_{ij} or c_{ijk} is odd. Hence, to check whether two diagonal CNOT + T circuits implement the same unitary up to Clifford operations, we can calculate their multilinear polynomials in Eq. 2 and take modulo 2 of their coefficients. That is, two CNOT + T circuits are equal up to Clifford operations if and only if they have the same polynomial (after taking modulo 2 of their coefficients). All this information about the multilinear polynomial can then be captured in a single symmetric 3-tensor $\mathcal{T}$ with entries in $\{0, 1\}$. Namely, we make the correspondence as follows: for $i < j < k$ an entry $\mathcal{T}_{ijk} = 1$ if we have $c_{ijk} = 1$ in the corresponding polynomial ϕ; for $i < j$, $\mathcal{T}_{ijj} = 1$ if $b_{ij} = 1$; and for any i, we have $\mathcal{T}_{iii} = 1$ if $a_i = 1$.

Rank of a tensor. We can write a parity expression like $x_1 \oplus x_3$ as a dot product with a vector $\vec{y} = (1, 0, 1)$ via $\vec{y} \cdot \vec{x} = y_1 x_1 \oplus y_2 x_2 \oplus y_3 x_3$, where we take the value of the dot product to be modulo 2 and the ones in $\vec{y}$ indicate which variables are part of the parity. Now, take an XOR phase polynomial consisting of a single term $\mathrm{e}^{i\frac{\pi}{4}\vec{y}\cdot\vec{x}}$. We can also write this as $\mathrm{e}^{i\frac{\pi}{4}x_{a_1}\oplus\cdots\oplus x_{a_k}}$ where the indices a_k indicate the positions wherer $y_i = 1$. Using Eq. 1 to decompose this polynomial into multilinear form, we get all combinations of the terms x_{a_i}, $2x_{a_i}x_{a_j}$ and $4x_{a_i}x_{a_j}x_{a_k}$ —for all the variables x_i where $y_i = 1$. Hence, we set the corresponding symmetric 3-tensor $\mathcal{T}_{ijk}$ as $\mathcal{T}_{ijk} = y_i y_j y_k$. This is then a *rank-one* symmetric tensor, as it is built out of a single vector $\vec{y}$. Let such a rank-one tensor be denoted as $\mathcal{T}_{\vec{y}}$. Any rank-one tensor is defined by a vector $\vec{y}$, and hence if a symmetric 3-tensor built from a phase polynomial has rank-one, then it corresponds to a single XOR term.

The translation from the XOR form into the 3-tensor is linear in the XOR terms. Hence, if the phase polynomial is a sum of parity terms, $\{\vec{y}^1 \cdot \vec{x}, \vec{y}^2 \cdot \vec{x}, \ldots, \vec{y}^k \cdot \vec{x}\}$, then its corresponding symmetric 3-tensor is $\mathcal{T} = \mathcal{T}_{\vec{y}^1} + \cdots + \mathcal{T}_{\vec{y}^k}$, where again the summation is under modulo 2. Conversely, each symmetric 3-tensor can be *decomposed* in this manner. The *rank* of the tensor is the number of terms in this decomposition. Thus, if we have a symmetric 3-tensor, finding a rank-R decomposition corresponds to finding a set of R XOR parity terms that result in the same tensor, which hence implement the same unitary up to Clifford operations. Given that implementing the unitary matrix corresponding to a single parity phase requires one T gate, the T-count of this circuit is then exactly equal to the rank of the decomposition.

Therefore, we have reduced the problem of T-count optimization of CNOT + T circuits to finding Waring decompositions of symmetric $N \times N \times N$ 3-tensors with elements in $\{0, 1\}$.

A.2 Quantum Chemistry

Due to the exponential computational cost associated with solving the Schrödinger equation for strongly correlated many-electron systems, state-of-the-art classical quantum chemistry methods struggle to accurately simulate strongly correlated molecules [44]. Quantum computers, in theory, can handle these calculations more efficiently, due to the logarithmic scaling of qubit number with system size. The FeMoco molecule, which is the active site in the nitrogenase enzyme responsible for nitrogen fixation, presents a compelling use case for quantum computing in strongly correlated quantum chemistry, due to its highly complex electronic structure [45]. The primary function of nitrogenase is to catalyze the conversion of atmospheric nitrogen (breaking a strong triple bond) at room temperature, into ammonia. Synthetically this is done via the Haber process which is incredibly energy intensive. The FeMoco molecule contains multiple atoms, including iron and molybdenum.

To understand complex reaction pathways such as those in FeMoCo one must first calculate the ground state energy of a molecule at a particular geometry. Of the techniques for fault-tolerant quantum computation of molecular ground states, the current most efficient method is the *linear combinations of unitaries* (LCU) method combined with *qubitization* for phase estimation, which leads to a linear T gate complexity in the outcome precision [33]. This can be further improved with Coulomb operator prepossessing that exploits sparsity [48] or the use of tensor hyper-contraction approaches [46]. Furthermore, beyond the Born-Oppenheimer approximation of fixed nuclei, a first quantized plane wave framework has been proposed as the optimal for molecules [64]. This approach has been extended to be used with pseudo potentials for simulating materials. Furthermore, these qubitized phase estimation techniques have been used to calculate molecular observables when combined with quantum signal processing [65].

The rest of this section assumes prior knowledge of the standard mathematical formulation of quantum computing and quantum algorithms. See de Wolf [66, Sections 4 & 9] for a more gentle introduction and Bauer et al. [67] for a more comprehensive reference on the applicability of quantum algorithms in chemistry.

Quantum phase estimation. The typical way to calculate ground state energies is to exploit the relation of a unitary matrix acting on its eigenvector $|k\rangle$, which when applied to a mixture of eigenstates $|\psi\rangle = \sum_k c_k|k\rangle$, gives a superposition

$$\langle\psi|U|\psi\rangle = \sum_j |c_k|^2 \mathrm{e}^{i\phi_k}\langle k|k\rangle.$$

where $\mathrm{e}^{i\phi_k}$ are the eigenvalues of U (and we call ϕ_k the eigenphases).

The canonical phase estimation algorithm is shown in Figure 9. The ancilla register is prepared in the Hadamard basis, then phase-kickback of controlled unitaries acting on their eigenbasis $U|k\rangle = \mathrm{e}^{\phi_k}|k\rangle$ is used to put the powers of eigenphases into the Fourier basis on the ancilla register. The eigenphases are then extracted into the computational basis from the Fourier basis using the inverse quantum Fourier transform. Readout of the ancilla bits gives the eigenphases in fixed-point binary representation. The eigenstates with the dominant initial overlap will have their eigenphases obtained with the largest probability. Therefore it is required that the initial state must have dominant overlap with the state of interest. Generating initial states with this property is an unresolved problem in quantum chemistry and has been discussed by Lee et al. [68].

In quantum chemistry applications, we encode the energies of the eigenstate $|k\rangle$ of the Hamiltonian describing the chemical system into the phase ϕ_k using the qubitized quantum walk unitary from Eq. 4 (see below). Qubitized quantum phase estimation [33] applies a controlled quantum-walk operator as opposed to a

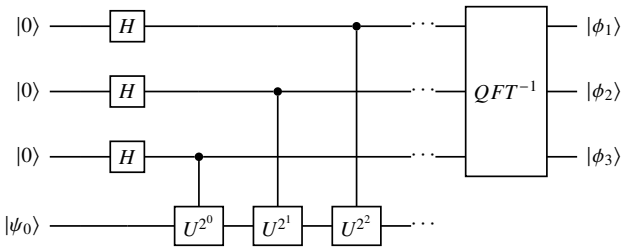


Figure 9: Canonical quantum phase estimation.

controlled time-evolution operator, where the Hamiltonian H of the system is encoded in the quantum walk operator $W = \mathrm{e}^{i \arccos(H)\hat{Y}}$, where $\hat{Y}$ is the Pauli Y operator applied to each eigenvector independently. The eigenphases of this walk operator are proportional to the Hamiltonian eigenvalues E_k and can be extracted when applied to eigenstates $|k\rangle$ of the Hamiltonian. Consider any state $|\psi\rangle = \sum_k c_k |k\rangle$, then:

$$\langle\psi|W|\psi\rangle = \langle\psi|\mathrm{e}^{i \arccos(H)\hat{Y}}|\psi\rangle = \sum_k |c_k|^2 \mathrm{e}^{\pm i \arccos(E_k)} \langle k|k\rangle.$$

A naive implementation of the controlled quantum walk circuit W is shown in Figure 7. Its complexity can be improved by using the techniques employed by Babbush et al. [33], which exploit adjacent cancellation of reflection operators and a different initial ancilla state.

Linear combination of unitaries. The U_{LCU} operator is a block encoding the Hamiltonian of interest, describing the molecule at a given geometry. It uses two registers, a state register and a prepare register. The block encoding is indexed by the unique bit strings of the PREPARE register, it can be thought of as there being a state block for each PREPARE register row and column bit strings.

A Hamiltonian operator can be decomposed into a LCU by

$$H = \sum_{\ell=0}^{L-1} \alpha_\ell H_\ell,$$

where H_ℓ is a unitary matrix and α_l a linear expansion coefficient. We can assume α_l to be a real positive number, since any complex phases in the coefficients can be absorbed into H_ℓ. The LCU circuit primitives were first introduced by Childs and Wiebe [69]. The operator H is encoded into a block of a larger unitary matrix U_{LCU}, and it is accessed by projecting into the block via post-selection – that is, we measure the qubits defining the block structure, and the encoding succeeds when this returns zero. To ensure the overall matrix U_{LCU} is unitary, the operator is re-scaled by $|\alpha|$, its L_1-norm , i.e.,

$$U_{LCU} = \begin{pmatrix} \frac{H}{|\alpha|} & * \\ * & * \end{pmatrix}, \quad \text{where} \quad |\alpha| = \sum_\ell |\alpha_\ell|.$$

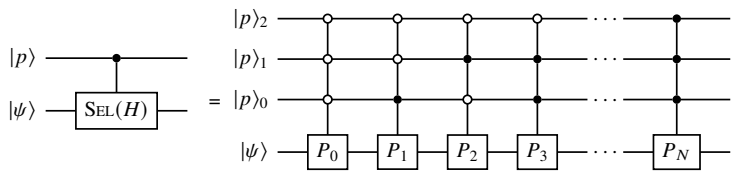


Figure 10: SELECT circuit implementing $\sum_\ell |\ell\rangle\langle\ell| \otimes P_\ell$.

The Hamiltonian is encoded into a quantum circuit via two oracles: SELECT(H) and PREPARE(α) which encode the unitaries H_ℓ and the coefficients α_ℓ respectively. The PREPARE(α) transforms the all zeros state of the control register to a state $|p\rangle$ (with positive real amplitudes) defined by the given coefficients in Eq. 3,

$$\text{PREPARE}(\alpha) : |0\rangle \mapsto \sum_{\ell=0}^{L-1} \sqrt{\frac{\alpha_\ell}{|\alpha|}} |\ell\rangle = \begin{bmatrix} \sqrt{\left(\frac{\alpha_0}{|\alpha|}\right.} & \cdot & \cdots \\ \sqrt{\left(\frac{\alpha_1}{|\alpha|}\right.} & \cdot & \cdots \\ \vdots & \ddots & \cdots \\ \sqrt{\left(\frac{\alpha_{k-1}}{|\alpha|}\right.} & \cdot & \cdots \end{bmatrix} \tag{3}$$

A leading fault-tolerant primitive was proposed by Low et al. [70]. The signs and phases of the coefficients are pulled into the SELECT.

It must be noted that upon an action of the SELECT, an orthogonal complement $|p_\perp\rangle$ will also be generated, which leads to success probabilities (of the block-encoding) of less than 1. The SELECT oracle is typically a multi-controlled unitary matrix that applies the Hamiltonian terms to the state register via a multi-control for each term, indexed by a unique PREPARE register bit string. A fault tolerant approach was proposed by Babbush et al. [33] built upon Gidney's work [31]. The multi-control multiplies the weighting coefficient from the PREPARE to the Pauli for that bit string and applies the weighted Pauli to the state register. The SELECT oracle is shown in Figure 10 and given by

$$\text{SELECT}(H) := \sum_{\ell=0}^{L-1} |\ell\rangle\langle\ell| \otimes H_\ell$$

While any operator can in principle be written as a linear combination of unitary matrices, second quantized Hamiltonians in the Pauli representation are naturally expressed in this framework. The SELECT(H) and PREP(α) oracles can be combined to create a probabilistic circuit for acting on an arbitrary quantum state $|\psi\rangle$ with an operator $\frac{H}{|\alpha|}$; this is shown in Figure 11.

The operation $\frac{H}{\alpha}|\psi\rangle$ is successful if a measurement on the prepare register returns $|\bar{0}\rangle$. The success probability is related to the closeness of H to being unitary and the initial state overlap,

$$P_{success} = \frac{1}{|\alpha|^2} \langle\psi|H^\dagger H|\psi\rangle$$

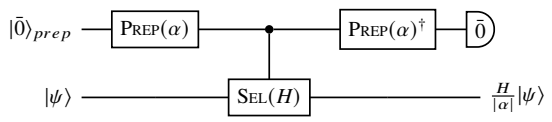


Figure 11: LCU Circuit acting on an arbitrary quantum state $|\psi\rangle$ with a rescaled operator $\frac{H}{|\alpha|}$, when a successful post-selection occurs.

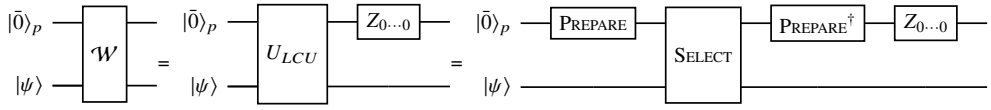
The L_1-norm of the operator has a significant effect on the success probability and this is the dominant factor in the complexity results analysis. Therefore the majority of work on trying to improve the complexity of LCU methods typical aiming to reduce the L_1-norm of the operator.

Qubitization. First proposed by Low and Chuang [71], qubitization is a powerful method allowing one to implement polynomials of block encoded operators. In block encoding methods we are typically only interested in the PREPARE $|0\cdots0\rangle\langle0\cdots0|$ block of the unitary matrix where the Hamiltonian is encoded. The rest of the rows and columns of the unitary matrix are refereed to as the orthogonal complement, $|0\cdots0_\perp\rangle$.

$$U_{LCU} = \begin{array}{c} \begin{matrix} |0\cdots0\rangle|\psi\rangle & |0\cdots0_\perp\rangle|\psi\rangle \end{matrix} \\ \begin{pmatrix} H & * \\ * & * \end{pmatrix} \end{array} \begin{matrix} |0\cdots0\rangle|\psi\rangle \\ |0\cdots0_\perp\rangle|\psi\rangle \end{matrix} = \bigoplus_k \begin{array}{c} \begin{matrix} |0\cdots0\rangle|k\rangle & |0\cdots0_\perp\rangle|k\rangle \end{matrix} \\ \begin{pmatrix} \lambda_k & \sqrt{1-\lambda_k^2} \\ \sqrt{1-\lambda_k^2} & -\lambda_k \end{pmatrix} \end{array} \begin{matrix} |0\cdots0\rangle|k\rangle \\ |0\cdots0_\perp\rangle|k\rangle \end{matrix}.$$

In the eigenbasis $|\psi\rangle = \sum_k \lambda_k|k\rangle$ the U_{LCU} becomes a direct sum of 2×2 blocks indexed on each eigenvector. Where the orthogonal compliment $|0\cdots0_\perp\rangle$ of the prepare register is given an arbitrary vector which just has to exist to generate the qubitized 2x2 subspace via some Gram-Schmidt projection. We can then turn this into a quantum walk operator by applying a reflection, which is typically implemented as a all-0 CZ gate with a minus phase,

$$W = \bigoplus_k \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} \lambda_k & \sqrt{1-\lambda_k^2} \\ \sqrt{1-\lambda_k^2} & -\lambda_k \end{pmatrix} = \bigoplus_k \begin{pmatrix} \lambda_k & \sqrt{1-\lambda_k^2} \\ -\sqrt{1-\lambda_k^2} & \lambda_k \end{pmatrix}.$$



Therefore, powers of the quantum walk operator take the form of increments of an $R_y(\arccos(\lambda_k))$ rotation in the 2×2 sub-spaces of $\{|0\cdots0\rangle|k\rangle, |0\cdots0_\perp\rangle|k\rangle\}$. These rotation matrices can be interpreted in terms of the Chebyshev polynomials of the first $T_d(\lambda_k)$ and second $U_d(\lambda_k)$ kind:

$$W^d = \bigoplus_k \begin{pmatrix} \cos(d\theta_k) & \sin(d\theta_k) \\ -\sin(d\theta_k) & \cos(d\theta_k) \end{pmatrix} = \bigoplus_k \begin{pmatrix} T_d(\lambda_k) & \sqrt{1-\lambda_k^2}U_{d-1}(\lambda_k) \\ -\sqrt{1-\lambda_k^2}U_{d-1}(\lambda_k) & T_d(\lambda_k) \end{pmatrix} = \begin{pmatrix} T_d(H) & \cdot \\ \cdot & \cdot \end{pmatrix}$$

The phase of this operator can be found in the same way as traditional phase estimation using a controlled quantum-walk:

$$\langle\psi|W|\psi\rangle = \langle\psi|e^{i\arccos(H)\hat{Y}}|\psi\rangle = \sum_k |c_k|^2 e^{\pm i\arccos(E_k)}\langle k|k\rangle \tag{4}$$

This operation is the compilation target for the quantum chemistry section of this work.

B Baselines

Methods for T-count optimization are broadly divided into two camps: those that treat all non-Clifford phase gates equivalently, and those that utilize specific optimizations for T gates. In the algorithms in the first category [e.g., 14, 17, 19, 37], non-Clifford rotation gates are treated as distinct black boxes, without any knowledge of transformations relating them, except that they can be merged together to reduce the number of gates. While limited in this way, they also have the ability to optimize all circuits, regardless of the gates used, and in particular they can optimize circuits containing Hadamard gates.

By contrast, algorithms in the second category [e.g., 16, 18, 23] are based (either directly or indirectly) on the representation of CNOT+T circuits as Waring decompositions of the signature tensor, as they are considered in AlphaTensor-Quantum. These generally perform better on such circuits than the black-box methods. However, since they cannot optimize around Hadamard gates, it is beneficial to first apply a black-box optimizer that can do so [34].

To obtain the baseline T-count for the circuit benchmarks used in this work, we applied the phase-teleportation algorithm of Kissinger and van de Wetering [19] followed by compiling the circuits to the Waring decomposition using the steps detailed in Appendix C.1. Then we apply TODD [16] and STOMP [23] and take the output with better T-count. (For STOMP, we took the best of twenty runs.) For the standard arithmetic benchmarks (i.e., the non-split circuits), we compared against the published T-count numbers and took whichever was better.

We chose phase-teleportation because it was shown to be essentially optimal for reducing T-count in the black-box setting [72], and TODD/STOMP were chosen because they consistently outperform other methods [18] in published benchmarks. While TODD and STOMP both have methods to compile Clifford+T circuits to a Waring decomposition, we opt to use the method in Appendix C.1 as it includes a more recent and better performing method from Vandaele et al. [34].

C Method Details

C.1 Compilation of the Quantum Circuits

To obtain signature tensors suitable for optimization with AlphaTensor-Quantum from quantum circuits, we proceed as follows:

1. First, we express the input circuit over the $\{\mathrm{H}, \mathrm{Z}, \mathrm{S}, \mathrm{T}, \mathrm{CNOT}\}$ gate set.

2. We apply the phase-teleportation optimization algorithm of Kissinger and van de Wetering [19] to reduce the initial number of T gates as much as possible. As discussed in Appendix B, this is particularly beneficial because phase-teleportation can optimize around Hadamard gates.

3. We rewrite the circuit to contain as few internal Hadamard gates as possible, using the algorithm of Vandaele et al. [34]. A Hadamard gate is internal if there are T gates between it and both the beginning and end of the circuit that it cannot commute with (for example, in the sequence $HTHTH$ the second Hadamard is internal but the first and third ones are not).

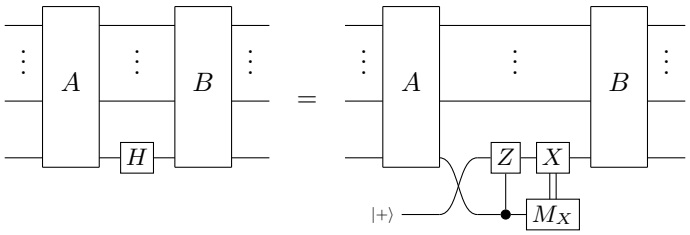
4. We split the circuit U into alternating blocks of Clifford circuits B_i and circuits A_i containing only CNOT and phase gates, such that $U = B_m A_{m-1} B_{m-1} \cdots A_1 B_1$. We accomplish this by scanning the circuit from left to right, collecting as many Clifford gates as possible into the first block B_1 until we encounter a T gate. After this, we collect gates into the block A_1 until we encounter a Hadamard gate. Then we switch to collecting Clifford gates into block B_2, etc. We proceed in this way until the whole circuit has been consumed.

5. We apply the Hadamard gadgetization process of Heyfron and Campbell [16] to remove all internal Hadamard gates by replacing them with ancilla and Clifford gates. In Section 3.1, if this process generates more than 60 qubits, we take the output of the previous step and use the technique in Appendix C.2 to split the circuit into smaller sections, and we repeat this compilation process on each. We discuss Hadamard gadgetization in further detail below.

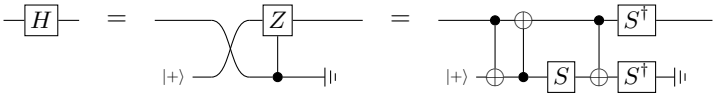
6. At this point we can partition the circuit into the form $U = U_3 U_2 U_1$ where U_1 and U_3 contain Clifford gates and non-internal Hadamard gates, and U_2 contains only CNOT and phase gates. Let C be the circuit formed by taking just the CNOT gates in U_2, then consider the transformation $U = U_3' U_2' U_1 = (U_3 C)(C^\dagger U_2) U_1$. Now, U_2' is a diagonal operator. Let V be the circuit formed by removing all T gates from U_2' and W be the circuit formed by removing all S and Z gates from U_2'. Then V is Clifford, W contains only CNOT and T gates, and it can be shown that $U_2' = VW$, so $U = (U_3 C V) W U_1$. Therefore, all the non-Clifford components of the original circuit U is now contained within W.

7. We map W to a Waring decomposition of the signature tensor as follows. Suppose W contains R T gates and N qubits, then let A be a $N \times R$ matrix in $\mathbb{F}_2$. For the ith T gate in W, set $A_{ji} = 1$ if it appears on the jth qubit, and $A_{ji} = 0$ otherwise. Scan over W from left to right: for each CNOT gate with control a and target b, if there are any T gates to its right, then let i' be the corresponding column of A and set $A_{ai} \leftarrow A_{ai} + A_{bi}$ for all $i' \leq i \leq R$. After all CNOT gates have been processed, A will be a Waring decomposition of the signature tensor of W, which we can calculate as $\mathcal{T}_{ijk} = \sum_{m=1}^{R} A_{im} A_{jm} A_{km}$. This is then ready to be optimized with AlphaTensor-Quantum. Overall, this procedure is the same as the phase polynomial method in Section 5.1, but phrased in terms of matrices.

Hadmard gadgetization. To remove the internal Hadamard gates from a circuit, we can use the Hadamard gadgetization method of Heyfron and Campbell [16]. In this method, each Hadamard gate is replaced with the following gadget circuit, consisting of an ancilla, Clifford gates, measurements, and classically-controlled gates (here, A and B represent the rest of the circuit):



However, in this implementation, we assume that the measurement always returns zero. Therefore, the corresponding gadget implementation is as follows (additionally rewritten to our target gate set):



This is justified in that it does not change the non-Clifford behaviour of a circuit—in fact, the classically controlled X gate correction factor can be translated into a controlled Clifford operator at the end of the circuit [16, 23], hence the T-count is unchanged by this assumption. Because computing these correction factors is complicated, and existing quantum circuit representations have poor support for classically-controlled operations; this decision was also made for the implementations of both TODD [16] and STOMP [23]. Since fault-tolerant quantum computing is still a number of years away from practical implementation, we argue that this is acceptable at this stage.

Reconstructing optimized circuits. In order to reconstruct an optimized form W' of the diagonal circuit W computed above, we can follow the steps in Appendix C.3 given a new Waring decomposition of the signature tensor $\mathcal{T}$. With W', we can recompose the original circuit $U = (U_3CV)WU_1$. However, note that W' may differ from the original W by a diagonal Clifford factor, so it must be corrected. This factor can be computed, given the original and optimized Waring decompositions along with V, by considering the difference between the signature tensors over $\mathbb{Z}$ rather than $\mathbb{F}_2$—see Heyfron and Campbell [16] for more details. As discussed above, in this process we discarded the classically-controlled Clifford gates incurred during Hadamard gadgetization. It is possible in principle (and computationally easy) to take these into account—see de Beaudrap et al. [23] for more details and Heyfron and Campbell [16] for the general method. Since in this work we only determine the optimized T-count rather than reconstructing the circuit, we do not need to implement either of these steps.

C.2 Splitting Larger Quantum Circuits

During the compilation process, each quantum circuit U is split into a list of blocks alternating between circuits A_i containing CNOT and phase gates, and Clifford circuits B_i, i.e.,

$$U = B_m A_{m-1} B_{m-1} \cdots A_2 B_2 A_1 B_1.$$

For most circuits, we can then proceed with Hadamard gadgetization of the B_i blocks as described above. However, for some circuits, this would result in a circuit with too many qubits. Hence, we choose some boundaries $1 = j_1 < j_2 < \cdots < j_k = m$ to break up the circuit into a set of subcircuits:

$$U_i = A_{j_{i+1}-1} B_{j_{i+1}-1} \cdots A_{j_i} B_{j_i} \text{ where } U = B_m U_k \cdots U_1.$$

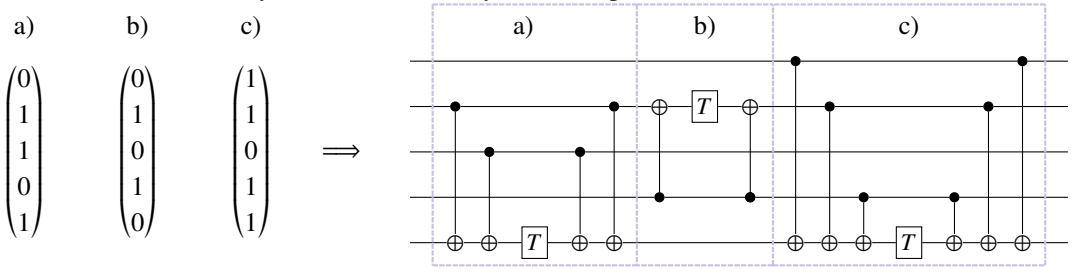
We want to minimize k in order to maximize the number of T gates that can be considered together in each block. To do this, we use a random-greedy algorithm: fix a threshold t on the maximum number of qubits allowed in any subcircuit (e.g., 60), and initially let $j_i = i$ so that $k = m$. For any consecutive pair U_i and U_{i+1}, we consider them to be *compatible* if

$$|\operatorname{supp} U_i \cup \operatorname{supp} U_{i+1}| \; + \sum_{j_i < n < j_{i+2}} |B_n|_H \leq t,$$

where $\operatorname{supp} U_i$ is the set of qubits used by gates on U_i, and $|C|_H$ is the number of Hadamard gates in C. While there is any compatible pair remaining, pick one at random and set $j_n \leftarrow j_{n+1}$ for $n \geq i+1$ and $k \leftarrow k-1$. In this way, we merge consecutive compatible pairs until there are only a few blocks remaining, and combining any two of them would result in too many qubits being generated. We repeat this process 1 000 times and take the run that results in the smallest value of k. We can then treat each U_i as its own target circuit, processing it back through the compilation pipeline, and optimizing it with AlphaTensor-Quantum. To composite the final circuit we combine the U_i along with the Clifford circuit B_m to form $U = B_m U_k \cdots U_1$.

C.3 Gadgetization

The conversion of phase-gadget circuits to Waring decompositions of the signature tensor as described in Appendix C.1 can be inverted to obtain a phase-gadget circuit from a decomposition by placing a CNOT-fanout structure conjugating a T gate for each factor, where the target corresponds to some non-zero entry in the factor, and the controls are on every other non-zero entry. For example:



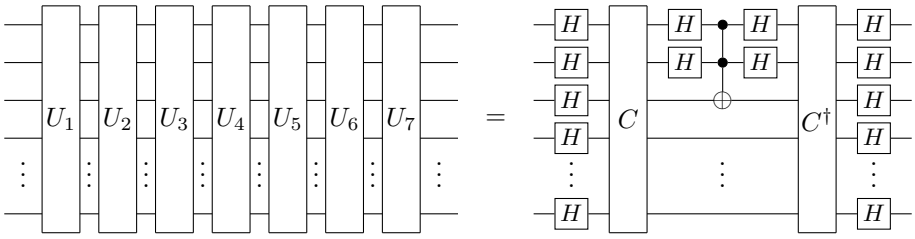
Note that this is not optimal with respect to the number of CNOT gates, and many strategies have been developed to improve this [73–75]. In this work we ignore the cost of Clifford gates, and so do not attempt

to optimize this step. If we are constrained to CNOT+T circuits, this method is optimal in terms of T gates, but some patterns of factors can be implemented with fewer non-Clifford resources if we make use of ancilla qubits, measurements, and classically-controlled Clifford gates.

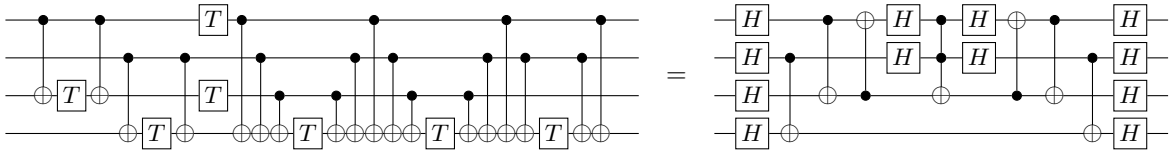
In particular, suppose we have factors $u^{(1)} \cdots u^{(7)}$ such that $u^{(1)}$, $u^{(2)}$ and $u^{(3)}$ are linearly independent, and the following equations are satisfied:

$$\begin{aligned} u^{(4)} &= u^{(1)} + u^{(2)} \\ u^{(5)} &= u^{(1)} + u^{(3)} \\ u^{(6)} &= u^{(1)} + u^{(2)} + u^{(3)} \\ u^{(7)} &= u^{(2)} + u^{(3)} \end{aligned}$$

where addition is considered elementwise over $\mathbb{F}_2$. We call such patterns *Toffoli gadgets*. They can be implemented using a single Toffoli gate conjugated by CNOT and Hadamard gates as follows,



where U_i denotes the CNOT-fanout circuit given above for factor $u^{(i)}$, and C is a CNOT circuit such that $C|u^{(1)}\rangle = |100\cdots\rangle$, $C|u^{(2)}\rangle = |010\cdots\rangle$, $C|u^{(3)}\rangle = |001\cdots\rangle$. This can be proven by considering this circuit in the sum-over-paths formalism [63]. For example:

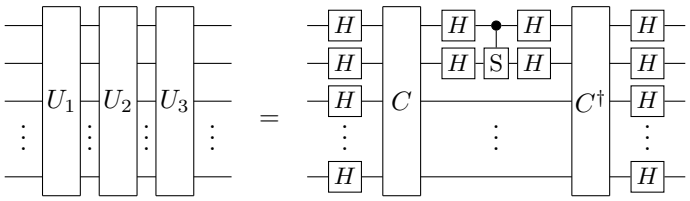


Since Toffoli gates can be implemented efficiently using either four T gates or a CCZ magic state which using state of the art factories costs the same as two T gates (see Figure 12), each Toffoli gadget can be implemented more efficiently than the seven T gates required to represent it as a phase-gadget circuit.

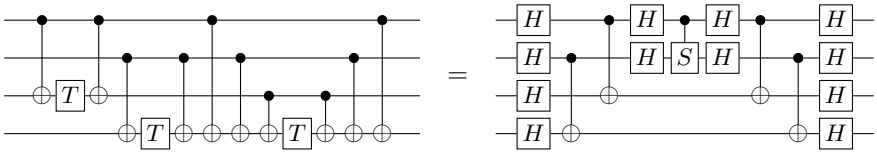
Because $u^{(1)}, u^{(2)}, u^{(3)}$ are linearly independent, such a circuit C must exist. To construct it, we first build an invertible matrix M over $\mathbb{F}_2$ such that the first three columns of M are $u^{(1)}, u^{(2)}, u^{(3)}$ respectively. The remaining columns of M can be computed as a basis for the kernel of the first three via Gaussian elimination. Then we can synthesize a CNOT circuit C such that $C|x\rangle = |M^{-1}x\rangle$ using the Patel-Markov-Hayes algorithm. By definition, this must send $|u^{(1)}\rangle$ to $|100\cdots\rangle$, etc., as required. Therefore, to encourage Toffoli gadgets to be played by AlphaTensor-Quantum, we assign a positive reward to moves which complete a group. To identify when this happens, we set $u^{(1)}$ to be the factor at step $s-6$, $u^{(2)}$ to be the factor at step $s-5$, and so

on, so that $u^{(7)}$ is the last played action (at step s).[1] Finally, to synthesize a quantum circuit from a Waring decomposition of the signature tensor, we apply this construction to any Toffoli gadgets that were identified by AlphaTensor-Quantum, and synthesize the remaining factors with the CNOT-fanout approach from above.

In the same way, we call groups of three factors $u^{(1)}, u^{(2)}, u^{(3)}$ such that $u^{(1)}$ and $u^{(2)}$ are linearly independent and $u^{(3)} = u^{(1)} + u^{(2)}$ *CS gadgets*. In this case, such a group can be implemented by a single CS gate conjugated by CNOT gates:



The CNOT circuit C is such that $C|u^{(1)}\rangle = |10\dots\rangle$ and $C|u^{(2)}\rangle = |01\cdots\rangle$, and it can be generated in exactly the same way as for the Toffoli gadget case. For example:



The central CS gate can be implemented using a CCZ magic state (see Figure 13), which costs the same as two T gates, and so the whole gadget can be implemented more efficiently than representing it as a phase-gadget circuit with three T gates. Note that in both of these gadgets, most of the Hadamard gates can be eliminated by commuting them with the CNOT gates in C.

C.4 Neural Network Architecture

As shown in Figure 8a, the neural network has three main components: the torso, the policy head, and the value head.

The input to the neural network is formed from the current game state, which, at step s of TensorGame, is composed of a (residual) tensor $\mathcal{T}^{(s)}$ and a list of all the previously played actions $u^{(s')}$, with $s' < s$. To form the input to the network, we only consider the last $T = 20$ actions; specifically we stack them into a tensor of size $N \times N \times T$, where each $N \times N$ slice is obtained by taking the outer product $u^{(s')} \otimes u^{(s')}$. This tensor is concatenated with $\mathcal{T}^{(s)}$, as well as with the output of a linear layer that takes as input a single scalar: the square root of the quotient between s and the maximum number of allowed moves, which is 250. Figure 14 shows an overview of the torso.

The concatenated inputs are first passed through a linear layer that maps from $\mathbb{R}^{N+T+1}$ to $\mathbb{R}^c$, with $c = 512$. The output of this layer is passed through $L = 4$ layers of symmetrized axial attention (see Figure 8b),

[1] Any permutation of the seven factors could be absorbed in C and therefore it would still correspond to a Toffoli gadget. Additionally, the seven factors do not need to be consecutive in order to form a Toffoli gadget. However we impose a specific ordering of the (consecutive) factors $u^{(1)}, \dots, u^{(7)}$ for two reasons. First, the sequence of seven factors does not include the pattern of the CS gadget as a subsequence. Second, this makes it easier for the RL agent to automatically detect and exploit the gadgetization patterns.

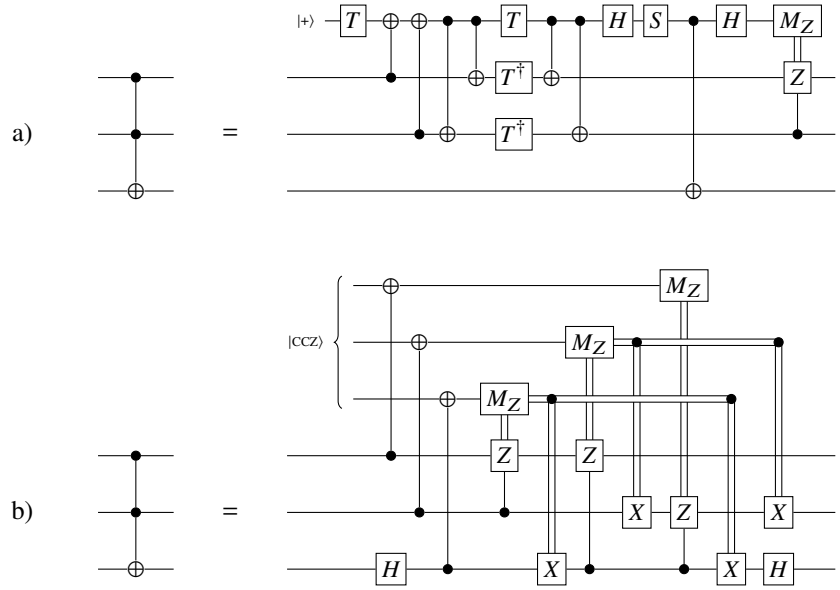


Figure 12: The Toffoli gate can be implemented efficiently using ancilla qubits, measurements, classically-controlled Clifford gates, and either four T gates (a) as in Jones [25], or a CCZ magic state (b) which are produced natively by the magic state factories of Gidney and Fowler [8] at a cost equivalent to two T gates.

which alternates self-attention operations with symmetrization layers. Each attention operation is depicted in Figure 15 and uses multi-head attention with 16 heads and head depth 32. The dense layer in Figure 15 consists in a feed-forward neural network with a single hidden layer that has twice as many hidden units as the input (and that applies the ReLU non-linearity).

Finally, the output of the torso is reshaped and passed to the policy and value heads (see Figure 14). The architecture of both the policy and value heads is taken from Fawzi et al. [24]. In particular, for the policy head, we use an autoregressive model that predicts $\widetilde{N}$ entries of $u^{(s)}$ at a time, where $\widetilde{N}$ divides N. For instance, we use $\widetilde{N} = 10$ for the experiments from Section 3.1 for which $N = 60$, and $\widetilde{N} = 12$ for the experiments on unary iteration for which $N = 72$.

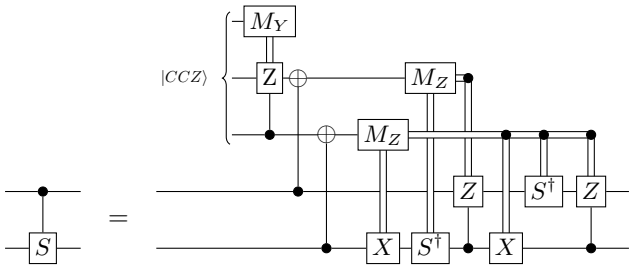


Figure 13: A CS gate can implemented efficiently [26, Appendix A] using ancilla qubits, measurements, classically-controlled Clifford gates and a CCZ magic state which is produced natively by the magic state factories of Gidney and Fowler [8] at a cost equivalent to two T gates.

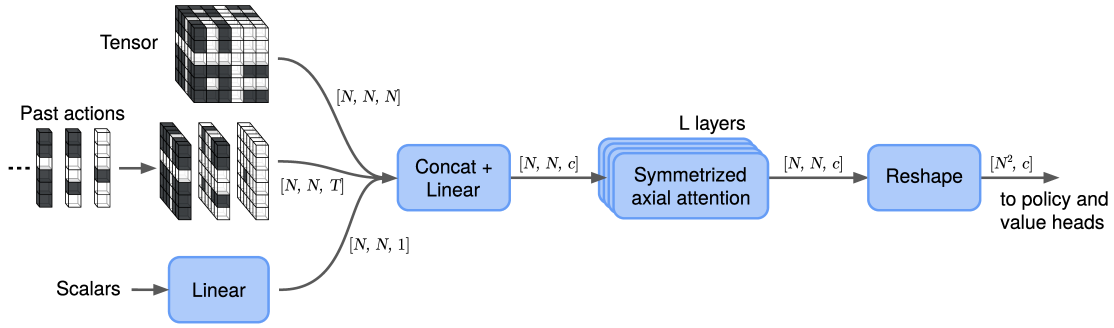


Figure 14: Overview of the neural network torso. Its main component is a stack of symmetrized axial attention layers.

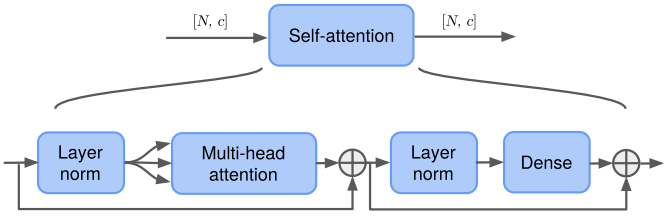


Figure 15: Overview of the self-attention operation. It features the standard components of a self-attention layer [60], including skip connections, layer normalization, multi-head attention, and a dense (fully connected) network with one hidden layer. When the input is a tensor of size $N \times N \times c$, the self-attention layer is applied independently over each row (or column) of the input.

D Additional Results

D.1 Additional Benchmark Circuits

The benchmark circuits GF(2^2)-mult and GF(2^3)-mult are derived from Cheung et al. [27] in the same way as the rest of the GF(2^m)-mult ($m \geq 4$) series of circuits available from Amy [36], using the irreducible polynomials $X^2 + X + 1$ and $X^3 + X + 1$ respectively. We include them to complete the series, since $m = 2$ is the smallest interesting instance ($m = 1$ corresponds to a single Toffoli gate). For reference, the source code is given in OpenQASM 2.0 format as follows:

Listing 1: GF(2^2)-mult

```
OPENQASM 2.0;
include "qelib1.inc";
qreg a[2];
qreg b[2];
qreg c[2];
ccx a[1], b[1], c[0];
cx c[0], c[1];
ccx a[0], b[0], c[0];
ccx a[1], b[0], c[1];
ccx a[0], b[1], c[1];
```

Listing 2: GF(2^3)-mult

```
OPENQASM 2.0;
include "qelib1.inc";
qreg a[3];
qreg b[3];
qreg c[3];
ccx a[1], b[2], c[0];
ccx a[2], b[1], c[0];
ccx a[2], b[2], c[1];
cx c[1], c[2];
cx c[0], c[1];
ccx a[0], b[0], c[0];
ccx a[1], b[0], c[1];
ccx a[0], b[1], c[1];
ccx a[2], b[0], c[2];
ccx a[1], b[1], c[2];
ccx a[0], b[2], c[2];
```

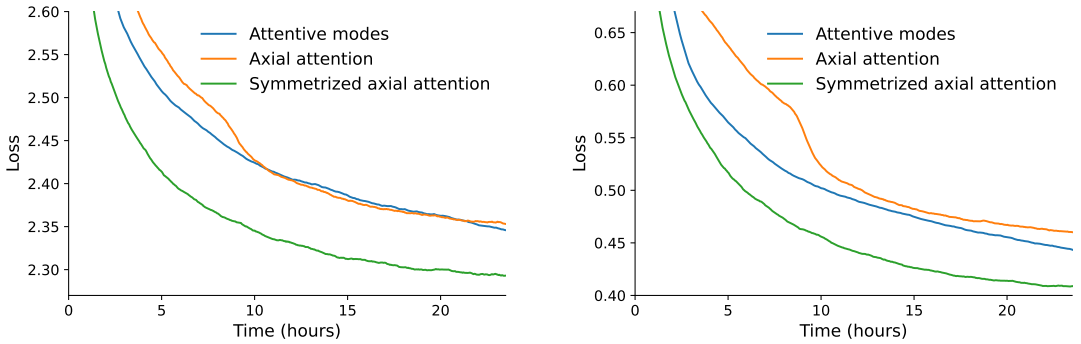
D.2 Benchmarks Circuits with a Larger Number of Qubits

Table 3 reports the T-count obtained on the circuits from Section 3.1 that were split into sub-circuits (i.e., the ones marked with * in Table 2), together with the T-count for each of the sub-circuits. For each sub-circuit, the baseline T-count was obtained as described in Appendix B.

D.3 Neural Network Ablations

Here we compare different architectures for the torso of the neural network (see Section 5.2 and appendix C.4).

Comparison across architectures. We first compare the following: (i) *attentive modes* [24], which uses attention operations across pairs of modes of the input tensor; (ii) *axial attention* [61], which we recover when we eliminate the symmetrization layers from our architecture in Figure 8b; and (iii) *symmetrized axial attention*, which is the architecture that we introduce in Section 5.2, featuring symmetrization layers.



(a) Policy loss (lower is better).

(b) Value loss (lower is better).

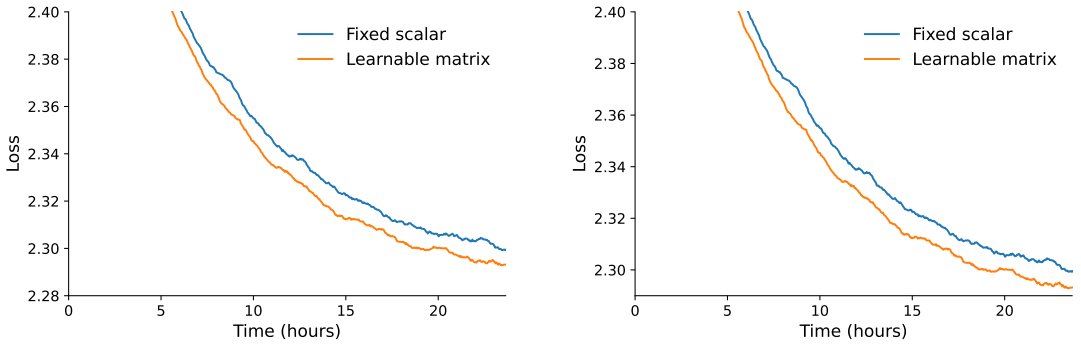
Figure 16: Comparison of different architectures for the torso of the neural network on a supervised experiment on $30 \times 30 \times 30$ tensors. Symmetrized axial attention performs best. The reported policy loss is the average cross-entropy loss for the first step of the autoregressive model. (For better visualization, all curves have been smoothed using an average moving window of size 20.)

We run each architecture on a *supervised* experiment with synthetic data only (i.e., a system without actors), using tensors of size $N = 30$, and compare the loss that they achieve. For each of the methods, we set the number of layers $L = 4$. For axial attention and symmetrized axial attention, this means 8 (batched) attention operations with N tokens each; for attentive modes, this means 12 (batched) attention operations with $2N$ tokens each. Therefore, we expect the architecture of Fawzi et al. [24] to run slower, and in fact we verify that its runtime is about 2x that of the other architectures: attentive modes runs at about 6 iterations per second, while axial attention and symmetrized axial attention complete 11.8 and 11.4 iterations per second, respectively.

We report in Figure 16 the training loss from each head of the neural network (the loss on a test set performs similarly). While axial attention and symmetrized axial attention run at approximately the same speed, the symmetrization layers bring a significant boost in performance. In contrast, although the architecture based on attentive modes performs almost as good as symmetrized axial attention if ran for longer, its decreased speed make it an impractical choice for AlphaTensor-Quantum (it also presents memory issues when applied on larger tensors).

Comparison of symmetrization layers. We now consider two different choices for the symmetrization layers. Recall that the symmetrization operation performs the operation $X \leftarrow \beta \odot X + (1 - \beta) \odot X^\top$. We refer to the choice of $\beta = \frac{1}{2}$ as *fixed scalar*, in contrast to letting $\beta \in \mathbb{R}^{N \times N}$ be a *learnable matrix*.

We compare both versions of symmetrization in Figure 17, where we report the training loss. Using a learnable matrix provides a small improvement over a fixed scalar (both choices run at approximately the same speed of 11.4 iterations per second). Letting β be a learnable matrix increases the number of model parameters by a negligible fraction (e.g., in this scenario, it adds 3 600 parameters to a model of approximately 132.7 million parameters).



(a) Policy loss (lower is better).

(b) Value loss (lower is better).

Figure 17: Comparison of different versions of the symmetrization operation, on a supervised experiment on $30 \times 30 \times 30$ tensors. Symmetrization layers with learnable parameters lead to some performance improvement. The reported policy loss is the average cross-entropy loss for the first step of the autoregressive model. (For better visualization, all curves have been smoothed using an average moving window of size 20.)

Circuit	#Qubits	Baselines	AlphaTensor-Quantum	
			Without gadgets	With gadgets
8-bit adder		129 [16]	139	**94** (33Tof + 28T)
block 1	31	50	**48**	**36** (11Tof + 14T)
block 2	50	95	**91**	**58** (22Tof + 14T)
Grover_5		44 [16]	152	66 (27Tof + 12T)
block 1	58	108	**107**	**44** (19Tof + 6T)
block 2	27	46	**45**	**22** (8Tof + 6T)
Hamming_{15} (high)		$787^{\dagger}$ (1010 [16])	**773**	**440** (173Tof + 2CS + 90T)
block 1	58	92	92	**52** (21Tof + 10T)
block 2	49	78	78	**44** (18Tof + 8T)
block 3	56	82	**81**	**50** (18Tof + 14T)
block 4	57	95	**94**	**55** (20Tof + 15T)
block 5	46	66	66	**40** (16Tof + 8T)
block 6	50	75	77	**45** (18Tof + 9T)
block 7	53	88	**86**	**50** (20Tof + 10T)
block 8	58	106	**100**	**53** (20Tof + 2CS + 9T)
block 9	59	105	**99**	**51** (22Tof + 7T)
Hamming_{15} (med)		$156^{\dagger}$ (162 [16])	156	**78** (35Tof + 8T)
block 1	25	38	38	**20** (8Tof + 4T)
block 2	59	118	118	**58** (27Tof + 4T)
Mod-Adder_{1024}		$798^{\dagger}$ (978 [16])	**762**	**500** (141Tof + 15CS + 188T)
block 1	54	77	**75**	**42** (16Tof + 10T)
block 2	52	71	**70**	**47** (12Tof + 23T)
block 3	53	80	**76**	**48** (16Tof + 16T)
block 4	55	86	**77**	**47** (15Tof + 17T)
block 5	54	75	**73**	**53** (15Tof + 5CS + 13T)
block 6	52	84	**83**	**58** (13Tof + 5CS + 22T)
block 7	59	89	**87**	**60** (19Tof + 22T)
block 8	56	80	**73**	**54** (7Tof + 5CS + 30T)
block 9	56	96	**89**	**58** (17Tof + 24T)
block 10	53	60	**59**	**33** (11Tof + 11T)
QCLA-Adder_{10}		116 [16]	135	**94** (28Tof + 5CS + 28T)
block 1	37	53	53	**43** (12Tof + 19T)
block 2	48	83	**82**	**51** (16Tof +5CS + 9T)
QCLA-Mod_7		165 [16]	199	**122** (43Tof + 36T)
block 1	50	95	95	**60** (21Tof + 18T)
block 2	59	107	**104**	**62** (22Tof + 18T)

Table 3: T-count achieved by different methods on a set of benchmark circuits, after splitting them into sub-circuits (blocks). AlphaTensor-Quantum finds good trade-offs between gadgets and T gates and outperforms the baseline methods for all circuits except Grover_5. The T-count from AlphaTensor-Quantum is obtained by additively combining all sub-circuits. The baseline T-count for the sub-circuits is given by a combination of methods (see Appendix B). For the shaded rows, the baseline corresponds to the lowest between the best reported value in the literature and the sum of the T-count across sub-circuits (the symbol † indicates it is the latter case and also includes in parenthesis the best reported value in the literature, for completeness). The notation "aTof+bCS +cT" indicates a Toffoli gates, b CS gates, and c T gates. Bold font highlights results with strictly better T-count than the baselines.

Structure optimization for parameterized quantum circuits

Mateusz Ostaszewski[1,2], Edward Grant[2,3], and Marcello Benedetti[2,4]

[1]Institute of Theoretical and Applied Informatics, Polish Academy of Sciences, Bałtycka 5, 44-100 Gliwice, Poland
[2]Department of Computer Science, University College London, WC1E 6BT London, United Kingdom
[3]Rahko Limited, N4 3JP London, United Kingdom
[4]Cambridge Quantum Computing Limited, CB2 1UB Cambridge, United Kingdom

We propose an efficient method for simultaneously optimizing both the structure and parameter values of quantum circuits with only a small computational overhead. Shallow circuits that use structure optimization perform significantly better than circuits that use parameter updates alone, making this method particularly suitable for noisy intermediate-scale quantum computers. We demonstrate the method for optimizing a variational quantum eigensolver for finding the ground states of Lithium Hydride and the Heisenberg model in simulation, and for finding the ground state of Hydrogen gas on the IBM Melbourne quantum computer.

1 Introduction

Methods for tuning the parameters of a quantum circuit to perform a specific task have several important applications. For example, these have been demonstrated in chemical simulation, combinatorial optimization, generative modeling and classification [1–8]. These tasks are usually performed by selecting a fixed circuit structure, parameterizing it using rotation gates, then iteratively updating the parameters to minimize an objective function estimated from measurements. Circuits of this type are known as parameterized quantum circuits. These methods are particularly promising for use with noisy intermediate-scale quantum (NISQ) computers because of their relative tolerance to noise compared to many other quantum algorithms.

In recent years a lot of progress has been made in improving the performance of parameterized quantum circuits including methods for calculating parameter gradients, hardware efficient ansätze, reducing the number of measurements required, and resolving problems with vanishing gradients [9–13]. Despite progress, many challenges remain. One of the most critical challenges is addressing the effects of noise. Generally, the effects of noise increase with the depth of the quantum circuit. For this reason it is highly desirable for a parameterized quantum circuit to be as shallow as possible whilst being expressive enough to perform the task at hand.

Few approaches have been proposed for optimizing the structure of a quantum circuit. In Ref. [14] the authors propose using a genetic algorithm for optimizing the circuit structure by selecting candidate gates from a set of allowed gates that are not parameterized. In Ref. [15] the authors propose growing the circuit by iteratively adding parameterized gates and re-optimizing the circuit using gradient descent.

In this work we propose a method for efficiently optimizing the structure of quantum circuits while optimizing an objective function, for example the energy given by a Hamiltonian operator. Gates are defined so that both the direction and angle of rotation are degrees of freedom. Whilst the angle is parameterized in a continuous manner, the direction is chosen from a set of generators, namely tensor products of Pauli matrices. Optimization is performed by selecting the first gate

Mateusz Ostaszewski: mm.ostaszewski@gmail.com
Edward Grant: edward.grant@rahko.ai
Marcello Benedetti: marcello.benedetti@cambridgequantum.com

and finding the direction and angle of rotation that yield minimum energy. This is performed iteratively for all the parameterized gates in the circuit. Once the last gate has been optimized the cycle is repeated until convergence.

In the Methods section we describe the approach in generality and then provide algorithms for the case of parameterized single-qubit gates and fixed two-qubit gates. This special case is interesting because it can be executed on all existing NISQ hardware and can provide a significant advantage as we show in the Results section. For single-qubit rotations about a fixed direction, the optimal angle can be found using three energy estimations. Independently from our work this property has been used as part of proposed methods for optimizing angles of rotation in quantum circuits [16, 17][‡]. Our method extends this and finds also the optimal direction within a set of canonical ones. This comes with very little overhead, since only seven energy estimations are required. Although we demonstrate the method on parameterized single-qubit gates and fixed entangling gates, the method can be applied to higher order gates. In the Discussion section we present possible extensions and ways to reduce the number or energy estimations required for our algorithms. We leave all the details to the Appendix.

2 Methods

Let us consider an optimization problem where the objective function is encoded in a Hermitian operator M and the candidate solution is encoded in a parameterized quantum circuit $U = U_D \cdots U_1$ acting on an n-qubit initial state ρ. Each gate is either fixed, e.g., a controlled-Z, or parameterized of the form $U_d = \exp\left(-i\frac{\theta_d}{2}H_d\right)$. Here, $\theta_d \in (-\pi, \pi]$ are angles of rotation and H_d are Hermitian and unitary matrices, e.g., tensor products of Pauli matrices. We collect these parameters into a real vector $\boldsymbol{\theta}$ and a vector of matrices $\boldsymbol{H}$, respectively.

Without loss of generality we consider the task of minimizing the objective function which we simply refer to as *the energy*. Using subscripts to indicate arguments of functions, we can state the problem as $(\boldsymbol{\theta}^*, \boldsymbol{H}^*) = \arg\min_{\boldsymbol{\theta},\boldsymbol{H}} \langle M\rangle_{\boldsymbol{\theta},\boldsymbol{H}}$, where $\langle M\rangle_{\boldsymbol{\theta},\boldsymbol{H}} = \mathrm{tr}\left(MU\rho U^\dagger\right)$.

To solve the problem we present two methods. The first fixes the circuit structure and optimizes only the angles; the second optimizes structure and angles simultaneously. Both methods rely on the fact that the expectation value as a function of an angle of rotation has sinusoidal form. That is, if we fix $\boldsymbol{H}$ as well as all the degrees of freedom in vector $\boldsymbol{\theta}$ except one, say θ_d, we can express the expectation as $\langle M\rangle_{\theta_d} = A\sin(\theta_d + B) + C$ where A, B and C are unknown coefficients. A detailed derivation is provided in Appendix A. Clearly, if we were able to estimate these coefficients, we could also characterize the sinusoidal form. This can be exploited to design opimization strategies.

Our first method is a coordinate optimization [18–20] algorithm applied to the angles of rotation. It finds the optimal angle for one gate while fixing all others to their current values, and sequentially cycles through all gates. This is rather simple to perform since at each step the energy has sinusoidal form with period 2π. For gate U_d, the optimal angle has a closed form expression

$$\begin{aligned}\theta_d^* &= \arg\min_{\theta_d} \langle M\rangle_{\theta_d} \\ &= \phi - \tfrac{\pi}{2} - \mathrm{arctan2}\left(2\langle M\rangle_{\phi} - \langle M\rangle_{\phi+\frac{\pi}{2}} - \langle M\rangle_{\phi-\frac{\pi}{2}},\ \langle M\rangle_{\phi+\frac{\pi}{2}} - \langle M\rangle_{\phi-\frac{\pi}{2}}\right) + 2\pi k,\end{aligned} \tag{1}$$

for any real ϕ and any integer k. In practice we select k such that $\theta_d^* \in (-\pi, \pi]$.

The optimal angle can be found for all $d = 1, \ldots, D$ in order to complete a cycle. Once all angles have been updated, a new cycle is initialized unless a stopping criterion is met. A number of potential stopping criteria could be used here. For example, one could stop after a fixed budget of K_1 cycles with the caveat that there may still be room for improvement. As another example, one could stop when the energy has not been significantly lowered for K_2 consecutive cycles. We call this algorithm `Rotosolve` and summarize it in Algorithm 1.

The choice of circuit structure is often based on prior knowledge about the problem as well as hardware constraints, e.g., qubit-to-qubit connectivity and gate set. It is reasonable to expect the chosen circuit structure to be suboptimal in the majority of cases. We believe there is large room

[‡]To the best of our knowledge this idea appeared in Nakanishi et al. (March 2019), Parrish et al. (April 2019) and this work (May 2019). Our manuscript was originally titled 'Quantum circuit structure learning'.

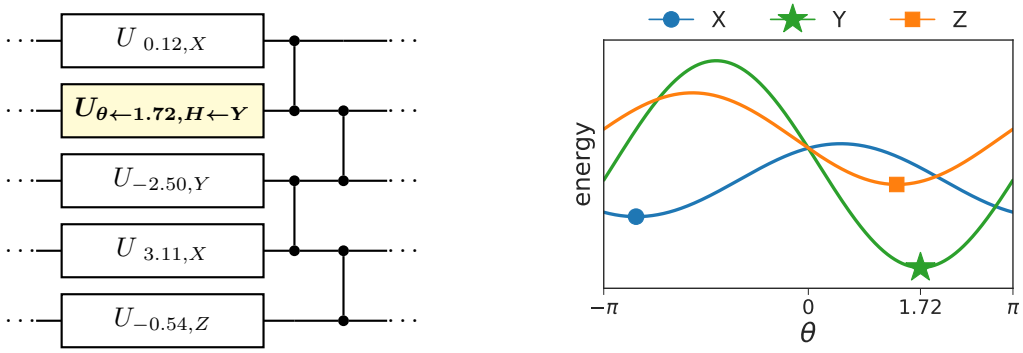


Figure 1: The gradient-free algorithm `Rotoselect` sequentially adjusts angles of rotation and circuit structure in order to efficiently minimize the energy. For each generator the energy is a sinusoidal function of θ with period 2π. In this example, the single-qubit gate coloured in yellow has been assigned generator Y and angle of rotation 1.72. This is the configuration attaining minimal energy, as shown in the right panel.

for improvement here. Our second method relaxes the constraint of the fixed circuit structure and optimizes it along with the angles. Similarly to the first method, we opt for a greedy approach optimizing one gate at a time.

Recall that in the parameterization considered here, n-qubit gates are generated by Hermitian and unitary matrices such as tensor products of Pauli matrices $H_d \in \{I, X, Y, Z\}^{\otimes n}$. Structure optimization aims at finding the optimal set of generators, which is clearly a daunting combinatorial problem. The general approach consists of using the expression in Eq. (1) to find minimizers $\theta_d^*(P) = \arg\min_{\theta_d} \langle M \rangle_{\theta_d, P}$ for all generators $P \in \{I, X, Y, Z\}^{\otimes n}$. Here the second subscript indicates that P is used as the generator for the d-th gate. Then, H_d is set to the generator giving lowest energy, and θ_d is set to the corresponding minimizer. This is repeated for $d = 1, \ldots, D$ to complete a cycle and the algorithm iterates until a stop criterion is triggered.

It is worth noting that to select the generator we need to know the energy attained by the corresponding optimal angle. In other words, given a generator P and its optimal angle $\theta_d^*(P)$, we need to calculate $\langle M \rangle_{\theta_d^*(P), P} = -A + C$. This can be extrapolated at no additional cost using the expressions for A and C provided in Appendix A.

The above is very general and indeed suffers from combinatorial explosion due to the 4^n possible choices for the generator of each gate. However, in practice we shall still comply with the constraints of the underlying NISQ hardware. Since we are not considering compilation of logical gates to physical ones, only a very small subset of generators is available, namely the native gate set of the hardware. For simplicity, we now consider optimizing the structure of single-qubit gates while employing a fixed layout of two-qubit entangling gates similar to the one in Ref. [21]. Figure 1 illustrates the circuit layer and shows an example of how the algorithm updates both the generator and the angle.

When structure optimization is limited to single-qubit gates, the generators can be selected such that $H_d \in \{X, Y, Z\}$. The identity generator is not required because it can be trivially obtained from any other generator by setting the angle to zero. In fact, we can exploit this to reduce the number of circuit evaluations per optimization step. According to Eq. (1), each of the three generators requires three circuit evaluations, for a total of 9 evaluations per optimization step. Recalling that ϕ can be chosen at will, we can set $\phi \leftarrow 0$ to obtain the identity gate for all three generators. Since $\langle M \rangle_{0,X} = \langle M \rangle_{0,Y} = \langle M \rangle_{0,Z}$, we can estimate this quantity once, effectively reducing the number of evaluations to 7 per optimization step. We call this algorithm `Rotoselect` and summarize it in Algorithm 2. We emphasize that `Rotosolve` is a classical optimization algorithm for circuit parameters. `Rotoselect` is an extension to `Rotosolve` that also allows for some optimization of the structure of the ansatz itself. Neither algorithm is tied to a specific ansatz.

Algorithm 1 `Rotosolve`

Input: Hermitian measurement operator M encoding the objective function, parameterized quantum circuit U with fixed structure, stopping criterion

Initialize $\theta_d \in (-\pi, \pi]$ for $d = 1, \ldots, D$ heuristically or at random
repeat
  for $d = 1, \ldots, D$ **do**
    Select a value $\phi \in \mathbb{R}$ heuristically or at random
    Fix all angles except the d-th one
    Estimate $\langle M \rangle_{\phi}$, $\langle M \rangle_{\phi+\frac{\pi}{2}}$ and $\langle M \rangle_{\phi-\frac{\pi}{2}}$ from samples
    $\theta_d \leftarrow \phi - \frac{\pi}{2} - \arctan2(2\langle M \rangle_{\phi} - \langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}},\ \langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}})$
until stopping criterion is met

Algorithm 2 `Rotoselect`

Input: Hermitian measurement operator M encoding the objective function, parameterized quantum circuit U, stopping criterion

Initialize $\theta_d \in (-\pi, \pi]$ and $H_d \in \{X, Y, Z\}$ for $d = 1, \ldots, D$ heuristically or at random
repeat
  for $d = 1, \ldots, D$ **do**
    Fix all angles and generators except for the d-th gate
    for $P \in \{X, Y, Z\}$ **do**
      Compute $\theta_d^*(P) = \arg\min_{\theta_d} \langle M \rangle_{\theta_d, P}$ using Eq. (1) with $\phi \leftarrow 0$
      Extrapolate $\langle M \rangle_{\theta_d^*(P), P}$ using the expressions in Appendix A
    $H_d \leftarrow \arg\min_P \langle M \rangle_{\theta_d^*(P), P}$
    $\theta_d \leftarrow \theta_d^*(H_d)$
until stopping criterion is met

We conclude this section with a note about convergence. On a quantum computer the objective function is stochastic because $\langle M \rangle_{\boldsymbol{\theta}, \boldsymbol{H}}$ is estimated from samples. For implementation in NISQ computers the operator is typically described as the weighted sum of a polynomial number of terms $M = \sum_i w_i M_i$. In this case the expectation is given by $\langle M \rangle_{\boldsymbol{\theta}, \boldsymbol{H}} = \sum_i w_i \langle M_i \rangle_{\boldsymbol{\theta}, \boldsymbol{H}}$. Assuming an infinite number of measurements the objective function is deterministic and easier to analyze. Bertsekas [18] provides convergence results for coordinate minimization algorithms under rather general conditions and including non-convex objective functions. An analysis along these lines could be attempted for `Rotosolve`. In `Rotoselect` the analysis is complicated by the fact that multiple generators could lead to equally good minima. That is, at each given optimization step the solution may not be unique.

3 Results

In this section we first study the impact of structure optimization on the variational quantum eigensolver (VQE) for the Heisenberg model and for Lithium Hydride. `Rotoselect` differs from `Rotosolve` only because it employs structure optimization. Any difference in performance between these two algorithms can therefore be attributed to structure optimization. Second, we demonstrate `Rotoselect` on the IBM Melbourne quantum computer by performing VQE for Hydrogen. Thirdly, we show that an unfavorable initial choice of circuit structure can be boosted using our method. Finally, we compare our algorithms with other state-of-the-art optimizers and find significant improvements in terms of performance and scaling.

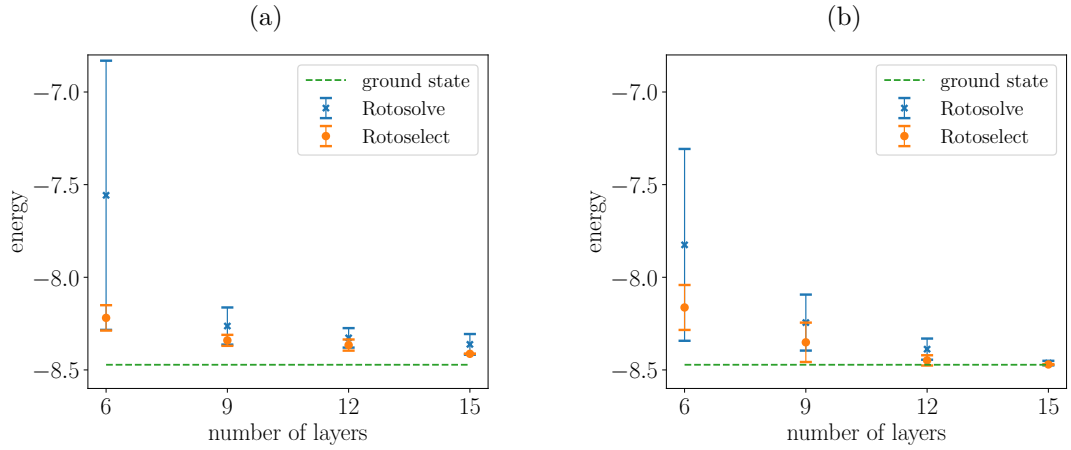


Figure 2: Mean and standard deviation of energy across trials as a function of number of circuit layers comparing `Rotosolve` and `Rotoselect` for optimizing a VQE to minimize the energy of the cyclic spin chain Heisenberg model on 5 qubits. In (a) 1000 measurements for each Hamiltonian term were used to approximate the energy. In (b) the exact energy was calculated.

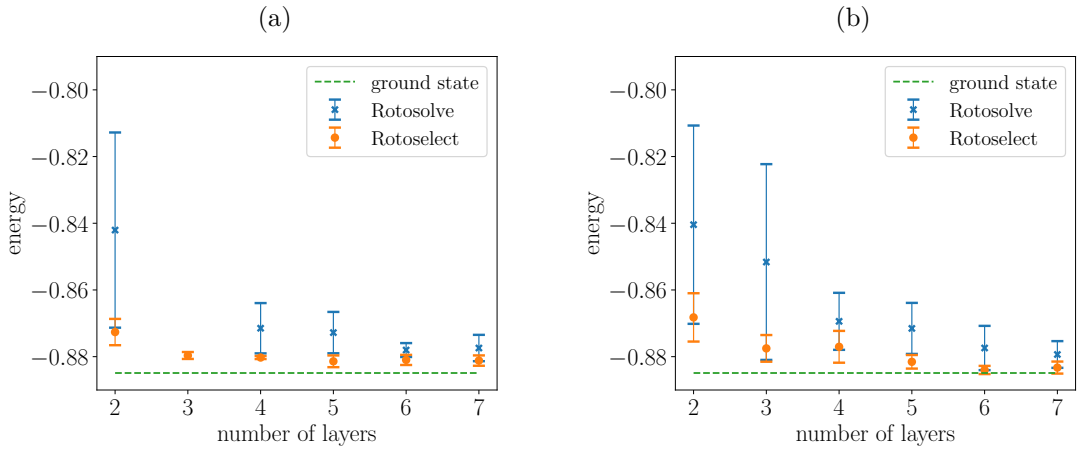


Figure 3: Mean and standard deviation of energy across trials as a function of number of circuit layers comparing `Rotosolve` and `Rotoselect` for optimizing a VQE to minimize the molecular Hamiltonian for LiH. In (a) 1000 measurements for each Hamiltonian term were used to approximate the energy. We do not include the results for `Rotosolve` on 3 layers because the mean energy lied above the plotted range due to the presence of an outlier. In (b) the exact energy was calculated.

3.1 Performance on the variational quantum eigensolver

We considered the problem of finding the ground state energy of the 5-qubit Heisenberg model on a 1D lattice with periodic boundary conditions and in the presence of an external magnetic field. The corresponding Hamiltonian reads

$$M = J \sum_{(i,j)\in\mathcal{E}} (X_i X_j + Y_i Y_j + Z_i Z_j) + h \sum_{i\in\mathcal{V}} Z_i, \tag{2}$$

where $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is the undirected graph of the lattice with 5 nodes, J is the strength of the spin-spin interactions, and h is to the strength of the magnetic field in the Z-direction. For $J/h = 1$ the ground state is known to be highly entangled (see Ref. [10] for VQE simulations on the Heisenberg model). We chose $J = h = 1$.

Circuits were initialized using the strategy described in Ref. [13]. The circuit form consisted of layers of parameterized single-qubit rotations followed by a ladder of controlled-Z gates [21]. An example of this type of layer is shown in Fig. 1. Optimization was performed for 1000 cycles.

Figure 2 reports mean and standard deviation of the lowest energy encountered during optimization across 10 trials for circuits with $6, 9, 12$ and 15 layers. Panel (a) shows results when 1000 measurements were used to estimate the expectation of each term in the sum in Eq. (2), while panel (b) shows oracular results in the limit of infinite measurements. For all number of layers, `Rotoselect` achieved better mean, standard deviation, and absolute lowest energy than `Rotosolve`. In particular for the lower depth circuits `Rotoselect` achieves a much smaller standard deviation than `Rotosolve`. This is likely due to the fact that the generators are initialized at random and `Rotosolve` is unable to change an initial bad condition. Given the limited capacity of low-depth circuits, a good choice for the generators appears to be particularly important. As the number of layers increases, both algorithms find better approximations to the ground state. Both algorithms exhibit robustness to the finite number of measurement employed here.

The experiment was repeated on the 4-qubit chemical Hamiltonian for Lithium Hydride (LiH) at bond distance. To construct the Hamiltonian we used the coefficients and the Pauli terms given in Ref. [10], Supplementary Material. Figure 3 shows mean and standard deviation of the lowest energy encountered across 10 trials for circuits up to 7 layers. Panel (a) shows results when expectations are estimated from 1000 measurements, while panel (b) shows results in the limit of infinite measurements. The results are consistent with the previous experiment.

3.2 Demonstration on the IBM Melbourne computer

We compared the performance of `Rotoselect` on the 14-qubit IBM Melbourne quantum computer against a quantum simulator. For this test we chose to find the ground state energy of the 2-qubit Hydrogen Hamiltonian, using the coefficients and the Pauli terms given in Ref. [10], Supplementary Material. The circuit had 2 layers and a total of 4 adjustable angles. In contrast to previous experiments where we used controlled-Z entangling gates, here we used CNOTs as they belong to the native gate alphabet of this device.

We also characterized the measurement noise using an off-the-shelf method provided by Qiskit Ignis [22], namely the tensored measurement calibration. A general measurement error calibration prepares each of the 2^n basis states and immediately measures them. The statistics are then used to calculate a calibration matrix which is applied in the classical post-processing of subsequent runs. Tensored measurement calibration assumes that errors are local to subsets of qubits, hence reducing the requirements for calculating the calibration matrix. In our experiments, we assumed that measurement errors are local to each qubit. In the following analysis, we do not include the overhead from the calibration since it was performed only once at the beginning of each trial.

We ran 5 trials with randomly initialized circuits and taken 1000 measurements for each Hamiltonian term when estimating the energy. Figure 4 shows the mean and one standard deviation of the energy as a function of the number of evaluations for two full cycles of `Rotoselect`. Despite error mitigation, results on the quantum device (blue dots) are in average slightly worse than those from the simulator (orange crosses). Yet we were able to get close to the ground state within the 56 evaluations required to complete two cycles.

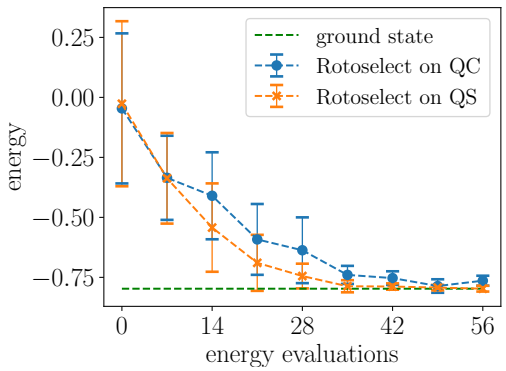


Figure 4: Mean and standard deviation of the energy as a function of the number of evaluations for the Hydrogen Hamiltonian. Blue dots correspond to experiments on the IBM Melbourne quantum computer (QC), and orange crosses corresponds to experiments on the quantum simulator (QS).

3.3 Optimization of circuits with limited expressibility

In Ref. [23] the authors define "expressibility" as the circuit's ability to generate pure states that are well representative of the Hilbert space. To estimate expressibility, they repeatedly sample the adjustable angles of the circuit at random in order to generate a distribution of quantum states; then, they use a suitable divergence to compare this distribution to the uniform distribution of quantum states. Small divergence means high expressibility.

As a function of the number of layers, expressibility improves at different rates depending on the circuit structure. For some choices of structure, additional layers do not yield improvements and expressibility saturates rather quickly. The authors suggest that information about expressibility saturation could be valuable when selecting the circuit structure.

In this section, we use expressibility to select an unfavorable circuit and show that structure optimization can turn it into a useful one. In particular, we required a circuit structure for which expressibility is poor and saturates quickly. We chose circuit #15 from the pool of circuits studied in Ref. [23]. Figure 5 (a) shows the corresponding circuit diagram for 4 qubits. Our task consisted of maximizing the overlap of the state generated by the circuit with a target state sampled uniformly at random. To avoid potential misunderstanding, we stress that our task is not that of maximizing the expressibility, although the two may be related. In practice, we minimized the energy for operator $M = -|\phi\rangle\langle\phi|$ where $|\phi\rangle$ is the random target state.

For this simulation, we used the exact energy and we varied the number of layers in $\{1, \ldots, 7\}$. We stopped the algorithms after 50 cycles. Figure 5 (b) shows the average trace distance and standard deviation as a function of layers across 10 random target states for circuit #15. `Rotosolve` is not able to take the trace distance below a certain value, even when adding more layers. On the other hand, `Rotoselect` achieves lower trace distance for all choices of the number of layers and approaches zero for 7 layers. This indicates that an unfavorable initial choice of circuit structure can be boosted using our method.

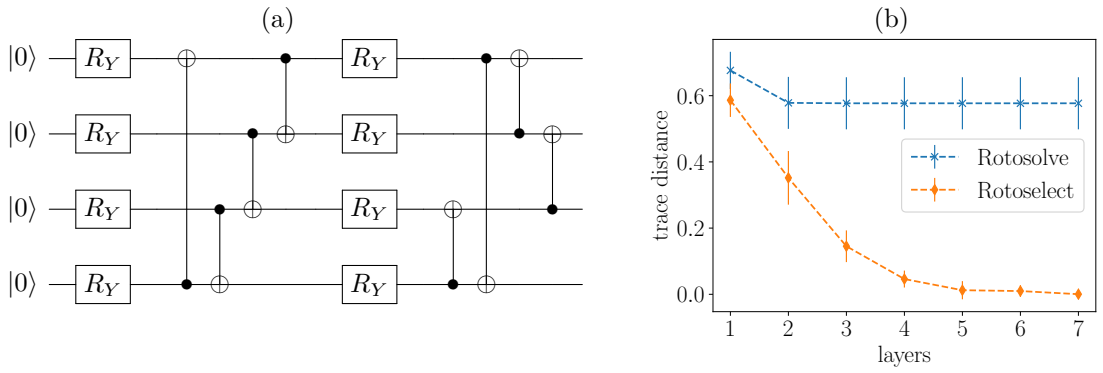


Figure 5: (a) Diagram for circuit #15 from Ref. [23]. (b) Mean and standard deviation of the trace distance to uniformly random states as a function of number of circuit layers for `Rotosolve` and `Rotoselect` on circuit #15.

3.4 Comparison of optimization algorithms and scaling

In this experiment we compare the optimization time and optimization time scaling in the system size for `Rotosolve`, `Rotoselect`, `SPSA` [24] and gradient descent with `Adam` [25].

Figure 6 (a) shows the energy and standard deviation as a function of the number of energy evaluations across 5 trials for the 5-qubit Heisenberg cyclic spin chain described in the Results section. The depth of the circuit was 30 layers. The exact energy was used to perform updates. The learning rate for `Adam` was set to 0.05. The hyperparameter for `SPSA` were the same as in Ref. [10]. Both `Rotosolve` and `Rotoselect` converged with significantly fewer energy evaluations than `Adam` or `SPSA`.

Figure 6 (b) shows the mean and standard deviation for the number of evaluations needed to find the ground state of the Heisenberg cyclic spin chain as a function of the size of the system. The ground state was considered found if the candidate solution had energy within 2% from the actual ground state energy. More precisely, this threshold is in relation to the normalized distance

between the candidate energy and the minimum eigenvalue expressed by $\frac{\langle M\rangle - E_{\min}}{E_{\max} - E_{\min}}$. It should be noted, that in the experiments there was an additional stop condition, which stopped algorithm after 100,000 cycles. Five trials were performed to estimate each mean and standard deviation. 1000 measurements were performed for each Hamiltonian term in order to approximate the energy for each evaluation. Circuit depth was $3n^2/2+2n$ for an even number of qubits and $3(n^2-1)/2+2n$ for odd where n was the number of qubits.

Both `Rotosolve` and `Rotoselect` converged faster than `Adam` or `SPSA` for the number of qubits tested, indicating a favorable scaling.

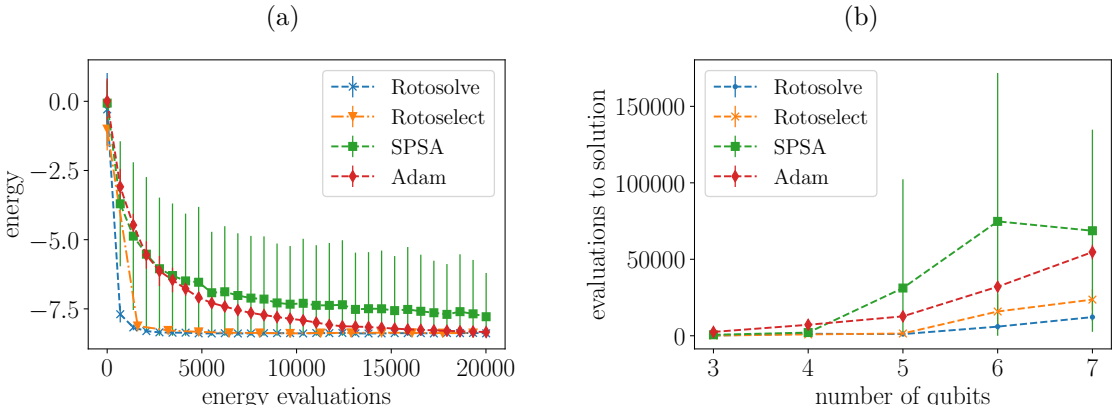


Figure 6: A comparison of `Rotosolve`, `Rotoselect`, `SPSA` and `Adam`. (a) Energy as a function of number of energy evaluations for the 5-qubit Heisenberg cyclic spin chain. (b) Number of energy evaluations to solution as a function of the number of qubits for the Heisenberg cyclic spin chain.

4 Discussion

The proposed algorithms can be extended in a number of ways. In the context of circuit optimization, `Rotosolve` can be generalized to find minimizers of K angles of rotation at the same time and at the cost of evaluating 3^K circuits. While this is an exponential cost, for small K this approach is computationally feasible and may provide an advantage. In Ref. [17] the authors explore this idea for $K \in \{1, 2, 3\}$ where subsets of angles are chosen at random. This approach belongs to the class of algorithms called coordinate block minimization [20].

In the context of circuit structure optimization, `Rotoselect` can be generalized to incrementally grow circuits from scratch rather than starting from random initial structures. This is similar in spirit to Adapt-VQE [15], but with the advantage that each new gate is optimized efficiently in closed form rather than using gradient-based optimization. Furthermore, we could efficiently remove gates by assessing whether they are contributing to the solution (e.g., a redundant gate would generate sinusoidal forms that appear to be flat).

Another interesting extension is to use `Rotoselect` to learn the optimal connectivity layout for the entangling gates. As an example consider trapped ion computers that can implement fully connected layers of Mølmer-Sørensen gates [26]. This choice provides high expressive power in low-depth circuits [4], but must be balanced against the potentially slow clock speed of these entangling gates [27] and potential gate errors. Our algorithm could find this sweet spot. Since with n qubits we must evaluate $n(n-1)/2$ choices for non-directional two-qubit gates in a layer, this approach is also efficient. We discuss other generalizations in Appendix B.

Heuristics can be used to speedup our methods. The most striking example consists of reusing information to reduce the number of energy evaluations. By appropriately choosing the offset ϕ in Eq. (1), one obtains an angle for which the energy is already known from the previous update. Thus one only needs to evaluate the remaining two energies to perform the current update. By applying this trick systematically one reduces the number of evaluations from 3 to 2 for `Rotosolve`, and from 7 to 6 for `Rotoselect`. While the result is exact in the limit of infinite number of measurement, in realistic scenarios this approximation introduces noise and its effects should be investigated in future work.

In our simulations we also found that `Rotoselect` tends to change the structure only in the early stage of optimization. One could detect this and switch from `Rotoselect` to `Rotosolve` further reducing the number of circuit evaluations.

Quantum circuit structure optimization provides an efficient means for improving the expressivity of low-depth quantum circuits with only a small computational overhead. This characteristic makes the described methods particularly suitable for deployment on near-term quantum computers where circuit depth and optimization time are significant bottlenecks.

Acknowledgements

M.B. is supported by the UK Engineering and Physical Sciences Research Council (EPSRC) and by Cambridge Quantum Computing Limited (CQC). E.G. is supported by ESPRC [EP/P510270/1]. M.O. acknowledges support from Polish National Science Center scholarship 2018/28/T/ST6/00429. We thank Leonard Wossnig for helpful discussions. We gratefully acknowledge the support of NVIDIA Corporation with the donation of the Titan Xp GPU used for this research. This research was supported in part by PLGrid Infrastructure.

References

[1] Alberto Peruzzo et al. "A variational eigenvalue solver on a photonic quantum processor". In: *Nature Communications* 5.1 (2014), p. 4213. DOI: 10.1038/ncomms5213.

[2] E. Farhi et al. *Quantum Algorithms for Fixed Qubit Architectures*. 2017. arXiv: 1703.06199 [quant-ph].

[3] Edward Farhi and Hartmut Neven. *Classification with Quantum Neural Networks on Near Term Processors*. 2018. arXiv: 1802.06002 [quant-ph].

[4] Marcello Benedetti et al. "A generative modeling approach for benchmarking and training shallow quantum circuits". In: *npj Quantum Information* 5.1 (May 2019). DOI: 10.1038/s41534-019-0157-8.

[5] Marcello Benedetti et al. "Adversarial quantum circuit learning for pure state approximation". In: *New Journal of Physics* 21.4 (Apr. 2019), p. 043023. DOI: 10.1088/1367-2630/ab14b5.

[6] Hongxiang Chen et al. *Universal discriminative quantum neural networks*. 2018. arXiv: 1805.08654 [quant-ph].

[7] Edward Grant et al. "Hierarchical quantum classifiers". In: *npj Quantum Information* 4.1 (2018), pp. 1–8. DOI: 10.1038/s41534-018-0116-9.

[8] Marcello Benedetti et al. "Parameterized quantum circuits as machine learning models". In: *Quantum Science and Technology* 4.4 (Nov. 2019), p. 043001. DOI: 10.1088/2058-9565/ab4eb5.

[9] K. Mitarai et al. "Quantum circuit learning". In: *Phys. Rev. A* 98 (3 Sept. 2018), p. 032309. DOI: 10.1103/PhysRevA.98.032309.

[10] Abhinav Kandala et al. "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets". In: *Nature* 549.7671 (Sept. 2017), 242–246. DOI: 10.1038/nature23879.

[11] Kosuke Mitarai, Tennin Yan, and Keisuke Fujii. "Generalization of the Output of a Variational Quantum Eigensolver by Parameter Interpolation with a Low-depth Ansatz". In: *Phys. Rev. Applied* 11 (4 Apr. 2019), p. 044087. DOI: 10.1103/PhysRevApplied.11.044087.

[12] Artur F. Izmaylov, Tzu-Ching Yen, and Ilya G. Ryabinkin. "Revising the measurement process in the variational quantum eigensolver: is it possible to reduce the number of separately measured operators?" In: *Chem. Sci.* 10 (13 2019), pp. 3746–3755. DOI: 10.1039/C8SC05592K.

[13] Edward Grant et al. "An initialization strategy for addressing barren plateaus in parametrized quantum circuits". In: *Quantum* 3 (Dec. 2019), p. 214. DOI: 10.22331/q-2019-12-09-214.

[14] Rui Li et al. "Approximate Quantum Adders with Genetic Algorithms: An IBM Quantum Experience". In: *Quantum Measurements and Quantum Metrology* 4.1 (26 Jul. 2017), pp. 1–7. DOI: https://doi.org/10.1515/qmetro-2017-0001.

[15] Harper R. Grimsley et al. "An adaptive variational algorithm for exact molecular simulations on a quantum computer". In: *Nature Communications* 10.1 (July 2019). DOI: 10.1038/s41467-019-10988-2.

[16] Ken M. Nakanishi, Keisuke Fujii, and Synge Todo. "Sequential minimal optimization for quantum-classical hybrid algorithms". In: *Phys. Rev. Research* 2 (4 Oct. 2020), p. 043158. DOI: 10.1103/PhysRevResearch.2.043158.

[17] Robert M. Parrish et al. *A Jacobi Diagonalization and Anderson Acceleration Algorithm For Variational Quantum Algorithm Parameter Optimization*. 2019. arXiv: 1904.03206 [quant-ph].

[18] D.P. Bertsekas. *Nonlinear Programming*. Athena Scientific, 1999.

[19] Ankan Saha and Ambuj Tewari. *On the Finite Time Convergence of Cyclic Coordinate Descent Methods*. 2010. arXiv: 1005.2146 [cs.LG].

[20] Stephen J Wright. "Coordinate descent algorithms". In: *Mathematical Programming* 151.1 (2015), pp. 3–34. DOI: 10.1007/s10107-015-0892-3.

[21] Jarrod R. McClean et al. "Barren plateaus in quantum neural network training landscapes". In: *Nature Communications* 9.1 (Nov. 2018). DOI: 10.1038/s41467-018-07090-4.

[22] Héctor Abraham et al. *Qiskit: An Open-source Framework for Quantum Computing*. 2019. DOI: 10.5281/zenodo.2562110.

[23] Sukin Sim, Peter D. Johnson, and Alán Aspuru-Guzik. "Expressibility and Entangling Capability of Parameterized Quantum Circuits for Hybrid Quantum-Classical Algorithms". In: *Advanced Quantum Technologies* 2.12 (2019), p. 1900070. DOI: https://doi.org/10.1002/qute.201900070.

[24] J. C. Spall. "Multivariate stochastic approximation using a simultaneous perturbation gradient approximation". In: *IEEE Transactions on Automatic Control* 37.3 (1992), pp. 332–341. DOI: 10.1109/9.119632.

[25] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2014. arXiv: 1412.6980 [cs.LG].

[26] Anders Sørensen and Klaus Mølmer. "Quantum Computation with Ions in Thermal Motion". In: *Phys. Rev. Lett.* 82 (9 Mar. 1999), pp. 1971–1974. DOI: 10.1103/PhysRevLett.82.1971.

[27] Norbert M. Linke et al. "Experimental comparison of two quantum computing architectures". In: *Proceedings of the National Academy of Sciences* 114.13 (2017), pp. 3305–3310. DOI: 10.1073/pnas.1618020114.

A Sinusoidal form of expectation values

Recall that we consider circuits $U = U_D \cdots U_1$ where each gate is either fixed, e.g., the controlled-Z, or parameterized. We consider parameterized gates of the kind $U_d = \exp\left(-i\frac{\theta_d}{2}H_d\right)$, where $\theta_d \in (-\pi, \pi]$ and the generator H_d is a Hermitian and unitary matrix such that $H_d^2 = I$. Using the definition of matrix exponential we obtain

$$\begin{aligned} U_d &= \sum_{k=0}^{\infty} \frac{(-i)^k (\frac{\theta_d}{2})^k H_d^k}{k!} \\ &= \sum_{k=0}^{\infty} \frac{(-i)^{2k} (\frac{\theta_d}{2})^{2k} H_d^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{(-i)^{2k+1} (\frac{\theta_d}{2})^{2k+1} H_d^{2k+1}}{(2k+1)!} \\ &= \sum_{k=0}^{\infty} \frac{(-1)^k (\frac{\theta_d}{2})^{2k}}{(2k)!} I - i \sum_{k=0}^{\infty} \frac{(-1)^k (\frac{\theta_d}{2})^{2k+1}}{(2k+1)!} H_d \\ &= \cos\left(\tfrac{\theta_d}{2}\right) I - i \sin\left(\tfrac{\theta_d}{2}\right) H_d. \end{aligned} \tag{3}$$

As an example of generators with this property, we could use tensor products of Pauli matrices $H_d \in \{I, X, Y, Z\}^{\otimes n}$, where n is the number of qubits.

We apply the circuit to a fiducial state $\bar{\rho}$ and then measure a Hermitian operator $\bar{M}$ encoding the objective function. From measurement outputs we can estimate the expectation

$$\langle \bar{M} \rangle = \mathrm{tr}\Big(\bar{M} U_D \cdots U_d \cdots U_1 \bar{\rho} U_1^\dagger \cdots U_d^\dagger \cdots U_D^\dagger\Big). \tag{4}$$

We want to analyze this expectation as a function of a single parameter θ_d. To simplify the notation we absorb all gates before U_d in the density operator, that is $\rho = U_{d-1} \cdots U_1 \bar{\rho} U_1^\dagger \cdots U_{d-1}^\dagger$. Similarly, we absorb all gates after U_d in the measurement operator $M = U_{d+1}^\dagger \cdots U_D^\dagger \bar{M} U_D \cdots U_{d+1}$. This can be done because unitary transformations preserve the Hermiticity of both density and measurement operators. Using this notation Eq. (4) can be written as $\langle M \rangle = \mathrm{tr}\Big(M U_d \rho U_d^\dagger\Big)$.

In the following discussion we will need to evaluate the expectation at different parameter values for gate U_d. A further change in notation helps. From now on, we drop index d and use subscripts to indicate parameter value. Using the new notation we write

$$\begin{aligned}
\langle M \rangle_\theta &= \mathrm{tr}\left(M U_\theta \rho U_\theta^\dagger\right) \\
&= \mathrm{tr}\left(M \left(\cos\left(\tfrac{\theta}{2}\right) I - i \sin\left(\tfrac{\theta}{2}\right) H\right) \rho \left(\cos\left(\tfrac{\theta}{2}\right) I + i \sin\left(\tfrac{\theta}{2}\right) H\right)\right) \\
&= \cos^2\left(\tfrac{\theta}{2}\right) \mathrm{tr}\left(M\rho\right) + i \sin\left(\tfrac{\theta}{2}\right) \cos\left(\tfrac{\theta}{2}\right) \mathrm{tr}\left(M\left[\rho, H\right]\right) + \sin^2\left(\tfrac{\theta}{2}\right) \mathrm{tr}\left(MH\rho H\right).
\end{aligned} \tag{5}$$

Let us inspect the third line of this equation. First, we note that $\mathrm{tr}(M\rho)$ is simply the expectation when the circuit is evaluated at $\theta = 0$. That is, $\langle M \rangle_0 = \mathrm{tr}(M\rho)$. Second, the term $i\,\mathrm{tr}(M[\rho, H])$ can be written as the difference of two expectations. Indeed, using two independent circuits $U_{\pm\frac{\pi}{2}} = \exp\left(\mp i \tfrac{\pi}{4} H\right) = \frac{1}{\sqrt{2}}(I \mp iH)$ we have

$$\begin{aligned}
\langle M \rangle_{\frac{\pi}{2}} - \langle M \rangle_{-\frac{\pi}{2}} &= \mathrm{tr}\left(M U_{\frac{\pi}{2}} \rho U_{\frac{\pi}{2}}^\dagger\right) - \mathrm{tr}\left(M U_{-\frac{\pi}{2}} \rho U_{-\frac{\pi}{2}}^\dagger\right) \\
&= \tfrac{1}{2}\left(\mathrm{tr}\left(M\left(I - iH\right)\rho\left(I + iH\right)\right) - \mathrm{tr}\left(M\left(I + iH\right)\rho\left(I - iH\right)\right)\right) \\
&= -i\,\mathrm{tr}\left(MH\rho\right) + i\,\mathrm{tr}\left(M\rho H\right) \\
&= i\,\mathrm{tr}\left(M[\rho, H]\right).
\end{aligned} \tag{6}$$

Third, the last term $\mathrm{tr}\left(MH\rho H\right)$ is the expectation obtained evaluating the circuit at $\theta = \pi$. That is, using circuit $U_\pi = \exp\left(-i\tfrac{\pi}{2} H\right) = -iH$ we have $\langle M \rangle_\pi = \mathrm{tr}\left(MH\rho H\right)$.

Putting these three pieces back in Eq. (4) and using well known trigonometric identities, we obtain

$$\begin{aligned}
\langle M \rangle_\theta &= \cos^2\left(\tfrac{\theta}{2}\right) \langle M \rangle_0 + \sin\left(\tfrac{\theta}{2}\right) \cos\left(\tfrac{\theta}{2}\right) \left(\langle M \rangle_{\frac{\pi}{2}} - \langle M \rangle_{-\frac{\pi}{2}}\right) + \sin^2\left(\tfrac{\theta}{2}\right) \langle M \rangle_\pi \\
&= \tfrac{1+\cos(\theta)}{2} \langle M \rangle_0 + \tfrac{\sin(\theta)}{2} \left(\langle M \rangle_{\frac{\pi}{2}} - \langle M \rangle_{-\frac{\pi}{2}}\right) + \tfrac{1-\cos(\theta)}{2} \langle M \rangle_\pi \\
&= \tfrac{\cos(\theta)}{2} \left(\langle M \rangle_0 - \langle M \rangle_\pi\right) + \tfrac{\sin(\theta)}{2} \left(\langle M \rangle_{\frac{\pi}{2}} - \langle M \rangle_{-\frac{\pi}{2}}\right) + \tfrac{1}{2}\left(\langle M \rangle_0 + \langle M \rangle_\pi\right).
\end{aligned} \tag{7}$$

We now use the identity $a\cos(x) + b\sin(x) = \sqrt{a^2 + b^2}\sin\left(x + \arctan\left(\tfrac{a}{b}\right)\right)$ to obtain a compact expression. The expectation value then reads

$$\begin{aligned}
\langle M \rangle_\theta &= A \sin(\theta + B) + C, \\
A &= \tfrac{1}{2}\sqrt{\left(\langle M \rangle_0 - \langle M \rangle_\pi\right)^2 + \left(\langle M \rangle_{\frac{\pi}{2}} - \langle M \rangle_{-\frac{\pi}{2}}\right)^2}, \\
B &= \mathrm{arctan2}\left(\langle M \rangle_0 - \langle M \rangle_\pi, \langle M \rangle_{\frac{\pi}{2}} - \langle M \rangle_{-\frac{\pi}{2}}\right), \\
C &= \tfrac{1}{2}\left(\langle M \rangle_0 + \langle M \rangle_\pi\right).
\end{aligned} \tag{8}$$

That is, the expectation of M as a function of a single parameter has sinusoidal form with amplitude A, phase B, intercept C, and period 2π. An example is shown in Figure 7.

Note that we must use the arctan2 function in order to correctly handle the sign of numerator and denominator, as well as the case where the denominator is zero. To see why, assume H commutes either with M or ρ. Then, Eq. (6) evaluates to zero and triggers a division by zero in the standard arctan.

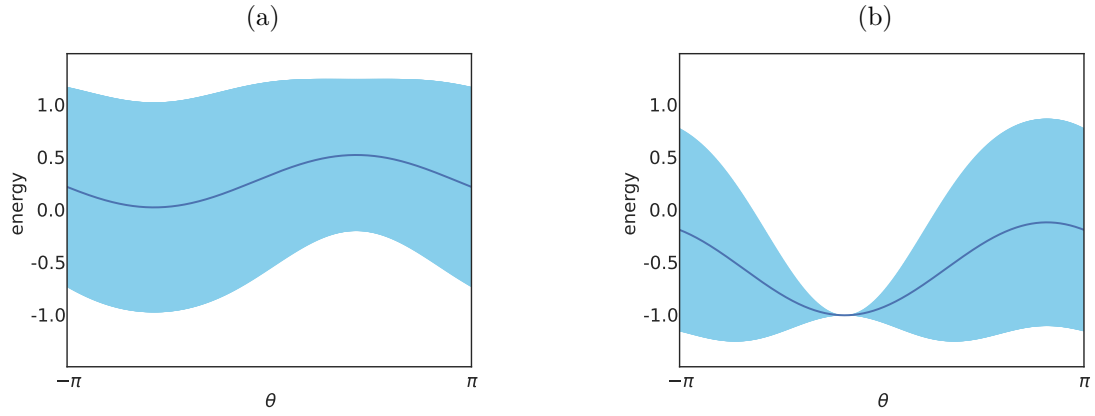


Figure 7: Expectation and variance of Hermitian observable $ZXZX$ as a function of two different angles of rotation in a random circuit. The expectation value is always of sinusoidal form with period 2π. (a) The variance is large because this angle cannot yield an eigenstate of $ZXZX$. (b) Both expectation and variance are minimized by setting this second angle to $\theta^* \approx -0.5$.

Now, from the graph of the sine function, it is easy to locate the minima at $\theta^* = -\frac{\pi}{2} - B + 2\pi k$ for all $k \in \mathbb{Z}$. The estimator for B in Eq. (8) seems to require four distinct circuit evaluations. Using simple trigonometry we generalize the estimator and show that only three evaluations are required. Let us write

$$\begin{aligned} \langle M \rangle_{\phi+\frac{\pi}{2}} &= A \sin\left(\phi + \tfrac{\pi}{2} + B\right) + C \\ &= -A \sin\left(\phi - \tfrac{\pi}{2} + B\right) + C \\ &= -\langle M \rangle_{\phi-\frac{\pi}{2}} + 2C. \end{aligned} \tag{9}$$

From this we obtain the general estimator for C

$$C = \tfrac{1}{2}\left(\langle M \rangle_{\phi+\frac{\pi}{2}} + \langle M \rangle_{\phi-\frac{\pi}{2}}\right). \tag{10}$$

We can write

$$\tan\left(\phi + B\right) = \frac{\sin\left(\phi + B\right)}{\sin\left(\phi + B - \frac{\pi}{2}\right)} = \frac{\langle M \rangle_{\phi} - C}{\langle M \rangle_{\phi-\frac{\pi}{2}} - C} = \frac{2\langle M \rangle_{\phi} - \langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}}}{\langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}}}. \tag{11}$$

Taking the inverse tangent we obtain the general estimator for B requiring only three circuit evaluations

$$B = \arctan2\left(2\langle M \rangle_{\phi} - \langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}},\ \langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}}\right) - \phi. \tag{12}$$

Finally, we express the minimizer in closed form

$$\begin{aligned} \theta^* &= \arg\min_{\theta} \langle M \rangle_{\theta} \\ &= -\tfrac{\pi}{2} - B + 2\pi k \\ &= \phi - \tfrac{\pi}{2} - \arctan2\left(2\langle M \rangle_{\phi} - \langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}},\ \langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}}\right) + 2\pi k, \end{aligned} \tag{13}$$

where $\phi \in \mathbb{R}$ and $k \in \mathbb{Z}$. Note that in practice we chose k such that $\theta^* \in (-\pi, \pi]$.
At this point we may also want a general estimator for A. It is easy to verify that

$$A = \tfrac{1}{2}\sqrt{\left(2\langle M \rangle_{\phi} - \langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}}\right)^2 + \left(\langle M \rangle_{\phi+\frac{\pi}{2}} - \langle M \rangle_{\phi-\frac{\pi}{2}}\right)^2}. \tag{14}$$

This is useful when we need to extrapolate the energy attained by the minimizer, $\langle M \rangle_{\theta^*} = -A + C$, and can be done at no additional cost. In summary, Eqs. (10),(12) and (14) completely characterize the sinusoidal form of the energy and allow us to estimate both minimizer and minimum in closed form.

B A few generalizations

We now discuss some generators H_d for which `Rotosolve` applies. Our derivation above is rather general, but in practice Algorithms 1 and 2 rely on the canonical axes of rotation, i.e, tensor products of Pauli matrices.

Our first generalization applies to single-qubits gates. Recall that in order to obtain a sinusoidal form of expectation values we need $H_d^2 = I$. For single-qubit gates this condition is met by any $H_d = c_x X + c_y Y + c_z Z$ such that $(c_x, c_y, c_z) \in \mathbb{R}^3$ is a unit vector. To see why,

$$\begin{aligned} H_d^2 &= (c_x X + c_y Y + c_z Z)(c_x X + c_y Y + c_z Z) \\ &= c_x^2 X^2 + c_x c_y XY + c_x c_z XZ + c_y c_x YX + c_y^2 Y^2 + c_y c_z YZ + c_z c_x ZX + c_z c_y ZY + c_z^2 Z^2 \\ &= c_x^2 I + i c_x c_y Z - i c_x c_z Y - i c_y c_x Z + c_y^2 I + i c_y c_z X + i c_z c_x Y - i c_z c_y X + c_z^2 I \\ &= (c_x^2 + c_y^2 + c_z^2) I = I, \end{aligned} \tag{15}$$

where in the third line we used well-known identities for the Pauli matrices. Therefore, $\langle M \rangle_{\theta_d}$ has sinusoidal form for single-qubit gates of the kind $U_d = \exp\big(-i\frac{\theta_d}{2}(c_x X + c_y Y + c_z Z)\big)$.

This suggests a version of `Rotosolve` where a unit vector of coefficients is sampled at random, and where θ_d is found using Algorithm 1. This version has the potential of finding interesting axes of rotation. Note that a NISQ implementation may require compilation of U_d to lower-level gates.

Our second generalization applies to gates acting on an arbitrary number of qubits. Starting from tensor products of n Pauli matrices, i.e., $H_d \in \{I, X, Y, Z\}^{\otimes n}$, any unitary transformation V provides an alternative valid generator $H'_d = V H_d V^\dagger$. To see why,

$$(H'_d)^2 = V H_d V^\dagger V H_d V^\dagger = V H_d^2 V^\dagger = V V^\dagger = I. \tag{16}$$

The result is an n-qubit gate of the form $U_d = \exp\big(-i\frac{\theta_d}{2} V H_d V^\dagger\big)$, where θ_d is again found using Algorithm 1. We leave the question of how to exploit gates of this kind for future work.